
Magicdog-W SDK Development Documentation

发布 *1.2.1-doc1*

MagicLab

2025 年 12 月 16 日

1	概述	1
1.1	产品 SKU	1
1.2	部件列表	2
1.3	部件名称	3
1.4	电气接口	4
1.5	雷达	6
1.6	指示灯	7
1.7	摄像头	8
1.8	关节	8
1.9	坐标系, 关节旋转轴与关节零点	9
2	机器人规格	11
2.1	尺寸	11
2.2	环境	11
2.3	内部相关	12
3	SDK 概述	15
3.1	SDK 通信接口介绍	15
3.2	获取 SDK	16
4	软硬件架构	17
4.1	软件系统架构图	18
4.2	硬件系统架构图	19
5	服务介绍	21
6	快速开始	23
6.1	SDK 支持概述	23
6.2	系统环境	23

6.3	网络环境	24
6.4	安装与编译	27
6.5	C++ 例程示例	28
6.6	Python 例程示例	29
6.7	其他	32
7	机器人主控制服务 C++	33
7.1	接口定义	33
8	高层运动控制服务 C++	39
8.1	接口定义	39
8.2	类型定义	45
8.3	枚举类型定义	46
8.4	遥控器示意图	47
8.5	高层运动控制机器人状态介绍	47
8.6	高层运动控制接口	48
9	底层运动控制服务 C++	51
9.1	接口定义	51
9.2	类型定义	53
9.3	URDF 参考	55
9.4	底层运动控制机器人状态介绍	55
9.5	注意事项	56
10	传感器控制服务 C++	57
10.1	接口定义	57
10.2	类型定义	66
10.3	3D 雷达数据	70
10.4	注意事项	70
11	语音控制服务 C++	71
11.1	接口定义	71
11.2	类型定义	77
11.3	结构体定义	78
11.4	注意事项	82
12	状态监控服务 C++	83
12.1	接口定义	83
12.2	错误码映射表	85
13	SLAM 导航控制服务 C++	87
13.1	接口定义	87
13.2	类型定义	94
13.3	SLAM 导航控制介绍	96

14 显示控制服务 C++	99
14.1 接口定义	99
14.2 结构体定义	101
14.3 注意事项	101
15 机器人主控制服务 Python	103
15.1 模块信息	103
15.2 接口定义	103
15.3 常量定义	108
15.4 数据结构定义	108
16 高层运动控制服务 Python	109
16.1 接口定义	109
16.2 枚举类型定义	114
16.3 数据结构定义	115
16.4 遥控器示意图	116
16.5 高层运动控制机器人状态介绍	117
16.6 高层运动控制接口	117
17 底层运动控制服务 Python	119
17.1 接口定义	119
17.2 数据结构定义	121
17.3 URDF 参考	124
17.4 底层运动控制机器人状态介绍	124
17.5 注意事项	125
18 传感器控制服务 Python	127
18.1 接口定义	127
18.2 数据结构定义	136
18.3 3D 雷达数据	142
18.4 注意事项	142
19 语音控制服务 Python	143
19.1 接口定义	143
19.2 枚举类型定义	150
19.3 数据结构定义	150
19.4 注意事项	155
20 状态监控服务 Python	157
20.1 接口定义	157
20.2 错误码映射表	158
20.3 枚举类型定义	160
20.4 数据结构定义	161

21 SLAM 导航控制服务 Python	163
21.1 接口定义	163
21.2 枚举类型定义	169
21.3 数据结构定义	170
21.4 SLAM 导航控制介绍	172
22 显示控制服务 Python	173
22.1 接口定义	173
22.2 数据结构定义	175
22.3 注意事项	175
23 高层运动控制示例	177
23.1 C++	177
23.2 Python	185
23.3 运行说明	193
24 底层运动控制示例	195
24.1 C++	195
24.2 Python	200
24.3 运行说明	204
25 传感器控制示例	205
25.1 C++	205
25.2 Python	220
25.3 运行说明	234
26 语音控制示例	237
26.1 C++	237
26.2 Python	247
26.3 运行说明	256
27 状态监控示例	259
27.1 C++	259
27.2 Python	261
27.3 运行说明	263
28 SLAM 导航示例	265
28.1 C++	265
28.2 Python	294
28.3 运行说明	326
29 显示示例	329
29.1 C++	329
29.2 Python	333

29.3 运行说明	338
30 ROS2 SDK 使用说明	339
30.1 SDK 介绍	339
31 机器狗 ROS2 SDK 文档	341
31.1 文档概述	341
31.2 系统架构	341
31.3 通信协议	342
31.4 接口定义	342
31.5 常见问题与故障排查	347
32 FAQ	349
32.1 开发准备环境	349
32.2 MagicDog_W 支持无线开发吗?	349
32.3 MagicDog_W 目前底层通讯频率是多少?	349
32.4 SDK 接口调用报错: Deadline Exceeded	349
32.5 报错: Failed to disable SDK.	350
32.6 SDK 内部配置和日志如何查看?	350
32.7 调用 high_level_motion_example 时, 打印以下错误 Fail initializing after {} seconds. \nPlease check the lcm connection between remote and hardware PC then try again.\n	350
32.8 调用 audio_example 或者 sensor_example 无法拿到对应的数据	350
32.9 如何检查双目相机图像延时	350
32.10 如何获取鱼眼相机或者 4K 相机的数据	351
32.11 SLAM 导航不执行?	351
32.12 运行报错: Invalid IP address	351
32.13 SDK 无法订阅话题数据或者话题数据不生效?	351
32.14 SDK 订阅话题数据频率不稳定?	352
32.15 日志报错打印: Call of pthread_setschedparam with insufficient privileges!	352
32.16 connect 返回报错: Failed to connect to robot, code: ErrorCode.SERVICE_ERROR, message: failed to connect to all addresses; last error: UNKNOWN: ipv4:192.168.55.200:50051: Failed to connect to remote host: Connection refused	352
32.17 如何查看机器本体软件版本?	353

概述

1.1 产品 SKU

参数	MagicDog-W 智能版	MagicDog-W 巡航版
整机尺寸	67 cm x 35 cm x 65 cm	67 cm x 35 cm x 65 cm
整机重量	23kg(含电池)	约 24.36kg（含电池、算力背包、LDS）
电池电压	29.6V	29.6V
电池容量	8200mAh	8200mAh
载重	10kg	10kg
速度范围	0~3m/s	0~3m/s
最小避障距离高度	< 60cm	< 60cm
最大爬坡角度	40°	40°
运动控制器	低功耗高性能 CPU	低功耗高性能 CPU
本体算力	6 TOPS	163 TOPS (Orin NX 16GB)
最大关节力矩 (单腿)	约 37.5N · m	约 37.5N · m
轮胎	七寸充气轮胎	七寸充气轮胎
4K 相机	●	●
双目相机	●	●
广角相机	●	●
超声波传感器	●	●
触摸传感器	●	●

下页继续

表 1 – 续上页

参数	MagicDog-W 智能版	MagicDog-W 巡航版
深度相机	●	●
2D 激光雷达	●	●
3D 激光雷达	○	览沃 Mid-360
智能回充传感器	○	●
APP 控制	●	●
选配遥控器 (T12/北通)	●	●
语音控制模组	●	●
语音指令	●	●
语音交互	●	●
定位建图	○	●
导航避障	○	●
视觉跟随	●	●
WiFi 模组	WIFI 5.0	WIFI 5.0
BT 模组	BT 5.0/4.2/2.1	BT 5.0/4.2/2.1
4G 模组	●	●
二次开发接口	○	●
拓展模块	○	标配算力模组 (Orin NX 157 TOPS)
续航时间	2~4h	2~4h
充电器	标配 (33.6V 6A)	标配 (33.6V 6A)

1.2 部件列表

包装部件如下：



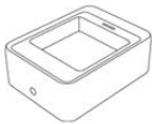
仿生四足机器人



电池



电源适配器



电池充电座

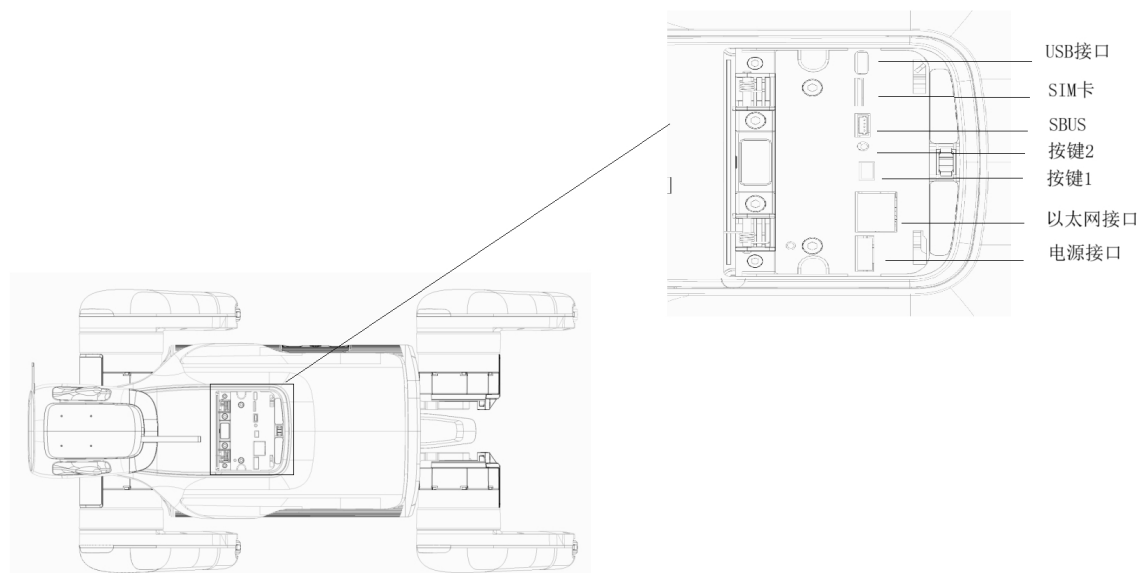


说明书

1.3 部件名称



1.4 电气接口

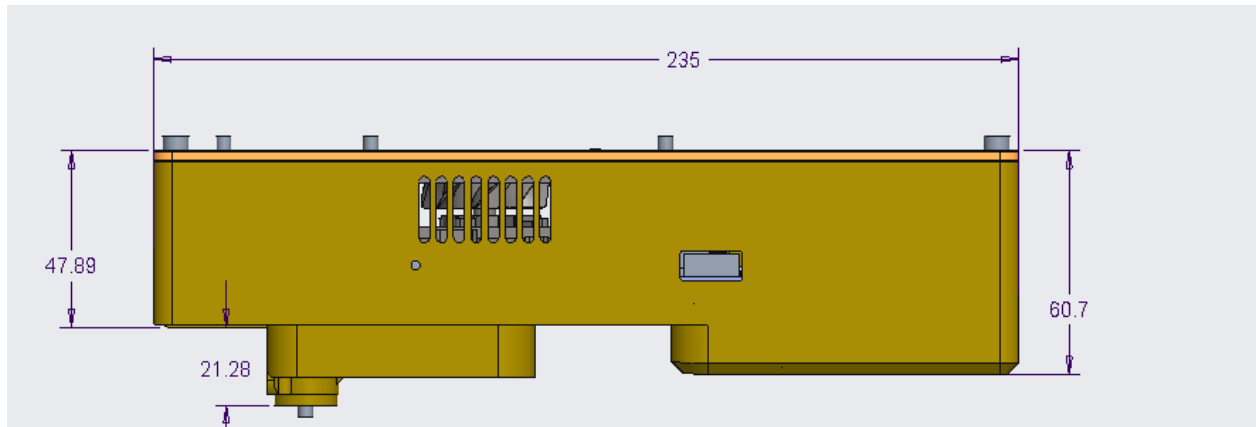


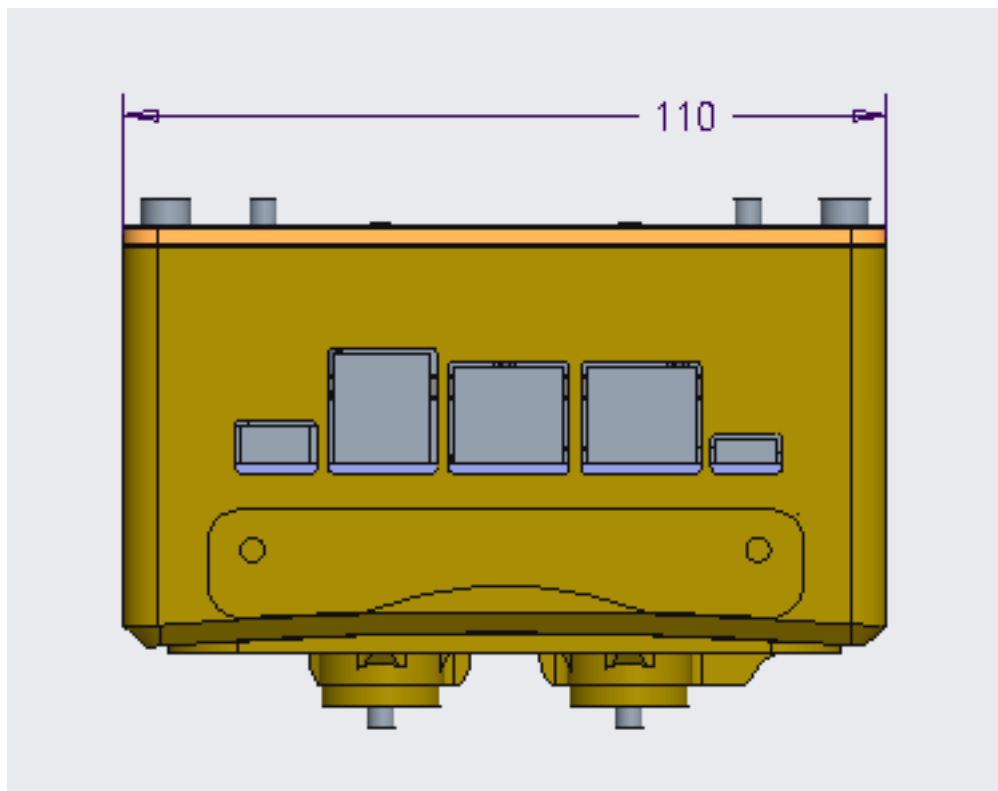
- **电源接口**输出范围 22V~33.6V，给外部设备供电。接口带负载检测引脚，用于控制电源输出。
- **以太网接口**标准 RJ45，连接 PC 等外部设备
- **按键 1** 群控/单控按键
- **按键 2** 配网按键
- **SBUS 接口** UART 通信接口
- **SIM 卡** SIM 卡接口，4G 联网使用。
- **USB 接口**标准 TypeC 接口，支持 USB2.0/USB3.0。

NX 算力盒子对外扩展标准接口

端口名称	接口型号	电压	电流	说明
DC IN	XT30 2+2	24-36V	5A	盒子电源输入接口
DC OUT	XT30	25-33V	3A	内部有限流保险丝
12V OUT	XT30	12.6V	3A	内部有限流保险丝
USB3.0	USB Type-A*2	5V	0.9A*2	2个USB A口
type-C OTG	type-C	5V	0.9A	烧录调试口
RJ45网口	RJ45*3			注意有3个千兆网口
HDMI	HDMI type-A			标准HDMI口

NX 算力盒子的防护等级 IP30，无防水能力





1.5 雷达

激光雷达 STL-19P 采用 DTOF 技术，可进行每秒 5000 次测距，用于运动避障。

技术规格如下：

参数	开发计算单元
测距范围	0.03-12 米（80% 反射率白靶测试）
测距精准度	±30mm @ 2-12m（白靶），标准差 15mm
扫描频率	最小 6 Hz，典型 10 Hz，最大 13 Hz
测距频率	5000 Hz（固定频率）
俯仰角误差	最小 0.5°，最大 2°
偏航角误差	最小 -1°，典型 0°，最大 1°
角度分辨率	典型 0.72° @ 10 Hz
抗环境光能力	最大 60 KLux（参照乐动的环境光测试规范）
雷达噪音	典型 45 dB（雷达正向水平放置，距离 30 厘米，噪声仪型号 AR824 测试）
整机寿命	10000 小时
工作温度	-10°C 至 45°C
存储温度	-30°C 至 70°C
防护等级	IP5X（详见备注“防尘防水”）

巡航版可搭载 Livox Mid-360 激光雷达。

技术规格如下：

参数	规格
测距范围	40 米 @ 10% 反射率, 70 米 @ 80% 反射率
测距精度	±2 厘米
测距频率	200,000 点/秒 (单回波) 480,000 点/秒 (双/三回波)
视场角 (FOV)	水平: 360° 垂直: 59° (-7° 至 52°)
角度分辨率	水平: 0.18° ~ 0.36° (@ 10Hz), 垂直: < 2°
近处盲区	0.1 米
数据接口	1Gbps 以太网
激光波长	905 纳米
人眼安全级别	Class 1 (IEC60825-1:2014)
抗环境光能力	100 klx
雷达噪音	40 dB @ 30 厘米
整机寿命	> 16,000 小时
工作温度	-20°C 至 65°C
存储温度	-40°C 至 85°C
防护等级 (IP)	IP67
尺寸	75 × 65 × 55 毫米
重量	约 265 克
供电电压	10V ~ 17V DC
功耗	典型: 7.5W, 峰值: 10W
时间同步	IEEE 1588-2008 (PTP v2)

1.6 指示灯

尾灯的状态含义如下：

参数	开发计算单元
红色灯常亮	电量低于 20%，设备异常
白色灯常亮	设备正常，任务完成
白色灯快闪 3 次	任务处理中
白色灯熄灭	关机状态
蓝色灯闪烁 3 次	模式切换
红色灯常亮	电量低，设备异常

1.7 摄像头

参数	开发计算单元
4K 相机	人形识别
双目相机	
超广角相机	避障
深度相机	避障

1.8 关节

四足机器人像动物一样，身体（Trunk）和四肢（Leg）都是左右对称的。四条腿，按照前后分成两组，这两组除了前后不同，坐标系和关节活动范围等都相同的。

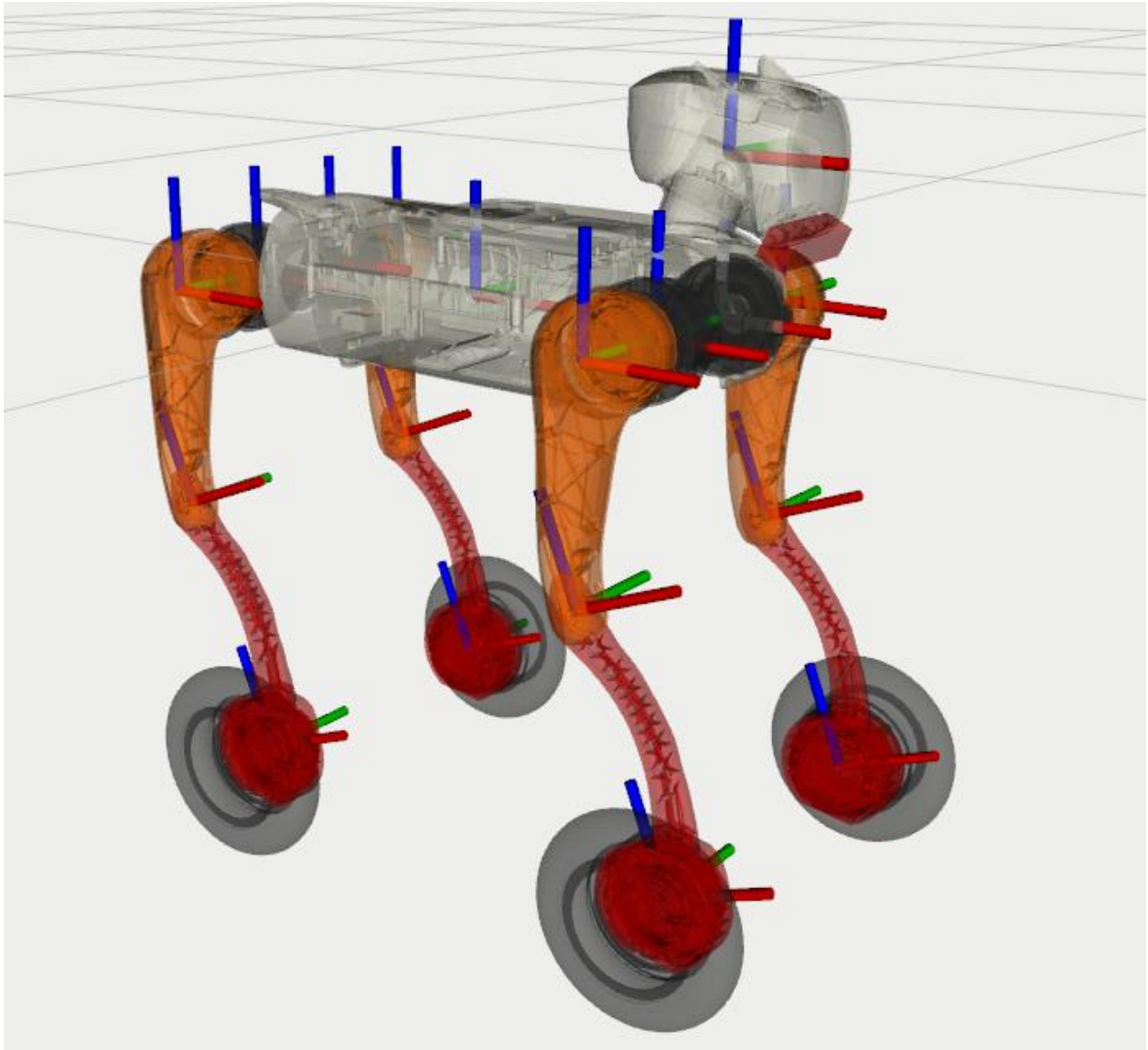
各腿的 index:

- 0: FR (front right)
- 1: FL (front left)
- 2: RR (rear right)
- 3: RL (rear left)

具体的参数表如下:

参数	关节限位
FR_hip_joint	-0.65 ~ 0.79 rad
FR_thigh_joint	-1.27 ~ 3.62 rad
FR_calf_joint	-2.37~ -0.39 rad
FL_hip_joint	-0.79 ~ 0.65 rad
FL_thigh_joint	-1.27 ~ 3.62 rad
FL_calf_joint	-2.37~ -0.39 rad
RR_hip_joint	-0.65 ~ 0.79 rad
RR_thigh_joint	-1.27 ~ 3.62 rad
RR_calf_joint	-2.37~ -0.39 rad
RL_hip_joint	-0.79 ~ 0.65 rad
RL_thigh_joint	-1.27 ~ 3.62 rad
RL_calf_joint	-2.37~ -0.39 rad

1.9 坐标系，关节旋转轴与关节零点



机身关节旋转轴为 x 轴，大腿关节和小腿关节的旋转轴为 y 轴，旋转正方向符合右手定则。

2.1 尺寸

参数	开发计算单元
裸机长宽高（站立）	67cm * 35cm * 65cm
裸机长宽高（趴下）	72cm * 50cm * 29cm
裸机净重（不含电池）	约 22.5kg
自由度	17
最大速度	3.0m/s

2.2 环境

参数	开发计算单元
工作温度	5°C-35°C，天气良好环境下运行
斜坡	+/- 30°
最大台阶高度	15cm

2.3 内部相关

参数	开发计算单元
处理芯片	8 核处理器
4G	可插 SIM 卡
Wi-Fi	WIFI 5.0
蓝牙	BT 5.0/4.2/2.1
存储空间	64G
输出电源	DC 28.8V(电池电压)
连接	RJ45 以太网口

2.3.1 深度相机

参数	开发计算单元
尺寸	124mm29mm26mm
最小深度距离	0.105 米
深度图像分辨率	1280720 @ 30fps; 848480 @ 90 fps
深度视场角	86° * 57° (±3°)

2.3.2 4K 相机

项目	参数说明
图像传感器	1/3.06inch CMOS Sensor
最大分辨率	3840H×2880V
像素大小	1.12um*1.12um
曝光系统	自动曝光 / Auto Exposure
电子快门	滚动快门 / Rolling Shutter
接口速度	480MB/S (USB2.0)
工作温度	0°C ~ 60°C
存储温度	0°C ~ 65°C
焦距	2.22MM
视场角	D:117° H:94.8° V:76.7°
畸变	<10.3%
光谱特性	无 IR (No filter)
光圈	2.2±5%

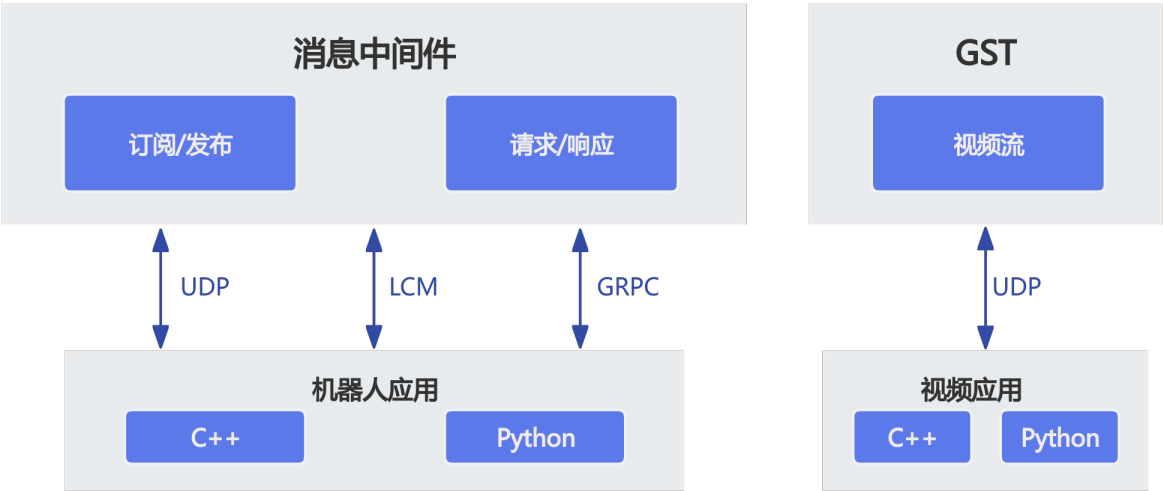
2.3.3 双目相机

参数	参数说明
图像传感器	1/4inch CMOS Sensor
最大分辨率	2560H×720V
像素大小	3.0um*3.0um
信噪比	38dB
动态范围	63.9dB
曝光系统	自动曝光 / Auto Exposure
电子快门	全局快门 / Global shutter
接口速度	480MB/S (USB2.0)
工作温度	0°C ~ 50°C
储存温度	0°C ~ 60°C
连接接口	5-pin 1.25mm USB2.0
图像显示方向	图像正立
焦距	3.15MM
视场角	D:162° H:135° V:73°
畸变	<14.3%
光谱特性	650nm (滤光片 Optical filter)
光圈	2.35

2.3.4 广角相机

参数	参数说明
模组型号	ZV-A0588-V5.1
模组尺寸	32.00mm × 32.00mm × 15mm
工作温度	-30°C to 80°C
储存温度	-10°C to 60°C
相机板装配技术	SMT (ROHS)
拍照焦距距离	30CM~ 无穷远
供电电压	USB_5V
连接接口	USB 2.0 -5P 1.25mm 间距 SMD 立贴
功率	MAX 1W, 电流 140mA~160mA
操作系统	Win XP、Win 7、Linux 2.6.20 以上版本
包装	Anti-electrostatic tray
传感器类型	(1/2.7")
最高有效像素	200 万像素 (WDR)
最大动态范围	85db
像素尺寸	3.0μm x 3.0μm
镜头类型	1/2.7 inch
镜头结构	6G+IR
光圈	2.0
有效焦距	2.0mm
视场角	D:200° H:200° V:200°
畸变	110%
滤光片	650±10nm
增益/曝光/白平衡	Auto

3.1 SDK 通信接口介绍



MagicDog_W 采用 LCM/GRPC/ROS2 作为消息中间件，主要数据交互模式：话题（订阅/发布）和 RPC（请求/响应）

- 话题（订阅/发布）：接收方订阅某个消息，发送方根据订阅列表向接收方发送消息，主要用于中高频率或持续性的数据交互。

- RPC (请求/响应): 采用问答模式, 通过请求实现数据获取或操作控制, 用于低频率或功能切换时的数据交互。

话题和 RPC 接口的主要调用方式: 函数式接口

- 函数式接口: 将 API 调用封装为函数调用, 方便用户使用。

3.2 获取 SDK

magicdog_w-sdk 是魔法原子新一代机器人开发 SDK。该 SDK 将高层运动控制、底层电机控制、语音控制等接口进行了完整封装, 并提供了相应的函数式接口。您可以参考我们提供的 SDK 教程, 学习机器人控制方法, 完成 MagicDog_W 的二次开发。

3.2.1 SDK 下载地址:

[magicdog_w-sdk](#)

SDK 版本与机器本体软件版本对齐记录, 请参考: [ChangeLog](#)

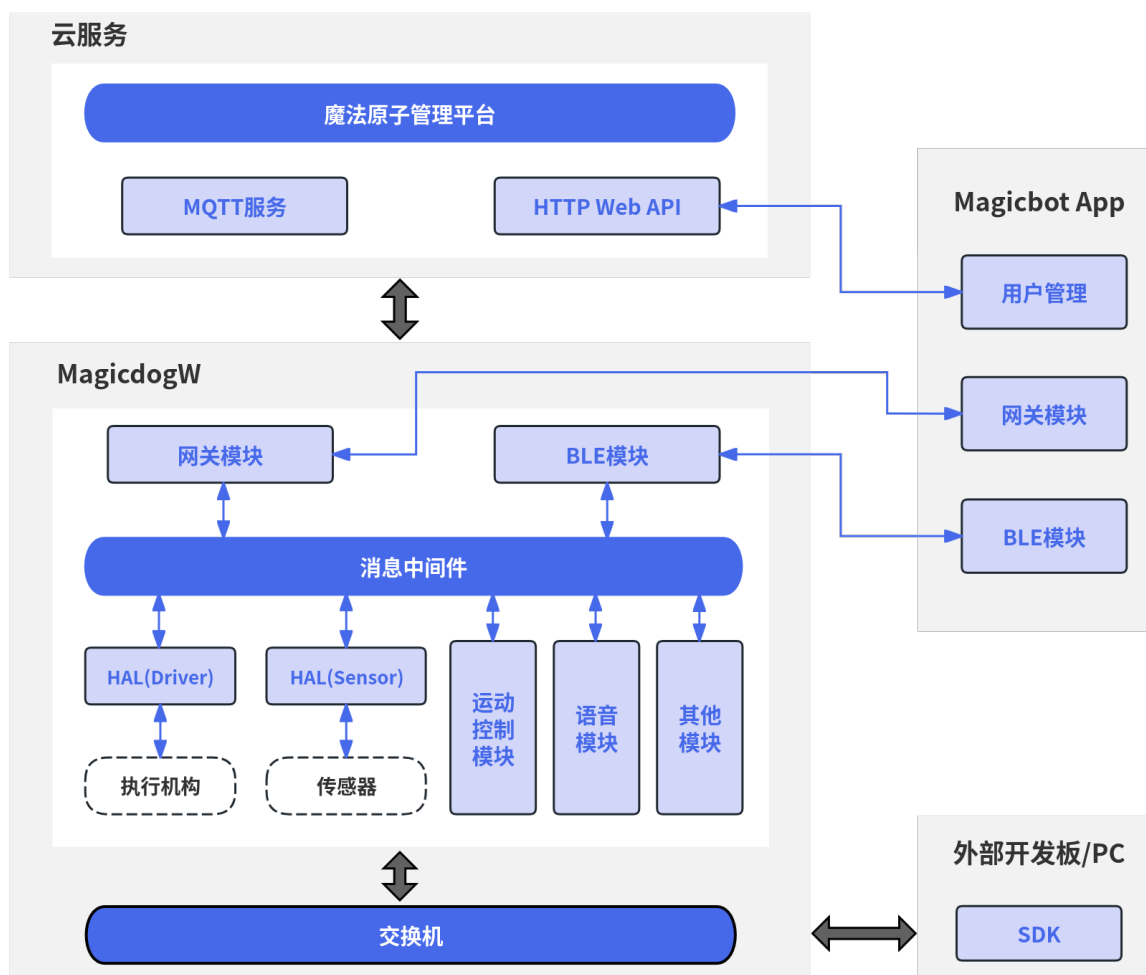
3.2.2 URDF/MJCF 下载地址:

[URDF/MJCF](#)

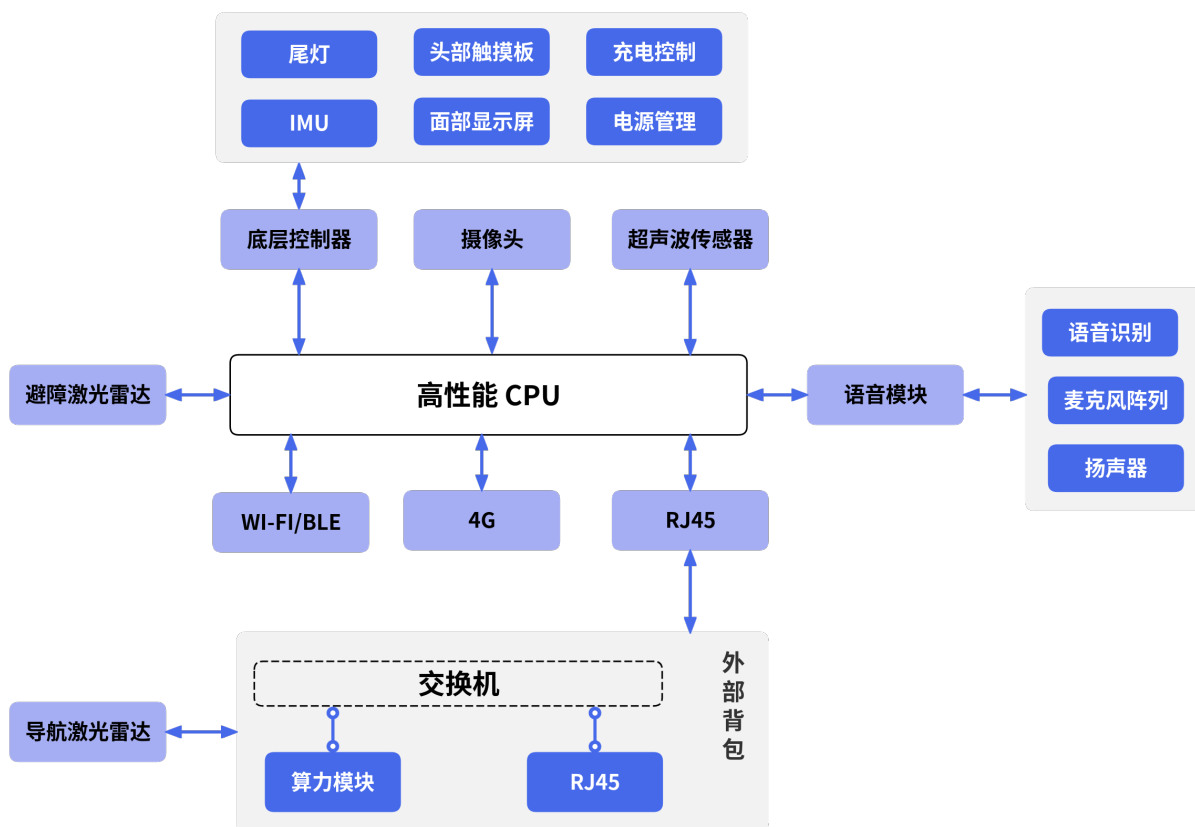
CHAPTER 4

软硬件架构

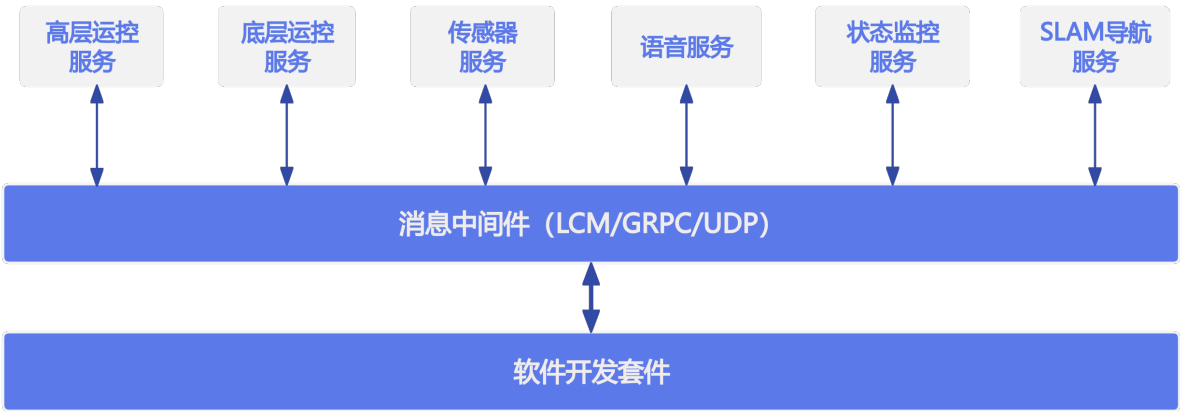
4.1 软件系统架构图



4.2 硬件系统架构图



服务介绍



MagicDog_W 通过消息中间件（LCM/GRPC/ROS2）提供如下服务：

- 高层运控服务

基于 MagicDog_W 内置步态控制器，提供步态切换、特技执行、姿态控制和速度控制等功能（等价于遥控器操作）。高层运控服务使用 GRPC 进行通信。

- 底层运控服务

通过服务接口可实时获取关节状态、IMU 数据等信息，同时可实时下发关节控制指令。底层运控服务使用 LCM 进行通信。

- 语音服务

通过服务接口可控制音量、语音播放、获取原始语音数据等。语音服务使用 GRPC 进行通信，主题数据通过 LCM 进行传输。

- **传感器服务**

支持雷达、RGBD 相机、双目相机等传感器的数据订阅。传感器服务使用 GRPC/ROS2/LCM 进行通信。

- **状态监控服务**

通过服务接口可订阅机器人本体的软硬件故障信息和 BMS 电池状态。状态监控服务使用 GRPC 进行通信。

- **SLAM 导航服务**

通过服务接口可进行机器人 SLAM 建图和导航控制。SLAM 导航服务使用 GRPC/LCM 进行通信。

6.1 SDK 支持概述

如果没有额外配置算力背包 + 3D Lidar，请按顺序配置开发电脑环境; 如果配置了算力背包 + 3D Lidar，登录到算力背包直接开发;

6.2 系统环境

推荐在 Ubuntu 22.04 系统下开发，暂不支持 Mac/Windows 系统下开发。机器人本体 PC 运行官方服务，不支持开发环境。

APP 和 SDK 不支持同时控制机器狗

6.2.1 开发环境要求

- GCC $\geq$ 11.4 (for Linux)
- CMake $\geq$ 3.16
- Make build system
- C++20 (minimum)
- Eigen3
- python3.10

6.2.2 系统配置

首先，为了实现普通用户下通信实时性，需要在/etc/security/limits.conf 文件中增加如下配置：

```
*      -   rtprio    98
```

其次，为了增加每个 socket 链接的接收缓存，需要在/etc/sysctl.conf 文件中增加如下配置，sudo sysctl -p 立即生效或重启生效：

```
net.core.rmem_max=20971520
net.core.rmem_default=20971520
net.core.wmem_max=20971520
net.core.wmem_default=20971520
```

6.3 网络环境

将用户计算机与机器人交换机接入统一网络。建议新用户使用网线将用户计算机接入机器人交换机，并将与机器人通信的网卡设置在 192.168.55.X 网段下，推荐使用 192.168.55.10。有经验的用户可自行配置网络环境。

假设 SDK 二次开发 PC 与机器人连接的网口为 eno1，需要进行如下配置以便 SDK 与机器人的底层通信：

```
sudo ifconfig eno1 multicast
sudo route add -net 224.0.0.0 netmask 240.0.0.0 dev eno1
```

为避免每次网卡拔插都需要重新配置网卡多播路由，用户可以选择在其二开 PC 上进行如下操作（要求操作系统：ubuntu22.04，并且本系统网络配置工具是 NetworkManager）：

- 检查服务状态：

```
$ systemctl is-active NetworkManager
active
```

当显示为：active，则可以继续下面操作；否则，跳过下面操作；

- 编辑配置，sudo vim /etc/NetworkManager/dispatcher.d/90-add-mcast-route:

```
#!/bin/bash

IFACE="$1"
STATUS="$2"

# 目标网卡
TARGET_IF="eno1"

# 仅在指定网卡 carrier on 或 up 时执行
if [ "$IFACE" = "$TARGET_IF" ] && [ "$STATUS" = "up" || "$STATUS" = "carrier" ]; then
```

(下页继续)

(续上页)

```
# 删除可能存在的旧路由
ip route del 224.0.0.0/4 dev "$TARGET_IF" 2>/dev/null

# 添加你需要的 route (带 scope link)
ip route add 224.0.0.0/4 dev "$TARGET_IF" scope link
fi
```

- 加上执行权限:

```
sudo chmod +x /etc/NetworkManager/dispatcher.d/90-add-mcast-route
```

- 启动生效:

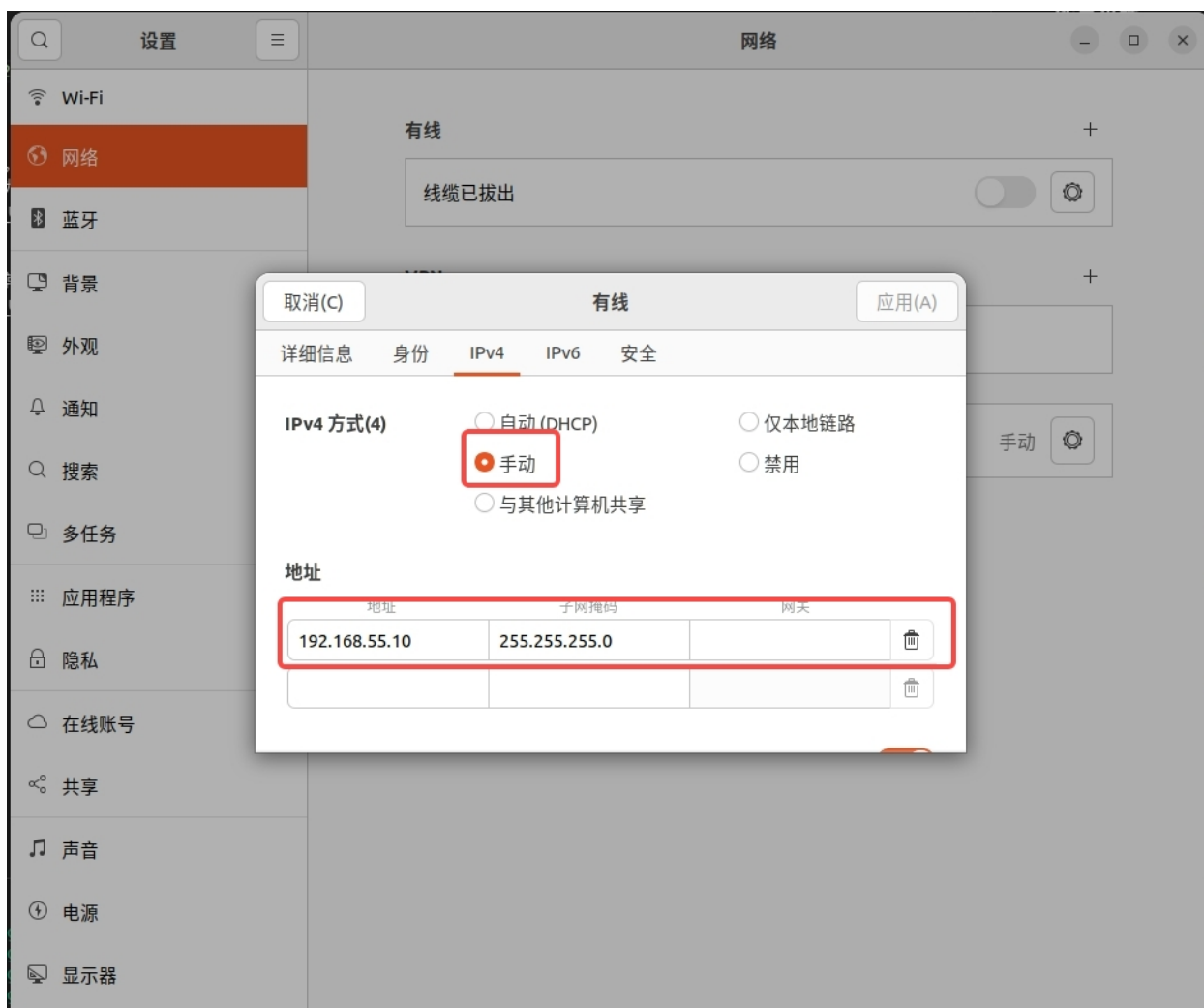
```
sudo systemctl restart NetworkManager
```

检查网卡路由配置是否正常:

```
$ ip route | grep 224.0.0.0
224.0.0.0/4 dev eno1 scope link
```

6.3.1 配置步骤

1. 用网线的一端连接机器人, 另一端连接用户电脑。机器人机载电脑的 IP 地址为 192.168.55.200, 所以需要将电脑 ip 设置为同一网段, 建议 192.168.55.10。



2. 为了测试通信连接是否正常，可以通过 ping 进行测试：

```
eame@eame:~$ ping 192.168.55.200
PING 192.168.55.200 (192.168.55.200) 56(84) bytes of data.
64 bytes from 192.168.55.200: icmp_seq=1 ttl=64 time=1.61 ms
64 bytes from 192.168.55.200: icmp_seq=2 ttl=64 time=0.960 ms
64 bytes from 192.168.55.200: icmp_seq=3 ttl=64 time=0.807 ms
64 bytes from 192.168.55.200: icmp_seq=4 ttl=64 time=0.823 ms
64 bytes from 192.168.55.200: icmp_seq=5 ttl=64 time=1.07 ms
64 bytes from 192.168.55.200: icmp_seq=6 ttl=64 time=0.984 ms
64 bytes from 192.168.55.200: icmp_seq=7 ttl=64 time=0.786 ms
64 bytes from 192.168.55.200: icmp_seq=8 ttl=64 time=1.09 ms
```

6.4 安装与编译

无算力背包，则在开发电脑上；有算力背包 + 3D Lidar，则在算力背包上以下步骤假设工作目录为 /home/magicbot/workspace

6.4.1 安装 magicdog_w-sdk

```
cd /home/magicbot/workspace/magicdog_w_sdk/
mkdir build
cd build
cmake .. -DCMAKE_INSTALL_PREFIX=/opt/magic_robotics/magicdog_w_sdk
make -j8
sudo make install
```

上述命令会把 magicdog_w_sdk 安装到 /opt/magic_robotics/magicdog_w_sdk 目录下

6.4.2 例程编译

```
cd /home/magicbot/workspace/magicdog_w_sdk/
mkdir build
cd build
cmake .. -DBUILD_EXAMPLES=ON
make -j8
```

运行上述命令后，若进度进行到 100% 且没有报错，则意味着编译成功。

```
[ 7%] Building CXX object example/monitor_example/CMakeFiles/monitor_example.dir/monitor_example.cpp.o
[ 14%] Linking CXX executable ../../monitor_example
[ 14%] Built target monitor_example
[ 21%] Building CXX object example/low_level_motion_example/CMakeFiles/low_level_motion_example.dir/low_level_motion_example.cpp.o
[ 28%] Linking CXX executable ../../low_level_motion_example
[ 28%] Built target low_level_motion_example
[ 35%] Building CXX object example/high_level_motion_example/CMakeFiles/recovery_stand_example.dir/recovery_stand_example.cpp.o
[ 42%] Linking CXX executable ../../recovery_stand_example
[ 42%] Built target recovery_stand_example
[ 50%] Building CXX object example/high_level_motion_example/CMakeFiles/execute_trick_action_example.dir/execute_trick_action_example.cpp.o
[ 57%] Linking CXX executable ../../execute_trick_action_example
[ 57%] Built target execute_trick_action_example
[ 64%] Building CXX object example/high_level_motion_example/CMakeFiles/joy_control_example.dir/joy_control_example.cpp.o
[ 71%] Linking CXX executable ../../joy_control_example
[ 71%] Built target joy_control_example
[ 78%] Building CXX object example/audio_example/CMakeFiles/audio_example.dir/audio_example.cpp.o
[ 85%] Linking CXX executable ../../audio_example
[ 85%] Built target audio_example
[ 92%] Building CXX object example/sensor_example/CMakeFiles/sensor_example.dir/sensor_example.cpp.o
[100%] Linking CXX executable ../../sensor_example
[100%] Built target sensor_example
Install the project
```

6.4.3 用户模块导入 magicdog_w-sdk

如果用户需要在自己的模块中引入 magicdog_w-sdk，可以参考 example/cmake_example/CMakeLists.txt

6.5 C++ 例程示例

magicdog_w_sdk/build 目录中：

- 语音示例：
 - audio_example
- 传感器示例：
 - sensor_example
- 状态监控示例：
 - monitor_example
- 底层运控示例：
 - low_level_motion_example
- 高层运控示例：
 - high_level_motion_example
- slam 示例：
 - slam_example
- 导航示例：
 - navigation_example
- 显示示例：
 - display_example

6.5.1 进入调试模式：

按照操作流程，确保机器人进入调试模式

6.5.2 运行例程

进入 magicdog_w_sdk/build 目录，执行如下命令：

```
cd /home/magicbot/workspace/magicdog_w_sdk/build
# 环境配置
sudo ifconfig eno1 multicast
sudo route add -net 224.0.0.0 netmask 240.0.0.0 dev eno1
export LD_LIBRARY_PATH=/opt/magic_robotics/magicdog_w_sdk/lib:$LD_LIBRARY_PATH

# 执行语音示例
./audio_example
```

6.6 Python 例程示例

magicdog_w_sdk/example/python 目录中：

- 语音示例:
 - audio_example.py
- 传感器示例:
 - sensor_example.py
- 状态监控示例:
 - monitor_example.py
- 底层运控示例:
 - low_level_motion_example.py
- 高层运控示例:
 - high_level_motion_example.py
- slam 示例:
 - slam_example.py
- 导航示例:
 - navigation_example.py
- 显示示例:
 - display_example.py

6.6.1 进入调试模式：

按照操作流程，确保机器人进入调试模式

6.6.2 运行例程

进入 magicdog_sdk/example/python 目录，执行如下命令：

```
cd /home/magicbot/workspace/magicdog_w_sdk/example/python/  
  
# 环境配置  
sudo ifconfig eno1 multicast  
sudo route add -net 224.0.0.0 netmask 240.0.0.0 dev eno1  
export PYTHONPATH=/opt/magic_robotics/magicdog_w_sdk/lib:$PYTHONPATH  
export LD_LIBRARY_PATH=/opt/magic_robotics/magicdog_w_sdk/lib:$LD_LIBRARY_PATH
```

(下页继续)

(续上页)

```
# 执行语音示例
python3 audio_example.py
```

注意：手动执行 `sudo route add -net 224.0.0.0 netmask 240.0.0.0 dev eno1` 只会生效一次，开机重启或者网线断连之后，需要重新执行

获取 python api 帮助信息：

```
cd /home/magicbot/workspace/magicdog_w_sdk/example/python
# 环境配置
export PYTHONPATH=/opt/magic_robotics/magicdog_w_sdk/lib:$PYTHONPATH
export LD_LIBRARY_PATH=/opt/magic_robotics/magicdog_w_sdk/lib:$LD_LIBRARY_PATH

$ python3
Python 3.10.12 (main, Aug 15 2025, 14:32:43) [GCC 11.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
# 导入Magicdog python SDK接口
>>> import magicdog_w_python
# 查看所有接口信息
>>> help(magicdog_w_python)
# 查看CameraInfo结构信息
>>> help(magicdog_w_python.CameraInfo)
# 查看HighLevelMotionController对象信息
>>> help(magicdog_w_python.HighLevelMotionController)
# 查看LowLevelMotionController对象信息
>>> help(magicdog_w_python.LowLevelMotionController)
# 查看SensorController对象信息
>>> help(magicdog_w_python.SensorController)
# 查看AudioController对象信息
>>> help(magicdog_w_python.AudioController)
# 查看StateMonitor对象信息
>>> help(magicdog_w_python.StateMonitor)
```

比如，如果想查看特技动作 `TrickAction` 的枚举值：

```
$ python3
Python 3.10.12 (main, Aug 15 2025, 14:32:43) [GCC 11.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import magicdog_w_python
>>> help(magicdog_w_python.TrickAction)

Help on class TrickAction in module magicdog_w_python:

class TrickAction(pybind11_builtins.pybind11_object)
```

(下页继续)

(续上页)

```
| Method resolution order:
|     TrickAction
|     pybind11_builtins.pybind11_object
|     builtins.object
|
| Methods defined here:
|
|     __eq__(...)
|         __eq__(self: object, other: object, /) -> bool
|
|     __getstate__(...)
|         __getstate__(self: object, /) -> int
|
|     __hash__(...)
|         __hash__(self: object, /) -> int
|
|     __index__(...)
|         __index__(self: magicdog_w_python.TrickAction, /) -> int
|
|     __init__(...)
|         __init__(self: magicdog_w_python.TrickAction, value: typing.SupportsInt) -> None
|
|     __int__(...)
|         __int__(self: magicdog_w_python.TrickAction, /) -> int
|
|     __ne__(...)
|         __ne__(self: object, other: object, /) -> bool
|
|     __repr__(...)
|         __repr__(self: object, /) -> str
|
|     __setstate__(...)
|         __setstate__(self: magicdog_w_python.TrickAction, state: typing.SupportsInt, /) -> None
|
|     __str__(...)
|         __str__(self: object, /) -> str
|
| -----
| Readonly properties defined here:
|
|     __members__
|
|     name
```

(下页继续)

(续上页)

```
|     name(self: object, /) -> str
|
|     value
|
|     -----
|     Data and other attributes defined here:
|
|     ACTION_LIE_DOWN = <TrickAction.ACTION_LIE_DOWN: 102>
|
|     ACTION_NONE = <TrickAction.ACTION_NONE: 0>
|
|     -----
|     Static methods inherited from pybind11_builtins.pybind11_object:
|
|     __new__(*args, **kwargs) from pybind11_builtins.pybind11_type
|         Create and return a new object.  See help(type) for accurate signature.
```

6.7 其他

6.7.1 SDK 配置文件

SDK 运行时默认会在/tmp 目录下生成其默认配置文件，如果配置文件已存在则使用已有配置：

```
$ ls /tmp/magicdog_w_mjrrt.yaml
/tmp/magicdog_w_mjrrt.yaml
```

6.7.2 SDK 日志

SDK 运行时内部的日志信息默认生成在/tmp/logs 目录下：

```
$ ls /tmp/logs/magicdog_w_sdk.log
/tmp/logs/magicdog_w_sdk.log
```


机器人主控制服务 C++

提供机器人系统主控制器，通过 **MagicRobot** 可以进行资源初始化、管理通信连接、访问各子控制器如运动控制器、音频控制器、状态监控以及传感器控制器等。

7.1 接口定义

MagicRobot 是机器人系统的统一入口类。

7.1.1 MagicRobot

项目	内容
函数名	MagicRobot
函数声明	<code>MagicRobot();</code>
功能概述	构造函数，创建 MagicRobot 实例。
备注	构造内部状态。

7.1.2 ~MagicRobot

项目	内容
函数名	<code>~MagicRobot</code>
函数声明	<code>~MagicRobot();</code>
功能概述	析构函数，释放 MagicRobot 实例资源。
备注	确保资源安全释放。

7.1.3 Initialize

项目	内容
函数名	<code>Initialize</code>
函数声明	<code>bool Initialize(const std::string& local_ip);</code>
功能概述	初始化机器人系统，包括控制器与网络模块。
参数说明	<code>local_ip</code> : 本地通信 IP 地址。
返回值	<code>true</code> 表示成功， <code>false</code> 表示失败。
备注	首次使用前必须调用。

7.1.4 Shutdown

项目	内容
函数名	<code>Shutdown</code>
函数声明	<code>void Shutdown();</code>
功能概述	关闭机器人系统，释放资源。
备注	配合 Initialize 使用。

7.1.5 Release

项目	内容
函数名	<code>Release</code>
函数声明	<code>void Release();</code>
功能概述	释放机器人系统资源。
备注	调用该函数可手动释放 SDK 内部占用的资源。

7.1.6 Connect

项目	内容
函数名	Connect
函数声明	Status Connect(int timeout_ms);
功能概述	与机器人服务建立通信连接。
参数	timeout_ms: 超时时间 (毫秒), 默认值为 5000。
返回值	Status::OK 表示成功。
备注	阻塞接口, 需初始化后调用。

7.1.7 Disconnect

项目	内容
函数名	Disconnect
函数声明	Status Disconnect(int timeout_ms);
功能概述	断开与机器人服务的连接。
参数	timeout_ms: 超时时间 (毫秒), 默认值为 5000。
返回值	gRPC 调用状态。
备注	阻塞接口, 与 Connect 配对使用。

7.1.8 GetSDKVersion

项目	内容
函数名	GetSDKVersion
函数声明	std::string GetSDKVersion() const;
功能概述	获取当前 SDK 的版本号。
返回值	SDK 版本字符串 (如 0.0.1)。
备注	非阻塞接口, 便于调试或日志标记。

7.1.9 GetMotionControlLevel

项目	内容
函数名	GetMotionControlLevel
函数声明	ControllerLevel GetMotionControlLevel();
功能概述	获取当前运动控制级别 (高层/低层)。
返回值	ControllerLevel 枚举值。
备注	非阻塞接口, 用于判断当前控制权限。

7.1.10 SetMotionControlLevel

项目	内容
函数名	SetMotionControlLevel
函数声明	Status SetMotionControlLevel(ControllerLevel level);
功能概述	设置当前运动控制级别。
参数说明	level: 控制权限枚举值。
返回值	Status::OK 表示设置成功, 其他代表设置失败。
备注	阻塞接口, 控制模式切换时使用。

7.1.11 GetHighLevelMotionController

项目	内容
函数名	GetHighLevelMotionController
函数声明	HighLevelMotionController& GetHighLevelMotionController();
功能概述	获取高层运动控制器对象。
返回值	引用类型, 用于调用高层控制接口。
备注	非阻塞接口, 封装了步态、特技、遥控等控制功能。

7.1.12 GetLowLevelMotionController

项目	内容
函数名	GetLowLevelMotionController
函数声明	LowLevelMotionController& GetLowLevelMotionController();
功能概述	获取低层运动控制器对象。
返回值	引用类型, 用于控制关节或电机。
备注	非阻塞接口, 可直接控制关节电机等。

7.1.13 GetAudioController

项目	内容
函数名	GetAudioController
函数声明	AudioController& GetAudioController();
功能概述	获取音频控制器对象。
返回值	引用类型, 用于语音控制。
备注	非阻塞接口, 可播放语音和控制音量。

7.1.14 GetSensorController

项目	内容
函数名	GetSensorController
函数声明	SensorController& GetSensorController();
功能概述	获取传感器控制器对象。
返回值	引用类型，用于访问传感器数据。
备注	非阻塞接口，封装了 IMU、RGBD 等传感器数据读取。

7.1.15 GetStateMonitor

项目	内容
函数名	GetStateMonitor
函数声明	StateMonitor& GetStateMonitor();
功能概述	获取状态监控器对象。
返回值	引用类型，用于获取机器人当前状态信息。
备注	非阻塞接口，封装了 BMS、故障等状态信息的读取。

7.1.16 GetSlamNavController

项目	内容
函数名	GetSlamNavController
函数声明	SlamNavController& GetSlamNavController();
功能概述	获取 SLAM/导航控制器对象。
返回值	引用类型，用于访问地图、定位等 SLAM 相关数据。
备注	非阻塞接口，封装地图构建、定位、导航等相关功能的使用。

7.1.17 OpenChannelSwitch

项目	内容
函数名	OpenChannelSwitch
函数声明	Status OpenChannelSwitch(int timeout_ms);
功能概述	打开 lcm 通道开关。
参数	timeout_ms (可选参数，单位毫秒)，操作超时时间，默认 5000 ms。
返回值	Status 枚举，表示操作状态。
备注	用于控制 SDK lcm 通信通道的启用。

7.1.18 CloseChannelSwitch

项目	内容
函数名	CloseChannelSwitch
函数声明	Status CloseChannelSwitch(int timeout_ms);
功能概述	关闭 lcm 通道开关。
参数	timeout_ms (可选参数, 单位毫秒), 操作超时时间, 默认 5000 ms。
返回值	Status 枚举, 表示操作状态。
备注	用于控制 SDK lcm 通信通道的关闭。

高层运动控制服务 C++

提供机器人系统高层运动控制服务，通过 HighLevelMotionController 可以通过 RPC 通信方式实现对机器人的步态、特技、遥控的控制。

8.1 接口定义

HighLevelMotionController 是面向语义控制的高层运动控制器，支持如行走、特技等控制操作，封装底层细节以供上层系统调用。

△ **Notice:** 某些特技动作，比如前空翻、后空翻对场地要求较大，执行时需要额外注意，建议所有特技动作在大的空场地测试后，再在小的场地使用。

8.1.1 HighLevelMotionController

项目	内容
函数名	HighLevelMotionController
函数声明	HighLevelMotionController();
功能概述	构造函数，初始化高层控制器状态。
备注	构造内部控制资源。

8.1.2 ~HighLevelMotionController

项目	内容
函数名	<code>~HighLevelMotionController</code>
函数声明	<code>virtual ~HighLevelMotionController();</code>
功能概述	析构函数，释放控制器资源。
备注	配合构造函数使用。

8.1.3 Initialize

项目	内容
函数名	<code>Initialize</code>
函数声明	<code>virtual bool Initialize() override;</code>
功能概述	初始化控制器，准备高层控制功能。
返回值	<code>true</code> 表示成功， <code>false</code> 表示失败。
备注	首次使用前必须调用。

8.1.4 Shutdown

项目	内容
函数名	<code>Shutdown</code>
函数声明	<code>virtual void Shutdown() override;</code>
功能概述	关闭控制器并释放资源。
备注	配合 <code>Initialize</code> 使用。

8.1.5 SetGait

项目	内容
函数名	<code>SetGait</code>
函数声明	<code>Status SetGait(const GaitMode gait_mode, int timeout_ms);</code>
功能概述	设置机器人步态模式（如位控站立、力控站立、慢跑等）。
参数说明	<code>gait_mode</code> : 步态控制枚举。 <code>timeout_ms</code> : 超时时间（毫秒），默认值为 5000。
返回值	<code>Status::OK</code> 表示成功，其他为失败。
备注	阻塞接口，支持多种步态模式切换。

8.1.6 GetGait

项目	内容
函数名	GetGait
函数声明	Status GetGait(GaitMode& gait_mode, int timeout_ms);
功能概述	获取机器人步态模式（如位控站立、力控站立、慢跑等）。timeout_ms: 超时时间（毫秒），默认为 5000。
参数说明	gait_mode: 步态控制枚举。
返回值	Status::OK 表示成功，其他为失败。
备注	阻塞接口，获取当前步态模式。

8.1.7 ExecuteTrick

项目	内容
函数名	ExecuteTrick
函数声明	Status ExecuteTrick(const TrickAction trick_action, int timeout_ms);
功能概述	执行特技动作（如扭屁股、趴下等）。
参数说明	trick_action: 特技动作标识。timeout_ms: 超时时间（毫秒），默认为 5000。
返回值	Status::OK 表示成功，其他为失败。
备注	阻塞接口，需确保机器人当前处于可执行特技的状态。

- 注意事项：特技动作必须在 GaitMode::GAIT_STAND_R(2) 位控站立步态下才能执行

8.1.8 EnableJoyStick

项目	内容
函数名	EnableJoyStick
函数声明	Status EnableJoyStick();
功能概述	开启摇杆控制命令。
参数说明	无。
返回值	Status::OK 表示操作成功，其他为失败。
备注	用于允许通过摇杆发送实时控制指令。

8.1.9 DisableJoyStick

项目	内容
函数名	DisableJoyStick
函数声明	Status DisableJoyStick();
功能概述	关闭摇杆控制命令。
参数说明	无。
返回值	Status::OK 表示操作成功，其他为失败。
备注	切换导航模式之前，必须先关闭摇杆控制命令。

8.1.10 SendJoyStickCommand

项目	内容
函数名	SendJoyStickCommand
函数声明	Status SendJoyStickCommand(const JoystickCommand& joy_command);
功能概述	发送实时摇杆控制指令。
参数说明	joy_command: 包含摇杆坐标的控制数据。
返回值	Status::OK 表示成功，其他为失败。
备注	非阻塞接口，建议发送频率为 20Hz。

8.1.11 GetAllGaitSpeedRatio

项目	内容
函数名	GetAllGaitSpeedRatio
函数声明	Status GetAllGaitSpeedRatio(AllGaitSpeedRatio& gait_speed_ratios, int timeout_ms);
功能概述	获取所有步态以及对应前进、横移、旋转速度比例。
参数说明	gait_speed_ratios: 所有步态以及对应前进、横移、旋转速度比例。timeout_ms: 超时时间(毫秒)，默认值为 5000。
返回值	Status::OK 表示成功，其他为失败。
备注	阻塞接口，用于获取所有步态的速度比例配置。

8.1.12 SetGaitSpeedRatio

项目	内容
函数名	SetGaitSpeedRatio
函数声明	Status SetGaitSpeedRatio(GaitMode gait_mode, const GaitSpeedRatio& gait_speed_ratio, int timeout_ms);
功能概述	设置步态以及对应前进、横移、旋转速度比例。
参数说明	gait_mode: 步态模式 gait_speed_ratio: 步态以及对应前进、横移、旋转速度比例 timeout_ms: 超时时间（毫秒），默认值为 5000。
返回值	Status::OK 表示成功，其他为失败。
备注	阻塞接口，用于设置特定步态的速度比例配置。

8.1.13 GetHeadMotorEnabled

项目	内容
函数名	GetHeadMotorEnabled
函数声明	Status GetHeadMotorEnabled(bool& enabled, int timeout_ms);
功能概述	获取头部电机使能状态。
参数说明	enabled: 返回参数，true 表示已使能，false 表示未使能。timeout_ms: 超时时间（毫秒），默认值为 5000。
返回值	Status::OK 表示成功，其他为失败。
备注	阻塞接口，用于查询头部电机当前使能状态。

8.1.14 EnableHeadMotor

项目	内容
函数名	EnableHeadMotor
函数声明	Status EnableHeadMotor(int timeout_ms);
功能概述	使能头部电机。
参数说明	timeout_ms: 超时时间（毫秒），默认值为 5000。
返回值	Status::OK 表示成功，其他为失败。
备注	阻塞接口，用于启用头部电机控制。

8.1.15 DisableHeadMotor

项目	内容
函数名	DisableHeadMotor
函数声明	Status DisableHeadMotor(int timeout_ms);
功能概述	关闭头部电机。
参数说明	timeout_ms: 超时时间（毫秒），默认值为 5000。
返回值	Status::OK 表示成功，其他为失败。
备注	阻塞接口，用于禁用头部电机控制。

8.1.16 GetHeadPosition

项目	内容
函数名	GetHeadPosition
函数声明	Status GetHeadPosition(EulerAngles& euler_angles, int timeout_ms);
功能概述	获取头部位置。
参数说明	euler_angles: 返回参数，头部当前绝对位置。timeout_ms: 超时时间（毫秒），默认值为 5000。
返回值	Status::OK 表示成功，其他为失败。
备注	阻塞接口，用于获取头部位置。

8.1.17 SetHeadPosition

项目	内容
函数名	SetHeadPosition
函数声明	Status SetHeadPosition(const EulerAngles& euler_angles, int timeout_ms);
功能概述	设置头部位置。
参数说明	euler_angles: 头部目标绝对位置。timeout_ms: 超时时间（毫秒），默认值为 5000。
返回值	Status::OK 表示成功，其他为失败。
备注	阻塞接口，用于设置头部位置。

8.2 类型定义

8.2.1 GaitSpeedRatio —步态速度比例结构体

字段名	类型	描述
straight_ratio	double	前进速度比例
turn_ratio	double	旋转速度比例
lateral_ratio	double	横移速度比例

8.2.2 AllGaitSpeedRatio —所有步态速度比例结构体

字段名	类型	描述
gait_speed_ratios	std::map<GaitMode, GaitSpeedRatio>	步态模式到速度比例的映射

8.2.3 JoystickCommand —高层运动控制摇杆指令结构体

字段名	类型	描述
left_x_axis	float	左侧摇杆的 X 轴方向值 (-1.0: 左, 1.0: 右)
left_y_axis	float	左侧摇杆的 Y 轴方向值 (-1.0: 下, 1.0: 上)
right_x_axis	float	右侧摇杆的 X 轴方向值 (旋转 -1.0: 左, 1.0: 右)
right_y_axis	float	右侧摇杆的 Y 轴方向值 (暂未定义用途)

8.2.4 EulerAngles —欧拉角结构体

字段名	类型	描述
roll	double	横滚角 (Roll), 绕 X 轴旋转的角度 (单位为弧度)
pitch	double	俯仰角 (Pitch), 绕 Y 轴旋转的角度 (单位为弧度)
yaw	double	偏航角 (Yaw), 绕 Z 轴旋转的角度 (单位为弧度)

8.3 枚举类型定义

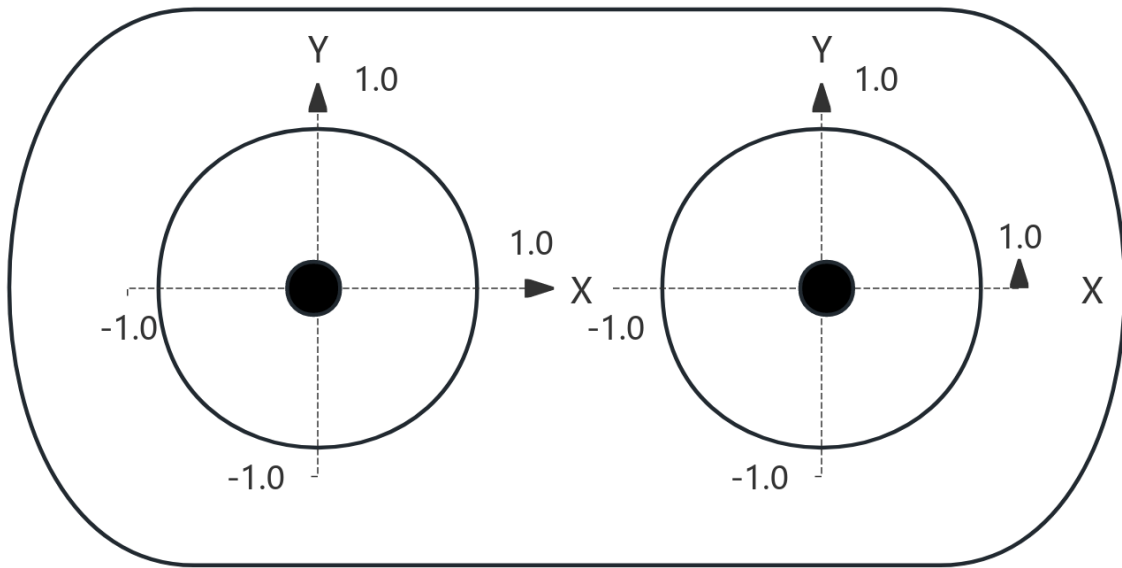
8.3.1 GaitMode —机器人步态模式枚举

枚举值	数值	描述
GAIT_PASSIVE	0	掉落（关闭电机使能）
GAIT_PURE_DAMPER	1	纯阻尼
GAIT_STAND_R	2	位控站立、RecoveryStand
GAIT_LOWLEVL_SDK	30	低层 SDK 步态
GAIT_RL_MOVE_QUICK	111	快速移动
GAIT_RL_TERRAIN	112	全地形
GAIT_RL_CLIMB	113	爬楼梯
GAIT_RL_HAND_STAND	114	倒立
GAIT_RL_FOOT_STAND	115	正立
GAIT_NONE	9999	无步态

8.3.2 TrickAction —特技动作指令枚举

枚举值	数值	描述
ACTION_NONE	0	无动作
ACTION_LIE_DOWN	102	趴下（Lie down）

8.4 遥控器示意图



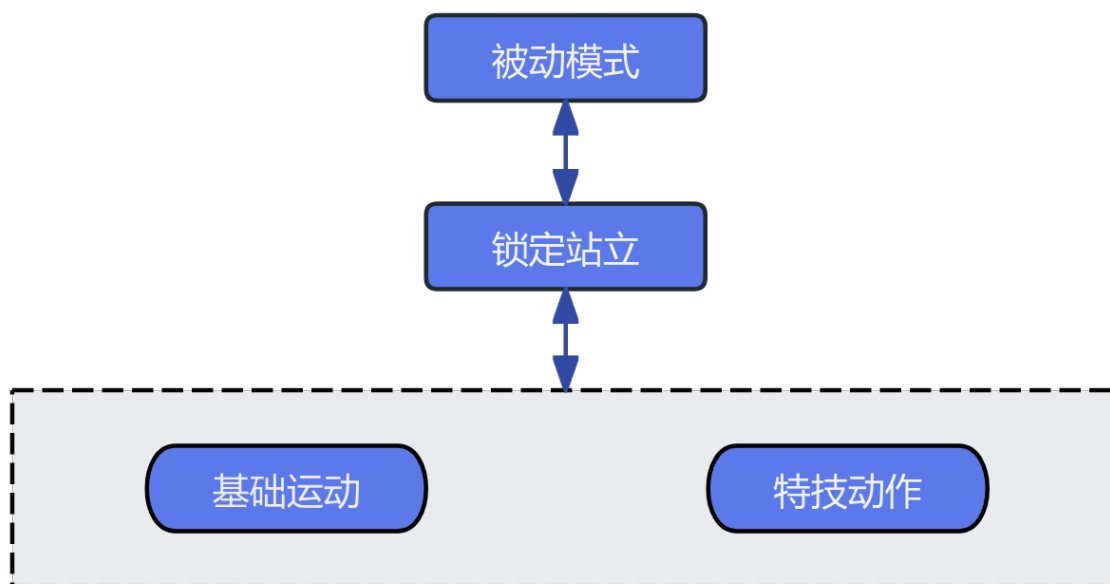
1. 左右摇杆 x 轴和 y 轴的取值范围为 $[-1.0, 1.0]$;
2. 左右摇杆 x 轴和 y 轴的方向上/右为正，如示意图所示；

8.5 高层运动控制机器人状态介绍

机器人的运动包含位控站立、基础运动、特技动作状态，机器人在运行过程中，通过状态机在不同状态之间进行切换，以实现不同的控制任务。各个状态的解释说明如下：

- 位控站立：在位控站立状态下，可调用 SDK 的各部分接口实现机器人的特技动作和基础运动控制。
- 基础运动：在运动执行过程中，可调用 SDK 接口，让机器人进入不同的步态。
- 特技动作：当进入特殊动作执行状态后，其他运动控制服务会先被挂起，等待当前动作执行完毕并进入位控站立状态后再生效。

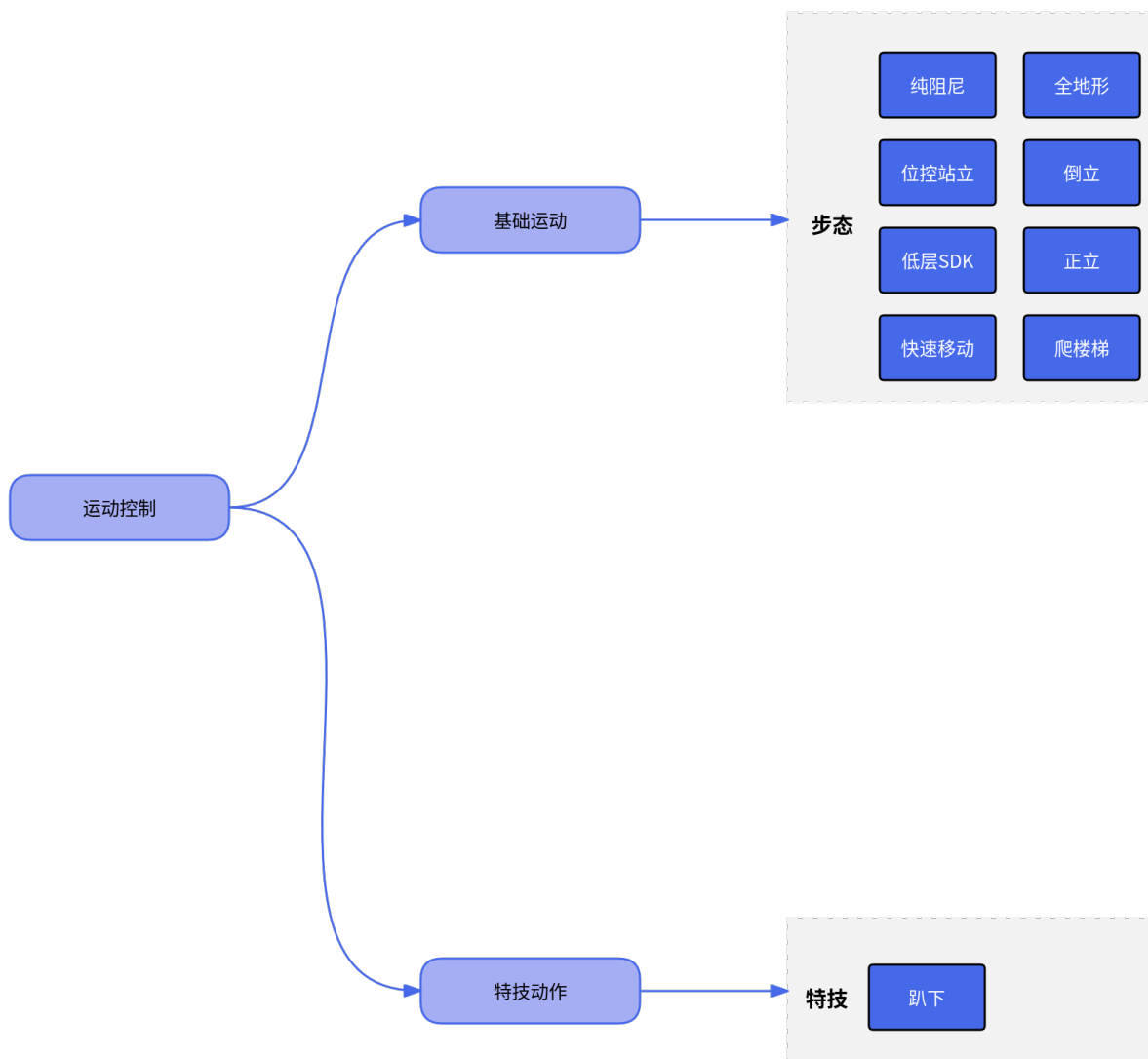
机器人状态切换机制；



8.6 高层运动控制接口

机器人的高层运动控制服务可分为基础运动控制和特技动作控制。

- 基础运动控制服务中，可调用相应的接口，根据不同的地形场景和任务需求切换机器人的行走步态。
- 特技动作控制服务中，可调用相应的接口，实现机器人内置的特殊特技，比如扭屁股、趴下等。



底层运动控制服务 C++

提供机器人系统底层运动控制服务，通过 LowLevelMotionController 可以使用话题通信方式实现对机器人关节的指令控制和状态获取。

9.1 接口定义

LowLevelMotionController 是面向底层开发的运动控制器，支持对腿等运动部件的直接控制与状态订阅。

△ **Notice:** 调用 PublishLegCommand 接口，用户命令才会下发到运控，因此需要用户保证调用 PublishLegCommand 的频率，建议是 500 hz。

9.1.1 LowLevelMotionController

项目	内容
函数名	LowLevelMotionController
函数声明	LowLevelMotionController();
功能概述	构造函数，初始化低层控制器对象。
备注	构造内部资源。

9.1.2 ~LowLevelMotionController

项目	内容
函数名	<code>~LowLevelMotionController</code>
函数声明	<code>virtual ~LowLevelMotionController();</code>
功能概述	析构函数，释放资源。
备注	清理底层资源。

9.1.3 Initialize

项目	内容
函数名	<code>Initialize</code>
函数声明	<code>virtual bool Initialize() override;</code>
功能概述	初始化控制器，建立底层连接。
返回值	<code>true</code> 表示成功， <code>false</code> 表示失败。
备注	首次调用必须初始化。

9.1.4 Shutdown

项目	内容
函数名	<code>Shutdown</code>
函数声明	<code>virtual void Shutdown() override;</code>
功能概述	关闭控制器，释放底层资源。
备注	配合 <code>Initialize</code> 使用。

9.1.5 SubscribeLegState

项目	内容
函数名	<code>SubscribeLegState</code>
函数声明	<code>void SubscribeLegState(LegJointStateCallback callback);</code>
功能概述	订阅下肢关节状态数据
参数说明	<code>callback</code> : 回调函数，用于处理下肢关节状态的接收数据
备注	非阻塞接口。

9.1.6 UnsubscribeLegState

项目	内容
函数名	UnsubscribeLegState
函数声明	<code>void UnsubscribeLegState();</code>
功能概述	取消订阅下肢关节状态数据
备注	非阻塞接口。与 SubscribeLegState 配合使用。

9.1.7 PublishLegCommand

项目	内容
函数名	PublishLegCommand
函数声明	<code>Status PublishLegCommand(const LegJointCommand& command);</code>
功能概述	发布下肢关节控制指令
参数说明	command: 包含目标角度/速度等控制信息的双关节控制指令
返回值	Status::OK 表示成功, 其他为失败。
备注	非阻塞接口。

9.2 类型定义

9.2.1 SingleLegJointCommand — 单个下肢关节的控制命令

字段名	类型	描述
q_des	float	期望关节位置
dq_des	float	期望关节速度
tau_des	float	期望前馈力矩
kp	float	P 增益, 必须为正数
kd	float	D 增益, 必须为正数

9.2.2 SingleWheelJointCommand —单个轮子关节的控制命令

字段名	类型	描述
q_des	double	期望关节位置
dq_des	double	期望关节速度
tau_des	double	期望前馈力矩
kp	double	P 增益，必须为正数
kd	double	D 增益，必须为正数

9.2.3 LegJointCommand —整个下肢控制命令

字段名	类型	描述
timestamp	int64_t	时间戳（单位：纳秒）
cmd	std::array<SingleLegJointCommand, kLegJointNum>;	关节控制命令数组
wheel_cmd	std::array<SingleWheelJointCommand, kWheelJointNum>;	轮子控制命令数组

9.2.4 SingleLegJointState —单个下肢关节的状态

字段名	类型	描述
q	float	实际关节位置
dq	float	实际关节速度
tau_est	float	估计力矩

9.2.5 SingleWheelJointState —单个轮子关节的状态

字段名	类型	描述
q	double	实际轮子关节位置
dq	double	实际轮子关节速度
tau_est	double	估计轮子关节力矩

9.2.6 LegState — 整个下肢状态信息

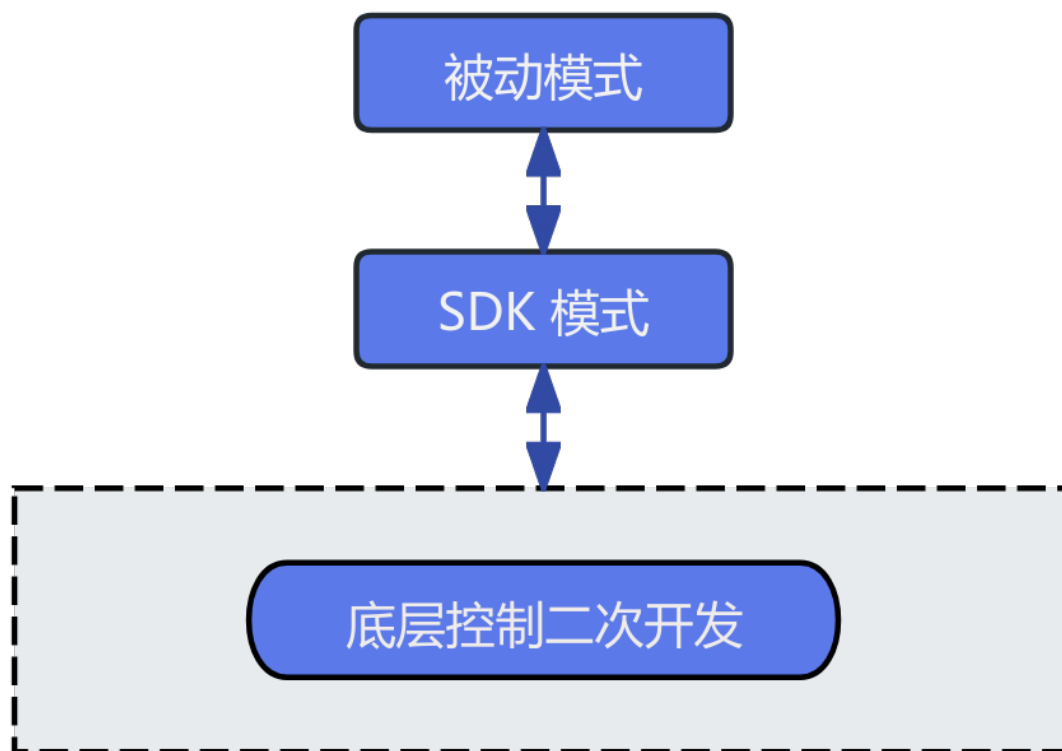
字段名	类型	描述
timestamp	int64_t	时间戳（单位：纳秒）
state	std::array<SingleLegJointState, kLegJointNum>;	所有下肢关节状态
wheel_state	std::array<SingleWheelJointState, kWheelJointNum>;	所有轮子关节状态

9.3 URDF 参考

机器人 URDF

9.4 底层运动控制机器人状态介绍

机器人底层运动主要是开发关节的三环控制给开发人员进行机器人运动能力的二次开发，基本的控制状态切换机制：



9.5 注意事项

在使用 Subscribe 等订阅类 SDK 接口时，请避免在回调函数中执行阻塞式的数据处理操作。否则，可能导致消息堆积、处理延迟，甚至引发不可预期的错误。

传感器控制服务 C++

提供机器人系统传感器（雷达/RGBD 相机/双目相机）服务，通过 `SensorController` 可以通过 RPC 和话题方式实现对机器人传感器的指令控制和状态获取。

10.1 接口定义

`SensorController` 是封装机器人各类传感器的管理类，支持 Laser Scan、RGBD 相机、双目相机的初始化、控制与数据订阅。

10.1.1 `SensorController`

项目	内容
函数名	<code>SensorController</code>
函数声明	<code>SensorController();</code>
功能概述	构造函数，初始化传感器控制器对象。
备注	构造内部状态。

10.1.2 ~SensorController

项目	内容
函数名	~SensorController
函数声明	virtual ~SensorController();
功能概述	析构函数，释放所有传感器资源。
备注	调用前应关闭传感器。

10.1.3 Initialize

项目	内容
函数名	Initialize
函数声明	bool Initialize();
功能概述	初始化控制器，包括资源申请和驱动加载。
返回值	true 表示成功，false 表示失败。
备注	调用前需先构造对象。

10.1.4 Shutdown

项目	内容
函数名	Shutdown
函数声明	void Shutdown();
功能概述	关闭所有传感器连接并释放资源。
备注	配合 Initialize 使用。

10.1.5 OpenLaserScan

项目	内容
函数名	OpenLaserScan
函数声明	Status OpenLaserScan(int timeout_ms);
功能概述	打开雷达。
参数说明	timeout_ms: 超时时间（毫秒），默认值为 5000。
返回值	Status::OK 表示成功，其他为失败。
备注	阻塞接口。

10.1.6 CloseLaserScan

项目	内容
函数名	CloseLaserScan
函数声明	Status CloseLaserScan(int timeout_ms);
功能概述	关闭雷达。
参数说明	timeout_ms: 超时时间 (毫秒), 默认值为 5000。
返回值	Status::OK 表示成功, 其他为失败。
备注	阻塞接口, 配合打开函数使用。

10.1.7 OpenRgbCamera

项目	内容
函数名	OpenRgbCamera
函数声明	Status OpenRgbCamera(int timeout_ms);
功能概述	打开 RGBD 相机。
参数说明	timeout_ms: 超时时间 (毫秒), 默认值为 5000。
返回值	Status::OK 表示成功, 其他为失败。
备注	阻塞接口。

10.1.8 CloseRgbCamera

项目	内容
函数名	CloseRgbCamera
函数声明	Status CloseRgbCamera(int timeout_ms);
功能概述	关闭 RGBD 相机。
参数说明	timeout_ms: 超时时间 (毫秒), 默认值为 5000。
返回值	Status::OK 表示成功, 其他为失败。
备注	阻塞接口, 配合打开函数使用。

10.1.9 OpenBinocularCamera

项目	内容
函数名	OpenBinocularCamera
函数声明	Status OpenBinocularCamera(int timeout_ms);
功能概述	打开双目相机。
参数说明	timeout_ms: 超时时间 (毫秒), 默认值为 5000。
返回值	Status::OK 表示成功, 其他为失败。
备注	阻塞接口。

10.1.10 CloseBinocularCamera

项目	内容
函数名	CloseBinocularCamera
函数声明	Status CloseBinocularCamera(int timeout_ms);
功能概述	关闭双目相机。
参数说明	timeout_ms: 超时时间 (毫秒), 默认值为 5000。
返回值	Status::OK 表示成功, 其他为失败。
备注	阻塞接口, 配合打开函数使用。

10.1.11 SubscribeUltra

项目	内容
函数名	SubscribeUltra
函数声明	void SubscribeUltra(const UltraCallback callback);
功能概述	订阅超声波数据
参数说明	callback: 接收到超声波数据后的处理回调
备注	非阻塞接口, 回调函数会在数据更新时被调用。

10.1.12 UnsubscribeUltra

项目	内容
函数名	UnsubscribeUltra
函数声明	void UnsubscribeUltra();
功能概述	取消订阅超声波数据
备注	非阻塞接口, 用于停止接收超声波数据。

10.1.13 SubscribeHeadTouch

项目	内容
函数名	SubscribeHeadTouch
函数声明	<code>void SubscribeHeadTouch(const HeadTouchCallback callback);</code>
功能概述	订阅头部触摸数据
参数说明	callback: 接收到头部触摸数据后的处理回调
备注	非阻塞接口, 回调函数会在数据更新时被调用。

10.1.14 UnsubscribeHeadTouch

项目	内容
函数名	UnsubscribeHeadTouch
函数声明	<code>void UnsubscribeHeadTouch();</code>
功能概述	取消订阅头部触摸数据
备注	非阻塞接口, 用于停止接收头部触摸数据。

10.1.15 SubscribeLaserScan

项目	内容
函数名	SubscribeLaserScan
函数声明	<code>void SubscribeLaserScan(const LaserScanCallback callback);</code>
功能概述	订阅激光雷达数据
参数说明	callback: 接收到激光雷达数据后的处理回调
备注	非阻塞接口, 回调函数会在数据更新时被调用。

10.1.16 UnsubscribeLaserScan

项目	内容
函数名	UnsubscribeLaserScan
函数声明	<code>void UnsubscribeLaserScan();</code>
功能概述	取消订阅激光雷达数据
备注	非阻塞接口, 用于停止接收激光雷达数据。

10.1.17 SubscribeRgbDepthCameraInfo

项目	内容
函数名	SubscribeRgbDepthCameraInfo
函数声明	<code>void SubscribeRgbDepthCameraInfo(const CameraInfoCallback callback);</code>
功能概述	订阅 RGBD 深度相机内参数数据
参数说明	callback: 接收到 RGBD 深度相机内参数数据后的处理回调
备注	非阻塞接口, 回调函数会在数据更新时被调用。

10.1.18 UnsubscribeRgbDepthCameraInfo

项目	内容
函数名	UnsubscribeRgbDepthCameraInfo
函数声明	<code>void UnsubscribeRgbDepthCameraInfo();</code>
功能概述	取消订阅 RGBD 深度相机内参数数据
备注	非阻塞接口, 用于停止接收 RGBD 深度相机内参数数据。

10.1.19 SubscribeRgbDepthImage

项目	内容
函数名	SubscribeRgbDepthImage
函数声明	<code>void SubscribeRgbDepthImage(const ImageCallback callback);</code>
功能概述	订阅 RGBD 深度图像数据
参数说明	callback: 接收到 RGBD 深度图像数据后的处理回调
备注	非阻塞接口, 回调函数会在数据更新时被调用。

10.1.20 UnsubscribeRgbDepthImage

项目	内容
函数名	UnsubscribeRgbDepthImage
函数声明	<code>void UnsubscribeRgbDepthImage();</code>
功能概述	取消订阅 RGBD 深度图像数据
备注	非阻塞接口, 用于停止接收 RGBD 深度图像数据。

10.1.21 SubscribeRgbColorCameraInfo

项目	内容
函数名	SubscribeRgbColorCameraInfo
函数声明	<code>void SubscribeRgbColorCameraInfo(const CameraInfoCallback callback);</code>
功能概述	订阅 RGBD 彩色图像内参数数据
参数说明	callback: 接收到 RGBD 彩色图像内参数数据后的处理回调
备注	非阻塞接口, 回调函数会在数据更新时被调用。

10.1.22 UnsubscribeRgbColorCameraInfo

项目	内容
函数名	UnsubscribeRgbColorCameraInfo
函数声明	<code>void UnsubscribeRgbColorCameraInfo();</code>
功能概述	取消订阅 RGBD 彩色图像内参数数据
备注	非阻塞接口, 用于停止接收 RGBD 彩色图像内参数数据。

10.1.23 SubscribeRgbColorImage

项目	内容
函数名	SubscribeRgbColorImage
函数声明	<code>void SubscribeRgbColorImage(const ImageCallback callback);</code>
功能概述	订阅 RGBD 彩色图像数据
参数说明	callback: 接收到 RGBD 彩色图像数据后的处理回调
备注	非阻塞接口, 回调函数会在数据更新时被调用。

10.1.24 UnsubscribeRgbColorImage

项目	内容
函数名	UnsubscribeRgbColorImage
函数声明	<code>void UnsubscribeRgbColorImage();</code>
功能概述	取消订阅 RGBD 彩色图像数据
备注	非阻塞接口, 用于停止接收 RGBD 彩色图像数据。

10.1.25 SubscribeImu

项目	内容
函数名	SubscribeImu
函数声明	<code>void SubscribeImu(const ImuCallback callback);</code>
功能概述	订阅 IMU 数据
参数说明	callback: 接收到 IMU 数据后的处理回调
备注	非阻塞接口, 回调函数会在数据更新时被调用。

10.1.26 UnsubscribeImu

项目	内容
函数名	UnsubscribeImu
函数声明	<code>void UnsubscribeImu();</code>
功能概述	取消订阅 IMU 数据
备注	非阻塞接口, 用于停止接收 IMU 数据。

10.1.27 SubscribeLeftBinocularHighImg

项目	内容
函数名	SubscribeLeftBinocularHighImg
函数声明	<code>void SubscribeLeftBinocularHighImg(const CompressedImageCallback callback);</code>
功能概述	订阅左侧高质量双目数据
参数说明	callback: 接收到左侧高质量双目数据后的处理回调
备注	非阻塞接口, 回调函数会在数据更新时被调用。

10.1.28 UnsubscribeLeftBinocularHighImg

项目	内容
函数名	UnsubscribeLeftBinocularHighImg
函数声明	<code>void UnsubscribeLeftBinocularHighImg();</code>
功能概述	取消订阅左侧高质量双目数据
备注	非阻塞接口, 用于停止接收左侧高质量双目数据。

10.1.29 SubscribeLeftBinocularLowImg

项目	内容
函数名	SubscribeLeftBinocularLowImg
函数声明	<code>void SubscribeLeftBinocularLowImg(const CompressedImageCallback callback);</code>
功能概述	订阅左侧低质量双目数据
参数说明	callback: 接收到左侧低质量双目数据后的处理回调
备注	非阻塞接口, 回调函数会在数据更新时被调用。

10.1.30 UnsubscribeLeftBinocularLowImg

项目	内容
函数名	UnsubscribeLeftBinocularLowImg
函数声明	<code>void UnsubscribeLeftBinocularLowImg();</code>
功能概述	取消订阅左侧低质量双目数据
备注	非阻塞接口, 用于停止接收左侧低质量双目数据。

10.1.31 SubscribeRightBinocularLowImg

项目	内容
函数名	SubscribeRightBinocularLowImg
函数声明	<code>void SubscribeRightBinocularLowImg(const CompressedImageCallback callback);</code>
功能概述	订阅右侧低质量双目数据
参数说明	callback: 接收到右侧低质量双目数据后的处理回调
备注	非阻塞接口, 回调函数会在数据更新时被调用。

10.1.32 UnsubscribeRightBinocularLowImg

项目	内容
函数名	UnsubscribeRightBinocularLowImg
函数声明	<code>void UnsubscribeRightBinocularLowImg();</code>
功能概述	取消订阅右侧低质量双目数据
备注	非阻塞接口, 用于停止接收右侧低质量双目数据。

10.1.33 SubscribeDepthImage

项目	内容
函数名	SubscribeDepthImage
函数声明	<code>void SubscribeDepthImage(const ImageCallback callback);</code>
功能概述	订阅深度图像
参数说明	callback : 接收到深度图像后的处理回调
备注	非阻塞接口，回调函数会在数据更新时被调用。

10.1.34 UnsubscribeDepthImage

项目	内容
函数名	UnsubscribeDepthImage
函数声明	<code>void UnsubscribeDepthImage();</code>
功能概述	取消订阅深度图像
备注	非阻塞接口，用于停止接收深度图像数据。

10.2 类型定义

10.2.1 Imu —IMU 数据结构体

字段名	类型	描述
timestamp	int64_t	时间戳（单位：纳秒）
orientation[4]	double[4]	姿态四元数（w, x, y, z）
angular_velocity[3]	double[3]	角速度（单位：rad/s）
linear_acceleration[3]	double[3]	线加速度（单位：m/s ² ）
temperature	float	温度（单位：摄氏度或其他，应明确单位）

10.2.2 Header —通用消息头结构体

字段名	类型	描述
stamp	int64_t	时间戳（单位：纳秒）
frame_id	std::string	坐标系名称

10.2.3 PointField 一点字段描述

字段名	类型	描述
name	std::string	字段名（如 x、y、z、intensity）
offset	int32_t	起始字节偏移
datatype	int8_t	数据类型（对应常量）
count	int32_t	每点此字段包含元素数量

10.2.4 PointCloud2 一点云数据结构体

字段名	类型	描述
header	Header	消息头
height	int32_t	行数
width	int32_t	列数
fields	std::vector<PointField>	点字段数组
is_bigendian	bool	字节序
point_step	int32_t	每个点的字节数
row_step	int32_t	每行的字节数
data	std::vector<uint8_t>	原始点云字节数据
is_dense	bool	是否稠密点云（无无效点）

10.2.5 Image 图像数据结构体

字段名	类型	描述
header	Header	消息头
height	int32_t	图像高度（像素）
width	int32_t	图像宽度（像素）
encoding	std::string	编码类型（如 rgb8, mono8）
is_bigendian	bool	是否为大端模式
step	int32_t	每行图像占用字节数
data	std::vector<uint8_t>	原始图像字节数据

10.2.6 CameraInfo —相机内参与畸变信息

字段名	类型	描述
header	Header	消息头
height	int32_t	图像高度
width	int32_t	图像宽度
distortion_model	std::string	畸变模型（如 plumb_bob）
D	std::vector<double>	畸变参数数组
K	double[9]	相机内参矩阵
R	double[9]	矫正矩阵
P	double[12]	投影矩阵
binning_x	int32_t	水平 binning 系数
binning_y	int32_t	垂直 binning 系数
roi_x_offset	int32_t	ROI 起始 x 坐标
roi_y_offset	int32_t	ROI 起始 y 坐标
roi_height	int32_t	ROI 高度
roi_width	int32_t	ROI 宽度
roi_do_rectify	bool	是否进行矫正

10.2.7 CompressedImage —压缩图像数据结构

字段名	类型	描述
header	Header	消息头
format	std::string	压缩格式（如 jpeg, png）
data	std::vector<uint8_t>	压缩后的图像数据

10.2.8 LaserScan — 激光雷达数据结构

字段名	类型	描述
header	Header	消息头
angle_min	int32_t	起始角度（弧度）
angle_max	int32_t	结束角度（弧度）
angle_increment	int32_t	角度增量（弧度）
time_increment	int32_t	时间增量（秒）
scan_time	int32_t	扫描时间（秒）
range_min	int32_t	最小距离（米）
range_max	int32_t	最大距离（米）
ranges	std::vector<double>	距离数据数组（单位：米）
intensities	std::vector<double>	强度数据数组（无量纲）

10.2.9 MultiArrayDimension — 多维数组维度描述

字段名	类型	描述
label	std::string	维度标签
size	int32_t	维度大小
stride	int32_t	步长

10.2.10 MultiArrayLayout — 多维数组布局描述

字段名	类型	描述
dim_size	int32_t	维度数量
dim	std::vector<MultiArrayDimension>	维度数组
data_offset	int32_t	数据偏移量

10.2.11 Float32MultiArray — 浮点数多维数组

字段名	类型	描述
layout	MultiArrayLayout	数组布局信息
data	std::vector<float>	浮点数数据数组

10.2.12 Int8 — 8 位整数数据结构

字段名	类型	描述
data	int8_t	8 位整数数据

10.2.13 HeadTouch — 头部触摸数据结构

项目	内容
类型	Int8
描述	头部触摸传感器数据，使用 8 位整数格式

10.3 3D 雷达数据

由于 3D Lidar 数据在 nx 上以 ros2 话题的方式发布，为了避免引入不必要的延迟，SDK 未对其进行封装。如果需要使用，则可以使用以下方法查找 3D Lidar 数据

```
source /etc/eame/ros2.env
ros2 topic echo /lidar_point_cloud
```

- 消息类型：sensor_msgs::msg::PointCloud2
- 该消息类型属于 ros2 标准消息类型，详情可以参考 https://docs.ros.org/en/noetic/api/sensor_msgs/html/msg/PointCloud2.html

10.4 注意事项

在使用 Subscribe 等订阅类 SDK 接口时，请避免在回调函数中执行阻塞式的数据处理操作。否则，可能导致消息堆积、处理延迟，甚至引发不可预期的错误。

提供机器人系统语音服务控制器，通过 `AudioController` 可以使用 RPC 方式实现对机器人音频的指令控制和状态获取。

11.1 接口定义

`AudioController` 是一个封装音频控制功能的 C++ 类，主要用于音频播放控制、TTS 播放、音量设置、查询，订阅语音原始数据等场景。

11.1.1 `AudioController`

项目	内容
函数名	<code>AudioController</code>
函数声明	<code>AudioController();</code>
功能概述	初始化音频控制器对象，构造内部状态，分配资源等。
备注	构造内部状态。

11.1.2 ~AudioController

项目	内容
函数名	<code>~AudioController</code>
函数声明	<code>~AudioController();</code>
功能概述	释放音频控制器资源，确保停止播放并清理底层资源。
备注	确保资源安全释放。

11.1.3 Initialize

项目	内容
函数名	<code>Initialize</code>
函数声明	<code>bool Initialize();</code>
功能概述	初始化音频控制模块，准备播放资源与设备。
返回值	<code>true</code> 表示成功， <code>false</code> 表示失败。
备注	与 <code>Shutdown()</code> 配对使用。

11.1.4 Shutdown

项目	内容
函数名	<code>Shutdown</code>
函数声明	<code>void Shutdown();</code>
功能概述	关闭音频控制器并释放资源。
备注	确保在销毁前调用。

11.1.5 GetVoiceConfig

项目	内容
函数名	GetVoiceConfig
函数声明	Status GetVoiceConfig(GetSpeechConfig& config, int timeout_ms);
功能概述	获取语音系统的完整配置信息。
参数说明	config: 通过引用返回语音系统配置信息。timeout_ms: 超时时间 (毫秒), 默认值为 5000。
返回值	Status::OK 表示成功, 其他为失败状态。
备注	阻塞接口, 获取的配置包含音色、智能体、唤醒、对话等所有子配置, 以及当前使用的语音模型类型。

11.1.6 SwitchTtsVoiceModel

项目	内容
函数名	SwitchTtsVoiceModel
函数声明	Status SwitchTtsVoiceModel(TtsType tts_type, int timeout_ms);
功能概述	切换 TTS 语音模型。
参数说明	tts_type: TTS 语音模型类型 timeout_ms: 超时时间 (毫秒), 默认值为 5000。
返回值	操作状态。
备注	切换模型属于阻塞操作。

11.1.7 SetVoiceConfig

项目	内容
函数名	SetVoiceConfig
函数声明	Status SetVoiceConfig(const SetSpeechConfig& config, int timeout_ms);
功能概述	设置语音系统的完整配置信息。
参数说明	config: 要设置的语音系统配置信息。timeout_ms: 超时时间 (毫秒), 默认值为 5000。
返回值	Status::OK 表示成功, 其他为失败状态。
备注	阻塞接口, 设置的配置将完全覆盖当前所有语音相关配置。

11.1.8 Play

项目	内容
函数名	Play
函数声明	Status Play(const TtsCommand& cmd, int timeout_ms);
功能概述	播放 TTS（文本转语音）语音命令。
参数说明	cmd: TTS 命令，包含文本、语速、语调等。timeout_ms: 超时时间（毫秒），默认值为 5000。
返回值	Status::OK 表示成功，其他为失败状态。
备注	阻塞接口，调用前需确保已初始化模块。

11.1.9 Stop

项目	内容
函数名	Stop
函数声明	Status Stop(int timeout_ms);
功能概述	停止当前音频播放。
参数说明	timeout_ms: 超时时间（毫秒），默认值为 5000。
返回值	Status::OK 表示成功，其他为失败状态。
备注	阻塞接口，通常用于中断当前语音。

11.1.10 SetVolume

项目	内容
函数名	SetVolume
函数声明	Status SetVolume(int volume, int timeout_ms);
功能概述	设置音频输出的音量。
参数说明	volume: 音量值，通常范围为 0~100。timeout_ms: 超时时间（毫秒），默认值为 5000。
返回值	Status::OK 表示成功，其他为失败状态。
备注	阻塞接口，设置后立即生效。

11.1.11 GetVolume

项目	内容
函数名	GetVolume
函数声明	Status GetVolume(int& volume, int timeout_ms);
功能概述	获取当前音频输出音量。
参数说明	volume: 通过引用返回当前音量值。timeout_ms: 超时时间 (毫秒), 默认值为 5000。
返回值	Status::OK 表示成功, 其他为失败状态。
备注	阻塞接口, 返回值需检查后再使用 volume。

11.1.12 ControlVoiceStream

项目	内容
函数名	ControlVoiceStream
函数声明	Status ControlVoiceStream(bool enable_raw_data, bool enable_bf_data, int timeout_ms);
功能概述	控制语音数据流。
参数说明	enable_raw_data: 是否开启原始语音数据流。enable_bf_data: 是否开启 BF 语音数据流。 timeout_ms: 超时时间 (毫秒), 默认值为 5000。
返回值	操作状态。Status::OK 表示成功, 其他为失败。
备注	阻塞接口, 用于控制语音数据流的开启和关闭。

11.1.13 SubscribeOriginVoiceData

项目	内容
函数名	SubscribeOriginVoiceData
函数声明	void SubscribeOriginVoiceData(const ByteMultiArrayCallback callback);
功能概述	订阅原始语音数据
参数说明	callback: 接收到原始语音数据后的处理回调
备注	非阻塞接口, 回调函数会在数据更新时被调用。

11.1.14 UnsubscribeOriginVoiceData

项目	内容
函数名	UnsubscribeOriginVoiceData
函数声明	<code>void UnsubscribeOriginVoiceData();</code>
功能概述	取消订阅原始语音数据
备注	非阻塞接口，用于停止接收原始语音数据。

11.1.15 SubscribeBfVoiceData

项目	内容
函数名	SubscribeBfVoiceData
函数声明	<code>void SubscribeBfVoiceData(const ByteMultiArrayCallback callback);</code>
功能概述	订阅 BF 语音数据
参数说明	callback: 接收到 BF 语音数据后的处理回调
备注	非阻塞接口，回调函数会在数据更新时被调用。

11.1.16 UnsubscribeBfVoiceData

项目	内容
函数名	UnsubscribeBfVoiceData
函数声明	<code>void UnsubscribeBfVoiceData();</code>
功能概述	取消订阅 BF 语音数据
备注	非阻塞接口，用于停止接收 BF 语音数据。

11.1.17 ControlSpeechIO

项目	内容
函数名	ControlSpeechIO
函数声明	<code>Status ControlSpeechIO(bool enable_data, int timeout_ms);</code>
功能概述	控制语音 io 数据流 (asr 和 tts)。
参数说明	enable_data: 打开或关闭语音 io 数据流。timeout_ms: 超时时间 (毫秒)，默认值为 5000。
返回值	Status::OK 表示成功，其他为失败。
备注	阻塞接口，用于控制语音 io 数据流 (asr 和 tts)。

11.1.18 SubscribeSpeechASRStream

项目	内容
函数名	SubscribeSpeechASRStream
函数声明	<code>void SubscribeSpeechASRStream(const SpeechASRStreamCallback callback);</code>
功能概述	订阅语音 asr 流。
参数说明	callback: 处理接收到的语音 asr 流的回调。
备注	阻塞接口，用于订阅语音 asr 流。

11.1.19 UnsubscribeSpeechASRStream

项目	内容
函数名	UnsubscribeSpeechASRStream
函数声明	<code>void UnsubscribeSpeechASRStream();</code>
功能概述	取消订阅语音 asr 流。
备注	阻塞接口，用于取消订阅语音 asr 流。

11.1.20 PublishSpeechTTSStream

项目	内容
函数名	PublishSpeechTTSStream
函数声明	<code>Status PublishSpeechTTSStream(const SpeechTTSStream& data);</code>
功能概述	发布语音 tts 流。
参数说明	data: 语音 tts 流。
备注	阻塞接口，用于发布语音 tts 流。数据 ID 应该与 ASR 输出的请求 ID 一致，起始类型为 begin，中间数据类型为 var，结束类型为 end。

11.2 类型定义

11.2.1 TtsPriority —TTS 播报优先级等级

用于控制不同 TTS 任务之间的中断行为。优先级越高的任务将中断当前低优先级任务的播放。

枚举值	数值	描述
TtsPriority::HIGH	0	最高优先级，例如：低电告警、紧急提醒
TtsPriority::MIDDLE	1	中优先级，例如：系统提示、状态播报
TtsPriority::LOW	2	最低优先级，例如：日常语音对话、背景播报

11.2.2 TtsMode —同一优先级下的任务调度策略

用于细化控制在相同优先级条件下多个 TTS 任务的播放顺序和清除逻辑。

枚举值	数值	描述
TtsMode::CLEARTOP	0	清空当前优先级所有任务（包括正在播放和等待队列），立即播放本次请求
TtsMode::ADD	1	将本次请求追加到当前优先级队列尾部，顺序播放（不中断当前播放）
TtsMode::CLEARBUFFER	2	清空队列中未播放的请求，保留当前播放，之后播放本次请求

11.3 结构体定义

11.3.1 TtsCommand —TTS 播放命令结构体

描述一次 TTS 播放请求的完整信息，支持设置唯一标识、文本内容、优先级控制以及同优先级下的调度模式。

字段名	类型	描述
id	std::string	TTS 任务唯一 ID，例如 "id_01"，用于追踪播放状态
content	std::string	要播放的文本内容，例如 " 你好，欢迎使用智能语音系统。"
priority	TtsPriority	播报优先级，控制是否中断正在播放的低优先级语音
mode	TtsMode	同优先级下的调度策略，控制任务是否追加、覆盖等

11.3.2 CustomBotInfo —自定义智能体配置结构体

描述一个自定义智能体的基本配置信息。

字段名	类型	描述
name	std::string	智能体名称
workflow	std::string	工作流 ID
token	std::string	用户授权 token

11.3.3 CustomBotMap —自定义智能体映射表

项目	内容
类型	<code>std::map<std::string, CustomBotInfo></code> ;
描述	自定义智能体映射表，键为智能体 ID，值为智能体配置信息

11.3.4 SetSpeechConfig —语音配置参数结构体

用于设置语音系统的各项配置参数。

字段名	类型	描述
speaker_id	<code>std::string</code>	音色 ID
region	<code>std::string</code>	音色地区
bot_id	<code>std::string</code>	模式 ID
is_front_doa	<code>bool</code>	是否强制正前方识别
is_fullduplex_enable	<code>bool</code>	自然对话开关
is_enable	<code>bool</code>	语音开关
is_doa_enable	<code>bool</code>	是否打开唤醒方位转头
speaker_speed	<code>double</code>	TTS 播放语速，范围 [1,2]
wakeup_name	<code>std::string</code>	唤醒名字
custom_bot	<code>CustomBotMap</code>	自定义智能体配置

11.3.5 SpeakerConfigSelected —音色选中配置结构体

当前音色配置

字段名	类型	描述
region	<code>std::string</code>	选中的地区
speaker_id	<code>std::string</code>	选中的音色 ID

11.3.6 SpeakerConfig —音色配置结构体

音色配置

字段名	类型	描述
data	<code>std::map<std::string, std::vector<std::array<std::string, 2>>></code>	音色数据：地区-> 音色 ID-> 音色名称
selected	<code>SpeakerConfigSelected</code>	当前选中的音色配置
speaker_sr	<code>double</code>	语速

11.3.7 BotInfo —智能体配置信息结构体

描述一个标准智能体的基本信息。

字段名	类型	描述
name	std::string	工作场景名称
workflow	std::string	工作流 ID

BotConfigSelected —智能体选中项结构体

字段名	类型	描述
bot_id	std::string	选中的智能体 ID

11.3.8 BotConfig —智能体配置结构体

描述智能体相关的配置信息，包括标准智能体和自定义智能体。

字段名	类型	描述
data	std::map<std::string, BotInfo>;	标准智能体数据：智能体 ID-> 智能体信息
custom_data	std::map<std::string, CustomBotInfo>;	自定义智能体数据：智能体 ID-> 自定义智能体信息
selected	BotConfigSelected	当前选中的智能体配置

11.3.9 WakeupConfig —唤醒配置结构体

描述唤醒相关的配置信息。

字段名	类型	描述
name	std::string	唤醒名称
data	std::map<std::string, std::string>;	唤醒词数据：唤醒词-> 拼音

11.3.10 DialogConfig —对话配置结构体

描述对话相关的配置参数。

字段名	类型	描述
is_front_doa	bool	是否强制正前方加强拾音
is_fulllduplex_enable	bool	是否打开全双工对话
is_enable	bool	是否打开语音
is_doa_enable	bool	是否打开唤醒方位转头

11.3.11 TtsType —语音模型枚举

描述可选的语音合成模型类型。

枚举值	数值	描述
TtsType::NONE	0	无语音模型
TtsType::DOUBAO	1	豆包语音模型
TtsType::GOOGLE	2	谷歌语音模型

11.3.12 GetSpeechConfig —语音系统完整配置结构体

描述语音系统的完整配置信息，包含所有子配置模块。

字段名	类型	描述
speaker_config	SpeakerConfig	音色配置
bot_config	BotConfig	智能体配置
wakeup_config	WakeupConfig	唤醒配置
dialog_config	DialogConfig	对话配置
tts_type	TtsType	语音模型，默认值为 TtsType::NONE

11.3.13 MultiArrayDimension —多维数组维度描述

字段名	类型	描述
label	std::string	维度标签
size	int32_t	维度大小
stride	int32_t	步长

11.3.14 MultiArrayLayout —多维数组布局描述

字段名	类型	描述
dim_size	int32_t	维度数量
dim	std::vector<MultiArrayDimension>;	维度数组
data_offset	int32_t	数据偏移量

11.3.15 ByteMultiArray —字节数组数据结构

字段名	类型	描述
layout	MultiArrayLayout	数组布局信息
data	std::vector<uint8_t>;	字节数据数组

11.3.16 SpeechASRStream —机器人 ASR 输出

字段名	类型	描述
id	std::string	ID - 请求 id
type	std::string	Type - 类型 request or cancel
text	std::string	Text - ASR 输出

11.3.17 SpeechTTSStream —机器人 TTS 输入

字段名	类型	描述
id	std::string	ID - 与请求 id 相同
type	std::string	Type - 类型 begin or var or end
text	std::string	Text - TTS 输入
end_session	bool	End Session - 结束会话并关闭 asr

11.4 注意事项

在使用 Subscribe 等订阅类 SDK 接口时，请避免在回调函数中执行阻塞式的数据处理操作。否则，可能导致消息堆积、处理延迟，甚至引发不可预期的错误。

提供机器人状态监控接口类，通过 `StateMonitor` 可以实现状态查询等功能。

12.1 接口定义

`StateMonitor` 是机器人系统状态监控管理类，该类通常用于控制机器人的状态管理，支持状态查询。

12.1.1 GetCurrentState

项目	内容
函数名	<code>GetCurrentState</code>
函数声明	<code>Status GetCurrentState(RobotState& robot_state);</code>
功能概述	获取当前机器人聚合状态。
参数说明	<code>robot_state</code> : 用于接收状态结构体。
返回值	<code>Status::OK</code> 表示成功，其他为失败状态。
备注	非阻塞接口，获取最近监控到的机器人状态数据，包括 BMS 电池状态、故障信息等，后续会持续扩展。

12.1.2 RobotState —机器人状态数据结构体

用于表示机器人总体状态信息：

字段名	类型	描述
faults	std::vector<Fault>;	故障信息列表
bms_data	BmsData	电池管理系统数据

12.1.3 BmsData —电池管理系统数据结构体

表示电池的状态信息：

字段名	类型	描述
battery_percentage	double	当前电池剩余电量（百分比，0~100）
battery_health	double	电池健康状态（数值越高表示健康状态越好）
battery_state	BatteryState	电池状态（见上方枚举）
power_supply_status	PowerSupplyStatus	电池充放电状态（见上方枚举）

12.1.4 枚举类型定义

BatteryState —电池状态枚举类型

用于表示电池当前的状态，用于系统中电池状态的判断和处理：

枚举值	数值	描述
BatteryState::UNKNOWN	0	未知状态
BatteryState::GOOD	1	电池状态良好
BatteryState::OVERHEAT	2	电池过热
BatteryState::DEAD	3	电池损坏
BatteryState::OVERVOLTAGE	4	电池过电压
BatteryState::UNSPEC_FAILURE	5	未知故障
BatteryState::COLD	6	电池过冷
BatteryState::WATCHDOG_TIMER_EXPIRE	7	看门狗定时器超时
BatteryState::SAFETY_TIMER_EXPIRE	8	安全定时器超时

PowerSupplyStatus —电池充放电状态

用于表示当前电池的充放电状态：

枚举值	数值	描述
PowerSupplyStatus::UNKNOWN	0	未知状态
PowerSupplyStatus::CHARGING	1	电池充电中
PowerSupplyStatus::DISCHARGING	2	电池放电中
PowerSupplyStatus::NOTCHARGING	3	电池未充放电
PowerSupplyStatus::FULL	4	电池充满

12.2 错误码映射表

错误码（十六进制）	错误描述
0x0000	无错误
0x1101	调用 ROS 服务失败
0x1301	中控节点丢失
0x1302	APP 节点丢失
0x1304	音频节点丢失
0x1305	运动节点丢失
0x1306	LCD 节点丢失
0x1307	realsense 节点丢失
0x1308	双目相机节点丢失
0x1309	LDS 节点丢失
0x130A	传感器板节点丢失
0x130B	触摸节点丢失
0x130C	SLAM 节点丢失
0x130D	导航节点丢失
0x130E	AI 节点丢失
0x130F	头部节点丢失
0x1310	云端处理器节点丢失
0x3201	激光无数据
0x3202	双目相机无数据
0x3203	双目相机数据错误
0x3204	双目相机初始化失败
0x320B	里程计无数据
0x320C	IMU 无数据
0x6101	机器人连接 APP 失败
0x6102	与 APP 断开连接
0x9201	打开 LCD 串口失败
0x7201	打开头部串口失败

下页继续

表 1 – 续上页

错误码（十六进制）	错误描述
0x7202	头部无数据
0x8201	打开传感器板串口失败
0x8202	传感器板无数据
0x2201	错误：导航未收到 tf 数据
0x2202	错误：导航未收到地图数据
0x2203	错误：导航未收到定位数据
0x2204	错误：导航未收到超声波数据
0x2205	错误：导航未收到激光数据
0x2206	错误：导航未收到 RGBD 数据
0x2207	错误：导航未收到多线激光数据
0x2208	错误：导航未收到点 TOF 数据
0x2209	错误：导航未收到面 TOF 数据
0x220A	错误：导航未收到里程计数据
0x2101	警告：导航未收到 tf 数据
0x2102	警告：导航未收到地图数据
0x2103	警告：导航未收到定位数据
0x2104	警告：导航未收到超声波数据
0x2105	警告：导航未收到激光数据
0x2106	警告：导航未收到 RGBD TOF 数据
0x2107	警告：导航未收到多线激光数据
0x2108	警告：导航未收到点 TOF 数据
0x2109	警告：导航未收到面 TOF 数据
0x210A	警告：导航未收到里程计数据
0x4201	SLAM 定位错误
0x4102	错误：SLAM 未收到激光数据
0x4103	错误：SLAM 未收到里程计数据
0x4205	SLAM 地图错误

SLAM 导航控制服务 C++

提供机器人系统 SLAM 导航控制服务，通过 `SlamNavController` 可以实现 SLAM 建图、定位、导航等功能。

13.1 接口定义

`SlamNavController` 是面向 SLAM 导航控制的控制器，支持建图、定位、导航等操作。

13.1.1 `SlamNavController`

项目	内容
函数名	<code>SlamNavController</code>
函数声明	<code>SlamNavController();</code>
功能概述	构造函数，创建 <code>SlamNavController</code> 实例。
备注	构造内部控制资源。

13.1.2 ~SlamNavController

项目	内容
函数名	<code>~SlamNavController</code>
函数声明	<code>~SlamNavController();</code>
功能概述	析构函数，释放 SlamNavController 实例资源。
备注	配合构造函数使用。

13.1.3 Initialize

项目	内容
函数名	<code>Initialize</code>
函数声明	<code>bool Initialize();</code>
功能概述	初始化 SLAM 导航控制器。
返回值	<code>true</code> 表示成功， <code>false</code> 表示失败。
备注	首次使用前必须调用。

13.1.4 Shutdown

项目	内容
函数名	<code>Shutdown</code>
函数声明	<code>void Shutdown();</code>
功能概述	释放资源，清理 SLAM 导航控制器。
备注	配合 Initialize 使用。

13.1.5 SwitchToIdle

项目	内容
函数名	<code>SwitchToIdle</code>
函数声明	<code>Status SwitchToIdle();</code>
功能概述	切换到空闲模式。
返回值	<code>Status::OK</code> 表示成功，其他为失败。
备注	阻塞接口，用于切换到空闲状态。

13.1.6 StartMapping

项目	内容
函数名	StartMapping
函数声明	Status StartMapping();
功能概述	开始建图模式。
返回值	Status::OK 表示成功，其他为失败。
备注	阻塞接口，用于启动建图任务。

13.1.7 CancelMapping

项目	内容
函数名	CancelMapping
函数声明	Status CancelMapping();
功能概述	取消当前建图任务。
返回值	Status::OK 表示成功，其他为失败。
备注	阻塞接口，用于取消建图任务。

13.1.8 SwitchToLocation

项目	内容
函数名	SwitchToLocation
函数声明	Status SwitchToLocation();
功能概述	切换到定位模式。
返回值	Status::OK 表示成功，其他为失败。
备注	阻塞接口，用于切换到定位状态。

13.1.9 SaveMap

项目	内容
函数名	SaveMap
函数声明	Status SaveMap(const std::string& map_name);
功能概述	结束建图并保存地图，当处于建图模式时。
参数说明	map_name: 地图名称。
返回值	Status::OK 表示成功，其他为失败。
备注	阻塞接口，用于保存当前建图结果。

13.1.10 LoadMap

项目	内容
函数名	LoadMap
函数声明	Status LoadMap(const std::string& map_name);
功能概述	加载地图并设置为当前地图。
参数说明	map_name: 地图名称。
返回值	Status::OK 表示成功, 其他为失败。
备注	阻塞接口, 用于加载已保存的地图。

13.1.11 DeleteMap

项目	内容
函数名	DeleteMap
函数声明	Status DeleteMap(const std::string& map_name);
功能概述	删除地图。
参数说明	map_name: 要删除的地图名称。
返回值	Status::OK 表示成功, 其他为失败。
备注	阻塞接口, 用于删除不需要的地图。

13.1.12 GetAllMapInfo

项目	内容
函数名	GetAllMapInfo
函数声明	Status GetAllMapInfo(AllMapInfo& all_map_info);
功能概述	获取所有地图信息。
参数说明	all_map_info: 所有地图信息 (输出参数)。
返回值	Status::OK 表示成功, 其他为失败。
备注	阻塞接口, 用于获取系统中所有地图的列表。

13.1.13 InitPose

项目	内容
函数名	InitPose
函数声明	Status InitPose(const Pose3DEuler& pose);
功能概述	初始化位姿。
参数说明	pose: 要发布的位姿信息。
返回值	Status::OK 表示成功, 其他为失败。
备注	阻塞接口, 用于设置机器人的初始位姿。

13.1.14 GetCurrentLocalizationInfo

项目	内容
函数名	GetCurrentLocalizationInfo
函数声明	Status GetCurrentLocalizationInfo(LocalizationInfo& localization_info);
功能概述	获取当前位姿信息。
参数说明	localization_info: 当前位置和姿态信息 (输出参数)。
返回值	Status::OK 表示成功, 其他为失败。
备注	阻塞接口, 用于获取机器人当前位姿。

13.1.15 ActivateNavMode

项目	内容
函数名	ActivateNavMode
函数声明	Status ActivateNavMode(NavMode mode);
功能概述	激活导航模式。
参数说明	mode: 目标导航模式 (NavMode 枚举类型)。
返回值	Status::OK 表示成功, 其他为失败。
备注	阻塞接口, 用于切换导航工作模式。

13.1.16 SetNavTarget

项目	内容
函数名	<code>SetNavTarget</code>
函数声明	<code>Status SetNavTarget(const NavTarget& goal);</code>
功能概述	设置全局导航目标点并开始导航任务。
参数说明	<code>goal</code> : 目标点的全局坐标。
返回值	<code>Status::OK</code> 表示成功, 其他为失败。
备注	阻塞接口, 用于设置导航目标。

13.1.17 PauseNavTask

项目	内容
函数名	<code>PauseNavTask</code>
函数声明	<code>Status PauseNavTask();</code>
功能概述	暂停当前导航任务。
返回值	<code>Status::OK</code> 表示成功, 其他为失败。
备注	阻塞接口, 用于暂停导航。

13.1.18 ResumeNavTask

项目	内容
函数名	<code>ResumeNavTask</code>
函数声明	<code>Status ResumeNavTask();</code>
功能概述	恢复暂停的导航任务。
返回值	<code>Status::OK</code> 表示成功, 其他为失败。
备注	阻塞接口, 用于恢复导航。

13.1.19 CancelNavTask

项目	内容
函数名	<code>CancelNavTask</code>
函数声明	<code>Status CancelNavTask();</code>
功能概述	取消当前导航任务。
返回值	<code>Status::OK</code> 表示成功, 其他为失败。
备注	阻塞接口, 用于取消导航。

13.1.20 GetNavTaskStatus

项目	内容
函数名	GetNavTaskStatus
函数声明	Status GetNavTaskStatus (NavStatus& status);
功能概述	获取当前导航任务状态。
参数说明	status: 导航状态。
返回值	操作状态, 成功返回 Status::OK。
备注	阻塞接口, 用于查询导航任务状态。

13.1.21 SubscribeOdometry

项目	内容
函数名	SubscribeOdometry
函数声明	Status SubscribeOdometry (OdometryCallback callback);
功能概述	订阅机器人里程计数据。
参数说明	callback: 里程计数据回调函数。
返回值	Status::OK 表示成功, 其他为失败。
备注	非阻塞接口, 用于获取实时里程计数据。

13.1.22 UnsubscribeOdometry

项目	内容
函数名	UnsubscribeOdometry
函数声明	void UnsubscribeOdometry();
功能概述	取消订阅机器人里程计数据。
返回值	无
备注	非阻塞接口, 用于取消订阅里程计数据。

13.1.23 GetPointCloudMap

项目	内容
函数名	GetPointCloudMap
函数声明	Status GetPointCloudMap(PointCloud2& point_cloud_map, int timeout_ms);
功能概述	获取当前构建的点云地图。
参数说明	point_cloud_map: 输出参数, 获取到的点云地图数据。timeout_ms: 操作超时(毫秒), 默认10000。
返回值	Status::OK 表示成功, 其他为失败。
备注	阻塞接口, 若在超时时间内未完成则返回错误状态。

13.2 类型定义

13.2.1 NavMode —导航模式枚举类型

枚举值	数值	描述
IDLE	0	空闲模式
GRID_MAP	1	栅格地图导航模式

13.2.2 Pose3DEuler —3D 位姿结构

字段名	类型	描述
position	std::array<double, 3>;	位置 (x, y, z), 用于表示空间位置
orientati	std::array<double, 3>;	欧拉角 (roll, pitch, yaw), 用于表示空间姿态, 避免欧拉角万向锁问题

13.2.3 NavTarget —全局导航目标点结构

字段名	类型	描述
id	int32_t	目标点 ID
frame_id	std::string	目标点坐标系 ID
goal	Pose3DEuler	目标点位姿

13.2.4 NavStatusType —导航状态类型枚举

枚举值	数值	描述
NONE	0	无状态
RUNNING	1	运行中
END_SUCCESS	2	结束成功
END_FAILED	3	结束失败
PAUSE	4	暂停
CONTINUE	5	继续
CANCEL	6	取消

13.2.5 NavStatus —导航状态结构

字段名	类型	描述
id	int32_t	目标点 ID, -1 表示无目标点
status	NavStatusType	导航状态
message	std::string	导航状态消息

13.2.6 MapImageData —建图图像数据结构, .pgm 格式

字段名	类型	描述
type	std::string	魔数, "P5": 二进制格式
width	uint32_t	图像宽度
height	uint32_t	图像高度
max_gray_value	uint32_t	最大灰度值, 255
image	std::vector<uint8_t>	图像数据

13.2.7 MapMetaData —建图地图元数据结构

字段名	类型	描述
resolution	double	地图分辨率, 单位: m/pixel
origin	Pose3DEuler	地图原点, 世界坐标系原点相对于地图左下角的位置
map_image_data	MapImageData	图像数据, .pgm 格式图像数据

13.2.8 MapInfo — 单个地图信息结构

字段名	类型	描述
map_name	std::string	地图名称
map_meta_data	MapMetaData	地图元数据

13.2.9 AllMapInfo — 所有地图信息结构

字段名	类型	描述
current_map_name	std::string	当前地图名称
map_infos	std::vector<MapInfo>;	所有地图信息

13.2.10 LocalizationInfo — 当前位置信息结构

字段名	类型	描述
is_localization	bool	是否已定位
pose	Pose3DEuler	欧拉角位姿

13.3 SLAM 导航控制介绍

机器人的 SLAM 导航控制服务可分为 SLAM 功能和导航功能。

- SLAM 功能中，可调用相应的接口，实现建图、定位、地图管理等功能。
- 导航功能中，可调用相应的接口，实现目标导航、导航控制等功能。

13.3.1 适用场景与范围

- 小于 20m * 20m 且特征丰富的静态室内平地场景，请勿超过建议范围

13.3.2 SLAM 功能流程

功能	操作流程
建图	开始建图 → 保存地图
地图管理	加载地图、获取地图信息、删除地图、获取地图绝对路径

13.3.3 导航功能流程

功能	操作流程
定位	加载地图 → 切换到定位模式 → 初始化位姿 → 获取定位状态
导航	定位成功 → 激活导航模式 → 设置目标位姿 (开始导航任务) → 获取导航状态
导航控制	设置目标姿态 (开始导航任务) → 暂停导航 → 恢复导航 → 取消导航

提供机器人系统显示服务控制器，通过 `DisplayController` 可以使用 RPC 方式实现对机器人显示的指令控制和状态获取。

14.1 接口定义

`DisplayController` 是一个封装显示控制功能的 C++ 类，主要用于显示控制等场景。

14.1.1 `DisplayController`

项目	内容
函数名	<code>DisplayController</code>
函数声明	<code>DisplayController();</code>
功能概述	初始化显示控制器对象，构造内部状态，分配资源等。
备注	构造内部状态。

14.1.2 ~DisplayController

项目	内容
函数名	<code>~DisplayController</code>
函数声明	<code>~DisplayController();</code>
功能概述	释放显示控制器资源。
备注	确保资源安全释放。

14.1.3 Initialize

项目	内容
函数名	<code>Initialize</code>
函数声明	<code>bool Initialize();</code>
功能概述	初始化显示控制模块。
返回值	<code>true</code> 表示成功, <code>false</code> 表示失败。
备注	与 <code>Shutdown()</code> 配对使用。

14.1.4 Shutdown

项目	内容
函数名	<code>Shutdown</code>
函数声明	<code>void Shutdown();</code>
功能概述	关闭显示控制器并释放资源。
备注	确保在销毁前调用。

14.1.5 GetAllFaceExpressions

项目	内容
函数名	<code>GetAllFaceExpressions</code>
函数声明	<code>Status GetAllFaceExpressions(std::vector& data, int timeout_ms);</code>
功能概述	获取所有面部表情。
参数说明	<code>data</code> : 所有面部表情。 <code>timeout_ms</code> : 超时时间 (毫秒), 默认值为 5000。
返回值	<code>Status::OK</code> 表示成功, 其他为失败状态。
备注	阻塞接口, 获取所有面部表情。

14.1.6 SetFaceExpression

项目	内容
函数名	SetFaceExpression
函数声明	Status SetFaceExpression(int expression_id, int timeout_ms);
功能概述	设置面部表情。
参数说明	expression_id: 表情 IDtimeout_ms: 超时时间 (毫秒), 默认值为 5000。
返回值	Status::OK 表示成功, 其他为失败状态。
备注	阻塞接口, 设置面部表情。

14.1.7 GetFaceExpression

项目	内容
函数名	GetFaceExpression
函数声明	Status GetFaceExpression(FaceExpression& data, int timeout_ms);
功能概述	获取当前面部表情。
参数说明	data: 当前面部表情。timeout_ms: 超时时间 (毫秒), 默认值为 5000。
返回值	Status::OK 表示成功, 其他为失败状态。
备注	阻塞接口, 获取当前面部表情。

14.2 结构体定义

14.2.1 FaceExpression — 机器人面部表情结构

描述机器人面部表情的完整信息, 支持唯一 ID、名称和描述。

字段名	类型	描述
id	int32_t	机器人面部表情 ID
name	std::string	机器人面部表情名称
description	std::string	机器人面部表情描述

14.3 注意事项

在使用 Subscribe 等订阅类 SDK 接口时, 请避免在回调函数中执行阻塞式的数据处理操作。否则, 可能导致消息堆积、处理延迟, 甚至引发不可预期的错误。

机器人主控制服务 Python

提供机器人系统主控制器，通过 **MagicRobot** 可以进行资源初始化、管理通信连接、访问各子控制器如运动控制器、音频控制器、状态监控以及传感器控制器等。

15.1 模块信息

`magicdog_w_python` 是 `MagicDog_W SDK` 的 Python 绑定模块，提供了完整的机器人控制、传感器数据获取、音频处理等功能接口。

项目	内容
模块名	<code>magicdog_w_python</code>
描述	<code>MagicDog_W SDK Python</code> 绑定模块
依赖	<code>pybind11</code> , C++ SDK

15.2 接口定义

`MagicRobot` 是机器人系统的统一入口类。

15.2.1 MagicRobot —主机器人类

项目	内容
类名	MagicRobot
功能概述	机器人主控制类，整合所有功能模块
主要功能	系统初始化、连接管理、控制器获取
使用场景	机器人应用开发、系统集成、功能测试

15.2.2 initialize

项目	内容
方法名	initialize
方法声明	bool initialize(const str& local_ip)
功能概述	初始化机器人系统，包括控制器与网络模块。
参数说明	local_ip: 本地通信 IP 地址。
返回值	true 表示成功，false 表示失败。
备注	首次使用前必须调用。

15.2.3 shutdown

项目	内容
方法名	shutdown
方法声明	void shutdown()
功能概述	关闭机器人系统，释放资源。
备注	配合 initialize 使用。

15.2.4 release

项目	内容
方法名	release
方法声明	void release()
功能概述	释放机器人系统资源。
备注	调用该方法可手动释放 SDK 内部占用的所有资源，通常在程序退出前调用。

15.2.5 connect

项目	内容
方法名	connect
方法声明	Status connect(int timeout_ms)
功能概述	与机器人服务建立通信连接。
参数说明	timeout_ms: 超时时间 (毫秒), 默认值为 5000。
返回值	Status::OK 表示成功。
备注	阻塞接口, 需初始化后调用。

15.2.6 disconnect

项目	内容
方法名	disconnect
方法声明	Status disconnect(int timeout_ms)
功能概述	断开与机器人服务的连接。
参数说明	timeout_ms: 超时时间 (毫秒), 默认值为 5000。
返回值	Status::OK 表示成功。
备注	阻塞接口, 与 connect 配对使用。

15.2.7 get_sdk_version

项目	内容
方法名	get_sdk_version
方法声明	str get_sdk_version()
功能概述	获取当前 SDK 的版本号。
返回值	SDK 版本字符串 (如 0.0.1)。
备注	非阻塞接口, 便于调试或日志标记。

15.2.8 get_motion_control_level

项目	内容
方法名	get_motion_control_level
方法声明	ControllerLevel get_motion_control_level()
功能概述	获取当前运动控制级别 (高层/低层)。
返回值	ControllerLevel 枚举值。
备注	非阻塞接口, 用于判断当前控制权限。

15.2.9 set_motion_control_level

项目	内容
方法名	set_motion_control_level
方法声明	Status set_motion_control_level(ControllerLevel level)
功能概述	设置当前运动控制级别。
参数说明	level: 控制权限枚举值。
返回值	Status::OK 表示设置成功, 其他代表设置失败。
备注	阻塞接口, 控制模式切换时使用。

15.2.10 get_high_level_motion_controller

项目	内容
方法名	get_high_level_motion_controller
方法声明	HighLevelMotionController get_high_level_motion_controller()
功能概述	获取高层运动控制器对象。
返回值	高层运动控制器实例, 用于调用高层控制接口。
备注	非阻塞接口, 封装了步态、特技、遥控等控制功能。

15.2.11 get_low_level_motion_controller

项目	内容
方法名	get_low_level_motion_controller
方法声明	LowLevelMotionController get_low_level_motion_controller()
功能概述	获取低层运动控制器对象。
返回值	低层运动控制器实例, 用于控制关节或电机。
备注	非阻塞接口, 可直接控制关节电机等。

15.2.12 get_audio_controller

项目	内容
方法名	get_audio_controller
方法声明	AudioController get_audio_controller()
功能概述	获取音频控制器对象。
返回值	音频控制器实例, 用于语音控制。
备注	非阻塞接口, 可播放语音和控制音量。

15.2.13 get_sensor_controller

项目	内容
方法名	get_sensor_controller
方法声明	SensorController get_sensor_controller()
功能概述	获取传感器控制器对象。
返回值	传感器控制器实例，用于访问传感器数据。
备注	非阻塞接口，封装了 IMU、RGBD 等传感器数据读取。

15.2.14 get_state_monitor

项目	内容
方法名	get_state_monitor
方法声明	StateMonitor get_state_monitor()
功能概述	获取状态监控器对象。
返回值	状态监控器实例，用于获取机器人当前状态信息。
备注	非阻塞接口，封装了 BMS、故障等状态信息的读取。

15.2.15 get_slam_nav_controller

项目	内容
方法名	get_slam_nav_controller
方法声明	SlamNavController get_slam_nav_controller()
功能概述	获取 SLAM/导航控制器对象。
返回值	SLAM/导航控制器实例，用于访问地图、定位等 SLAM 相关数据。
备注	非阻塞接口，封装地图构建、定位、导航等相关功能的使用。

15.2.16 open_channel_switch

项目	内容
方法名	open_channel_switch
方法声明	Status open_channel_switch(int timeout_ms)
功能概述	打开 lcm 通道开关。
参数说明	timeout_ms (可选参数，单位毫秒)，操作超时时间，默认 5000 ms。
返回值	Status 结构体，表示操作状态。
备注	用于控制 SDK lcm 通信通道的启用。

15.2.17 close_channel_switch

项目	内容
方法名	close_channel_switch
方法声明	Status close_channel_switch(int timeout_ms)
功能概述	关闭 lcm 通道开关。
参数说明	timeout_ms (可选参数, 单位毫秒), 操作超时时间, 默认 5000 ms。
返回值	Status 结构体, 表示操作状态。
备注	用于控制 SDK lcm 通信通道的关闭。

15.3 常量定义

常量名	类型	描述
LEG_JOINT_NUM	int	腿部关节数量
PERIOD_MS	int	控制周期 (毫秒)

15.4 数据结构定义

15.4.1 Status — 状态返回结构体

字段名	类型	描述
code	ErrorCode	错误代码
message	str	状态消息

15.4.2 ErrorCode — 错误代码枚举

枚举值	数值	描述
OK	0	操作成功
SERVICE_NOT_READY	1	服务未就绪
TIMEOUT	2	操作超时
INTERNAL_ERROR	3	内部错误
SERVICE_ERROR	4	服务错误

高层运动控制服务 Python

提供机器人系统高层运动控制服务，通过 `HighLevelMotionController` 可以通过 RPC 通信方式实现对机器人的步态、特技、遥控的控制。

16.1 接口定义

`HighLevelMotionController` 是面向语义控制的高层运动控制器，支持如行走、特技等控制操作，封装底层细节以供上层系统调用。

△ **Notice:** 某些特技动作，比如前空翻、后空翻对场地要求较大，执行时需要额外注意，建议所有特技动作在大的空场地测试后，再在小的场地使用。

16.1.1 `HighLevelMotionController` —高级运动控制器

项目	内容
类名	<code>HighLevelMotionController</code>
功能概述	面向语义控制的高层运动控制器，支持行走、特技等控制操作
主要功能	步态控制、特技执行、摇杆命令发送
使用场景	机器人高层运动控制、特技表演、遥控操作

16.1.2 initialize

项目	内容
方法名	initialize
方法声明	bool initialize()
功能概述	初始化控制器，准备高层控制功能。
返回值	true 表示成功，false 表示失败。
备注	首次使用前必须调用。

16.1.3 shutdown

项目	内容
方法名	shutdown
方法声明	void shutdown()
功能概述	关闭控制器并释放资源。
备注	配合 initialize 使用。

16.1.4 set_gait

项目	内容
方法名	set_gait
方法声明	Status set_gait(GaitMode gait_mode, int timeout_ms)
功能概述	设置机器人步态模式（如位控站立、力控站立、慢跑等）。
参数说明	gait_mode: 步态控制枚举。timeout_ms: 超时时间（毫秒），默认值为 5000。
返回值	Status::OK 表示成功，其他为失败。
备注	阻塞接口，支持多种步态模式切换。

16.1.5 get_gait

项目	内容
方法名	get_gait
方法声明	GaitMode get_gait(int timeout_ms)
功能概述	获取机器人步态模式（如位控站立、力控站立、慢跑等）。timeout_ms: 超时时间（毫秒），默认值为 5000。
返回值	当前步态模式，失败返回 GAIT_NONE。
备注	阻塞接口，获取当前步态模式。

16.1.6 execute_trick

项目	内容
方法名	execute_trick
方法声明	Status execute_trick(TrickAction trick_action, int timeout_ms)
功能概述	执行特技动作（如扭屁股、趴下等）。
参数说明	trick_action: 特技动作标识。timeout_ms: 超时时间（毫秒），默认值为 5000。
返回值	Status::OK 表示成功，其他为失败。
备注	阻塞接口，需确保机器人当前处于可执行特技的状态。

注意事项：特技动作必须在 GaitMode::GAIT_STAND_R(2) 位控站立步态下才能执行

16.1.7 enable_joy_stick

方法名	内容
方法名	enable_joy_stick
方法声明	Status enable_joy_stick()
功能概述	开启摇杆控制命令。
参数说明	无。
返回值	Status.OK 表示操作成功，其他为失败。
备注	用于允许通过摇杆发送实时控制指令。

16.1.8 disable_joy_stick

方法名	内容
方法名	disable_joy_stick
方法声明	Status disable_joy_stick()
功能概述	关闭摇杆控制命令。
参数说明	无。
返回值	Status.OK 表示操作成功，其他为失败。
备注	切换导航模式之前，必须先关闭摇杆控制命令。

16.1.9 send_joystick_command

项目	内容
方法名	send_joystick_command
方法声明	Status send_joystick_command(JoystickCommand command)
功能概述	发送实时摇杆控制指令。
参数说明	command: 包含摇杆坐标的控制数据。
返回值	Status::OK 表示成功, 其他为失败。
备注	非阻塞接口, 建议发送频率为 20Hz。

16.1.10 get_all_gait_speed_ratio

项目	内容
方法名	get_all_gait_speed_ratio
方法声明	AllGaitSpeedRatio get_all_gait_speed_ratio(int timeout_ms)
功能概述	获取所有步态以及对应前进、横移、旋转速度比例。
参数说明	timeout_ms: 超时时间 (毫秒), 默认值为 5000。
返回值	成功返回 AllGaitSpeedRatio 对象, 失败返回空对象。
备注	阻塞接口, 用于获取所有步态的速度比例配置。

16.1.11 set_gait_speed_ratio

项目	内容
方法名	set_gait_speed_ratio
方法声明	Status set_gait_speed_ratio(GaitMode gait_mode, GaitSpeedRatio gait_speed_ratio, int timeout_ms)
功能概述	设置步态以及对应前进、横移、旋转速度比例。
参数说明	gait_mode: 步态模式; gait_speed_ratio: 速度比例配置。timeout_ms: 超时时间 (毫秒), 默认值为 5000。
返回值	Status::OK 表示成功, 其他为失败。
备注	阻塞接口, 用于设置特定步态的速度比例配置。

16.1.12 get_head_motor_enabled

项目	内容
方法名	get_head_motor_enabled
方法声明	bool get_head_motor_enabled(int timeout_ms)
功能概述	获取头部电机使能状态。
参数说明	timeout_ms: 超时时间（毫秒），默认值为 5000。
返回值	成功返回 true（已使能）或 false（未使能），失败返回 false。
备注	阻塞接口，用于查询头部电机的使能状态。

16.1.13 enable_head_motor

项目	内容
方法名	enable_head_motor
方法声明	Status enable_head_motor(int timeout_ms)
功能概述	使能头部电机。
参数说明	timeout_ms: 超时时间（毫秒），默认值为 5000。
返回值	Status::OK 表示成功，其他为失败。
备注	阻塞接口，用于启用头部电机控制。

16.1.14 disable_head_motor

项目	内容
方法名	disable_head_motor
方法声明	Status disable_head_motor(int timeout_ms)
功能概述	关闭头部电机。
参数说明	timeout_ms: 超时时间（毫秒），默认值为 5000。
返回值	Status::OK 表示成功，其他为失败。
备注	阻塞接口，用于禁用头部电机控制。

16.1.15 get_current_head_position

项目	内容
方法名	get_current_head_position
方法声明	Status get_current_head_position(EulerAngles euler_angles, int timeout_ms);
功能概述	获取头部位置。
参数说明	euler_angles: 返回参数，头部当前绝对位置。timeout_ms: 超时时间（毫秒），默认值为 5000。
返回值	Status::OK 表示成功，其他为失败。
备注	阻塞接口，用于获取头部位置。

16.1.16 set_head_position

项目	内容
方法名	set_head_position
方法声明	Status set_head_position(EulerAngles euler_angles, int timeout_ms);
功能概述	设置头部位置。
参数说明	euler_angles: 头部目标绝对位置。timeout_ms: 超时时间（毫秒），默认值为 5000。
返回值	Status::OK 表示成功，其他为失败。
备注	阻塞接口，用于设置头部位置。

16.2 枚举类型定义

16.2.1 GaitMode 一步态模式枚举

枚举值	数值	描述
GAIT_PASSIVE	0	掉落（关闭电机使能）
GAIT_PURE_DAMPER	1	纯阻尼
GAIT_STAND_R	2	位控站立、RecoveryStand
GAIT_LOWLEVEL_SDK	30	低层 SDK 步态
GAIT_RL_MOVE_QUICK	111	快速移动
GAIT_RL_TERRAIN	112	全地形
GAIT_RL_CLIMB	113	爬楼梯
GAIT_RL_HAND_STAND	114	倒立
GAIT_RL_FOOT_STAND	115	正立
GAIT_NONE	9999	无步态

16.2.2 TrickAction —特技动作指令枚举

枚举值	数值	描述
ACTION_NONE	0	无动作
ACTION_LIE_DOWN	102	趴下 (Lie down)

16.3 数据结构定义

16.3.1 GaitSpeedRatio —步态速度比例结构体

字段名	类型	描述
straight_ratio	double	前进速度比例
turn_ratio	double	旋转速度比例
lateral_ratio	double	横移速度比例

16.3.2 AllGaitSpeedRatio —所有步态速度比例结构体

字段名	类型	描述
gait_speed_ratios	GaitModeGaitSpeedRatioMap	步态模式到速度比例的映射

16.3.3 GaitModeGaitSpeedRatioMap —步态模式到速度比例映射类型

项目	内容
类型名	GaitModeGaitSpeedRatioMap
类型定义	std::map<GaitMode, GaitSpeedRatio>;
功能概述	步态模式到速度比例配置的映射容器
键类型	GaitMode - 步态模式枚举
值类型	GaitSpeedRatio - 速度比例结构体
使用场景	存储和管理所有步态的速度比例配置

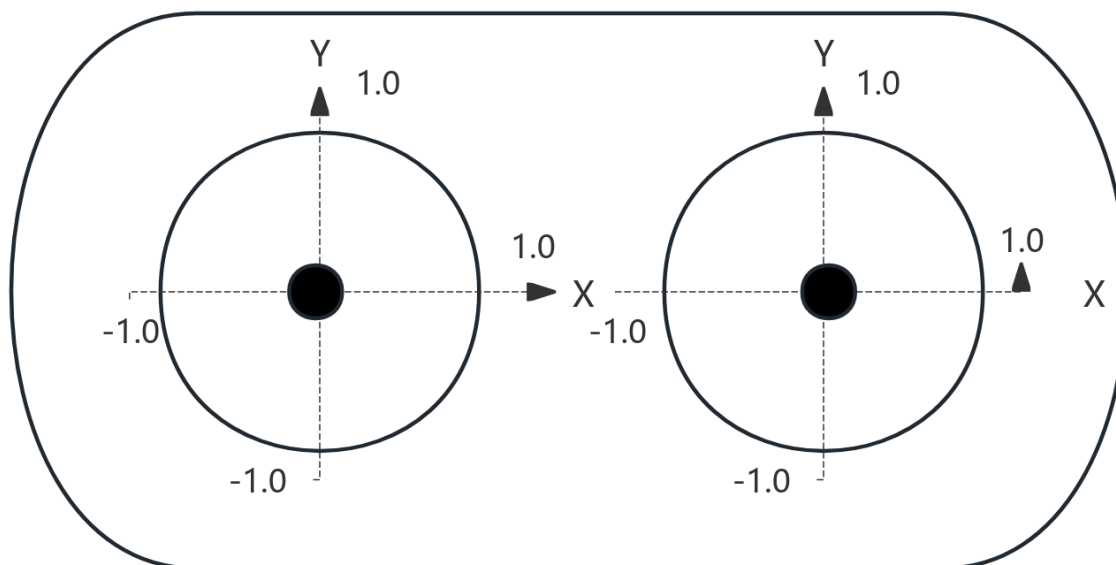
16.3.4 JoystickCommand —摇杆命令结构体

字段名	类型	描述
left_x_axis	float	左侧摇杆的 X 轴方向值 (-1.0: 左, 1.0: 右)
left_y_axis	float	左侧摇杆的 Y 轴方向值 (-1.0: 下, 1.0: 上)
right_x_axis	float	右侧摇杆的 X 轴方向值 (旋转 -1.0: 左, 1.0: 右)
right_y_axis	float	右侧摇杆的 Y 轴方向值 (暂未定义用途)

16.3.5 EulerAngles —欧拉角结构体

字段名	类型	描述
roll	double	横滚角 (Roll), 绕 X 轴旋转的角度 (单位为弧度)
pitch	double	俯仰角 (Pitch), 绕 Y 轴旋转的角度 (单位为弧度)
yaw	double	偏航角 (Yaw), 绕 Z 轴旋转的角度 (单位为弧度)

16.4 遥控器示意图



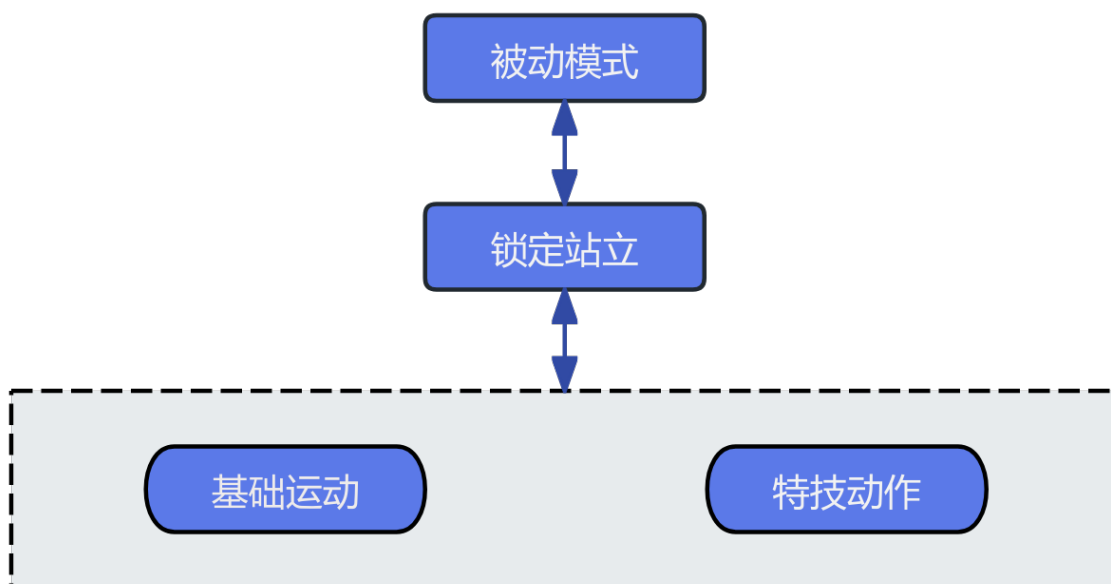
1. 左右摇杆 x 轴和 y 轴的取值范围为 [-1.0, 1.0];
2. 左右摇杆 x 轴和 y 轴的方向上/右为正, 如示意图所示;

16.5 高层运动控制机器人状态介绍

机器人的运动包含位控站立、基础运动、特技动作状态，机器人在运行过程中，通过状态机在不同状态之间进行切换，以实现不同的控制任务。各个状态的解释说明如下：

- 位控站立：在位控站立状态下，可调用 SDK 的各部分接口实现机器人的特技动作和基础运动控制。
- 基础运动：在运动执行过程中，可调用 SDK 接口，让机器人进入不同的步态。
- 特技动作：当进入特殊动作执行状态后，其他运动控制服务会先被挂起，等待当前动作执行完毕并进入位控站立状态后再生效。

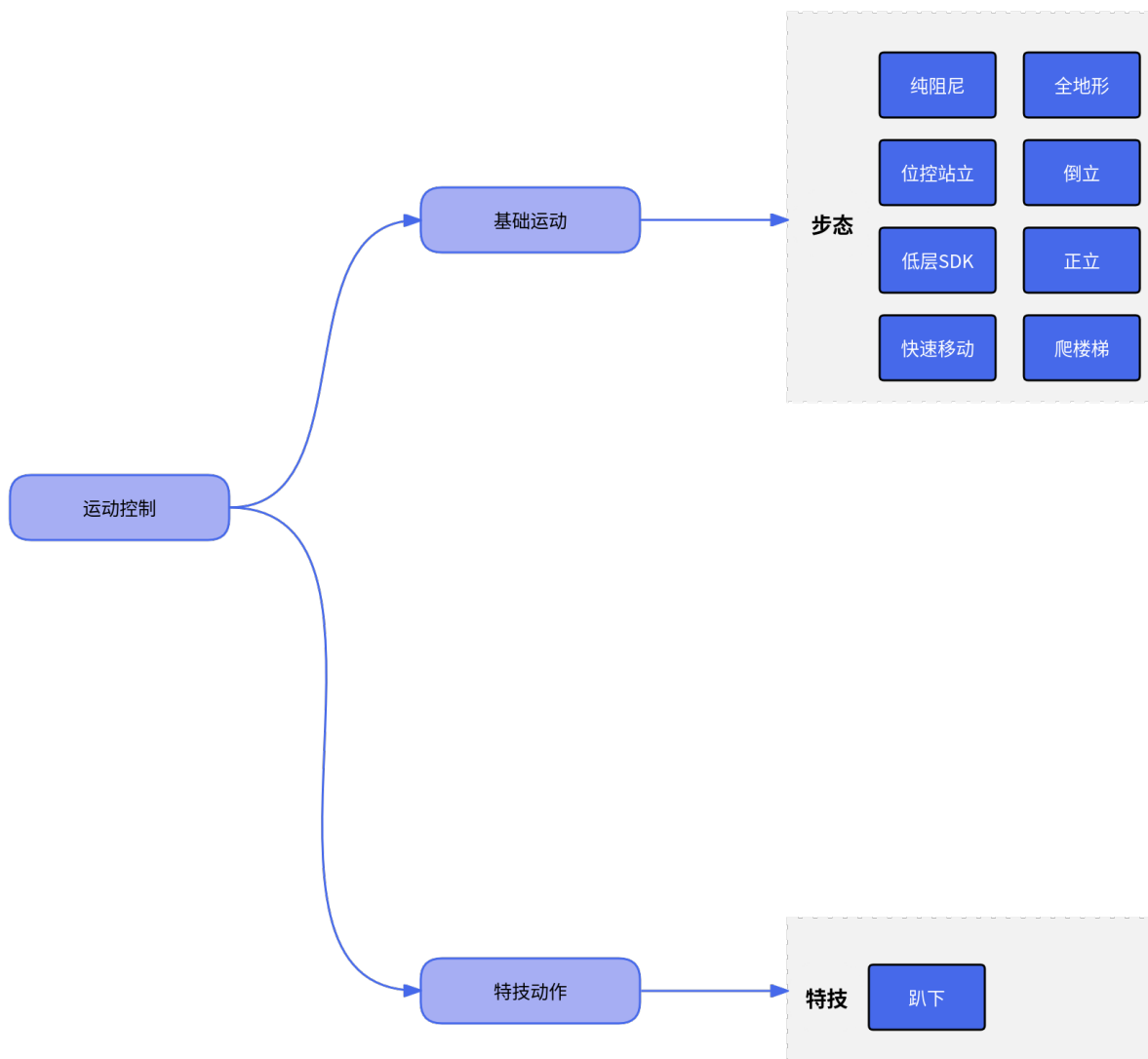
机器人状态切换机制：



16.6 高层运动控制接口

机器人的高层运动控制服务可分为基础运动控制和特技动作控制。

- 基础运动控制服务中，可调用相应的接口，根据不同的地形场景和任务需求切换机器人的行走步态。
- 特技动作控制服务中，可调用相应的接口，实现机器人内置的特殊特技，比如扭屁股、趴下等。



底层运动控制服务 Python

提供机器人系统底层运动控制服务，通过 `LowLevelMotionController` 可以使用话题通信方式实现对机器人关节的指令控制和状态获取。

17.1 接口定义

`LowLevelMotionController` 是面向底层开发的运动控制器，支持对腿等运动部件的直接控制与状态订阅。

△ **Notice:** 调用 `publish_leg_command` 接口，用户命令才会下发到运控，因此需要用户保证调用 `publish_leg_command` 的频率，建议是 500 hz。

17.1.1 LowLevelMotionController —低级运动控制器

项目	内容
类名	<code>LowLevelMotionController</code>
功能概述	底层关节级运动控制器，提供精确的关节控制
主要功能	关节状态订阅、关节命令发布、控制周期设置
使用场景	精确运动控制、自定义步态、研究开发

17.1.2 initialize

项目	内容
方法名	initialize
方法声明	bool initialize()
功能概述	初始化控制器，建立底层连接。
返回值	true 表示成功，false 表示失败。
备注	首次调用必须初始化。

17.1.3 shutdown

项目	内容
方法名	shutdown
方法声明	void shutdown()
功能概述	关闭控制器，释放底层资源。
备注	配合 initialize 使用。

17.1.4 subscribe_leg_state

项目	内容
方法名	subscribe_leg_state
方法声明	void subscribe_leg_state(callback)
功能概述	订阅下肢关节状态数据。
参数说明	callback: 回调函数，用于处理下肢关节状态的接收数据。函数签名: callback(data : LegState) -> None
备注	非阻塞接口。

17.1.5 unsubscribe_leg_state

项目	内容
方法名	unsubscribe_leg_state
方法声明	void unsubscribe_leg_state()
功能概述	取消订阅下肢关节状态数据。
备注	非阻塞接口。与 subscribe_leg_state 配合使用。

17.1.6 publish_leg_command

项目	内容
方法名	publish_leg_command
方法声明	Status publish_leg_command(LegJointCommand command)
功能概述	发布下肢关节控制指令。
参数说明	command: 包含目标角度/速度等控制信息的下肢关节控制指令。
返回值	Status::OK 表示成功, 其他为失败。
备注	非阻塞接口。

17.2 数据结构定义

17.2.1 LegJointCommandArray —腿部关节命令数组

项目	内容
类型	std::array<magic::dog::SingleLegJointCommand, magic::dog::kLegJointNum>; 的 Python 绑定
功能概述	固定大小的腿部关节命令数组, 包含所有腿部关节的控制指令
主要方法	支持索引访问、迭代、长度查询, 长度固定为腿部关节数量
使用场景	腿部运动控制、关节协调控制、步态规划

17.2.2 WheelJointCommandArray —轮子关节命令数组

项目	内容
类型	std::array<magic::dog_w::SingleWheelJointCommand, magic::dog_w::kWheelJointNum>; 的 Python 绑定
功能概述	固定大小的轮子关节命令数组, 包含所有轮子关节的控制指令
主要方法	支持索引访问、迭代、长度查询, 长度固定为轮子关节数量
使用场景	轮式运动控制、关节协调控制、底盘控制

17.2.3 LegJointStateArray —腿部关节状态数组

项目	内容
类型	<code>std::array<magic::dog::SingleLegJointState, magic::dog::kLegJointNum></code> ; 的 Python 绑定
功能概述	固定大小的腿部关节状态数组，包含所有腿部关节的实时状态信息
主要方法	支持索引访问、迭代、长度查询，长度固定为腿部关节数量
使用场景	关节状态监控、运动反馈、安全检测

17.2.4 WheelJointStateArray —轮子关节状态数组

项目	内容
类型	<code>std::array<magic::dog_w::SingleWheelJointState, magic::dog_w::kWheelJointNum></code> ; 的 Python 绑定
功能概述	固定大小的轮子关节状态数组，包含所有轮子关节的实时状态信息
主要方法	支持索引访问、迭代、长度查询，长度固定为轮子关节数量
使用场景	轮子关节状态监控、运动反馈、安全检测

17.2.5 SingleLegJointCommand —单腿关节命令结构体

字段名	类型	描述
<code>q_des</code>	<code>float</code>	期望关节角度
<code>dq_des</code>	<code>float</code>	期望关节角速度
<code>tau_des</code>	<code>float</code>	期望关节力矩
<code>kp</code>	<code>float</code>	位置增益
<code>kd</code>	<code>float</code>	速度增益

17.2.6 SingleWheelJointCommand — 单个轮子关节的控制命令

字段名	类型	描述
q_des	float	期望关节位置
dq_des	float	期望关节速度
tau_des	float	期望前馈力矩
kp	float	P 增益，必须为正数
kd	float	D 增益，必须为正数

17.2.7 LegJointCommand — 整个腿部和轮子关节控制命令结构体

字段名	类型	描述
timestamp	int	时间戳（单位：纳秒）
cmd	SingleLegJointCommand[n]	关节控制命令数组，长度为腿部关节数量，可索引或用列表或 array 设置
wheel_cmd	SingleWheelJointCommand	轮子控制命令数组，长度为轮子数量，用于 WheeledDog 型号

17.2.8 SingleLegJointState — 单腿关节状态结构体

字段名	类型	描述
q	float	关节角度
dq	float	关节角速度
tau_est	float	估计关节力矩

17.2.9 SingleWheelJointState — 单个轮子关节状态结构体

字段名	类型	描述
q	float	实际轮子关节位置
dq	float	实际轮子关节速度
tau_est	float	估计轮子关节力矩

17.2.10 LegState — 整个下肢状态信息

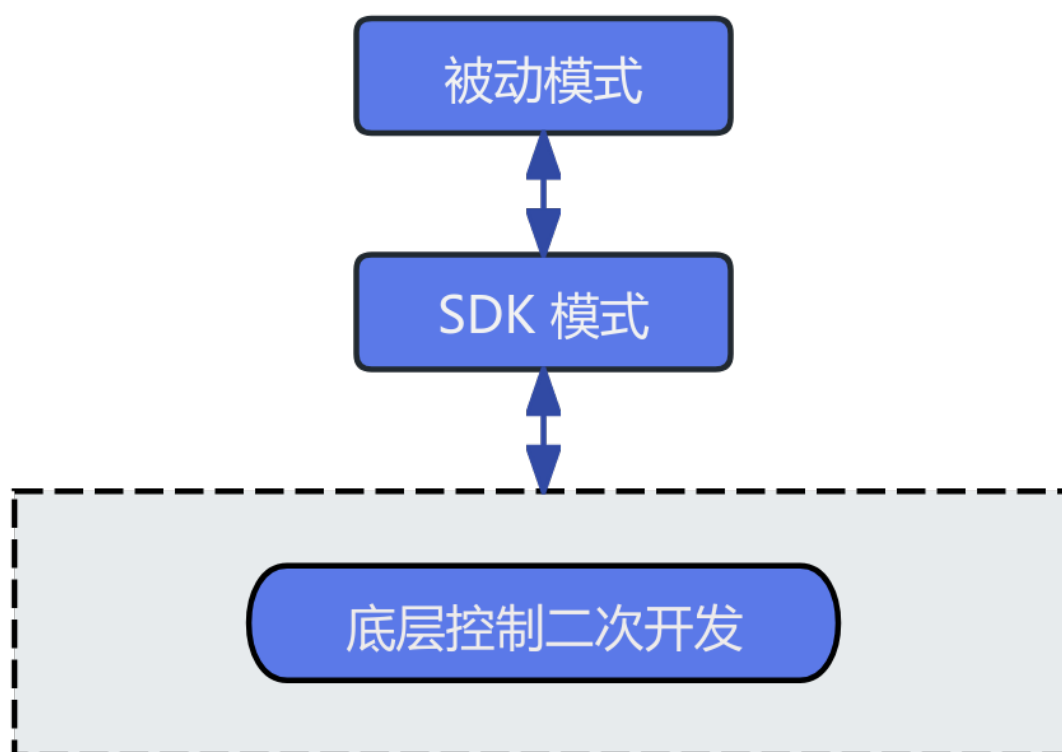
字段名	类型	描述
timestamp	int	时间戳（单位：纳秒）
state	SingleLegJointState[	所有下肢关节状态，n 为腿关节数量，可以用 list 或 array 读写
wheel_stat	SingleWheelJointStat	所有轮子关节状态，m 为轮子数量，仅 WheeledDog 型号支持，可用 list 或 array 读写

17.3 URDF 参考

机器人 URDF

17.4 底层运动控制机器人状态介绍

机器人底层运动主要是开发关节的三环控制给开发人员进行机器人运动能力的二次开发，基本的控制状态切换机制：



17.5 注意事项

在使用 Subscribe 等订阅类 SDK 接口时，请避免在回调函数中执行阻塞式的数据处理操作。否则，可能导致消息堆积、处理延迟，甚至引发不可预期的错误。

提供机器人系统传感器数据管理，包括相机、IMU、激光雷达等。

18.1 接口定义

SensorController 是机器人传感器数据管理，包括相机、IMU、激光雷达等。

18.1.1 SensorController — 传感器控制器

项目	内容
类名	SensorController
功能概述	机器人传感器数据管理，包括相机、IMU、激光雷达等
主要功能	传感器开关控制、数据订阅、多模态感知
使用场景	环境感知、导航定位、状态监控

18.1.2 initialize

项目	内容
方法名	initialize
方法声明	bool initialize()
功能概述	初始化传感器控制器。
返回值	true 表示成功，false 表示失败。
备注	首次使用前必须调用。

18.1.3 shutdown

项目	内容
方法名	shutdown
方法声明	void shutdown()
功能概述	关闭传感器控制器。
备注	配合 initialize 使用。

18.1.4 open_laser_scan

项目	内容
方法名	open_laser_scan
方法声明	Status open_laser_scan(int timeout_ms)
功能概述	打开雷达。
参数说明	timeout_ms: 超时时间（毫秒），默认值为 5000。
返回值	Status::OK 表示成功，其他为失败。
备注	阻塞接口。

18.1.5 close_laser_scan

项目	内容
方法名	close_laser_scan
方法声明	Status close_laser_scan(int timeout_ms)
功能概述	关闭雷达。
参数说明	timeout_ms: 超时时间（毫秒），默认值为 5000。
返回值	Status::OK 表示成功，其他为失败。
备注	阻塞接口，配合打开函数使用。

18.1.6 open_rgbd_camera

项目	内容
方法名	open_rgbd_camera
方法声明	Status open_rgbd_camera(int timeout_ms)
功能概述	打开 RGBD 相机。
参数说明	timeout_ms: 超时时间 (毫秒), 默认值为 5000。
返回值	Status::OK 表示成功, 其他为失败。
备注	阻塞接口。

18.1.7 close_rgbd_camera

项目	内容
方法名	close_rgbd_camera
方法声明	Status close_rgbd_camera(int timeout_ms)
功能概述	关闭 RGBD 相机。
参数说明	timeout_ms: 超时时间 (毫秒), 默认值为 5000。
返回值	Status::OK 表示成功, 其他为失败。
备注	阻塞接口, 配合打开函数使用。

18.1.8 open_binocular_camera

项目	内容
方法名	open_binocular_camera
方法声明	Status open_binocular_camera(int timeout_ms)
功能概述	打开双目相机。
参数说明	timeout_ms: 超时时间 (毫秒), 默认值为 5000。
返回值	Status::OK 表示成功, 其他为失败。
备注	阻塞接口。

18.1.9 close_binocular_camera

项目	内容
方法名	close_binocular_camera
方法声明	Status close_binocular_camera(int timeout_ms)
功能概述	关闭双目相机。
参数说明	timeout_ms: 超时时间 (毫秒), 默认值为 5000。
返回值	Status::OK 表示成功, 其他为失败。
备注	阻塞接口, 配合打开函数使用。

18.1.10 subscribe_imu

项目	内容
方法名	subscribe_imu
方法声明	void subscribe_imu(callback)
功能概述	订阅 IMU 数据。
参数说明	callback: 回调函数, 接收 IMU 数据。函数签名为: callback(data : Imu) -> None
备注	非阻塞接口, 通过回调函数异步接收数据。

18.1.11 unsubscribe_imu

项目	内容
方法名	unsubscribe_imu
方法声明	void unsubscribe_imu()
功能概述	取消订阅 IMU 数据。
备注	非阻塞接口。与 subscribe_imu 配合使用。

18.1.12 subscribe_ultra

项目	内容
方法名	subscribe_ultra
方法声明	void subscribe_ultra(callback)
功能概述	订阅超声波数据。
参数说明	callback: 回调函数, 接收超声波数据。函数签名为: callback(data : Float32MultiArray) -> None
备注	非阻塞接口, 通过回调函数异步接收数据。

18.1.13 unsubscribe_ultra

项目	内容
方法名	unsubscribe_ultra
方法声明	void unsubscribe_ultra()
功能概述	取消订阅超声波数据。
备注	非阻塞接口。与 subscribe_ultra 配合使用。

18.1.14 subscribe_head_touch

项目	内容
方法名	subscribe_head_touch
方法声明	void subscribe_head_touch(callback)
功能概述	订阅头部触摸数据。
参数说明	callback: 回调函数, 接收头部触摸数据。函数签名为: callback(data : HeadTouch) -> None
备注	非阻塞接口, 通过回调函数异步接收数据。

18.1.15 unsubscribe_head_touch

项目	内容
方法名	unsubscribe_head_touch
方法声明	void unsubscribe_head_touch()
功能概述	取消订阅头部触摸数据。
备注	非阻塞接口。与 subscribe_head_touch 配合使用。

18.1.16 subscribe_laser_scan

项目	内容
方法名	subscribe_laser_scan
方法声明	void subscribe_laser_scan(callback)
功能概述	订阅激光雷达数据。
参数说明	callback: 回调函数, 接收激光雷达数据。函数签名为: callback(data : LaserScan) -> None
备注	非阻塞接口, 通过回调函数异步接收数据。

18.1.17 unsubscribe_laser_scan

项目	内容
方法名	unsubscribe_laser_scan
方法声明	void unsubscribe_laser_scan()
功能概述	取消订阅激光雷达数据。
备注	非阻塞接口。与 subscribe_laser_scan 配合使用。

18.1.18 subscribe_rgb_depth_camera_info

项目	内容
方法名	subscribe_rgb_depth_camera_info
方法声明	void subscribe_rgb_depth_camera_info(callback)
功能概述	订阅 RGBD 深度相机内参数据。
参数说明	callback: 回调函数, 接收相机内参数据。函数签名为: callback(data : CameraInfo) -> None
备注	非阻塞接口, 通过回调函数异步接收数据。

18.1.19 unsubscribe_rgb_depth_camera_info

项目	内容
方法名	unsubscribe_rgb_depth_camera_info
方法声明	void unsubscribe_rgb_depth_camera_info()
功能概述	取消订阅 RGBD 深度相机内参数据。
备注	非阻塞接口。与 subscribe_rgb_depth_camera_info 配合使用。

18.1.20 subscribe_rgbd_depth_image

项目	内容
方法名	subscribe_rgbd_depth_image
方法声明	void subscribe_rgbd_depth_image(callback)
功能概述	订阅 RGBD 深度图像数据。
参数说明	callback: 回调函数, 接收深度图像数据。函数签名为: callback(data : Image) -> None
备注	非阻塞接口, 通过回调函数异步接收数据。

18.1.21 unsubscribe_rgbd_depth_image

项目	内容
方法名	unsubscribe_rgbd_depth_image
方法声明	void unsubscribe_rgbd_depth_image()
功能概述	取消订阅 RGBD 深度图像数据。
备注	非阻塞接口。与 subscribe_rgbd_depth_image 配合使用。

18.1.22 subscribe_rgbd_color_camera_info

项目	内容
方法名	subscribe_rgbd_color_camera_info
方法声明	void subscribe_rgbd_color_camera_info(callback)
功能概述	订阅 RGBD 彩色图像内参数据。
参数说明	callback: 回调函数, 接收相机内参数据。函数签名为: callback(data : CameraInfo) -> None
备注	非阻塞接口, 通过回调函数异步接收数据。

18.1.23 unsubscribe_rgbd_color_camera_info

项目	内容
方法名	unsubscribe_rgbd_color_camera_info
方法声明	void unsubscribe_rgbd_color_camera_info()
功能概述	取消订阅 RGBD 彩色图像内参数据。
备注	非阻塞接口。与 subscribe_rgbd_color_camera_info 配合使用。

18.1.24 subscribe_rgbd_color_image

项目	内容
方法名	subscribe_rgbd_color_image
方法声明	void subscribe_rgbd_color_image(callback)
功能概述	订阅 RGBD 彩色图像数据。
参数说明	callback: 回调函数, 接收彩色图像数据。函数签名为: callback(data : Image) -> None
备注	非阻塞接口, 通过回调函数异步接收数据。

18.1.25 unsubscribe_rgbd_color_image

项目	内容
方法名	unsubscribe_rgbd_color_image
方法声明	void unsubscribe_rgbd_color_image()
功能概述	取消订阅 RGBD 彩色图像数据。
备注	非阻塞接口。与 subscribe_rgbd_color_image 配合使用。

18.1.26 subscribe_left_binocular_high_img

项目	内容
方法名	subscribe_left_binocular_high_img
方法声明	void subscribe_left_binocular_high_img(callback)
功能概述	订阅左侧高质量双目数据。
参数说明	callback: 回调函数, 接收左侧高质量双目数据。函数签名为: callback(data : CompressedImage) -> None
备注	非阻塞接口, 通过回调函数异步接收数据。

18.1.27 unsubscribe_left_binocular_high_img

项目	内容
方法名	unsubscribe_left_binocular_high_img
方法声明	void unsubscribe_left_binocular_high_img()
功能概述	取消订阅左侧高质量双目数据。
备注	非阻塞接口。与 subscribe_left_binocular_high_img 配合使用。

18.1.28 subscribe_left_binocular_low_img

项目	内容
方法名	subscribe_left_binocular_low_img
方法声明	void subscribe_left_binocular_low_img(callback)
功能概述	订阅左侧低质量双目数据。
参数说明	callback: 回调函数, 接收左侧低质量双目数据。函数签名为: callback(data: CompressedImage) -> None
备注	非阻塞接口, 通过回调函数异步接收数据。

18.1.29 unsubscribe_left_binocular_low_img

项目	内容
方法名	unsubscribe_left_binocular_low_img
方法声明	void unsubscribe_left_binocular_low_img()
功能概述	取消订阅左侧低质量双目数据。
备注	非阻塞接口。与 subscribe_left_binocular_low_img 配合使用。

18.1.30 subscribe_right_binocular_low_img

项目	内容
方法名	subscribe_right_binocular_low_img
方法声明	void subscribe_right_binocular_low_img(callback)
功能概述	订阅右侧低质量双目数据。
参数说明	callback: 回调函数, 接收右侧低质量双目数据。函数签名为: callback(data: CompressedImage) -> None
备注	非阻塞接口, 通过回调函数异步接收数据。

18.1.31 unsubscribe_right_binocular_low_img

项目	内容
方法名	unsubscribe_right_binocular_low_img
方法声明	void unsubscribe_right_binocular_low_img()
功能概述	取消订阅右侧低质量双目数据。
备注	非阻塞接口。与 subscribe_right_binocular_low_img 配合使用。

18.1.32 subscribe_depth_image

项目	内容
方法名	subscribe_depth_image
方法声明	void subscribe_depth_image(callback)
功能概述	订阅深度图像数据。
参数说明	callback: 回调函数, 接收深度图像数据。函数签名为: callback(data : Image) -> None
备注	非阻塞接口, 通过回调函数异步接收数据。

18.1.33 unsubscribe_depth_image

项目	内容
方法名	unsubscribe_depth_image
方法声明	void unsubscribe_depth_image()
功能概述	取消订阅深度图像数据。
备注	非阻塞接口。与 subscribe_depth_image 配合使用。

18.2 数据结构定义

18.2.1 UInt8Vector — 无符号 8 位整数向量

项目	内容
类型	std::vector<uint8_t> 的 Python 绑定
功能概述	存储无符号 8 位整数的动态数组, 常用于存储原始字节数据
主要方法	支持标准 Python 列表操作: 索引访问、切片、迭代、长度查询等
使用场景	图像数据、音频数据、二进制数据存储

18.2.2 PointFieldVector 一点云字段向量

项目	内容
类型	<code>std::vector<magic::dog::PointField></code> 的 Python 绑定
功能概述	存储点云字段描述的动态数组，用于定义点云数据的结构
主要方法	支持标准 Python 列表操作，每个元素为 PointField 结构体
使用场景	点云数据处理、3D 传感器数据解析

18.2.3 MultiArrayDimensionVector 多维数组维度向量

项目	内容
类型	<code>std::vector<magic::dog::MultiArrayDimension></code> 的 Python 绑定
功能概述	存储多维数组维度信息的动态数组，用于描述数组的形状和布局
主要方法	支持标准 Python 列表操作，每个元素为 MultiArrayDimension 结构体
使用场景	多维传感器数据、矩阵数据处理

18.2.4 FloatVector 浮点数向量

项目	内容
类型	<code>std::vector<float></code> 的 Python 绑定
功能概述	存储单精度浮点数的动态数组，用于高精度数值计算
主要方法	支持标准 Python 列表操作，支持浮点数运算
使用场景	传感器数据、坐标数据、物理量数据

18.2.5 DoubleVector 双精度浮点数向量

项目	内容
类型	<code>std::vector<double></code> 的 Python 绑定
功能概述	存储双精度浮点数的动态数组，提供最高精度的数值计算
主要方法	支持标准 Python 列表操作，支持高精度浮点数运算
使用场景	高精度传感器数据、科学计算、坐标变换

18.2.6 Double3Array — 三维双精度浮点数数组

项目	内容
类型	<code>std::array<double, 3></code> 的 Python 绑定
功能概述	固定大小的三维双精度浮点数数组，常用于 3D 坐标表示
主要方法	支持索引访问、迭代、长度查询，长度固定为 3
使用场景	3D 坐标、RGB 值、欧拉角、位置向量

18.2.7 Double4Array — 四维双精度浮点数数组

项目	内容
类型	<code>std::array<double, 4></code> 的 Python 绑定
功能概述	固定大小的四维双精度浮点数数组，常用于四元数表示
主要方法	支持索引访问、迭代、长度查询，长度固定为 4
使用场景	四元数、RGBA 值、姿态表示

18.2.8 Double9Array — 九维双精度浮点数数组

项目	内容
类型	<code>std::array<double, 9></code> 的 Python 绑定
功能概述	固定大小的九维双精度浮点数数组，常用于 3x3 矩阵表示
主要方法	支持索引访问、迭代、长度查询，长度固定为 9
使用场景	3x3 旋转矩阵、协方差矩阵、变换矩阵

18.2.9 Double12Array — 十二维双精度浮点数数组

项目	内容
类型	<code>std::array<double, 12></code> 的 Python 绑定
功能概述	固定大小的十二维双精度浮点数数组，常用于 3x4 投影矩阵表示
主要方法	支持索引访问、迭代、长度查询，长度固定为 12
使用场景	3x4 投影矩阵、相机参数矩阵

18.2.10 Header —头部信息结构体

字段名	类型	描述
stamp	int	时间戳
frame_id	str	坐标系 ID

18.2.11 Image —图像结构体

字段名	类型	描述
header	Header	头部信息
height	int	图像高度
width	int	图像宽度
encoding	str	编码格式
is_bigendian	bool	是否大端序
step	int	步长
data	UInt8Vector	图像数据

18.2.12 Imu —惯性测量单元结构体

字段名	类型	描述
timestamp	int	时间戳
orientation	Double4Array	姿态四元数（只读）
angular_velocity	Double3Array	角速度（只读）
linear_acceleration	Double3Array	线性加速度（只读）
temperature	float	温度

18.2.13 PointField —点云字段描述结构体

字段名	类型	描述
name	str	字段名，如"x"、"y"、"z"、"intensity" 等
offset	int	起始字节偏移
datatype	int	数据类型（对应常量）
count	int	该字段包含的元素数量

18.2.14 PointCloud2 —通用点云数据结构

字段名	类型	描述
header	Header	标准消息头
height	int	行数
width	int	列数
fields	PointFieldVector	点字段数组
is_bigendian	bool	字节序
point_step	int	每个点占用的字节数
row_step	int	每行占用的字节数
data	UInt8Vector	原始点云数据（按字段打包）
is_dense	bool	是否为稠密点云（无无效点）

18.2.15 CameraInfo —相机内参与畸变信息结构体

字段名	类型	描述
header	Header	通用消息头
height	int	图像高度（行数）
width	int	图像宽度（列数）
distortion_model	str	畸变模型，如“plumb_bob”
D	FloatVector	畸变参数数组
K	FloatVector	相机内参矩阵（9 个元素）
R	FloatVector	矫正矩阵（9 个元素）
P	FloatVector	投影矩阵（12 个元素）
binning_x	int	水平 binning 系数
binning_y	int	垂直 binning 系数
roi_x_offset	int	ROI 起始 x
roi_y_offset	int	ROI 起始 y
roi_height	int	ROI 高度
roi_width	int	ROI 宽度
roi_do_rectify	bool	是否进行矫正

18.2.16 CompressedImage —压缩图像结构体

字段名	类型	描述
header	Header	通用消息头
format	str	压缩格式
data	UInt8Vector	压缩后的图像数据

18.2.17 LaserScan — 激光雷达数据结构体

字段名	类型	描述
header	Header	消息头
angle_min	int	起始角度（弧度）
angle_max	int	结束角度（弧度）
angle_increment	int	角度增量（弧度）
time_increment	int	时间增量（秒）
scan_time	int	扫描时间（秒）
range_min	int	最小距离（米）
range_max	int	最大距离（米）
ranges	FloatVector	距离数据数组（单位：米）
intensities	FloatVector	强度数据数组（无量纲）

18.2.18 MultiArrayDimension — 多维数组维度描述结构体

字段名	类型	描述
label	str	维度标签
size	int	维度大小
stride	int	步长

18.2.19 MultiArrayLayout — 多维数组布局描述结构体

字段名	类型	描述
dim_size	int	维度数量
dim	MultiArrayDimensionVector	维度描述数组
data_offset	int	数据偏移

18.2.20 Float32MultiArray — 浮点数多维数组结构体

字段名	类型	描述
layout	MultiArrayLayout	数组布局描述
data	FloatVector	浮点数数据数组

18.2.21 ByteMultiArray —字节数组结构体

字段名	类型	描述
layout	MultiArrayLayout	数组布局描述
data	Uint8Vector	字节数据数组

18.2.22 HeadTouch —头部触摸数据结构体

字段名	类型	描述
data	int	触摸状态数据

18.3 3D 雷达数据

由于 3D Lidar 数据在 nx 上以 ros2 话题的方式发布，为了避免引入不必要的延迟，SDK 未对其进行封装。如果需要使用，则可以使用以下方法查找 3D Lidar 数据

```
source /etc/eame/ros2.env
ros2 topic echo /lidar_point_cloud
```

- 消息类型：sensor_msgs::msg::PointCloud2
- 该消息类型属于 ros2 标准消息类型，详情可以参考 https://docs.ros.org/en/noetic/api/sensor_msgs/html/msg/PointCloud2.html

18.4 注意事项

在使用 Subscribe 等订阅类 SDK 接口时，请避免在回调函数中执行阻塞式的数据处理操作。否则，可能导致消息堆积、处理延迟，甚至引发不可预期的错误。

提供机器人系统音频控制，包括 TTS 语音合成、音频播放、语音配置。

19.1 接口定义

AudioController 是机器人音频控制，包括 TTS 语音合成、音频播放、语音配置。

19.1.1 AudioController — 音频控制器

项目	内容
类名	AudioController
功能概述	机器人音频控制，包括 TTS 语音合成、音频播放、语音配置
主要功能	语音播放、音量控制、语音模型切换、原始音频数据订阅
使用场景	语音交互、音频播放、语音识别

19.1.2 initialize

项目	内容
方法名	initialize
方法声明	bool initialize()
功能概述	初始化音频控制器。
返回值	true 表示成功，false 表示失败。
备注	首次使用前必须调用。

19.1.3 shutdown

项目	内容
方法名	shutdown
方法声明	void shutdown()
功能概述	关闭音频控制器。
备注	配合 initialize 使用。

19.1.4 play

项目	内容
方法名	play
方法声明	Status play(TtsCommand command, int timeout_ms)
功能概述	播放 TTS（文本转语音）语音命令。
参数说明	command: TTS 命令，包含文本、语速、语调等。timeout_ms: 超时时间（毫秒），默认值为 5000。
返回值	Status::OK 表示成功，其他为失败状态。
备注	阻塞接口，调用前需确保已初始化模块。

19.1.5 stop

项目	内容
方法名	stop
方法声明	Status stop(int timeout_ms)
功能概述	停止当前音频播放。
参数说明	timeout_ms: 超时时间（毫秒），默认值为 5000。
返回值	Status::OK 表示成功，其他为失败状态。
备注	阻塞接口，通常用于中断当前语音。

19.1.6 set_volume

项目	内容
方法名	set_volume
方法声明	Status set_volume(int volume, int timeout_ms)
功能概述	设置音频输出的音量。
参数说明	volume: 音量值，通常范围为 0~100。timeout_ms: 超时时间（毫秒），默认值为 5000。
返回值	Status::OK 表示成功，其他为失败状态。
备注	阻塞接口，设置后立即生效。

19.1.7 get_volume

项目	内容
方法名	get_volume
方法声明	tuple[Status, int] get_volume(int timeout_ms)
功能概述	获取当前音频输出音量。
参数说明	timeout_ms: 超时时间（毫秒），默认值为 5000。
返回值	成功，返回当前音量值。
备注	非阻塞接口。

19.1.8 switch_tts_voice_model

项目	内容
方法名	switch_tts_voice_model
方法声明	Status switch_tts_voice_model(TtsType tts_type, int timeout_ms)
功能概述	切换 TTS 语音模型。
参数说明	tts_type: TTS 语音模型类型 timeout_ms: 超时时间 (毫秒), 默认值为 5000。
返回值	操作状态。
备注	切换模型属于阻塞操作。

19.1.9 get_voice_config

项目	内容
方法名	get_voice_config
方法声明	tuple[Status, GetSpeechConfig] get_voice_config(int timeout_ms)
功能概述	获取语音系统的完整配置信息。
参数说明	timeout_ms: 超时时间 (毫秒), 默认值为 5000。
返回值	语音系统配置信息, 失败返回空配置对象。
备注	阻塞接口, 获取的配置包含音色、智能体、唤醒、对话等所有子配置。

19.1.10 set_voice_config

项目	内容
方法名	set_voice_config
方法声明	Status set_voice_config(SetSpeechConfig config, int timeout_ms)
功能概述	设置语音系统的完整配置信息。
参数说明	config: 要设置的语音系统配置信息。timeout_ms: 超时时间 (毫秒), 默认值为 5000。
返回值	Status::OK 表示成功, 其他为失败状态。
备注	阻塞接口, 设置的配置将完全覆盖当前所有语音相关配置。

19.1.11 control_voice_stream

项目	内容
方法名	control_voice_stream
方法声明	Status control_voice_stream(bool raw_data, bool bf_data, int timeout_ms)
功能概述	控制语音数据流。
参数说明	raw_data: 是否开启原始语音数据流。bf_data: 是否开启 BF 语音数据流。timeout_ms: 超时时间 (毫秒), 默认值为 5000。
返回值	操作状态。Status::OK 表示成功, 其他为失败。
备注	阻塞接口, 用于控制语音数据流的开启和关闭。

19.1.12 subscribe_origin_voice_data

项目	内容
方法名	subscribe_origin_voice_data
方法声明	void subscribe_origin_voice_data(callback)
功能概述	订阅原始语音数据。
参数说明	callback: 接收到原始语音数据后的处理回调。函数签名为: callback(data : ByteMultiArray) -> None
备注	非阻塞接口, 回调函数会在数据更新时被调用。

19.1.13 unsubscribe_origin_voice_data

项目	内容
方法名	unsubscribe_origin_voice_data
方法声明	void unsubscribe_origin_voice_data()
功能概述	取消订阅原始语音数据。
备注	非阻塞接口。与 subscribe_origin_voice_data 配合使用。

19.1.14 subscribe_bf_voice_data

项目	内容
方法名	subscribe_bf_voice_data
方法声明	void subscribe_bf_voice_data(callback)
功能概述	订阅 BF 语音数据。
参数说明	callback: 接收到 BF 语音数据后的处理回调。函数签名为: callback(data : ByteMultiArray) -> None
备注	非阻塞接口, 回调函数会在数据更新时被调用。

19.1.15 unsubscribe_bf_voice_data

项目	内容
方法名	unsubscribe_bf_voice_data
方法声明	void unsubscribe_bf_voice_data()
功能概述	取消订阅 BF 语音数据。
备注	非阻塞接口。与 subscribe_bf_voice_data 配合使用。

19.1.16 control_speech_io

项目	内容
方法名	control_speech_io
方法声明	Status control_speech_io(bool enable_data, int timeout_ms);
功能概述	控制语音 io 数据流 (asr 和 tts)。
参数说明	enable_data: 打开或关闭语音 io 数据流。timeout_ms: 超时时间 (毫秒), 默认值为 5000。
返回值	Status::OK 表示成功, 其他为失败。
备注	阻塞接口, 用于控制语音 io 数据流 (asr 和 tts)。

19.1.17 subscribe_speech_asr

项目	内容
方法名	subscribe_speech_asr
方法声明	void subscribe_speech_asr(callback);
功能概述	订阅语音 asr 流。
参数说明	callback: 处理接收到的语音 asr 流的回调。
备注	阻塞接口，用于订阅语音 asr 流。

19.1.18 unsubscribe_speech_asr

项目	内容
方法名	unsubscribe_speech_asr
方法声明	void unsubscribe_speech_asr();
功能概述	取消订阅语音 asr 流。
备注	阻塞接口，用于取消订阅语音 asr 流。

19.1.19 publish_speech_tts

项目	内容
方 法 名	publish_speech_tts
方 法 声明	Status publish_speech_tts(SpeechTTSStream data);
功 能 概述	发布语音 tts 流。
参 数 说明	data: 语音 tts 流。
备注	阻塞接口，用于发布语音 tts 流。数据 ID 应该与 ASR 输出的请求 ID 一致，起始类型为 begin，中间数据类型为 var，结束类型为 end。

19.2 枚举类型定义

19.2.1 TtsPriority —TTS 优先级枚举

枚举值	数值	描述
HIGH	0	高优先级
MIDDLE	1	中优先级
LOW	2	低优先级

19.2.2 TtsMode —TTS 模式枚举

枚举值	数值	描述
CLEARTOP	0	清除顶部
ADD	1	添加
CLEARBUFFER	2	清除缓冲区

19.2.3 TtsType —TTS 类型枚举

枚举值	数值	描述
NONE	0	无
DOUBAO	1	豆包
GOOGLE	2	谷歌

19.3 数据结构定义

19.3.1 S2DString —二维字符串数组

项目	内容
类型	<code>std::array<std::string, 2></code> 的 Python 绑定
功能概述	固定大小的二维字符串数组，包含两个字符串元素
主要方法	支持索引访问、迭代、长度查询，长度固定为 2
使用场景	键值对存储、配置参数、状态标识

19.3.2 S2DStringVector — 二维字符串数组

项目	内容
类型	<code>std::vector<std::array<std::string, 2>>></code> 的 Python 绑定
功能概述	可变长度的二维字符串数组，每个元素为包含两个字符串的数组
主要方法	支持索引访问、迭代、长度查询，支持动态添加和删除元素，每个元素长度固定为 2
使用场景	批量键值对存储、配置参数集合、状态标识列表等

19.3.3 String2DStringVectorMap — 字符串到二维字符串数组向量的映射

项目	内容
类型	<code>std::map<std::string, std::vector<std::array<std::string, 2>>>></code> 的 Python 绑定
功能概述	字符串键到二维字符串数组向量的映射，用于存储配置参数
主要方法	支持标准 Python 字典操作：键值访问、迭代、长度查询等
使用场景	音色配置、智能体配置、系统参数存储

19.3.4 StringBotInfoMap — 字符串到智能体信息的映射

项目	内容
类型	<code>std::map<std::string, magic::dog::BotInfo></code> 的 Python 绑定
功能概述	字符串键到智能体信息的映射，用于管理多个智能体配置
主要方法	支持标准 Python 字典操作，每个值为 BotInfo 结构体
使用场景	智能体管理、配置存储、多智能体系统

19.3.5 StringCustomBotMap — 自定义智能体映射

项目	内容
类型	<code>std::map<std::string, magic::dog::CustomBotInfo></code> 的 Python 绑定
功能概述	字符串键到自定义智能体信息的映射，用于管理用户自定义的智能体
主要方法	支持标准 Python 字典操作，每个值为 CustomBotInfo 结构体
使用场景	自定义智能体管理、用户配置存储

19.3.6 StringStringMap —字符串到字符串的映射

项目	内容
类型	<code>std::map<std::string, std::string></code> 的 Python 绑定
功能概述	字符串键到字符串值的简单映射，用于存储配置参数
主要方法	支持标准 Python 字典操作，键值均为字符串类型
使用场景	简单配置存储、参数映射、状态标识

19.3.7 TtsCommand —TTS 命令结构体

字段名	类型	描述
id	int	命令 ID
content	str	内容
priority	TtsPriority	优先级
mode	TtsMode	模式

19.3.8 GetSpeechConfig —语音配置获取结构体

字段名	类型	描述
tts_type	TtsType	TTS 类型
speaker_config	SpeakerConfig	音色配置
bot_config	BotConfig	智能体配置
wakeup_config	WakeupConfig	唤醒配置
dialog_config	DialogConfig	对话配置

19.3.9 SetSpeechConfig —语音配置设置结构体

字段名	类型	描述
speaker_id	str	音色 ID
region	str	音色区域
bot_id	str	智能体 ID
is_front_doa	bool	是否启用前向声源定位
is_full duplex_enable	bool	是否启用全双工对话
is_enable	bool	是否启用对话功能
is_doa_enable	bool	是否启用声源定位
speaker_speed	double	音色语速
wakeup_name	str	唤醒词名称
custom_bot	CustomBotInfo	自定义智能体信息

19.3.10 SpeakerConfig —音色配置结构体

字段名	类型	描述
data	MapStringVectorArray2DString	音色数据映射
selected	SpeakerConfigSelected	选中的音色配置
speaker_speed	double	音色语速

19.3.11 BotConfig —智能体配置结构体

字段名	类型	描述
data	MapStringBotInfo	智能体数据映射
custom_data	CustomBotMap	自定义智能体数据
selected	BotInfo	选中的智能体

19.3.12 WakeupConfig —唤醒配置结构体

字段名	类型	描述
name	str	唤醒词名称
data	MapStringString	唤醒数据映射

19.3.13 DialogConfig —对话配置结构体

字段名	类型	描述
is_front_doa	bool	是否启用前向声源定位
is_full duplex_enable	bool	是否启用全双工对话
is_enable	bool	是否启用对话功能
is_doa_enable	bool	是否启用声源定位

19.3.14 SpeakerConfigSelected —音色配置选择结构体

字段名	类型	描述
speaker_id	str	音色 ID
region	str	音色区域

19.3.15 BotInfo —智能体信息结构体

字段名	类型	描述
name	str	智能体名称
workflow	str	工作流 ID

19.3.16 CustomBotInfo —自定义智能体信息结构体

字段名	类型	描述
name	str	自定义智能体名称
workflow	str	自定义工作流 ID
token	str	访问令牌

19.3.17 SpeechASRStream —机器人 ASR 输出

字段名	类型	描述
id	str	ID - 请求 id
type	str	Type - 类型 request or cancel
text	str	Text - ASR 输出

19.3.18 SpeechTTSStream —机器人 TTS 输入

字段名	类型	描述
id	str	ID - 与请求 id 相同
type	str	Type - 类型 begin or var or end
text	str	Text - TTS 输入
end_session	bool	End Session - 结束会话并关闭 asr

19.4 注意事项

在使用 `Subscribe` 等订阅类 SDK 接口时，请避免在回调函数中执行阻塞式的数据处理操作。否则，可能导致消息堆积、处理延迟，甚至引发不可预期的错误。

提供机器人系统运行状态监控，包括故障检测、电池状态等。

20.1 接口定义

StateMonitor 是机器人运行状态监控，包括故障检测、电池状态等。

20.1.1 StateMonitor —状态监控器

项目	内容
类名	StateMonitor
功能概述	机器人运行状态监控，包括故障检测、电池状态等
主要功能	状态查询、故障监控、健康检查
使用场景	系统监控、故障诊断、状态报告

20.1.2 initialize

项目	内容
方法名	<code>initialize</code>
方法声明	<code>bool initialize()</code>
功能概述	初始化状态监控器。
返回值	<code>true</code> 表示成功， <code>false</code> 表示失败。
备注	首次使用前必须调用。

20.1.3 shutdown

项目	内容
方法名	<code>shutdown</code>
方法声明	<code>void shutdown()</code>
功能概述	关闭状态监控器。
备注	配合 <code>initialize</code> 使用。

20.1.4 get_current_state

项目	内容
方法名	<code>get_current_state</code>
方法声明	<code>RobotState get_current_state()</code>
功能概述	获取机器人当前状态信息。
返回值	<code>tuple[Status, RobotState]</code> 对象
备注	非阻塞接口，用于获取机器人当前状态信息。

20.2 错误码映射表

错误码（十六进制）	错误描述
0x0000	无错误
0x1101	调用 ROS 服务失败
0x1301	中控节点丢失
0x1302	APP 节点丢失
0x1304	音频节点丢失
0x1305	运动节点丢失

下页继续

表 1 - 续上页

错误码（十六进制）	错误描述
0x1306	LCD 节点丢失
0x1307	realsense 节点丢失
0x1308	双目相机节点丢失
0x1309	LDS 节点丢失
0x130A	传感器板节点丢失
0x130B	触摸节点丢失
0x130C	SLAM 节点丢失
0x130D	导航节点丢失
0x130E	AI 节点丢失
0x130F	头部节点丢失
0x1310	云端处理器节点丢失
0x3201	激光无数据
0x3202	双目相机无数据
0x3203	双目相机数据错误
0x3204	双目相机初始化失败
0x320B	里程计无数据
0x320C	IMU 无数据
0x6101	机器人连接 APP 失败
0x6102	与 APP 断开连接
0x9201	打开 LCD 串口失败
0x7201	打开头部串口失败
0x7202	头部无数据
0x8201	打开传感器板串口失败
0x8202	传感器板无数据
0x2201	错误：导航未收到 tf 数据
0x2202	错误：导航未收到地图数据
0x2203	错误：导航未收到定位数据
0x2204	错误：导航未收到超声波数据
0x2205	错误：导航未收到激光数据
0x2206	错误：导航未收到 RGBD 数据
0x2207	错误：导航未收到多线激光数据
0x2208	错误：导航未收到点 TOF 数据
0x2209	错误：导航未收到面 TOF 数据
0x220A	错误：导航未收到里程计数据
0x2101	警告：导航未收到 tf 数据
0x2102	警告：导航未收到地图数据
0x2103	警告：导航未收到定位数据
0x2104	警告：导航未收到超声波数据

下页继续

表 1 – 续上页

错误码（十六进制）	错误描述
0x2105	警告：导航未收到激光数据
0x2106	警告：导航未收到 RGBD TOF 数据
0x2107	警告：导航未收到多线激光数据
0x2108	警告：导航未收到点 TOF 数据
0x2109	警告：导航未收到面 TOF 数据
0x210A	警告：导航未收到里程计数据
0x4201	SLAM 定位错误
0x4102	错误：SLAM 未收到激光数据
0x4103	错误：SLAM 未收到里程计数据
0x4205	SLAM 地图错误

20.3 枚举类型定义

20.3.1 BatteryState — 电池状态枚举

枚举值	数值	描述
UNKNOWN	0	未知状态
GOOD	1	良好
OVERHEAT	2	过热
DEAD	3	电量耗尽
OVERVOLTAGE	4	过压
UNSPEC_FAILURE	5	未指定故障
COLD	6	过冷
WATCHDOG_TIMER_EXPIRE	7	看门狗定时器超时
SAFETY_TIMER_EXPIRE	8	安全定时器超时

20.3.2 PowerSupplyStatus — 电源状态枚举

枚举值	数值	描述
UNKNOWN	0	未知状态
CHARGING	1	充电中
DISCHARGING	2	放电中
NOTCHARGING	3	未充电
FULL	4	电量满

20.4 数据结构定义

20.4.1 FaultVector —故障信息向量

项目	内容
类型	<code>std::vector<magic::dog::Fault></code> 的 Python 绑定
功能概述	存储故障信息的动态数组，用于系统状态监控和错误处理
主要方法	支持标准 Python 列表操作，每个元素为 Fault 结构体
使用场景	故障诊断、系统监控、错误日志记录

20.4.2 Fault —故障信息结构体

字段名	类型	描述
<code>error_code</code>	<code>int</code>	错误代码
<code>error_message</code>	<code>str</code>	错误消息

20.4.3 BmsData —电池管理系统数据结构体

字段名	类型	描述
<code>battery_percentage</code>	<code>double</code>	电池电量百分比
<code>battery_health</code>	<code>double</code>	电池健康度
<code>battery_state</code>	<code>BatteryState</code>	电池状态
<code>power_supply_status</code>	<code>PowerSupplyStatus</code>	电源状态

20.4.4 RobotState —机器人状态结构体

字段名	类型	描述
<code>faults</code>	<code>FaultVector</code>	故障列表
<code>bms_data</code>	<code>BmsData</code>	电池数据

SLAM 导航控制服务 Python

提供机器人系统 SLAM 导航控制服务，通过 SlamNavController 可以实现 SLAM 建图、定位、导航等功能。

21.1 接口定义

SlamNavController 是面向 SLAM 导航控制的控制器，支持建图、定位、导航等操作。

21.1.1 SlamNavController —SLAM 导航控制器

项目	内容
类名	SlamNavController
功能概述	面向 SLAM 导航控制的控制器，支持建图、定位、导航等操作
主要功能	SLAM 建图、定位、导航、地图管理
使用场景	机器人自主导航、环境建图、路径规划

21.1.2 initialize

项目	内容
方法名	<code>initialize</code>
方法声明	<code>bool initialize()</code>
功能概述	初始化 SLAM 导航控制器。
返回值	<code>true</code> 表示成功， <code>false</code> 表示失败。
备注	首次使用前必须调用。

21.1.3 shutdown

项目	内容
方法名	<code>shutdown</code>
方法声明	<code>void shutdown()</code>
功能概述	释放资源，清理 SLAM 导航控制器。
备注	配合 <code>initialize</code> 使用。

21.1.4 switch_to_idle

项目	内容
方法名	<code>switch_to_idle</code>
方法声明	<code>Status switch_to_idle()</code>
功能概述	切换到空闲模式。
返回值	<code>Status::OK</code> 表示成功，其他为失败。
备注	阻塞接口，用于切换到空闲状态。

21.1.5 switch_to_location

项目	内容
方法名	<code>switch_to_location</code>
方法声明	<code>Status switch_to_location()</code>
功能概述	切换到定位模式。
返回值	<code>Status::OK</code> 表示成功，其他为失败。
备注	阻塞接口，用于切换到定位状态。

21.1.6 start_mapping

项目	内容
方法名	start_mapping
方法声明	Status start_mapping()
功能概述	开始建图模式。
返回值	Status::OK 表示成功，其他为失败。
备注	阻塞接口，用于启动建图任务。

21.1.7 cancel_mapping

项目	内容
方法名	cancel_mapping
方法声明	Status cancel_mapping()
功能概述	取消当前建图任务。
返回值	Status::OK 表示成功，其他为失败。
备注	阻塞接口，用于取消建图任务。

21.1.8 save_map

项目	内容
方法名	save_map
方法声明	Status save_map(const str& map_name)
功能概述	结束建图并保存地图，当处于建图模式时。
参数说明	map_name: 地图名称。
返回值	Status::OK 表示成功，其他为失败。
备注	阻塞接口，用于保存当前建图结果。

21.1.9 load_map

项目	内容
方法名	load_map
方法声明	Status load_map(const str& map_name)
功能概述	加载地图并设置为当前地图。
参数说明	map_name: 地图名称。
返回值	Status::OK 表示成功，其他为失败。
备注	阻塞接口，用于加载已保存的地图。

21.1.10 delete_map

项目	内容
方法名	delete_map
方法声明	Status delete_map(const str& map_name)
功能概述	删除地图。
参数说明	map_name: 要删除的地图名称。
返回值	Status::OK 表示成功，其他为失败。
备注	阻塞接口，用于删除不需要的地图。

21.1.11 get_all_map_info

项目	内容
方法名	get_all_map_info
方法声明	tuple[Status, AllMapInfo] get_all_map_info(int timeout_ms)
功能概述	获取所有地图信息
参数说明	timeout_ms: 超时时间（毫秒），默认值为 10000
返回值	Status::OK: 表示成功，其他为失败 AllMapInfo: 所有地图信息（输出参数）
备注	阻塞接口，获取系统中所有地图的详细信息

21.1.12 init_pose

项目	内容
方法名	init_pose
方法声明	Status init_pose(Pose3DEuler pose)
功能概述	初始化位姿。
参数说明	pose: 要发布的位姿信息。
返回值	Status::OK 表示成功，其他为失败。
备注	阻塞接口，用于设置机器人的初始位姿。

21.1.13 get_current_localization_info

项目	内容
方法名	get_current_localization_info
方法声明	tuple[Status, LocalizationInfo] get_current_localization_info()
功能概述	获取当前位姿信息。
返回值	Status::OK 表示成功，其他为失败。LocalizationInfo 对象，失败返回空对象。
备注	阻塞接口，用于获取机器人当前位姿。

21.1.14 activate_nav_mode

项目	内容
方法名	activate_nav_mode
方法声明	Status activate_nav_mode (NavMode mode)
功能概述	激活导航模式。
参数说明	mode: 目标导航模式 (NavMode 枚举类型)。
返回值	Status::OK 表示成功，其他为失败。
备注	阻塞接口，用于切换导航工作模式。

21.1.15 set_nav_target

项目	内容
方法名	set_nav_target
方法声明	Status set_nav_target (NavTarget goal)
功能概述	设置全局导航目标点并开始导航任务。
参数说明	goal: 目标点的全局坐标。
返回值	Status::OK 表示成功，其他为失败。
备注	阻塞接口，用于设置导航目标。

21.1.16 pause_nav_task

项目	内容
方法名	pause_nav_task
方法声明	Status pause_nav_task()
功能概述	暂停当前导航任务。
返回值	Status::OK 表示成功，其他为失败。
备注	阻塞接口，用于暂停导航。

21.1.17 resume_nav_task

项目	内容
方法名	resume_nav_task
方法声明	Status resume_nav_task()
功能概述	恢复暂停的导航任务。
返回值	Status::OK 表示成功，其他为失败。
备注	阻塞接口，用于恢复导航。

21.1.18 cancel_nav_task

项目	内容
方法名	cancel_nav_task
方法声明	Status cancel_nav_task()
功能概述	取消当前导航任务。
返回值	Status::OK 表示成功，其他为失败。
备注	阻塞接口，用于取消导航。

21.1.19 get_nav_task_status

项目	内容
方法名	get_nav_task_status
方法声明	tuple[Status, NavStatus] get_nav_task_status()
功能概述	获取当前导航任务的状态信息。
返回值	Status::OK 表示成功，其他为失败，同时返回 NavStatus 对象（失败时为空对象）。
备注	非阻塞接口，用于查询导航任务状态。调用后返回 Status 状态码与 NavStatus 对象，可用于检测导航进度及状态。

21.1.20 subscribe_odometry

项目	内容
方法名	subscribe_odometry
方法声明	Status subscribe_odometry(callback: Callable[[Odometry], None])
功能概述	订阅机器人里程计数据。
参数说明	callback: 里程计数据回调函数。
返回值	Status::OK 表示成功，其他为失败。
备注	非阻塞接口，用于获取实时里程计数据。

21.1.21 unsubscribe_odometry

项目	内容
方法名	unsubscribe_odometry
方法声明	void unsubscribe_odometry()
功能概述	取消订阅机器人里程计数据。
返回值	无
备注	非阻塞接口，用于取消订阅里程计数据。

21.1.22 get_point_cloud_map

项目	内容
方法名	get_point_cloud_map
方法声明	tuple[Status, PointCloud2] get_point_cloud_map(int timeout_ms)
功能概述	获取当前构建的点云地图。
参数说明	timeout_ms: 操作超时（毫秒），默认 5000。
返回值	返回 Status 状态码与 PointCloud2 对象（失败时为默认空对象）。
备注	阻塞接口，若在超时内未完成则返回错误状态。

21.2 枚举类型定义

21.2.1 NavMode — 导航模式枚举类型

枚举值	数值	描述
IDLE	0	空闲模式
GRID_MAP	1	栅格地图导航模式

21.2.2 NavStatusType —导航状态类型枚举

枚举值	数值	描述
NONE	0	无状态
RUNNING	1	运行中
END_SUCCESS	2	结束成功
END_FAILED	3	结束失败
PAUSE	4	暂停
CONTINUE	5	继续
CANCEL	6	取消

21.3 数据结构定义

21.3.1 Pose3DEuler —3D 位姿结构

字段名	类型	描述
position	list[double]	位置 (x, y, z)，用于表示空间位置
orientation	list[double]	欧拉角 (roll, pitch, yaw)，用于表示空间姿态，避免欧拉角万向锁问题

21.3.2 NavTarget —全局导航目标点结构

字段名	类型	描述
id	int	目标点 ID
frame_id	str	目标点坐标系 ID
goal	Pose3DEuler	目标点位姿

21.3.3 NavStatus —导航状态结构

字段名	类型	描述
id	int	目标点 ID，-1 表示无目标点
status	NavStatusType	导航状态
message	str	导航状态消息

21.3.4 MapImageData —建图图像数据结构，.pgm 格式

字段名	类型	描述
type	str	魔数，"P5"：二进制格式
width	int	图像宽度
height	int	图像高度
max_gray_value	int	最大灰度值，255
image	list[int]	图像数据

21.3.5 MapMetaData —建图地图元数据结构

字段名	类型	描述
resolution	double	地图分辨率，单位：m/pixel
origin	Pose3DEuler	地图原点，世界坐标系原点相对于地图左下角的位置
map_image_data	MapImageData	图像数据，.pgm 格式图像数据

21.3.6 MapInfo —单个地图信息结构

字段名	类型	描述
map_name	str	地图名称
map_meta_data	MapMetaData	地图元数据

21.3.7 AllMapInfo —所有地图信息结构

字段名	类型	描述
current_map_name	str	当前地图名称
map_infos	list[MapInfo]	所有地图信息

21.3.8 LocalizationInfo —当前位置信息结构

字段名	类型	描述
is_localization	bool	是否已定位
pose	Pose3DEuler	欧拉角位姿

21.4 SLAM 导航控制介绍

机器人的 SLAM 导航控制服务可分为 SLAM 功能和导航功能。

- SLAM 功能中，可调用相应的接口，实现建图、定位、地图管理等功能。
- 导航功能中，可调用相应的接口，实现目标导航、导航控制等功能。

21.4.1 适用场景与范围

- 小于 20m * 20m 且特征丰富的静态室内平地场景，请勿超过建议范围

21.4.2 SLAM 功能流程

功能	操作流程
建图	开始建图 → 保存地图
地图管理	加载地图、获取地图信息、删除地图、获取地图绝对路径

21.4.3 导航功能流程

功能	操作流程
定位	加载地图 → 切换到定位模式 → 初始化位姿 → 获取定位状态
导航	定位成功 → 激活导航模式 → 设置目标位姿 (开始导航任务) → 获取导航状态
导航控制	设置目标姿态 (开始导航任务) → 暂停导航 → 恢复导航 → 取消导航

提供机器人系统显示服务控制器，通过 DisplayController 可以使用 RPC 方式实现对机器人显示的指令控制和状态获取。

22.1 接口定义

DisplayController 是显示控制功能，主要用于显示控制等场景。

22.1.1 DisplayController —显示控制器

项目	内容
类名	DisplayController
功能概述	机器人显示控制
主要功能	机器人显示控制
使用场景	机器人显示控制

22.1.2 initialize

项目	内容
方法名	initialize
方法声明	bool initialize()
功能概述	初始化显示控制器。
返回值	true 表示成功，false 表示失败。
备注	首次使用前必须调用。

22.1.3 shutdown

项目	内容
方法名	shutdown
方法声明	void shutdown()
功能概述	关闭显示控制器。
备注	配合 initialize 使用。

22.1.4 get_all_face_expressions

项目	内容
方法名	get_all_face_expressions
方法声明	Status get_all_face_expressions(FaceExpressionVector data, int timeout_ms);
功能概述	获取所有面部表情。
参数说明	data: 所有面部表情。timeout_ms: 超时时间（毫秒），默认值为 5000。
返回值	Status::OK 表示成功，其他为失败状态。
备注	阻塞接口，获取所有面部表情。

22.1.5 set_face_expression

项目	内容
方法名	set_face_expression
方法声明	Status set_face_expression(int expression_id, int timeout_ms);
功能概述	设置面部表情。
参数说明	expression_id: 表情 ID timeout_ms: 超时时间（毫秒），默认值为 5000。
返回值	Status::OK 表示成功，其他为失败状态。
备注	阻塞接口，设置面部表情。

22.1.6 get_current_face_expression

项目	内容
方法名	GetFaceExpression
方法声明	Status GetFaceExpression(FaceExpression data, int timeout_ms);
功能概述	获取当前面部表情。
参数说明	data: 当前面部表情。timeout_ms: 超时时间（毫秒），默认值为 5000。
返回值	Status::OK 表示成功，其他为失败状态。
备注	阻塞接口，获取当前面部表情。

22.2 数据结构定义

22.2.1 FaceExpression —机器人面部表情结构

描述机器人面部表情的完整信息，支持唯一 ID、名称和描述。

字段名	类型	描述
id	int	机器人面部表情 ID
name	str	机器人面部表情名称
description	str	机器人面部表情描述

22.3 注意事项

在使用 Subscribe 等订阅类 SDK 接口时，请避免在回调函数中执行阻塞式的数据处理操作。否则，可能导致消息堆积、处理延迟，甚至引发不可预期的错误。

高层运动控制示例

该示例展示了如何使用 MagicDog_W SDK 进行初始化、连接机器人、高层运动控制（步态、特技、遥控）等基本操作。

23.1 C++

示例文件：high_level_motion_example.cpp

```
#include "magic_robot.h"
#include "magic_sdk_version.h"

#include <termios.h>
#include <unistd.h>
#include <atomic>
#include <chrono>
#include <csignal>
#include <cstdlib>
#include <iostream>
#include <memory>
#include <thread>
#include <math.h>

using namespace magic::dog_w;
```

(下页继续)

(续上页)

```
// Global variables
std::unique_ptr<MagicRobot> robot = nullptr;
std::atomic<bool> running(true);

void signalHandler(int signum) {
    std::cout << "\nInterrupt signal (" << signum << ") received." << std::endl;
    running = false;
    if (robot) {
        robot->Shutdown();
    }
    exit(signum);
}

void print_help() {
    std::cout << "Key function description:" << std::endl;
    std::cout << " " << std::endl;
    std::cout << "Gait and Trick Functions:" << std::endl;
    std::cout << " 1      Function 1: Recovery Stand" << std::endl;
    std::cout << " 3      Function 3: Execute trick - lie down" << std::endl;
    std::cout << " " << std::endl;
    std::cout << "Joystick Functions:" << std::endl;
    std::cout << " w      Move forward" << std::endl;
    std::cout << " a      Move left" << std::endl;
    std::cout << " s      Move backward" << std::endl;
    std::cout << " d      Move right" << std::endl;
    std::cout << " t      Turn left" << std::endl;
    std::cout << " g      Turn right" << std::endl;
    std::cout << " x      Stop movement" << std::endl;
    std::cout << " " << std::endl;
    std::cout << "Head Position Functions:" << std::endl;
    std::cout << " 4      Function 4: Get head position" << std::endl;
    std::cout << " 5      Function 5: Set head position" << std::endl;
    std::cout << " " << std::endl;
    std::cout << " ?      Function ?: Print help" << std::endl;
    std::cout << " ESC    Exit program" << std::endl;
}

int getch() {
    struct termios oldt, newt;
    int ch;
    tcgetattr(STDIN_FILENO, &oldt);
    newt = oldt;
    newt.c_lflag &= ~(ICANON | ECHO);
```

(下页继续)

(续上页)

```

tcsetattr(STDIN_FILENO, TCSANOW, &newt);
ch = getchar();
tcsetattr(STDIN_FILENO, TCSANOW, &oldt);
return ch;
}

// Recovery Stand
void recovery_stand() {
    try {
        auto& high_controller = robot->GetHighLevelMotionController();
        auto status = high_controller.SetGait(GaitMode::GAIT_STAND_R);
        if (status.code != ErrorCode::OK) {
            std::cerr << "Failed to set position control standing: " << status.message << std::endl;
        } else {
            std::cout << "Robot set to position control standing" << std::endl;
        }
    } catch (const std::exception& e) {
        std::cerr << "Error executing position control standing: " << e.what() << std::endl;
    }
}

// Execute trick
void execute_trick(const std::string& cmd) {
    try {
        TrickAction action;
        if (cmd == "102") {
            action = TrickAction::ACTION_LIE_DOWN;
        }

        auto& high_controller = robot->GetHighLevelMotionController();
        auto status = high_controller.ExecuteTrick(action);
        if (status.code != ErrorCode::OK) {
            std::cerr << "Failed to execute trick: " << status.message << std::endl;
        } else {
            std::cout << "Trick executed successfully" << std::endl;
        }
    } catch (const std::exception& e) {
        std::cerr << "Error executing trick: " << e.what() << std::endl;
    }
}

// Switch gait to down climb stairs mode
bool change_gait_to_down_climb_stairs() {

```

(下页继续)

(续上页)

```

try {
    GaitMode current_gait;
    auto& high_controller = robot->GetHighLevelMotionController();
    auto status = high_controller.GetGait(current_gait);
    if (status.code != ErrorCode::OK) {
        std::cerr << "Failed to get current gait: " << status.message << std::endl;
        return false;
    }

    if (current_gait != GaitMode::GAIT_RL_TERRAIN) {
        status = high_controller.SetGait(GaitMode::GAIT_RL_TERRAIN);
        if (status.code != ErrorCode::OK) {
            std::cerr << "Failed to set down climb stairs gait: " << status.message << std::endl;
            return false;
        }

        // Wait for gait switch to complete
        while (current_gait != GaitMode::GAIT_RL_TERRAIN) {
            std::this_thread::sleep_for(std::chrono::milliseconds(10));
            status = high_controller.GetGait(current_gait);
            if (status.code != ErrorCode::OK) {
                std::cerr << "Failed to get gait during transition: " << status.message << std::endl;
                return false;
            }
        }
    }

    std::cout << "Gait changed to down climb stairs" << std::endl;
    return true;
} catch (const std::exception& e) {
    std::cerr << "Error changing gait: " << e.what() << std::endl;
    return false;
}

}

void send_joystick_command(float left_x, float left_y, float right_x, float right_y) {
    if (!change_gait_to_down_climb_stairs()) {
        return;
    }

    JoystickCommand joy_command;
    joy_command.left_x_axis = left_x;
    joy_command.left_y_axis = left_y;
    joy_command.right_x_axis = right_x;

```

(下页继续)

(续上页)

```

joy_command.right_y_axis = right_y;

auto& high_controller = robot->GetHighLevelMotionController();
auto status = high_controller.SendJoyStickCommand(joy_command);
if (status.code != ErrorCode::OK) {
    std::cerr << "Failed to send joystick command: " << status.message << std::endl;
}
}

void get_current_head_position() {
    auto& high_controller = robot->GetHighLevelMotionController();

    EulerAngles euler_angles;
    auto status = high_controller.GetHeadPosition(euler_angles);
    if (status.code != ErrorCode::OK) {
        std::cerr << "get current head position failed, code: " << status.code
            << ", message: " << status.message << std::endl;
        return;
    }

    std::cout << "Head Position Roll Angle: " << euler_angles.roll << std::endl;
    std::cout << "Head Position Pitch Angle: " << euler_angles.pitch << std::endl;
    std::cout << "Head Position Yaw Angle: " << euler_angles.yaw << std::endl;
}

void set_head_position(const EulerAngles& euler_angles) {
    auto& high_controller = robot->GetHighLevelMotionController();

    auto status = high_controller.SetHeadPosition(euler_angles);
    if (status.code != ErrorCode::OK) {
        std::cerr << "set head position failed, code: " << status.code
            << ", message: " << status.message << std::endl;
        return;
    }

    std::cout << "set head position success" << std::endl;
}

// Angle to radian function
constexpr double deg2rad(double deg) {
    return deg * M_PI / 180.0;
}

int main(int argc, char* argv[]) {

```

(下页继续)

(续上页)

```
print_help();

std::cout << "MagicDog_W SDK C++ Example Program" << std::endl;

robot = std::make_unique<MagicRobot>();
if (!robot->Initialize("192.168.55.10")) {
    std::cerr << "Initialization failed" << std::endl;
    return -1;
}

auto status = robot->Connect();
if (status.code != ErrorCode::OK) {
    std::cerr << "Connection failed, code: " << status.code
                << ", message: " << status.message << std::endl;
    robot->Shutdown();
    return -1;
}

// Set motion control level to high level
status = robot->SetMotionControlLevel(ControllerLevel::HighLevel);
if (status.code != ErrorCode::OK) {
    std::cerr << "Failed to set motion control level: " << status.message << std::endl;
    robot->Shutdown();
    return -1;
}

std::cout << "Program started, please use keys to control robot..." << std::endl;

while (running) {
    std::cout << "Enter command: ";
    int key = getch();
    if (key == 27) { // ESC key
        break;
    }

    if (key == '3') {
        std::string str_input;
        std::cout << "Enter parameters: ";
        std::getline(std::cin, str_input);
        if (str_input.empty()) {
            continue;
        }
    }
}
```

(下页继续)

(续上页)

```
// Parse parameters
std::string cmd = str_input.substr(0, str_input.find(' '));
if (cmd.empty()) {
    cmd = "102";
}
std::cout << "Execute trick: " << cmd << std::endl;
execute_trick(cmd);
continue;
}

switch (key) {
    case '1':
        recovery_stand();
        break;
    case 'w':
        send_joystick_command(0.0, 0.5, 0.0, 0.0);
        break;
    case 's':
        send_joystick_command(0.0, -0.5, 0.0, 0.0);
        break;
    case 'a':
        send_joystick_command(-0.3, 0.0, 0.0, 0.0);
        break;
    case 'd':
        send_joystick_command(0.3, 0.0, 0.0, 0.0);
        break;
    case 't':
        send_joystick_command(0.0, 0.0, -0.5, 0.0);
        break;
    case 'g':
        send_joystick_command(0.0, 0.0, 0.5, 0.0);
        break;
    case 'x':
        send_joystick_command(0.0, 0.0, 0.0, 0.0);
        break;
    case '4':
        get_current_head_position();
        break;
    case '5':{
        EulerAngles angles;
        double roll_deg, pitch_deg, yaw_deg;
        std::cout << "Please enter three angles (roll pitch yaw, separated by spaces, -60_
↪degrees to 60 degrees): ";
```

(下页继续)

(续上页)

```

std::cin >> roll_deg >> pitch_deg >> yaw_deg;
// Convert to radian
angles.roll = deg2rad(roll_deg);
angles.pitch = deg2rad(pitch_deg);
angles.yaw = deg2rad(yaw_deg);
// Output result
std::cout << "\nConversion result:" << std::endl;
std::cout << "Roll:  " << roll_deg << "° = " << angles.roll << " rad" << std::endl;
std::cout << "Pitch:  " << pitch_deg << "° = " << angles.pitch << " rad" << std::endl;
std::cout << "Yaw:   " << yaw_deg << "° = " << angles.yaw << " rad" << std::endl;

set_head_position(angles);
std::cin.ignore();
}
break;
case '?':
    print_help();
    break;
default:
    std::cout << "Unknown key: " << key << std::endl;
    break;
}
}

// Disconnect from robot
status = robot->Disconnect();
if (status.code != ErrorCode::OK) {
    std::cerr << "Disconnect robot failed, code: " << status.code
               << ", message: " << status.message << std::endl;
} else {
    std::cout << "Robot disconnected" << std::endl;
}

// Shutdown robot
robot->Shutdown();
std::cout << "Robot shutdown" << std::endl;

return 0;
}

```


23.2 Python

示例文件: high_level_motion_example.py

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import sys
import time
import threading
import tty
import termios
import os
import logging
from typing import Optional
import math
import magicdog_w_python as magicdog_w

logging.basicConfig(
    level=logging.INFO, # Minimum log level
    format="%(asctime)s [%(levelname)s] %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
)

running = True
robot = None
high_controller = None

# Get single character input (no echo)
def getch():
    fd = sys.stdin.fileno()
    old_settings = termios.tcgetattr(fd)
    try:
        tty.setraw(sys.stdin.fileno())
        ch = sys.stdin.read(1)
        logging.info(f"Received character: {ch}")

        sys.stdout.write("\r")
        sys.stdout.flush()
    finally:
        termios.tcsetattr(fd, termios.TCSADRAIN, old_settings)
    return ch
```

(下页继续)

(续上页)

```
# Recovery stand
def recovery_stand():
    global high_controller
    try:
        # Set gait to position control standing
        status = high_controller.set_gait(magicdog_w.GaitMode.GAIT_STAND_R)
        if status.code != magicdog_w.ErrorCode.OK:
            logging.error(f"Failed to set position control standing: {status.message}")
        else:
            logging.info("Robot set to position control standing")
    except Exception as e:
        logging.error(f"Error executing position control standing: {e}")

# Execute trick
def execute_trick(cmd):
    global high_controller
    try:
        action = get_action(cmd)
        # Execute lie down trick
        status = high_controller.execute_trick(action)
        if status.code != magicdog_w.ErrorCode.OK:
            logging.error(f"Failed to execute trick: {status.message}")
        else:
            logging.info("Trick executed successfully")
    except Exception as e:
        logging.error(f"Error executing trick: {e}")

# Switch gait to down climb stairs mode
def change_gait_to_down_climb_stairs():
    global high_controller
    try:
        current_gait = high_controller.get_gait()
        if current_gait != magicdog_w.GaitMode.GAIT_RL_TERRAIN:
            status = high_controller.set_gait(magicdog_w.GaitMode.GAIT_RL_TERRAIN)
            if status.code != magicdog_w.ErrorCode.OK:
                logging.error(f"Failed to set down climb stairs gait: {status.message}")
                return False

        # Wait for gait switch to complete
        while high_controller.get_gait() != magicdog_w.GaitMode.GAIT_RL_TERRAIN:
            time.sleep(0.01)
```

(下页继续)

(续上页)

```

        logging.info("Gait changed to down climb stairs")
        return True
    except Exception as e:
        logging.error(f"Error changing gait: {e}")
        return False

def print_help():
    logging.info("Key function description:")
    logging.info("")
    logging.info("Gait and Trick Functions:")
    logging.info(" 1      Function 1: Recovery stand")
    logging.info(" 3      Function 3: Execute trick")
    logging.info("")
    logging.info("Joystick Functions:")
    logging.info(" w      Function w: Move forward")
    logging.info(" a      Function a: Move left")
    logging.info(" s      Function s: Move backward")
    logging.info(" d      Function d: Move right")
    logging.info(" t      Function t: Turn left")
    logging.info(" g      Function g: Turn right")
    logging.info(" x      Function x: Stop movement")
    logging.info("")
    logging.info("Head Position Functions:")
    logging.info(" 4      Function 4: Get head position")
    logging.info(" 5      Function 5: Set head position")
    logging.info("")
    logging.info(" ?      Function ?: Print help")
    logging.info(" ESC    Exit program")

def send_joystick_command(high_controller, left_x, left_y, right_x, right_y):
    if not change_gait_to_down_climb_stairs():
        return

    joy_command = magicdog_w.JoystickCommand()
    joy_command.left_x_axis = left_x
    joy_command.left_y_axis = left_y
    joy_command.right_x_axis = right_x
    joy_command.right_y_axis = right_y

    status = high_controller.send_joystick_command(joy_command)
    if status.code != magicdog_w.ErrorCode.OK:
        logging.error(f"Failed to send joystick command: {status.message}")

```

(下页继续)

(续上页)

```
def get_current_head_position(high_controller):
    status, euler_angles = high_controller.get_current_head_position()
    if status.code != magicdog_w.ErrorCode.OK:
        logging.error(
            "Failed to get current head position, code: %s, message: %s",
            status.code,
            status.message,
        )
        return

    logging.info("Head Position Roll Angle: %f", euler_angles.roll)
    logging.info(" Pitch Angle: %f", euler_angles.pitch)
    logging.info(" Yaw Angle: %f", euler_angles.yaw)

def set_head_position(high_controller, euler_angles):
    status = high_controller.set_head_position(euler_angles)
    if status.code != magicdog_w.ErrorCode.OK:
        logging.error(
            "Failed to set head position, code: %s, message: %s",
            status.code,
            status.message,
        )
        return

def deg2rad(deg):
    """Angle to radian"""
    return deg * math.pi / 180.0

def signal_handler(signum, frame):
    """Signal handler function for graceful exit"""
    global robot, running
    running = False
    logging.info("Received interrupt signal (%s), exiting...", signum)
    if robot:
        robot.shutdown()
        logging.info("Robot shutdown")
    exit(-1)
```

(下页继续)

(续上页)

```
def get_action(cmd):
    if cmd == "102":
        return magicdog_w.TrickAction.ACTION_LIE_DOWN
    else:
        return magicdog_w.TrickAction.ACTION_NONE

def main():
    """Main function"""
    print_help()

    global robot, high_controller, running

    logging.info("MagicDog_W SDK Python Example Program")

    robot = magicdog_w.MagicRobot()
    if not robot.initialize("192.168.55.10"):
        logging.error("Initialization failed")
        return

    if not robot.connect():
        logging.error("Connection failed")
        robot.shutdown()
        return

    # Set motion control level to high level
    status = robot.set_motion_control_level(magicdog_w.ControllerLevel.HIGH_LEVEL)
    if status.code != magicdog_w.ErrorCode.OK:
        logging.error(f"Failed to set motion control level: {status.message}")
        robot.shutdown()
        return

    # Get high level motion controller
    high_controller = robot.get_high_level_motion_controller()

    logging.info("Program started, please use keys to control robot...")

    while running:
        logging.info("Enter command: ")
        key = getch()
        if key == "\x1b": # ESC key
            break
```

(下页继续)

(续上页)

```
# 1. Gait and Trick Functions
# 1.1 Recovery stand
if key == "1":
    recovery_stand()
# 1.3 Execute trick
elif key == "3":
    str_input = input("Enter parameters: ").strip()
    if not str_input:
        continue
    # Split input parameters by space
    parts = str_input.strip().split()

    # Parse parameters
    cmd = parts[0] if parts else "102"
    logging.info(f"Execute trick: {cmd}")
    execute_trick(cmd)
# 2. Joystick Functions
# 2.1 Move forward
elif key.lower() == "w":
    left_y = 0.5
    left_x = 0.0
    right_x = 0.0
    right_y = 0.0
    send_joystick_command(high_controller, left_x, left_y, right_x, right_y)
# 2.2 Move backward
elif key.lower() == "s":
    left_y = -0.5
    left_x = 0.0
    right_x = 0.0
    right_y = 0.0
    send_joystick_command(high_controller, left_x, left_y, right_x, right_y)
# 2.3 Move left
elif key.lower() == "a":
    left_x = -0.3
    left_y = 0.0
    right_x = 0.0
    right_y = 0.0
    send_joystick_command(high_controller, left_x, left_y, right_x, right_y)
# 2.4 Move right
elif key.lower() == "d":
    left_x = 0.3
    left_y = 0.0
```

(下页继续)

(续上页)

```

        right_x = 0.0
        right_y = 0.0
        send_joystick_command(high_controller, left_x, left_y, right_x, right_y)
# 2.5 Turn left
    elif key.lower() == "t":
        left_x = 0.0
        left_y = 0.0
        right_x = -0.5
        right_y = 0.0
        send_joystick_command(high_controller, left_x, left_y, right_x, right_y)
# 2.6 Turn right
    elif key.lower() == "g":
        left_x = 0.0
        left_y = 0.0
        right_x = 0.5
        right_y = 0.0
        send_joystick_command(high_controller, left_x, left_y, right_x, right_y)
# 2.7 Stop movement
    elif key.lower() == "x":
        left_x = 0.0
        left_y = 0.0
        right_x = 0.0
        right_y = 0.0
        send_joystick_command(high_controller, left_x, left_y, right_x, right_y)
# 3. Head Position Functions
# 3.1 Get current head position
    elif key == "4":
        get_current_head_position(high_controller)
# 3.2 Set head position
    elif key == "5":
        # Processing head position setting command
        try:
            # Get user input
            user_input = input(
                "Please enter three angles (roll pitch yaw, separated by spaces, -60_
↪degrees to 60 degrees): "
            )
            values = user_input.split()
            if len(values) != 3:
                logging.error("Error: Three numbers are required.")
            return
            # Convert to floating point number
            roll_deg = float(values[0])

```

(下页继续)

(续上页)

```

pitch_deg = float(values[1])
yaw_deg = float(values[2])
# Verification angle range
if (
    not (-60 <= roll_deg <= 60)
    or not (-60 <= pitch_deg <= 60)
    or not (-60 <= yaw_deg <= 60)
):
    logging.error("Warning: The angle should be in the range of -60 to 60_
↪degrees.")

# Convert to radian
euler_angles = magicrobot_w.EulerAngles()
euler_angles.roll = deg2rad(roll_deg)
euler_angles.pitch = deg2rad(pitch_deg)
euler_angles.yaw = deg2rad(yaw_deg)
# Output result
logging.info("\nConversion result:")
logging.info(f"Roll: {roll_deg}° = {euler_angles.roll:.6f} rad")
logging.info(f"Pitch: {pitch_deg}° = {euler_angles.pitch:.6f} rad")
logging.info(f"Yaw: {yaw_deg}° = {euler_angles.yaw:.6f} rad")
# Set the head position
set_head_position(high_controller, euler_angles)
except ValueError:
    logging.error("Error: Please enter a valid number.")
except Exception as e:
    logging.error(f"An error occurred:{e}")

# Help
elif key.upper() == "?":
    print_help()
else:
    logging.info(f"Unknown key: {key}")

# Shutdown high level motion controller
high_controller.shutdown()
logging.info("High level motion controller shutdown")

# Disconnect from robot
robot.disconnect()
logging.info("Robot disconnected")

# Shutdown robot
robot.shutdown()
logging.info("Robot shutdown")

```

(下页继续)

(续上页)

```
if __name__ == "__main__":  
    main()
```

23.3 运行说明

23.3.1 环境准备

```
export PYTHONPATH=/opt/magic_robotics/magicdog_w_sdk/lib:$PYTHONPATH  
export LD_LIBRARY_PATH=/opt/magic_robotics/magicdog_w_sdk/lib:$LD_LIBRARY_PATH
```

23.3.2 运行示例

```
# C++  
./high_level_motion_example  
# Python  
python3 high_level_motion_example.py
```

23.3.3 控制说明

- 步态和特技控制:
 - 1: 恢复站立姿态
 - 2: 力控站立
 - 3: 执行特技动作 - 趴下
- 遥控控制:
 - w: 前进
 - s: 后退
 - a: 左移
 - d: 右移
 - t: 左转
 - g: 右转
 - x: 停止移动
- 头部控制:
 - 4: 获取头部位置

- 5: 设置头部位置
- 其他功能:
 - ?: 打印帮助信息

23.3.4 停止程序

- 按 ESC 可以安全停止程序
- 程序会自动清理所有资源

底层运动控制示例

该示例展示了如何使用 MagicDog_W SDK 进行初始化、连接机器人、控制机器人重复站立、蹲下等基本操作。

24.1 C++

示例文件：low_level_motion_example.cpp

```
#include "magic_robot.h"
#include "magic_sdk_version.h"
#include "magic_type.h"

#include <unistd.h>
#include <csignal>
#include <iostream>
#include <memory>
#include <mutex>
#include <chrono>
#include <thread>
#include <cstdlib>

using namespace magic::dog_w;

// Global robot instance
```

(下页继续)

(续上页)

```
std::unique_ptr<MagicRobot> robot = nullptr;

void signalHandler(int signum) {
    std::cout << "Interrupt signal (" << signum << ") received." << std::endl;
    if (robot) {
        robot->Shutdown();
    }
    exit(signum);
}

int main() {
    // Bind SIGINT (Ctrl+C)
    signal(SIGINT, signalHandler);

    std::cout << "MagicDog_W SDK C++ Example Program" << std::endl;

    robot = std::make_unique<MagicRobot>();
    if (!robot->Initialize("192.168.55.10")) {
        std::cerr << "Initialization failed" << std::endl;
        return -1;
    }

    auto status = robot->Connect();
    if (status.code != ErrorCode::OK) {
        std::cerr << "Connection failed, code: " << status.code
            << ", message: " << status.message << std::endl;
        robot->Shutdown();
        return -1;
    }

    std::cout << "Setting motion control level to high level" << std::endl;
    status = robot->SetMotionControlLevel(ControllerLevel::HighLevel);
    if (status.code != ErrorCode::OK) {
        std::cerr << "Failed to set motion control level: " << status.message << std::endl;
        robot->Shutdown();
        return -1;
    }

    std::cout << "Getting high level motion controller" << std::endl;
    auto& high_controller = robot->GetHighLevelMotionController();

    std::cout << "Setting motion mode to passive" << std::endl;
    status = high_controller.SetGait(GaitMode::GAIT_PASSIVE);
```

(下页继续)

(续上页)

```

    if (status.code != ErrorCode::OK) {
        std::cerr << "Failed to set motion mode: " << status.message << std::endl;
        robot->Shutdown();
        return -1;
    }

    std::cout << "Waiting for motion mode to change to passive" << std::endl;
    GaitMode current_mode = GaitMode::GAIT_NONE;
    while (current_mode != GaitMode::GAIT_PASSIVE) {
        status = high_controller.GetGait(current_mode);
        if (status.code != ErrorCode::OK) {
            std::cerr << "Failed to get gait: " << status.message << std::endl;
            robot->Shutdown();
            return -1;
        }
        std::this_thread::sleep_for(std::chrono::milliseconds(100));
    }

    std::this_thread::sleep_for(std::chrono::seconds(2));

    std::cout << "Setting motion control level to low level" << std::endl;
    status = robot->SetMotionControlLevel(ControllerLevel::LowLevel);
    if (status.code != ErrorCode::OK) {
        std::cerr << "Failed to set motion control level: " << status.message << std::endl;
        robot->Shutdown();
        return -1;
    }

    std::cout << "Waiting for motion mode to change to low level" << std::endl;
    while (current_mode != GaitMode::GAIT_LOWLEVL_SDK) {
        status = high_controller.GetGait(current_mode);
        if (status.code != ErrorCode::OK) {
            std::cerr << "Failed to get gait: " << status.message << std::endl;
            robot->Shutdown();
            return -1;
        }
        std::this_thread::sleep_for(std::chrono::milliseconds(100));
    }

    std::this_thread::sleep_for(std::chrono::seconds(2));

    std::cout << "Getting low level motion controller" << std::endl;
    auto& low_controller = robot->GetLowLevelMotionController();

```

(下页继续)

(续上页)

```

bool is_had_receive_leg_state = false;
std::mutex mut;
LegState receive_state;
int count = 0;

auto leg_state_callback = [&](const std::shared_ptr<LegState> msg) {
    if (!is_had_receive_leg_state) {
        std::lock_guard<std::mutex> guard(mut);
        is_had_receive_leg_state = true;
        receive_state = *msg;
    }
    if (count % 1000 == 0) {
        std::cout << "Received leg state data." << std::endl;
    }
    count++;
};

low_controller.SubscribeLegState(leg_state_callback);

std::cout << "Waiting to receive leg state data" << std::endl;
while (!is_had_receive_leg_state) {
    std::this_thread::sleep_for(std::chrono::milliseconds(2));
}

std::this_thread::sleep_for(std::chrono::seconds(10));

// Joint angles for different poses
double j1[] = {0.0000, 1.0477, -2.0944}; // base height 0.2
double j2[] = {0.0000, 0.7231, -1.4455}; // base height 0.3

// Get initial joint positions
double initial_q[12] = {
    receive_state.state[0].q, receive_state.state[1].q, receive_state.state[2].q,
    receive_state.state[3].q, receive_state.state[4].q, receive_state.state[5].q,
    receive_state.state[6].q, receive_state.state[7].q, receive_state.state[8].q,
    receive_state.state[9].q, receive_state.state[10].q, receive_state.state[11].q
};

LegJointCommand command;
int cnt = 0;

std::cout << "Starting joint control loop..." << std::endl;

```

(下页继续)

(续上页)

```

// 计算第一个周期的时间点
auto interval = std::chrono::milliseconds(2);
auto next_cycle = std::chrono::steady_clock::now() + interval;
while (true) {
    double t;

    if (cnt < 1000) {
        t = 1.0 * cnt / 1000.0;
        t = std::min(std::max(t, 0.0), 1.0);
        for (int i = 0; i < 12; ++i) {
            command.cmd[i].q_des = (1 - t) * initial_q[i] + t * j1[i % 3];
        }
    } else if (cnt < 1750) {
        t = 1.0 * (cnt - 1000) / 700.0;
        t = std::min(std::max(t, 0.0), 1.0);
        for (int i = 0; i < 12; ++i) {
            command.cmd[i].q_des = (1 - t) * j1[i % 3] + t * j2[i % 3];
        }
    } else if (cnt < 2500) {
        t = 1.0 * (cnt - 1750) / 700.0;
        t = std::min(std::max(t, 0.0), 1.0);
        for (int i = 0; i < 12; ++i) {
            command.cmd[i].q_des = (1 - t) * j2[i % 3] + t * j1[i % 3];
        }
    } else {
        cnt = 1000;
    }

    // Set control gains
    for (int i = 0; i < 12; ++i) {
        command.cmd[i].kp = (i % 3 == 0) ? 40 : 100;
        command.cmd[i].kd = 1.2;
    }

    low_controller.PublishLegCommand(command);
    // 精确休眠到下一个周期
    std::this_thread::sleep_until(next_cycle);
    // 更新下一个周期的时间点
    next_cycle += interval;
    cnt++;
}

```

(下页继续)

(续上页)

```
// Disconnect from robot
status = robot->Disconnect();
if (status.code != ErrorCode::OK) {
    std::cerr << "Disconnect robot failed, code: " << status.code
                << ", message: " << status.message << std::endl;
} else {
    std::cout << "Robot disconnected" << std::endl;
}

robot->Shutdown();
std::cout << "Robot shutdown" << std::endl;

std::cout << "\nExample program execution completed!" << std::endl;

return 0;
}
```

24.2 Python

示例文件: low_level_motion_example.py

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import sys
import time
import threading
import logging
from typing import Optional
import magicdog_w_python as magicdog_w

logging.basicConfig(
    level=logging.INFO, # Minimum log level
    format="%(asctime)s [%(levelname)s] %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
)

def main():
    """Main function"""
    logging.info("MagicDog_W SDK Python Example Program")
```

(下页继续)

(续上页)

```

robot = magicdog_w.MagicRobot()
if not robot.initialize("192.168.55.10"):
    logging.error("Initialization failed")
    return

if not robot.connect():
    logging.error("Connection failed")
    robot.shutdown()
    return

logging.info("Setting motion control level to high level")
status = robot.set_motion_control_level(magicdog_w.ControllerLevel.HIGH_LEVEL)
if status.code != magicdog_w.ErrorCode.OK:
    logging.error(f"Failed to set motion control level: {status.message}")
    robot.shutdown()
    return

logging.info("Getting high level motion controller")
high_controller = robot.get_high_level_motion_controller()

logging.info("Setting motion mode to passive")
status = high_controller.set_gait(magicdog_w.GaitMode.GAIT_PASSIVE)
if status.code != magicdog_w.ErrorCode.OK:
    logging.error(f"Failed to set motion mode: {status.message}")
    robot.shutdown()
    return

logging.info("Waiting for motion mode to change to passive")
current_mode = magicdog_w.GaitMode.GAIT_NONE
while current_mode != magicdog_w.GaitMode.GAIT_PASSIVE:
    current_mode = high_controller.get_gait()
    time.sleep(0.1)

time.sleep(2)

logging.info("Setting motion control level to low level")
status = robot.set_motion_control_level(magicdog_w.ControllerLevel.LOW_LEVEL)
if status.code != magicdog_w.ErrorCode.OK:
    logging.error(f"Failed to set motion control level: {status.message}")
    robot.shutdown()
    return

logging.info("Waiting for motion mode to change to low level")

```

(下页继续)

(续上页)

```

while current_mode != magicdog_w.GaitMode.GAIT_LOWLEVEL_SDK:
    current_mode = high_controller.get_gait()
    time.sleep(0.1)

time.sleep(2)

logging.info("Getting low level motion controller")
low_controller = robot.get_low_level_motion_controller()

is_had_receive_leg_state = False
mut = threading.Lock()
receive_state = None
count = 0

def leg_state_callback(msg):
    nonlocal is_had_receive_leg_state, receive_state, count
    if not is_had_receive_leg_state:
        with mut:
            is_had_receive_leg_state = True
            receive_state = msg
    if count % 1000 == 0:
        logging.info("Received leg state data.")
        count += 1

low_controller.subscribe_leg_state(leg_state_callback)

logging.info("Waiting to receive leg state data")
while not is_had_receive_leg_state:
    time.sleep(0.002)

time.sleep(10)

j1 = [0.0000, 1.0477, -2.0944]
j2 = [0.0000, 0.7231, -1.4455]
inital_q = [
    receive_state.state[0].q,
    receive_state.state[1].q,
    receive_state.state[2].q,
    receive_state.state[3].q,
    receive_state.state[4].q,
    receive_state.state[5].q,
    receive_state.state[6].q,
    receive_state.state[7].q,

```

(下页继续)

(续上页)

```

        receive_state.state[8].q,
        receive_state.state[9].q,
        receive_state.state[10].q,
        receive_state.state[11].q,
    ]

    command = magicdog_w.LegJointCommand()
    cnt = 0
    interval = 0.002 # 2ms控制周期
    next_t = time.perf_counter() + interval
    while True:
        if cnt < 1000:
            t = 1.0 * cnt / 1000.0
            t = min(max(t, 0.0), 1.0)
            for i in range(12):
                command.cmd[i].q_des = (1 - t) * initial_q[i] + t * j1[i % 3]
        elif cnt < 1750:
            t = 1.0 * (cnt - 1000) / 700.0
            t = min(max(t, 0.0), 1.0)
            for i in range(12):
                command.cmd[i].q_des = (1 - t) * j1[i % 3] + t * j2[i % 3]
        elif cnt < 2500:
            t = 1.0 * (cnt - 1750) / 700.0
            t = min(max(t, 0.0), 1.0)
            for i in range(12):
                command.cmd[i].q_des = (1 - t) * j2[i % 3] + t * j1[i % 3]
        else:
            cnt = 1000

        for i in range(12):
            command.cmd[i].kp = 40 if i % 3 == 0 else 100
            command.cmd[i].kd = 1.2

        low_controller.publish_leg_command(command)
        # 等待到下一个执行时间点
        next_t += interval
        sleep_time = next_t - time.perf_counter()
        if sleep_time > 0:
            time.sleep(sleep_time)
        cnt += 1

    robot.disconnect()
    robot.shutdown()

```

(下页继续)

(续上页)

```
logging.info("\nExample program execution completed!")

if __name__ == "__main__":
    main()
```

24.3 运行说明

24.3.1 环境准备

```
export PYTHONPATH=/opt/magic_robotics/magicdog_w_sdk/lib:$PYTHONPATH
export LD_LIBRARY_PATH=/opt/magic_robotics/magicdog_w_sdk/lib:$LD_LIBRARY_PATH
```

24.3.2 运行示例

```
# C++
./low_level_motion_example

# Python
python3 low_level_motion_example.py
```

24.3.3 停止程序

- 按 Ctrl+C 可以安全停止程序
- 程序会自动清理所有资源

传感器控制示例

该示例展示了如何使用 MagicDog_W SDK 进行初始化、连接机器人、机器人传感器控制（雷达、RGBD 相机、双目相机）等基本操作。

25.1 C++

示例文件：sensor_example.cpp

```
#include "magic_robot.h"
#include "magic_sdk_version.h"
#include "magic_type.h"

#include <unistd.h>
#include <csignal>
#include <iostream>
#include <memory>
#include <chrono>
#include <thread>
#include <map>
#include <string>
#include <cstdlib>

using namespace magic::dog_w;
```

(下页继续)

(续上页)

```
// Global robot instance
std::unique_ptr<MagicRobot> robot = nullptr;

// Sensor manager class
class SensorManager {
public:
    SensorManager(SensorController& controller) : controller_(controller) {}

    // Channel Control
    bool open_channel() {
        if (channel_opened_) {
            std::cout << "Channel already opened" << std::endl;
            return true;
        }

        auto status = robot->OpenChannelSwitch();
        if (status.code != ErrorCode::OK) {
            std::cerr << "Failed to open channel: " << status.message << std::endl;
            return false;
        }

        channel_opened_ = true;
        std::cout << "✓ Channel opened successfully" << std::endl;
        return true;
    }

    bool close_channel() {
        if (!channel_opened_) {
            std::cout << "Channel already closed" << std::endl;
            return true;
        }

        auto status = robot->CloseChannelSwitch();
        if (status.code != ErrorCode::OK) {
            std::cerr << "Failed to close channel: " << status.message << std::endl;
            return false;
        }

        channel_opened_ = false;
        std::cout << "✓ Channel closed successfully" << std::endl;
        return true;
    }
}
```

(下页继续)

(续上页)

```
// Sensor Open/Close
bool open_laser_scan() {
    if (sensors_state_["laser_scan"]) {
        std::cout << "Laser scan already opened" << std::endl;
        return true;
    }

    auto status = controller_.OpenLaserScan();
    if (status.code != ErrorCode::OK) {
        std::cerr << "Failed to open laser scan: " << status.message << std::endl;
        return false;
    }

    sensors_state_["laser_scan"] = true;
    std::cout << "✓ Laser scan opened" << std::endl;
    return true;
}

bool close_laser_scan() {
    if (!sensors_state_["laser_scan"]) {
        std::cout << "Laser scan already closed" << std::endl;
        return true;
    }

    // Unsubscribe if subscribed
    if (subscriptions_["laser_scan"]) {
        toggle_laser_scan_subscription();
    }

    auto status = controller_.CloseLaserScan();
    if (status.code != ErrorCode::OK) {
        std::cerr << "Failed to close laser scan: " << status.message << std::endl;
        return false;
    }

    sensors_state_["laser_scan"] = false;
    std::cout << "✓ Laser scan closed" << std::endl;
    return true;
}

bool open_rgbd_camera() {
    if (sensors_state_["rgbd_camera"]) {
        std::cout << "RGBD camera already opened" << std::endl;
    }
}
```

(下页继续)

(续上页)

```

        return true;
    }

    auto status = controller_.OpenRgbdCamera();
    if (status.code != ErrorCode::OK) {
        std::cerr << "Failed to open RGBD camera: " << status.message << std::endl;
        return false;
    }

    sensors_state_["rgbd_camera"] = true;
    std::cout << "✓ RGBD camera opened" << std::endl;
    return true;
}

bool close_rgbd_camera() {
    if (!sensors_state_["rgbd_camera"]) {
        std::cout << "RGBD camera already closed" << std::endl;
        return true;
    }

    // Unsubscribe all RGBD subscriptions if subscribed
    if (subscriptions_["rgbd_color_info"]) {
        toggle_rgbd_color_info_subscription();
    }
    if (subscriptions_["rgbd_depth_image"]) {
        toggle_rgbd_depth_image_subscription();
    }
    if (subscriptions_["rgbd_color_image"]) {
        toggle_rgbd_color_image_subscription();
    }
    if (subscriptions_["rgb_depth_info"]) {
        toggle_rgb_depth_info_subscription();
    }

    auto status = controller_.CloseRgbdCamera();
    if (status.code != ErrorCode::OK) {
        std::cerr << "Failed to close RGBD camera: " << status.message << std::endl;
        return false;
    }

    sensors_state_["rgbd_camera"] = false;
    std::cout << "✓ RGBD camera closed" << std::endl;
    return true;
}

```

(下页继续)

(续上页)

```

}

bool open_binocular_camera() {
    if (sensors_state_["binocular_camera"]) {
        std::cout << "Binocular camera already opened" << std::endl;
        return true;
    }

    auto status = controller_.OpenBinocularCamera();
    if (status.code != ErrorCode::OK) {
        std::cerr << "Failed to open binocular camera: " << status.message << std::endl;
        return false;
    }

    sensors_state_["binocular_camera"] = true;
    std::cout << "✓ Binocular camera opened" << std::endl;
    return true;
}

bool close_binocular_camera() {
    if (!sensors_state_["binocular_camera"]) {
        std::cout << "Binocular camera already closed" << std::endl;
        return true;
    }

    // Unsubscribe all binocular subscriptions if subscribed
    if (subscriptions_["left_binocular_high"]) {
        toggle_left_binocular_high_subscription();
    }
    if (subscriptions_["left_binocular_low"]) {
        toggle_left_binocular_low_subscription();
    }
    if (subscriptions_["right_binocular_low"]) {
        toggle_right_binocular_low_subscription();
    }

    auto status = controller_.CloseBinocularCamera();
    if (status.code != ErrorCode::OK) {
        std::cerr << "Failed to close binocular camera: " << status.message << std::endl;
        return false;
    }

    sensors_state_["binocular_camera"] = false;
}

```

(下页继续)

(续上页)

```

std::cout << "✓ Binocular camera closed" << std::endl;
return true;
}

// Subscribe/Unsubscribe Methods
void toggle_ultra_subscription() {
    if (subscriptions_["ultra"]) {
        controller_.UnsubscribeUltra();
        subscriptions_["ultra"] = false;
        std::cout << "✓ Ultra sensor unsubscribed" << std::endl;
    } else {
        controller_.SubscribeUltra([](const std::shared_ptr<Float32MultiArray> ultra) {
            std::cout << "Ultra: " << ultra->data.size() << std::endl;
        });
        subscriptions_["ultra"] = true;
        std::cout << "✓ Ultra sensor subscribed" << std::endl;
    }
}

void toggle_head_touch_subscription() {
    if (subscriptions_["head_touch"]) {
        controller_.UnsubscribeHeadTouch();
        subscriptions_["head_touch"] = false;
        std::cout << "✓ Head touch sensor unsubscribed" << std::endl;
    } else {
        controller_.SubscribeHeadTouch([](const std::shared_ptr<HeadTouch> touch) {
            std::cout << "Head Touch: " << int(touch->data) << std::endl;
        });
        subscriptions_["head_touch"] = true;
        std::cout << "✓ Head touch sensor subscribed" << std::endl;
    }
}

void toggle_imu_subscription() {
    if (subscriptions_["imu"]) {
        controller_.UnsubscribeImu();
        subscriptions_["imu"] = false;
        std::cout << "✓ IMU sensor unsubscribed" << std::endl;
    } else {
        static int imu_counter = 0;
        controller_.SubscribeImu([&](const std::shared_ptr<Imu> imu) {
            imu_counter++;
            if (imu_counter % 500 == 0) {

```

(下页继续)

(续上页)

```

        std::cout << "IMU: " << imu->temperature << std::endl;
    }
});
subscriptions_["imu"] = true;
std::cout << "✓ IMU sensor subscribed" << std::endl;
}
}

void toggle_laser_scan_subscription() {
    if (subscriptions_["laser_scan"]) {
        controller_.UnsubscribeLaserScan();
        subscriptions_["laser_scan"] = false;
        std::cout << "✓ Laser scan unsubscribed" << std::endl;
    } else {
        controller_.SubscribeLaserScan([](const std::shared_ptr<LaserScan> scan) {
            std::cout << "Laser Scan: " << scan->ranges.size() << " ranges" << std::endl;
        });
        subscriptions_["laser_scan"] = true;
        std::cout << "✓ Laser scan subscribed" << std::endl;
    }
}

void toggle_rgbd_color_info_subscription() {
    if (subscriptions_["rgbd_color_info"]) {
        controller_.UnsubscribeRgbdColorCameraInfo();
        subscriptions_["rgbd_color_info"] = false;
        std::cout << "✓ RGBD color camera info unsubscribed" << std::endl;
    } else {
        controller_.SubscribeRgbdColorCameraInfo([](const std::shared_ptr<CameraInfo> info)
↪{
            std::cout << "RGBD Color Info: K=" << info->K[0] << ", " << info->K[1] << ", " <<
↪info->K[2] << ", " << info->K[3] << ", " << info->K[4] << ", " << info->K[5] << ", " <<
↪info->K[6] << ", " << info->K[7] << ", " << info->K[8] << std::endl;
        });
        subscriptions_["rgbd_color_info"] = true;
        std::cout << "✓ RGBD color camera info subscribed" << std::endl;
    }
}

void toggle_rgbd_depth_image_subscription() {
    if (subscriptions_["rgbd_depth_image"]) {
        controller_.UnsubscribeRgbdDepthImage();
        subscriptions_["rgbd_depth_image"] = false;
    }
}

```

(下页继续)

(续上页)

```

        std::cout << "✓ RGBD depth image unsubscribed" << std::endl;
    } else {
        controller_.SubscribeRgbdDepthImage([](const std::shared_ptr<Image> img) {
            std::cout << "RGBD Depth Image: " << img->data.size() << " bytes" << std::endl;
        });
        subscriptions_["rgbd_depth_image"] = true;
        std::cout << "✓ RGBD depth image subscribed" << std::endl;
    }
}

void toggle_rgbd_color_image_subscription() {
    if (subscriptions_["rgbd_color_image"]) {
        controller_.UnsubscribeRgbdColorImage();
        subscriptions_["rgbd_color_image"] = false;
        std::cout << "✓ RGBD color image unsubscribed" << std::endl;
    } else {
        controller_.SubscribeRgbdColorImage([](const std::shared_ptr<Image> img) {
            std::cout << "RGBD Color Image: " << img->data.size() << " bytes" << std::endl;
        });
        subscriptions_["rgbd_color_image"] = true;
        std::cout << "✓ RGBD color image subscribed" << std::endl;
    }
}

void toggle_rgb_depth_info_subscription() {
    if (subscriptions_["rgb_depth_info"]) {
        controller_.UnsubscribeRgbDepthCameraInfo();
        subscriptions_["rgb_depth_info"] = false;
        std::cout << "✓ RGB depth camera info unsubscribed" << std::endl;
    } else {
        controller_.SubscribeRgbDepthCameraInfo([](const std::shared_ptr<CameraInfo> info) {
            std::cout << "RGB Depth Info: K=" << info->K[0] << ", " << info->K[1] << ", " <<
            info->K[2] << ", " << info->K[3] << ", " << info->K[4] << ", " << info->K[5] << ", " <<
            info->K[6] << ", " << info->K[7] << ", " << info->K[8] << std::endl;
        });
        subscriptions_["rgb_depth_info"] = true;
        std::cout << "✓ RGB depth camera info subscribed" << std::endl;
    }
}

void toggle_left_binocular_high_subscription() {
    if (subscriptions_["left_binocular_high"]) {
        controller_.UnsubscribeLeftBinocularHighImg();
    }
}

```

(下页继续)

(续上页)

```

        subscriptions_["left_binocular_high"] = false;
        std::cout << "✓ Left binocular high image unsubscribed" << std::endl;
    } else {
        controller_.SubscribeLeftBinocularHighImg([](const std::shared_ptr<CompressedImage>&
↵img) {
            std::cout << "Left Binocular High: " << img->data.size() << " bytes, format: " <
↵< img->format << std::endl;
            });
        subscriptions_["left_binocular_high"] = true;
        std::cout << "✓ Left binocular high image subscribed" << std::endl;
    }
}

void toggle_left_binocular_low_subscription() {
    if (subscriptions_["left_binocular_low"]) {
        controller_.UnsubscribeLeftBinocularLowImg();
        subscriptions_["left_binocular_low"] = false;
        std::cout << "✓ Left binocular low image unsubscribed" << std::endl;
    } else {
        controller_.SubscribeLeftBinocularLowImg([](const std::shared_ptr<CompressedImage>&
↵img) {
            std::cout << "Left Binocular Low: " << img->data.size() << " bytes, format: " <
↵< img->format << std::endl;
            });
        subscriptions_["left_binocular_low"] = true;
        std::cout << "✓ Left binocular low image subscribed" << std::endl;
    }
}

void toggle_right_binocular_low_subscription() {
    if (subscriptions_["right_binocular_low"]) {
        controller_.UnsubscribeRightBinocularLowImg();
        subscriptions_["right_binocular_low"] = false;
        std::cout << "✓ Right binocular low image unsubscribed" << std::endl;
    } else {
        controller_.SubscribeRightBinocularLowImg([](const std::shared_ptr<CompressedImage>&
↵img) {
            std::cout << "Right Binocular Low: " << img->data.size() << " bytes, format: " <
↵< img->format << std::endl;
            });
        subscriptions_["right_binocular_low"] = true;
        std::cout << "✓ Right binocular low image subscribed" << std::endl;
    }
}

```

(下页继续)

(续上页)

```

}

void toggle_depth_image_subscription() {
    if (subscriptions_["depth_image"]) {
        controller_.UnsubscribeDepthImage();
        subscriptions_["depth_image"] = false;
        std::cout << "✓ Depth image unsubscribed" << std::endl;
    } else {
        controller_.SubscribeDepthImage([](const std::shared_ptr<Image> img) {
            std::cout << "Depth Image: " << img->data.size() << " bytes" << std::endl;
        });
        subscriptions_["depth_image"] = true;
        std::cout << "✓ Depth image subscribed" << std::endl;
    }
}

void show_status() {
    std::cout << "\n" << std::string(70, '=') << std::endl;
    std::cout << "SENSOR STATUS" << std::endl;
    std::cout << std::string(70, '=') << std::endl;
    std::cout << "Channel Switch:                " << (channel_opened_ ? "OPEN" : "CLOSED")
↪<< std::endl;
    std::cout << "Laser Scan:                " << (sensors_state_["laser_scan"] ? "OPEN
↪" : "CLOSED") << std::endl;
    std::cout << "RGBD Camera:                " << (sensors_state_["rgb_d_camera"] ? "OPEN
↪" : "CLOSED") << std::endl;
    std::cout << "Binocular Camera:          " << (sensors_state_["binocular_camera"] ?
↪"OPEN" : "CLOSED") << std::endl;
    std::cout << "\nSUBSCRIPTIONS:" << std::endl;
    std::cout << "  Ultra:                " << (subscriptions_["ultra"] ? "✓_
↪SUBSCRIBED" : "❏ UNSUBSCRIBED") << std::endl;
    std::cout << "  Head Touch:           " << (subscriptions_["head_touch"] ? "✓_
↪SUBSCRIBED" : "❏ UNSUBSCRIBED") << std::endl;
    std::cout << "  IMU:                  " << (subscriptions_["imu"] ? "✓ SUBSCRIBED
↪" : "❏ UNSUBSCRIBED") << std::endl;
    std::cout << "  Laser Scan:           " << (subscriptions_["laser_scan"] ? "✓_
↪SUBSCRIBED" : "❏ UNSUBSCRIBED") << std::endl;
    std::cout << "  RGBD Color Info:       " << (subscriptions_["rgb_d_color_info"] ?
↪"✓ SUBSCRIBED" : "❏ UNSUBSCRIBED") << std::endl;
    std::cout << "  RGBD Depth Image:     " << (subscriptions_["rgb_d_depth_image"] ?
↪"✓ SUBSCRIBED" : "❏ UNSUBSCRIBED") << std::endl;
    std::cout << "  RGBD Color Image:     " << (subscriptions_["rgb_d_color_image"] ?
↪"✓ SUBSCRIBED" : "❏ UNSUBSCRIBED") << std::endl;

```

(下页继续)

(续上页)

```

        std::cout << "   RGB Depth Info:           " << (subscriptions_["rgb_depth_info"] ?
↪ "✓ SUBSCRIBED" : "❑ UNSUBSCRIBED") << std::endl;

        std::cout << "   Left Binocular High:       " << (subscriptions_["left_binocular_high
↪ "] ? "✓ SUBSCRIBED" : "❑ UNSUBSCRIBED") << std::endl;

        std::cout << "   Left Binocular Low:         " << (subscriptions_["left_binocular_low"]_
↪ ? "✓ SUBSCRIBED" : "❑ UNSUBSCRIBED") << std::endl;

        std::cout << "   Right Binocular Low:        " << (subscriptions_["right_binocular_low
↪ "] ? "✓ SUBSCRIBED" : "❑ UNSUBSCRIBED") << std::endl;

        std::cout << "   Depth Image:               " << (subscriptions_["depth_image"] ? "✓_
↪ SUBSCRIBED" : "❑ UNSUBSCRIBED") << std::endl;

        std::cout << std::string(70, '=') << "\n" << std::endl;
    }

private:
    SensorController& controller_;
    bool channel_opened_ = false;
    std::map<std::string, bool> sensors_state_ = {
        {"laser_scan", false},
        {"rgb_camera", false},
        {"binocular_camera", false}
    };

    std::map<std::string, bool> subscriptions_ = {
        {"ultra", false},
        {"head_touch", false},
        {"imu", false},
        {"laser_scan", false},
        {"rgb_color_info", false},
        {"rgb_depth_image", false},
        {"rgb_color_image", false},
        {"rgb_depth_info", false},
        {"left_binocular_high", false},
        {"left_binocular_low", false},
        {"right_binocular_low", false},
        {"depth_image", false}
    };
};

void print_menu() {
    std::cout << "\n" << std::string(70, '=') << std::endl;
    std::cout << "SENSOR CONTROL MENU" << std::endl;
    std::cout << std::string(70, '=') << std::endl;
    std::cout << "Channel Control:" << std::endl;
    std::cout << "   1 - Open Channel Switch           2 - Close Channel Switch" << std::endl;

```

(下页继续)

(续上页)

```

std::cout << "\nSensor Control:" << std::endl;
std::cout << "  3 - Open Laser Scan          4 - Close Laser Scan" << std::endl;
std::cout << "  5 - Open RGBD Camera          6 - Close RGBD Camera" << std::endl;
std::cout << "  7 - Open Binocular Camera      8 - Close Binocular Camera" << std::endl;
std::cout << "\nSubscription Toggle (lowercase=toggle, UPPERCASE=unsubscribe):" <<
↪std::endl;
std::cout << "  u/U - Ultra" << std::endl;
std::cout << "  h/H - Head Touch          i/I - IMU" << std::endl;
std::cout << "  l/L - Laser Scan Data" << std::endl;
std::cout << "\nRGBD Subscriptions (lowercase=toggle, UPPERCASE=unsubscribe):" << std::endl;
std::cout << "  r/R - RGBD Color Info          d/D - RGBD Depth Image" << std::endl;
std::cout << "  c/C - RGBD Color Image          p/P - RGB Depth Info" << std::endl;
std::cout << "\nBinocular Subscriptions:" << std::endl;
std::cout << "  b/B - Left Binocular High      n/N - Left Binocular Low" << std::endl;
std::cout << "  m/M - Right Binocular Low" << std::endl;
std::cout << "\nOther Subscriptions:" << std::endl;
std::cout << "  e/E - Depth Image" << std::endl;
std::cout << "\nCommands:" << std::endl;
std::cout << "  s - Show Status          ESC - Quit          ? - Help" <<
↪std::endl;
std::cout << std::string(70, '=') << std::endl;
}

void signalHandler(int signal) {
std::cout << "\nInterrupt signal (" << signal << ") received." << std::endl;
if (robot) {
robot->Shutdown();
}
exit(signal);
}

int main() {
// Bind SIGINT (Ctrl+C)
signal(SIGINT, signalHandler);

std::cout << "\n" << std::string(70, '=') << std::endl;
std::cout << "MagicDog_W SDK Sensor Interactive Example" << std::endl;
std::cout << std::string(70, '=') << "\n" << std::endl;

robot = std::make_unique<MagicRobot>();
if (!robot->Initialize("192.168.55.10")) {
std::cerr << "Robot initialization failed" << std::endl;
return -1;
}

```

(下页继续)

(续上页)

```

}

auto status = robot->Connect();
if (status.code != ErrorCode::OK) {
    std::cerr << "Robot connection failed, code: " << status.code
                << ", message: " << status.message << std::endl;
    robot->Shutdown();
    return -1;
}

std::cout << "✓ Robot connected successfully\n" << std::endl;

SensorManager sensor_manager(robot->GetSensorController());

print_menu();

try {
    while (true) {
        std::cout << "\nEnter your choice: ";
        std::string choice;
        std::getline(std::cin, choice);

        if (choice.empty()) {
            continue;
        }

        char ch = choice[0];

        if (ch == 27) { // ESC key
            std::cout << "ESC key pressed, exiting program..." << std::endl;
            break;
        }

        // Channel control
        if (choice == "1") {
            sensor_manager.open_channel();
        } else if (choice == "2") {
            sensor_manager.close_channel();
        }

        // Sensor control
        else if (choice == "3") {
            sensor_manager.open_laser_scan();
        } else if (choice == "4") {

```

(下页继续)

(续上页)

```

        sensor_manager.close_laser_scan();
    } else if (choice == "5") {
        sensor_manager.open_rgbd_camera();
    } else if (choice == "6") {
        sensor_manager.close_rgbd_camera();
    } else if (choice == "7") {
        sensor_manager.open_binocular_camera();
    } else if (choice == "8") {
        sensor_manager.close_binocular_camera();
    }
    // Basic sensor subscriptions
    else if (ch == 'u') { // ~20HZ
        sensor_manager.toggle_ultra_subscription();
    } else if (ch == 'U') { // ~20HZ
        sensor_manager.toggle_ultra_subscription();
    } else if (ch == 'h') { // trigger
        sensor_manager.toggle_head_touch_subscription();
    } else if (ch == 'H') { // trigger
        sensor_manager.toggle_head_touch_subscription();
    } else if (ch == 'i') { // ~500HZ
        sensor_manager.toggle_imu_subscription();
    } else if (ch == 'I') { // ~500HZ
        sensor_manager.toggle_imu_subscription();
    } else if (ch == 'l') {
        sensor_manager.toggle_laser_scan_subscription();
    } else if (ch == 'L') {
        sensor_manager.toggle_laser_scan_subscription();
    }
    // RGBD subscriptions
    else if (ch == 'r') {
        sensor_manager.toggle_rgbd_color_info_subscription();
    } else if (ch == 'R') {
        sensor_manager.toggle_rgbd_color_info_subscription();
    } else if (ch == 'd') {
        sensor_manager.toggle_rgbd_depth_image_subscription();
    } else if (ch == 'D') {
        sensor_manager.toggle_rgbd_depth_image_subscription();
    } else if (ch == 'c') {
        sensor_manager.toggle_rgbd_color_image_subscription();
    } else if (ch == 'C') {
        sensor_manager.toggle_rgbd_color_image_subscription();
    } else if (ch == 'p') {
        sensor_manager.toggle_rgb_depth_info_subscription();
    }

```

(下页继续)

(续上页)

```

    } else if (ch == 'P') {
        sensor_manager.toggle_rgb_depth_info_subscription();
    }
    // Binocular subscriptions
    else if (ch == 'b') {
        sensor_manager.toggle_left_binocular_high_subscription();
    } else if (ch == 'B') {
        sensor_manager.toggle_left_binocular_high_subscription();
    } else if (ch == 'n') {
        sensor_manager.toggle_left_binocular_low_subscription();
    } else if (ch == 'N') {
        sensor_manager.toggle_left_binocular_low_subscription();
    } else if (ch == 'm') {
        sensor_manager.toggle_right_binocular_low_subscription();
    } else if (ch == 'M') {
        sensor_manager.toggle_right_binocular_low_subscription();
    } else if (ch == 'e') {
        sensor_manager.toggle_depth_image_subscription();
    } else if (ch == 'E') {
        sensor_manager.toggle_depth_image_subscription();
    }
    // Commands
    else if (ch == 's') {
        sensor_manager.show_status();
    } else if (choice == "?" || choice == "help") {
        print_menu();
    } else {
        std::cout << "Invalid choice: '" << choice << "'. Press '?' for help." <<
↵std::endl;
    }
}

} catch (const std::exception& e) {
    std::cerr << "Error occurred: " << e.what() << std::endl;
}

// Cleanup: unsubscribe all and close all sensors
std::cout << "Cleaning up..." << std::endl;

status = robot->Disconnect();
if (status.code != ErrorCode::OK) {
    std::cerr << "robot disconnect failed, code: " << status.code
        << ", message: " << status.message << std::endl;
} else {

```

(下页继续)

(续上页)

```
std::cout << "robot disconnect" << std::endl;
}

robot->Shutdown();
std::cout << "robot shutdown" << std::endl;

return 0;
}
```

25.2 Python

示例文件: sensor_example.py

示例文档: Python API/sensor_reference.md

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import sys
import time
import logging
from typing import Optional, Dict
import magicrobot_w_python as magicrobot_w
from magicrobot_w_python import ErrorCode

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
)

imu_counter = 0

class SensorManager:
    """Manages sensor subscriptions and channels"""

    def __init__(self, robot, sensor_controller):
        self.robot = robot
        self.sensor_controller = sensor_controller
        self.channel_opened = False
        self.sensors_state = {
```

(下页继续)

(续上页)

```

        "laser_scan": False,
        "rgbd_camera": False,
        "binocular_camera": False,
    }

    self.subscriptions = {
        "ultra": False,
        "head_touch": False,
        "imu": False,
        "laser_scan": False,
        "rgbd_color_info": False,
        "rgbd_depth_image": False,
        "rgbd_color_image": False,
        "rgb_depth_info": False,
        "left_binocular_high": False,
        "left_binocular_low": False,
        "right_binocular_low": False,
        "depth_image": False,
    }

# === Channel Control ===
def open_channel(self) -> bool:
    """Open sensor channel switch"""
    if self.channel_opened:
        logging.warning("Channel already opened")
        return True

    status = self.robot.open_channel_switch()
    if status.code != ErrorCode.OK:
        logging.error(f"Failed to open channel: {status.message}")
        return False

    self.channel_opened = True
    logging.info("✓ Channel opened successfully")
    return True

def close_channel(self) -> bool:
    """Close sensor channel switch"""
    if not self.channel_opened:
        logging.warning("Channel already closed")
        return True

    status = self.robot.close_channel_switch()
    if status.code != ErrorCode.OK:

```

(下页继续)

(续上页)

```

        logging.error(f"Failed to close channel: {status.message}")
        return False

    self.channel_opened = False
    logging.info("✓ Channel closed successfully")
    return True

# === Sensor Open/Close ===
def open_laser_scan(self) -> bool:
    """Open laser scan sensor"""
    if self.sensors_state["laser_scan"]:
        logging.warning("Laser scan already opened")
        return True

    status = self.sensor_controller.open_laser_scan()
    if status.code != ErrorCode.OK:
        logging.error(f"Failed to open laser scan: {status.message}")
        return False

    self.sensors_state["laser_scan"] = True
    logging.info("✓ Laser scan opened")
    return True

def close_laser_scan(self) -> bool:
    """Close laser scan sensor"""
    if not self.sensors_state["laser_scan"]:
        logging.warning("Laser scan already closed")
        return True

    # Unsubscribe if subscribed
    if self.subscriptions["laser_scan"]:
        self.toggle_laser_scan_subscription()

    status = self.sensor_controller.close_laser_scan()
    if status.code != ErrorCode.OK:
        logging.error(f"Failed to close laser scan: {status.message}")
        return False

    self.sensors_state["laser_scan"] = False
    logging.info("✓ Laser scan closed")
    return True

def open_rgbd_camera(self) -> bool:

```

(下页继续)

(续上页)

```

"""Open RGBD camera"""
if self.sensors_state["rgb_d_camera"]:
    logging.warning("RGBD camera already opened")
    return True

status = self.sensor_controller.open_rgb_d_camera()
if status.code != ErrorCode.OK:
    logging.error(f"Failed to open RGBD camera: {status.message}")
    return False

self.sensors_state["rgb_d_camera"] = True
logging.info("✓ RGBD camera opened")
return True

def close_rgb_d_camera(self) -> bool:
    """Close RGBD camera"""
    if not self.sensors_state["rgb_d_camera"]:
        logging.warning("RGBD camera already closed")
        return True

    # Unsubscribe all RGBD subscriptions if subscribed
    if self.subscriptions["rgb_d_color_info"]:
        self.toggle_rgb_d_color_info_subscription()
    if self.subscriptions["rgb_d_depth_image"]:
        self.toggle_rgb_d_depth_image_subscription()
    if self.subscriptions["rgb_d_color_image"]:
        self.toggle_rgb_d_color_image_subscription()
    if self.subscriptions["rgb_depth_info"]:
        self.toggle_rgb_depth_info_subscription()

    status = self.sensor_controller.close_rgb_d_camera()
    if status.code != ErrorCode.OK:
        logging.error(f"Failed to close RGBD camera: {status.message}")
        return False

    self.sensors_state["rgb_d_camera"] = False
    logging.info("✓ RGBD camera closed")
    return True

def open_binocular_camera(self) -> bool:
    """Open binocular camera"""
    if self.sensors_state["binocular_camera"]:
        logging.warning("Binocular camera already opened")

```

(下页继续)

(续上页)

```

        return True

    status = self.sensor_controller.open_binocular_camera()
    if status.code != ErrorCode.OK:
        logging.error(f"Failed to open binocular camera: {status.message}")
        return False

    self.sensors_state["binocular_camera"] = True
    logging.info("✓ Binocular camera opened")
    return True

def close_binocular_camera(self) -> bool:
    """Close binocular camera"""
    if not self.sensors_state["binocular_camera"]:
        logging.warning("Binocular camera already closed")
        return True

    # Unsubscribe all binocular subscriptions if subscribed
    if self.subscriptions["left_binocular_high"]:
        self.toggle_left_binocular_high_subscription()
    if self.subscriptions["left_binocular_low"]:
        self.toggle_left_binocular_low_subscription()
    if self.subscriptions["right_binocular_low"]:
        self.toggle_right_binocular_low_subscription()

    status = self.sensor_controller.close_binocular_camera()
    if status.code != ErrorCode.OK:
        logging.error(f"Failed to close binocular camera: {status.message}")
        return False

    self.sensors_state["binocular_camera"] = False
    logging.info("✓ Binocular camera closed")
    return True

# === Subscribe/Unsubscribe Methods ===
def toggle_ultra_subscription(self):
    """Toggle ultra sensor subscription"""
    if self.subscriptions["ultra"]:
        self.sensor_controller.unsubscribe_ultra()
        self.subscriptions["ultra"] = False
        logging.info("✓ Ultra sensor unsubscribed")
    else:
        self.sensor_controller.subscribe_ultra(

```

(下页继续)

(续上页)

```

        lambda ultra: logging.info(f"Ultra: {len(ultra.data)}")
    )
    self.subscriptions["ultra"] = True
    logging.info("✓ Ultra sensor subscribed")

def toggle_head_touch_subscription(self):
    """Toggle head touch sensor subscription"""
    if self.subscriptions["head_touch"]:
        self.sensor_controller.unsubscribe_head_touch()
        self.subscriptions["head_touch"] = False
        logging.info("✓ Head touch sensor unsubscribed")
    else:
        self.sensor_controller.subscribe_head_touch(
            lambda touch: logging.info(f"Head Touch: {touch}")
        )
        self.subscriptions["head_touch"] = True
        logging.info("✓ Head touch sensor subscribed")

def toggle_imu_subscription(self):
    """Toggle IMU sensor subscription"""
    if self.subscriptions["imu"]:
        self.sensor_controller.unsubscribe_imu()
        self.subscriptions["imu"] = False
        logging.info("✓ IMU sensor unsubscribed")
    else:
        def imu_callback(imu):
            global imu_counter
            imu_counter += 1
            if imu_counter % 500 == 0:
                logging.info(f"IMU: {imu}")

        self.sensor_controller.subscribe_imu(imu_callback)
        self.subscriptions["imu"] = True
        logging.info("✓ IMU sensor subscribed")

def toggle_laser_scan_subscription(self):
    """Toggle laser scan subscription"""
    if self.subscriptions["laser_scan"]:
        self.sensor_controller.unsubscribe_laser_scan()
        self.subscriptions["laser_scan"] = False
        logging.info("✓ Laser scan unsubscribed")
    else:

```

(下页继续)

(续上页)

```

        self.sensor_controller.subscribe_laser_scan(
            lambda scan: logging.info(f"Laser Scan: {len(scan.ranges)} ranges")
        )
        self.subscriptions["laser_scan"] = True
        logging.info("✓ Laser scan subscribed")

def toggle_rgbd_color_info_subscription(self):
    """Toggle RGBD color camera info subscription"""
    if self.subscriptions["rgbd_color_info"]:
        self.sensor_controller.unsubscribe_rgbd_color_camera_info()
        self.subscriptions["rgbd_color_info"] = False
        logging.info("✓ RGBD color camera info unsubscribed")
    else:
        self.sensor_controller.subscribe_rgbd_color_camera_info(
            lambda info: logging.info(f"RGBD Color Info: K={info.K}")
        )
        self.subscriptions["rgbd_color_info"] = True
        logging.info("✓ RGBD color camera info subscribed")

def toggle_rgbd_depth_image_subscription(self):
    """Toggle RGBD depth image subscription"""
    if self.subscriptions["rgbd_depth_image"]:
        self.sensor_controller.unsubscribe_rgbd_depth_image()
        self.subscriptions["rgbd_depth_image"] = False
        logging.info("✓ RGBD depth image unsubscribed")
    else:
        self.sensor_controller.subscribe_rgbd_depth_image(
            lambda img: logging.info(f"RGBD Depth Image: {len(img.data)} bytes")
        )
        self.subscriptions["rgbd_depth_image"] = True
        logging.info("✓ RGBD depth image subscribed")

def unsubscribe_rgbd_depth_image_subscription(self):
    """Unsubscribe RGBD depth image subscription"""
    self.sensor_controller.unsubscribe_rgbd_depth_image()
    self.subscriptions["rgbd_depth_image"] = False
    logging.info("✓ RGBD depth image unsubscribed")

def toggle_rgbd_color_image_subscription(self):
    """Toggle RGBD color image subscription"""
    if self.subscriptions["rgbd_color_image"]:
        self.sensor_controller.unsubscribe_rgbd_color_image()
        self.subscriptions["rgbd_color_image"] = False

```

(下页继续)

(续上页)

```

        logging.info("✓ RGBD color image unsubscribed")
    else:
        self.sensor_controller.subscribe_rgb_d_color_image(
            lambda img: logging.info(f"RGBD Color Image: {len(img.data)} bytes")
        )
        self.subscriptions["rgb_d_color_image"] = True
        logging.info("✓ RGBD color image subscribed")

def toggle_rgb_depth_info_subscription(self):
    """Toggle RGB depth camera info subscription"""
    if self.subscriptions["rgb_depth_info"]:
        self.sensor_controller.unsubscribe_rgb_depth_camera_info()
        self.subscriptions["rgb_depth_info"] = False
        logging.info("✓ RGB depth camera info unsubscribed")
    else:
        self.sensor_controller.subscribe_rgb_depth_camera_info(
            lambda info: logging.info(f"RGB Depth Info: K={info.K}")
        )
        self.subscriptions["rgb_depth_info"] = True
        logging.info("✓ RGB depth camera info subscribed")

def toggle_left_binocular_high_subscription(self):
    """Toggle left binocular high image subscription"""
    if self.subscriptions["left_binocular_high"]:
        self.sensor_controller.unsubscribe_left_binocular_high_img()
        self.subscriptions["left_binocular_high"] = False
        logging.info("✓ Left binocular high image unsubscribed")
    else:
        self.sensor_controller.subscribe_left_binocular_high_img(
            lambda img: logging.info(
                f"Left Binocular High: {len(img.data)} bytes, format: {img.format}"
            )
        )
        self.subscriptions["left_binocular_high"] = True
        logging.info("✓ Left binocular high image subscribed")

def toggle_left_binocular_low_subscription(self):
    """Toggle left binocular low image subscription"""
    if self.subscriptions["left_binocular_low"]:
        self.sensor_controller.unsubscribe_left_binocular_low_img()
        self.subscriptions["left_binocular_low"] = False
        logging.info("✓ Left binocular low image unsubscribed")
    else:

```

(下页继续)

(续上页)

```

        self.sensor_controller.subscribe_left_binocular_low_img(
            lambda img: logging.info(
                f"Left Binocular Low: {len(img.data)} bytes, format: {img.format}"
            )
        )
        self.subscriptions["left_binocular_low"] = True
        logging.info("✓ Left binocular low image subscribed")

def toggle_right_binocular_low_subscription(self):
    """Toggle right binocular low image subscription"""
    if self.subscriptions["right_binocular_low"]:
        self.sensor_controller.unsubscribe_right_binocular_low_img()
        self.subscriptions["right_binocular_low"] = False
        logging.info("✓ Right binocular low image unsubscribed")
    else:
        self.sensor_controller.subscribe_right_binocular_low_img(
            lambda img: logging.info(
                f"Right Binocular Low: {len(img.data)} bytes, format: {img.format}"
            )
        )
        self.subscriptions["right_binocular_low"] = True
        logging.info("✓ Right binocular low image subscribed")

def toggle_depth_image_subscription(self):
    """Toggle depth image subscription"""
    if self.subscriptions["depth_image"]:
        self.sensor_controller.unsubscribe_depth_image()
        self.subscriptions["depth_image"] = False
        logging.info("✓ Depth image unsubscribed")
    else:
        self.sensor_controller.subscribe_depth_image(
            lambda img: logging.info(f"Depth Image: {len(img.data)} bytes")
        )
        self.subscriptions["depth_image"] = True
        logging.info("✓ Depth image subscribed")

def show_status(self):
    """Display current sensor status"""
    logging.info("\n" + "=" * 70)
    logging.info("SENSOR STATUS")
    logging.info("=" * 70)
    logging.info(
        f"Channel Switch:                                {'OPEN' if self.channel_opened else 'CLOSED'}"
    )

```

(下页继续)

(续上页)

```

    )
    logging.info(
        f"Laser Scan:                                {'OPEN' if self.sensors_state['laser_scan'] else
↪ 'CLOSED'}"
    )
    logging.info(
        f"RGBD Camera:                                {'OPEN' if self.sensors_state['rgb_d_camera'] else
↪ 'CLOSED'}"
    )
    logging.info(
        f"Binocular Camera:                            {'OPEN' if self.sensors_state['binocular_camera'] else
↪ 'CLOSED'}"
    )
    logging.info("\nSUBSCRIPTIONS:")
    logging.info(
        f" Ultra:                                       {'✓ SUBSCRIBED' if self.subscriptions['ultra'] else
↪ '✗ UNSUBSCRIBED'}"
    )
    logging.info(
        f" Head Touch:                                 {'✓ SUBSCRIBED' if self.subscriptions['head_touch
↪ '] else '✗ UNSUBSCRIBED'}"
    )
    logging.info(
        f" IMU:                                          {'✓ SUBSCRIBED' if self.subscriptions['imu'] else
↪ '✗ UNSUBSCRIBED'}"
    )
    logging.info(
        f" Laser Scan:                                  {'✓ SUBSCRIBED' if self.subscriptions['laser_scan
↪ '] else '✗ UNSUBSCRIBED'}"
    )
    logging.info(
        f" RGBD Color Info:                             {'✓ SUBSCRIBED' if self.subscriptions['rgb_d_color_
↪ info'] else '✗ UNSUBSCRIBED'}"
    )
    logging.info(
        f" RGBD Depth Image:                           {'✓ SUBSCRIBED' if self.subscriptions['rgb_d_depth_
↪ image'] else '✗ UNSUBSCRIBED'}"
    )
    logging.info(
        f" RGBD Color Image:                             {'✓ SUBSCRIBED' if self.subscriptions['rgb_d_color_
↪ image'] else '✗ UNSUBSCRIBED'}"
    )
    logging.info(

```

(下页继续)

(续上页)

```

        f" RGB Depth Info:                {'✓ SUBSCRIBED' if self.subscriptions['rgb_depth_
↪info'] else '✗ UNSUBSCRIBED'}"
    )
    logging.info(
        f" Left Binocular High:           {'✓ SUBSCRIBED' if self.subscriptions['left_
↪binocular_high'] else '✗ UNSUBSCRIBED'}"
    )
    logging.info(
        f" Left Binocular Low:            {'✓ SUBSCRIBED' if self.subscriptions['left_
↪binocular_low'] else '✗ UNSUBSCRIBED'}"
    )
    logging.info(
        f" Right Binocular Low:           {'✓ SUBSCRIBED' if self.subscriptions['right_
↪binocular_low'] else '✗ UNSUBSCRIBED'}"
    )
    logging.info(
        f" Depth Image:                   {'✓ SUBSCRIBED' if self.subscriptions['depth_image
↪'] else '✗ UNSUBSCRIBED'}"
    )
    logging.info("=" * 70 + "\n")

def print_menu():
    """Print interactive menu"""
    logging.info("\n" + "=" * 70)
    logging.info("SENSOR CONTROL MENU")
    logging.info("=" * 70)
    logging.info("Channel Control:")
    logging.info("  1 - Open Channel Switch          2 - Close Channel Switch")
    logging.info("\nSensor Control:")
    logging.info("  3 - Open Laser Scan              4 - Close Laser Scan")
    logging.info("  5 - Open RGBD Camera             6 - Close RGBD Camera")
    logging.info("  7 - Open Binocular Camera        8 - Close Binocular Camera")
    logging.info("\nSubscription Toggle (lowercase=toggle, UPPERCASE=unsubscribe):")
    logging.info("  u/U - Ultra")
    logging.info("  h/H - Head Touch                i/I - IMU")
    logging.info("  l/L - Laser Scan Data")
    logging.info("\nRGBD Subscriptions (lowercase=toggle, UPPERCASE=unsubscribe):")
    logging.info("  r/R - RGBD Color Info            d/D - RGBD Depth Image")
    logging.info("  c/C - RGBD Color Image           p/P - RGB Depth Info")
    logging.info("\nBinocular Subscriptions:")
    logging.info("  b/B - Left Binocular High        n/N - Left Binocular Low")
    logging.info("  m/M - Right Binocular Low")

```

(下页继续)

(续上页)

```

logging.info("\nOther Subscriptions:")
logging.info("  e/E - Depth Image")
logging.info("\nCommands:")
logging.info("  s - Show Status          ESC - Quit          ? - Help")
logging.info("=" * 70)

def main():
    """Main function"""
    logging.info("\n" + "=" * 70)
    logging.info("MagicDog_W SDK Sensor Interactive Example")
    logging.info("=" * 70 + "\n")

    local_ip = "192.168.55.10"
    robot = magicdog_w.MagicRobot()

    if not robot.initialize(local_ip):
        logging.error("Robot initialization failed")
        return

    if not robot.connect():
        logging.error("Robot connection failed")
        robot.shutdown()
        return

    logging.info("✓ Robot connected successfully\n")

    sensor_controller = robot.get_sensor_controller()
    sensor_manager = SensorManager(robot, sensor_controller)

    print_menu()

    try:
        while True:
            choice = input("\nEnter your choice: ").strip().lower()

            if choice == "\x1b": # ESC key
                logging.info("ESC key pressed, exiting program...")
                break

            # Channel control
            if choice == "1":
                sensor_manager.open_channel()

```

(下页继续)

(续上页)

```

elif choice == "2":
    sensor_manager.close_channel()

# Sensor control
elif choice == "3":
    sensor_manager.open_laser_scan()
elif choice == "4":
    sensor_manager.close_laser_scan()
elif choice == "5":
    sensor_manager.open_rgbd_camera()
elif choice == "6":
    sensor_manager.close_rgbd_camera()
elif choice == "7":
    sensor_manager.open_binocular_camera()
elif choice == "8":
    sensor_manager.close_binocular_camera()

# Basic sensor subscriptions
elif choice.lower() == "u":
    sensor_manager.toggle_ultra_subscription()
elif choice.lower() == "h":
    sensor_manager.toggle_head_touch_subscription()
elif choice.lower() == "i":
    sensor_manager.toggle_imu_subscription()
elif choice.lower() == "l":
    sensor_manager.toggle_laser_scan_subscription()
# RGBD subscriptions
elif choice.lower() == "r":
    sensor_manager.toggle_rgbd_color_info_subscription()
elif choice.lower() == "d":
    sensor_manager.toggle_rgbd_depth_image_subscription()
elif choice.lower() == "c":
    sensor_manager.toggle_rgbd_color_image_subscription()
elif choice.lower() == "p":
    sensor_manager.toggle_rgb_depth_info_subscription()
# Binocular subscriptions
elif choice.lower() == "b":
    sensor_manager.toggle_left_binocular_high_subscription()
elif choice.lower() == "n":
    sensor_manager.toggle_left_binocular_low_subscription()
elif choice.lower() == "m":
    sensor_manager.toggle_right_binocular_low_subscription()
elif choice.lower() == "e":

```

(下页继续)

(续上页)

```

        sensor_manager.toggle_depth_image_subscription()

    # Commands
    elif choice.lower() == "s":
        sensor_manager.show_status()
    elif choice == "?" or choice == "help":
        print_menu()
    else:
        logging.warning(f"Invalid choice: '{choice}'. Press '?' for help.")

except KeyboardInterrupt:
    logging.info("\nReceived keyboard interrupt, shutting down...")
except Exception as e:
    logging.error(f"Error occurred: {e}")
    import traceback

    traceback.print_exc()
finally:
    # Cleanup: unsubscribe all and close all sensors
    logging.info("Cleaning up...")

    # Unsubscribe from all active subscriptions
    for sensor_name, is_subscribed in sensor_manager.subscriptions.items():
        if is_subscribed:
            logging.info(f"Unsubscribing from {sensor_name}...")

    if sensor_manager.sensors_state["laser_scan"]:
        sensor_manager.close_laser_scan()
    if sensor_manager.sensors_state["rgbd_camera"]:
        sensor_manager.close_rgbd_camera()
    if sensor_manager.sensors_state["binocular_camera"]:
        sensor_manager.close_binocular_camera()
    if sensor_manager.channel_opened:
        sensor_manager.close_channel()

    # Allow time for cleanup
    time.sleep(1)

    sensor_controller.shutdown()
    logging.info("sensor controller shutdown")

    robot.disconnect()
    logging.info("robot disconnect")

```

(下页继续)

(续上页)

```
robot.shutdown()
logging.info("robot shutdown")

if __name__ == "__main__":
    main()
```

25.3 运行说明

25.3.1 环境准备

```
export PYTHONPATH=/opt/magic_robotics/magicrobot_w_sdk/lib:$PYTHONPATH
export LD_LIBRARY_PATH=/opt/magic_robotics/magicrobot_w_sdk/lib:$LD_LIBRARY_PATH
```

25.3.2 运行示例

```
# C++
./sensor_example
# Python
python3 sensor_example.py
```

25.3.3 控制说明

- 通道控制：
 - 按键 1 - 打开通道开关
 - 按键 2 - 关闭通道开关
- 传感器控制：
 - 按键 3 - 打开激光扫描
 - 按键 4 - 关闭激光扫描
 - 按键 5 - 打开 RGBD 相机
 - 按键 6 - 关闭 RGBD 相机
 - 按键 7 - 打开双目相机
 - 按键 8 - 关闭双目相机
- 基本传感器订阅：

- 按键 `u/U` - 切换/取消订阅超声波传感器
- 按键 `h/H` - 切换/取消订阅头部触摸传感器
- 按键 `i/I` - 切换/取消订阅 IMU 传感器
- 按键 `l/L` - 切换/取消订阅激光扫描数据
- RGBD 相机订阅：
 - 按键 `r/R` - 切换/取消订阅 RGBD 彩色信息
 - 按键 `d/D` - 切换/取消订阅 RGBD 深度图像
 - 按键 `c/C` - 切换/取消订阅 RGBD 彩色图像
 - 按键 `p/P` - 切换/取消订阅 RGB 深度信息
- 双目相机订阅：
 - 按键 `b/B` - 切换/取消订阅左眼高分辨率图像
 - 按键 `n/N` - 切换/取消订阅左眼低分辨率图像
 - 按键 `m/M` - 切换/取消订阅右眼低分辨率图像
 - 按键 `e/E` - 切换/取消订阅深度图像
- 其他功能：
 - 按键 `s` - 显示状态
 - 按键 `?` - 显示帮助菜单

25.3.4 停止程序

- 按 `ESC` 可以安全停止程序
- 程序会自动清理所有资源

语音控制示例

该示例展示了如何使用 MagicDog_W SDK 进行初始化、连接机器人、音频控制（获取音量、设置音量、TTS 播放）等基本操作。

26.1 C++

示例文件：audio_example.cpp

```
#include "magic_robot.h"
#include "magic_sdk_version.h"

#include <termios.h>
#include <unistd.h>
#include <chrono>
#include <csignal>
#include <cstdlib>
#include <iostream>
#include <memory>
#include <thread>

using namespace magic::dog_w;

// Global robot instance
std::unique_ptr<MagicRobot> robot = nullptr;
```

(下页继续)

(续上页)

```
void signalHandler(int signum) {
    std::cout << "\nInterrupt signal (" << signum << ") received." << std::endl;
    if (robot) {
        robot->Shutdown();
    }
    exit(signum);
}

void print_help() {
    std::cout << "Key Function Demo Program\n"
                << std::endl;
    std::cout << "Key Function Description:" << std::endl;
    std::cout << "Audio Functions:" << std::endl;
    std::cout << "  1      Function 1: Get volume" << std::endl;
    std::cout << "  2      Function 2: Set volume" << std::endl;
    std::cout << "  3      Function 3: Play TTS" << std::endl;
    std::cout << "  4      Function 4: Stop playback" << std::endl;
    std::cout << "  5      Function 5: Get voice config" << std::endl;
    std::cout << "" << std::endl;
    std::cout << "Audio stream Functions:" << std::endl;
    std::cout << "  6      Function 6: Open audio stream" << std::endl;
    std::cout << "  7      Function 7: Close audio stream" << std::endl;
    std::cout << "  8      Function 8: Subscribe to audio stream" << std::endl;
    std::cout << "  9      Function 9: Unsubscribe from audio stream" << std::endl;
    std::cout << "" << std::endl;
    std::cout << "Speech IO Functions:" << std::endl;
    std::cout << "  d      Function d: Open speech io" << std::endl;
    std::cout << "  e      Function e: Close speech io" << std::endl;
    std::cout << "  f      Function f: Subscribe to speech asr" << std::endl;
    std::cout << "  g      Function g: Unsubscribe from speech asr" << std::endl;
    std::cout << "  h      Function f: Publish speech tts" << std::endl;
    std::cout << "" << std::endl;
    std::cout << "  ?      Function ?: Print help" << std::endl;
    std::cout << "  ESC    Exit program" << std::endl;
}

int getch() {
    struct termios oldt, newt;
    int ch;
    tcgetattr(STDIN_FILENO, &oldt); // Get current terminal settings
    newt = oldt;
    newt.c_lflag &= ~(ICANON | ECHO); // Disable buffering and echo
```

(下页继续)

(续上页)

```

tcsetattr(STDIN_FILENO, TCSANOW, &newt);
ch = getchar(); // Read key press
tcsetattr(STDIN_FILENO, TCSANOW, &oldt); // Restore settings
return ch;
}

void get_volume() {
    auto& audio_controller = robot->GetAudioController();

    int volume = 0;
    auto status = audio_controller.GetVolume(volume);
    if (status.code != ErrorCode::OK) {
        std::cerr << "get volume failed, code: " << status.code
                    << ", message: " << status.message << std::endl;
        return;
    }
    std::cout << "get volume success, volume: " << volume << std::endl;
}

void set_volume(int volume) {
    auto& audio_controller = robot->GetAudioController();

    auto status = audio_controller.SetVolume(volume);
    if (status.code != ErrorCode::OK) {
        std::cerr << "set volume failed, code: " << status.code
                    << ", message: " << status.message << std::endl;
        return;
    }
    std::cout << "set volume success" << std::endl;
}

void play_tts() {
    auto& audio_controller = robot->GetAudioController();

    TtsCommand tts;
    tts.id = "100000000001";
    tts.content = "How's the weather today!";
    tts.priority = TtsPriority::HIGH;
    tts.mode = TtsMode::CLEARTOP;

    auto status = audio_controller.Play(tts);
    if (status.code != ErrorCode::OK) {
        std::cerr << "play tts failed, code: " << status.code

```

(下页继续)

(续上页)

```

        << " , message: " << status.message << std::endl;

        return;
    }

    std::cout << "play tts success" << std::endl;
}

void stop_tts() {
    auto& audio_controller = robot->GetAudioController();

    auto status = audio_controller.Stop();
    if (status.code != ErrorCode::OK) {
        std::cerr << "stop tts failed, code: " << status.code
            << " , message: " << status.message << std::endl;

        return;
    }

    std::cout << "stop tts success" << std::endl;
}

void open_audio_stream() {
    auto& audio_controller = robot->GetAudioController();

    auto status = audio_controller.ControlVoiceStream(true, true);
    if (status.code != ErrorCode::OK) {
        std::cerr << "open audio stream failed, code: " << status.code
            << " , message: " << status.message << std::endl;

        return;
    }

    std::cout << "open audio stream success" << std::endl;
}

void close_audio_stream() {
    auto& audio_controller = robot->GetAudioController();

    auto status = audio_controller.ControlVoiceStream(false, false);
    if (status.code != ErrorCode::OK) {
        std::cerr << "close audio stream failed, code: " << status.code
            << " , message: " << status.message << std::endl;

        return;
    }

    std::cout << "close audio stream success" << std::endl;
}

// Global counters for audio stream callbacks

```

(下页继续)

(续上页)

```
static int origin_counter = 0;
static int bf_counter = 0;

void subscribe_audio_stream() {
    auto& audio_controller = robot->GetAudioController();

    // Subscribe to origin voice data
    audio_controller.SubscribeOriginVoiceData([](const std::shared_ptr<ByteMultiArray> data) {
        if (origin_counter % 30 == 0) {
            std::cout << "Received origin voice data, size: " << data->data.size() << std::endl;
            std::cout << "\r";
            std::cout.flush();
        }
        origin_counter++;
    });

    // Subscribe to bf voice data
    audio_controller.SubscribeBfVoiceData([](const std::shared_ptr<ByteMultiArray> data) {
        if (bf_counter % 30 == 0) {
            std::cout << "Received bf voice data, size: " << data->data.size() << std::endl;
            std::cout << "\r";
            std::cout.flush();
        }
        bf_counter++;
    });

    std::cout << "Subscribed to audio streams" << std::endl;
}

void unsubscribe_audio_stream() {
    auto& audio_controller = robot->GetAudioController();

    audio_controller.UnsubscribeOriginVoiceData();
    audio_controller.UnsubscribeBfVoiceData();
    std::cout << "Unsubscribed from audio streams" << std::endl;
}

void get_voice_config() {
    auto& audio_controller = robot->GetAudioController();

    GetSpeechConfig voice_config;
    auto status = audio_controller.GetVoiceConfig(voice_config);
    if (status.code != ErrorCode::OK) {
```

(下页继续)

(续上页)

```

        std::cerr << "get voice config failed, code: " << status.code
            << ", message: " << status.message << std::endl;

        return;
    }

    std::cout << "Get voice config success" << std::endl;
    std::cout << "TTS type: " << static_cast<int>(voice_config.tts_type) << std::endl;
    std::cout << "Speaker: " << voice_config.speaker_config.selected.speaker_id << std::endl;
    std::cout << "Bot config: " << voice_config.bot_config.selected.bot_id << std::endl;
    std::cout << "Wake word: " << voice_config.wakeup_config.name << std::endl;
    std::cout << "Dialog config - Front DOA: " << voice_config.dialog_config.is_front_doa <<
    ↪std::endl;
    std::cout << "Dialog config - Full duplex: " << voice_config.dialog_config.is_fullduplex_
    ↪enable << std::endl;
    std::cout << "Dialog config - Voice enable: " << voice_config.dialog_config.is_enable <<
    ↪std::endl;
    std::cout << "Dialog config - DOA enable: " << voice_config.dialog_config.is_doa_enable <<
    ↪std::endl;
}

void open_speech_io() {
    auto& audio_controller = robot->GetAudioController();

    auto status = audio_controller.ControlSpeechIO(true);
    if (status.code != ErrorCode::OK) {
        std::cerr << "open speech io failed, code: " << status.code
            << ", message: " << status.message << std::endl;

        return;
    }

    std::cout << "open speech io success" << std::endl;
}

void close_speech_io() {
    auto& audio_controller = robot->GetAudioController();

    auto status = audio_controller.ControlSpeechIO(false);
    if (status.code != ErrorCode::OK) {
        std::cerr << "close speech io failed, code: " << status.code
            << ", message: " << status.message << std::endl;

        return;
    }

    std::cout << "close speech io success" << std::endl;
}

```

(下页继续)

(续上页)

```

void subscribe_speech_asr() {
    auto& audio_controller = robot->GetAudioController();

    audio_controller.SubscribeSpeechASRStream([](const std::shared_ptr<SpeechASRStream> data) {
        std::cout << "Received speech asr data, id: " << data->id << std::endl;
        std::cout << "                                type: " << data->type << std::endl;
        std::cout << "                                text: " << data->text << std::endl;
        std::cout << "\r";
        std::cout.flush();
    });

    std::cout << "Subscribed to speech asr" << std::endl;
}

void unsubscribe_speech_asr() {
    auto& audio_controller = robot->GetAudioController();

    audio_controller.UnsubscribeSpeechASRStream();
    std::cout << "Unsubscribed from speech asr" << std::endl;
}

void publish_speech_tts(const SpeechTTSStream& data) {
    auto& audio_controller = robot->GetAudioController();
    // begin
    // var
    // end
    audio_controller.PublishSpeechTTSStream(data);
    std::cout << "Published to speech tts" << std::endl;
}

SpeechTTSStream get_speech_tts_input() {
    SpeechTTSStream stream;

    std::cout << "=== Speech TTS Stream Input ===" << std::endl;

    // Enter ID
    std::cout << "Enter request ID: ";
    std::getline(std::cin, stream.id);

    // Enter Type and verify
    while (true) {
        std::cout << "Enter type (begin/var/end): ";
    }
}

```

(下页继续)

(续上页)

```

std::getline(std::cin, stream.type);
if (stream.type == "begin" || stream.type == "var" || stream.type == "end") {
    break;
}
std::cout << "Invalid type! Please enter 'begin', 'var', or 'end'." << std::endl;
}

// Enter Text
std::cout << "Enter TTS text: ";
std::getline(std::cin, stream.text);

// Enter End Session
std::cout << "End session? (y/n): ";
std::string end_session_input;
std::getline(std::cin, end_session_input);
stream.end_session = (end_session_input == "y" || end_session_input == "Y");

return stream;
}

int main(int argc, char* argv[]) {
    // Bind SIGINT (Ctrl+C)
    signal(SIGINT, signalHandler);

    print_help();

    std::string local_ip = "192.168.54.111";
    robot = std::make_unique<MagicRobot>();

    // Configure local IP address for direct network connection and initialize SDK
    if (!robot->Initialize(local_ip)) {
        std::cerr << "robot sdk initialize failed." << std::endl;
        robot->Shutdown();
        return -1;
    }

    // Connect to robot
    auto status = robot->Connect();
    if (status.code != ErrorCode::OK) {
        std::cerr << "connect robot failed, code: " << status.code
            << ", message: " << status.message << std::endl;
        robot->Shutdown();
        return -1;
    }
}

```

(下页继续)

(续上页)

```

}

std::cout << "Press any key to continue (ESC to exit)..." << std::endl;

// Wait for user input
while (true) {
    int key = getch();
    if (key == 27) { // ESC key
        break;
    }

    std::cout << "Key ASCII: " << key << ", Character: " << static_cast<char>(key) << std::endl;

    switch (key) {
        case '1':
            get_volume();
            break;
        case '2': {
            int volume = 50;
            std::cout << "Please input volume: ";
            std::cin >> volume;
            set_volume(volume);
            std::cin.ignore();
        } break;
        case '3':
            play_tts();
            break;
        case '4':
            stop_tts();
            break;
        case '5':
            get_voice_config();
            break;
        case '6':
            open_audio_stream();
            break;
        case '7':
            close_audio_stream();
            break;
        case '8':
            subscribe_audio_stream();
            break;
        case '9':

```

(下页继续)

(续上页)

```

        unsubscribe_audio_stream();
        break;
    case 'd':
        open_speech_io();
        break;
    case 'e':
        close_speech_io();
        break;
    case 'f':
        subscribe_speech_asr();
        break;
    case 'g':
        unsubscribe_speech_asr();
        break;
    case 'h': {
        SpeechTTSStream data;
        data = get_speech_tts_input();
        publish_speech_tts(data);
    } break;
    case '?':
        print_help();
        break;
    default:
        std::cout << "Unknown key: " << key << std::endl;
        break;
}

// Sleep 10ms, equivalent to usleep(10000)
std::this_thread::sleep_for(std::chrono::milliseconds(10));
}

// Disconnect from robot
status = robot->Disconnect();
if (status.code != ErrorCode::OK) {
    std::cerr << "disconnect robot failed, code: " << status.code
                << ", message: " << status.message << std::endl;
    robot->Shutdown();
    return -1;
}

std::cout << "disconnect robot success" << std::endl;

robot->Shutdown();
    
```

(下页继续)

(续上页)

```
std::cout << "robot shutdown" << std::endl;

return 0;
}
```

26.2 Python

示例文件: audio_example.py

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import sys
import time
import signal
import termios
import tty
import logging
from typing import Optional
import magicdog_w_python as magicdog_w
from magicdog_w_python import TtsCommand, TtsPriority, TtsMode, GetSpeechConfig

# Configure logging
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
)

# Global robot instance
robot = None

def signal_handler(signum, frame):
    """Handle Ctrl+C signal"""
    logging.info(f"\nInterrupt signal ({signum}) received.")
    if robot:
        robot.shutdown()
    sys.exit(signum)

def print_help():
    """Print help information"""
```

(下页继续)

(续上页)

```

logging.info("Key Function Description:")
logging.info("")
logging.info("Audio Functions:")
logging.info(" 1      Function 1: Get volume")
logging.info(" 2      Function 2: Set volume")
logging.info(" 3      Function 3: Play TTS")
logging.info(" 4      Function 4: Stop playback")
logging.info(" 5      Function 5: Get voice config")
logging.info("")
logging.info("Audio stream Functions:")
logging.info(" 6      Function 6: Open audio stream")
logging.info(" 7      Function 7: Close audio stream")
logging.info(" 8      Function 8: Subscribe to audio stream")
logging.info(" 9      Function 9: Unsubscribe from audio stream")
logging.info("")
logging.info("Speech IO Functions:")
logging.info(" d      Function d: Open speech io")
logging.info(" e      Function e: Close speech io")
logging.info(" f      Function f: Subscribe to speech asr")
logging.info(" g      Function g: Unsubscribe from speech asr")
logging.info(" h      Function h: Publish speech tts")
logging.info("")
logging.info(" ?      Function ?: Print help")
logging.info(" ESC    Exit program")

def getch():
    """Get a single character from standard input"""
    fd = sys.stdin.fileno()
    old_settings = termios.tcgetattr(fd)
    try:
        tty.setraw(fd)
        ch = sys.stdin.read(1)
    finally:
        termios.tcsetattr(fd, termios.TCSADRAIN, old_settings)
    return ch

def get_volume(audio_controller):
    """Get current volume"""
    status, volume = audio_controller.get_volume()
    if status.code != magicrog_w.ErrorCode.OK:
        logging.error("get volume failed")

```

(下页继续)

(续上页)

```

        return

    logging.info(f"get volume success, volume: {volume}")

def set_volume(audio_controller):
    """Set volume"""
    status = audio_controller.set_volume(50)
    if status.code != magicdog_w.ErrorCode.OK:
        logging.error(
            f"set volume failed, code: {status.code}, message: {status.message}"
        )
    return
    logging.info("set volume success")

def play_tts(audio_controller):
    """Play TTS"""
    tts = TtsCommand()
    tts.id = "100000000001"
    tts.content = "How's the weather today!"
    tts.priority = TtsPriority.HIGH
    tts.mode = TtsMode.CLEARTOP
    status = audio_controller.play(tts)
    if status.code != magicdog_w.ErrorCode.OK:
        logging.error(
            f"play tts failed, code: {status.code}, message: {status.message}"
        )
    return
    logging.info("play tts success")

def stop_tts(audio_controller):
    """Stop TTS playback"""
    status = audio_controller.stop()
    if status.code != magicdog_w.ErrorCode.OK:
        logging.error(
            f"stop tts failed, code: {status.code}, message: {status.message}"
        )
    return
    logging.info("stop tts success")

def open_audio_stream(audio_controller):

```

(下页继续)

(续上页)

```

"""Open audio stream"""
status = audio_controller.control_voice_stream(True, True)
if status.code != magicdog_w.ErrorCode.OK:
    logging.error(
        f"open audio stream failed, code: {status.code}, message: {status.message}"
    )
    return
logging.info("open audio stream success")

def close_audio_stream(audio_controller):
    """Close audio stream"""
    status = audio_controller.control_voice_stream(False, False)
    if status.code != magicdog_w.ErrorCode.OK:
        logging.error(
            f"close audio stream failed, code: {status.code}, message: {status.message}"
        )
        return
    logging.info("close audio stream success")

# Global counter for audio stream callbacks
origin_counter = [0]
bf_counter = [0]

def subscribe_audio_stream(audio_controller):
    """Subscribe to audio streams"""

    def origin_callback(data):
        if origin_counter[0] % 30 == 0:
            logging.info(f"Received origin voice data, size: {len(data.data)}")
            sys.stdout.write("\r")
            sys.stdout.flush()
            origin_counter[0] += 1

    def bf_callback(data):
        if bf_counter[0] % 30 == 0:
            logging.info(f"Received bf voice data, size: {len(data.data)}")
            sys.stdout.write("\r")
            sys.stdout.flush()
            bf_counter[0] += 1

```

(下页继续)

(续上页)

```

audio_controller.subscribe_origin_voice_data(origin_callback)
audio_controller.subscribe_bf_voice_data(bf_callback)
logging.info("Subscribed to audio streams")

def unsubscribe_audio_stream(audio_controller):
    """Unsubscribe from audio streams"""
    audio_controller.unsubscribe_origin_voice_data()
    audio_controller.unsubscribe_bf_voice_data()
    logging.info("Unsubscribed from audio streams")

def get_voice_config(audio_controller):
    """Get voice configuration"""
    status, voice_config = audio_controller.get_voice_config()
    if status.code != magicdog_w.ErrorCode.OK:
        logging.error("get voice config failed")
        return

    logging.info("Get voice config success")
    logging.info(f"voice config tts type: {int(voice_config.tts_type)}")
    logging.info(
        f"voice config speaker config: {voice_config.speaker_config.selected.speaker_id}"
    )
    logging.info(f"voice config bot config: {voice_config.bot_config.selected.bot_id}")
    logging.info(f"voice config wakeup config: {voice_config.wakeup_config.name}")
    logging.info(
        f"voice config dialog config: {voice_config.dialog_config.is_front_doa}"
    )
    logging.info(
        f"voice config dialog config: {voice_config.dialog_config.is_full duplex_enable}"
    )
    logging.info(f"voice config dialog config: {voice_config.dialog_config.is_enable}")
    logging.info(
        f"voice config dialog config: {voice_config.dialog_config.is_doa_enable}"
    )

def open_speech_io(audio_controller):
    """Open speech IO"""
    status = audio_controller.control_speech_io(True)
    if status.code != magicdog_w.ErrorCode.OK:
        logging.error(

```

(下页继续)

(续上页)

```

        f"open speech io failed, code: {status.code}, message: {status.message}"
    )
    return
logging.info("open speech io success")

def close_speech_io(audio_controller):
    """Close speech IO"""
    status = audio_controller.control_speech_io(False)
    if status.code != magicdog_w.ErrorCode.OK:
        logging.error(
            f"close speech io failed, code: {status.code}, message: {status.message}"
        )
    return
logging.info("close speech io success")

def subscribe_speech_asr(audio_controller):
    """Subscribe to speech ASR"""

    def asr_callback(data):
        logging.info(
            f"Received speech asr, id: {data.id}, type: {data.type}, text: {data.text}"
        )
        sys.stdout.write("\r")
        sys.stdout.flush()

    audio_controller.subscribe_speech_asr(asr_callback)
    logging.info("Subscribed to speech asr")

def unsubscribe_speech_asr(audio_controller):
    """Unsubscribe from speech ASR"""
    audio_controller.unsubscribe_speech_asr()
    logging.info("Unsubscribed from speech ASR")

def get_speech_tts_input() -> magicdog_w.SpeechTTSStream:
    """Obtaining TTS stream data from user input"""
    print("=== Speech TTS Stream Input ===")

    # Enter ID
    stream_id = input("Enter request ID: ").strip()

```

(下页继续)

(续上页)

```
# Enter Type and verify.
while True:
    stream_type = input("Enter type (begin/var/end): ").strip().lower()
    if stream_type in ["begin", "var", "end"]:
        break
    print("Invalid type! Please enter 'begin', 'var', or 'end'.")

# Enter Text
text = input("Enter TTS text: ").strip()

# Enter End Session
end_session_input = input("End session? (y/n): ").strip().lower()
end_session = end_session_input in ["y", "yes"]

stream = magicdog_w.SpeechTTSStream()
stream.id = stream_id
stream.type = stream_type
stream.text = text
stream.end_session = end_session

return stream

def main():
    """Main function"""
    global robot

    # Bind SIGINT (Ctrl+C)
    signal.signal(signal.SIGINT, signal_handler)

    print_help()

    local_ip = "192.168.55.10"
    robot = magicdog_w.MagicRobot()

    # Configure local IP address for direct network connection and initialize SDK
    if not robot.initialize(local_ip):
        logging.error("robot sdk initialize failed.")
        robot.shutdown()
        return -1

    # Connect to robot
```

(下页继续)

(续上页)

```

status = robot.connect()

if status.code != magicdog_w.ErrorCode.OK:
    logging.error(
        f"connect robot failed, code: {status.code}, message: {status.message}"
    )
    robot.shutdown()
    return -1

logging.info("Press any key to continue (ESC to exit)...")

audio_controller = robot.get_audio_controller()

# Wait for user input
while True:
    key = getch()
    if key == "\x1b": # ESC key
        break

    key_ascii = ord(key)
    logging.info(f"Key ASCII: {key_ascii}, Character: {key}")

    # 1. Audio Functions
    # 1.1 Get volume
    if key == "1":
        get_volume(audio_controller)
    # 1.2 Set volume
    elif key == "2":
        set_volume(audio_controller)
    # 1.3 Play TTS
    elif key == "3":
        play_tts(audio_controller)
    # 1.4 Stop TTS
    elif key == "4":
        stop_tts(audio_controller)
    # 1.5 Get voice config
    elif key == "5":
        get_voice_config(audio_controller)

    # 2. Audio stream Functions
    # 2.1 Open audio stream
    elif key == "6":
        open_audio_stream(audio_controller)
    # 2.2 Close audio stream
    elif key == "7":

```

(下页继续)

(续上页)

```

        close_audio_stream(audio_controller)
# 2.3 Subscribe to audio stream
    elif key == "8":
        subscribe_audio_stream(audio_controller)
# 2.4 Unsubscribe from audio stream
    elif key == "9":
        unsubscribe_audio_stream(audio_controller)

# 4. Speech IO Functions
# 4.1 Open speech IO
    elif key == "d":
        open_speech_io(audio_controller)
# 4.2 Close speech IO
    elif key == "e":
        close_speech_io(audio_controller)
# 4.3 Subscribe to speech ASR
    elif key == "f":
        subscribe_speech_asr(audio_controller)
# 4.4 Unsubscribe from speech ASR
    elif key == "g":
        unsubscribe_speech_asr(audio_controller)
# 4.5 Publish speech TTS
    elif key == "h":
        data = get_speech_tts_input()
        audio_controller.publish_speech_tts(data)

# Help
    elif key == "?":
        print_help()
    else:
        logging.info(f"Unknown key: {key_ascii}")

# Sleep 10ms, equivalent to usleep(10000)
    time.sleep(0.01)

audio_controller.shutdown()
logging.info("audio controller shutdown")

# Disconnect from robot
robot.disconnect()
logging.info("disconnect robot success")

robot.shutdown()
logging.info("robot shutdown")

```

(下页继续)

(续上页)

```
    return 0

if __name__ == "__main__":
    sys.exit(main())
```

26.3 运行说明

26.3.1 环境准备

```
export PYTHONPATH=/opt/magic_robotics/magicrobot_w_sdk/lib:$PYTHONPATH
export LD_LIBRARY_PATH=/opt/magic_robotics/magicrobot_w_sdk/lib:$LD_LIBRARY_PATH
```

26.3.2 运行示例

```
# C++
./audio_example

# Python
python3 audio_example.py
```

26.3.3 控制说明：

- 音频功能：
 - 按键 1 - 获取音量
 - 按键 2 - 设置音量
 - 按键 3 - 播放 TTS 语音合成
 - 按键 4 - 停止播放
 - 按键 5 - 获取语音配置
- 音频流功能：
 - 按键 6 - 打开音频流
 - 按键 7 - 关闭音频流
 - 按键 8 - 订阅音频流
 - 按键 9 - 取消订阅音频流
- 语音 IO 功能：
 - 按键 d - 打开语音 IO

- 按键 e - 关闭语音 IO
- 按键 f - 订阅语音 asr
- 按键 g - 取消订阅语音 asr
- 按键 h - 发布语音 tts
- 其他功能：
 - 按键 ? - 显示帮助菜单

26.3.4 停止程序

- 按 ESC 可以安全停止程序
- 程序会自动清理所有资源

状态监控示例

该示例展示了如何使用 MagicDog_W SDK 进行初始化、连接机器人、机器人状态监控 (故障、BMS) 等基本操作。

27.1 C++

示例文件: monitor_example.cpp

```
#include "magic_robot.h"
#include "magic_sdk_version.h"

#include <unistd.h>
#include <csignal>
#include <iostream>
#include <memory>
#include <chrono>
#include <thread>
#include <cstdlib>

using namespace magic::dog_w;

// Global robot instance
std::unique_ptr<MagicRobot> robot = nullptr;
```

(下页继续)

(续上页)

```

void signalHandler(int signum) {
    std::cout << "Interrupt signal ( " << signum << " ) received." << std::endl;
    if (robot) {
        robot->Shutdown();
    }
    exit(signum);
}

int main() {
    // Bind SIGINT (Ctrl+C)
    signal(SIGINT, signalHandler);

    std::cout << "MagicDog_W SDK C++ Example Program" << std::endl;

    robot = std::make_unique<MagicRobot>();
    if (!robot->Initialize("192.168.55.10")) {
        std::cerr << "Initialization failed" << std::endl;
        return -1;
    }

    auto status = robot->Connect();
    if (status.code != ErrorCode::OK) {
        std::cerr << "Connection failed, code: " << status.code
            << ", message: " << status.message << std::endl;
        robot->Shutdown();
        return -1;
    }

    std::this_thread::sleep_for(std::chrono::seconds(5));

    auto& monitor = robot->GetStateMonitor();

    RobotState robot_state;
    status = monitor.GetCurrentState(robot_state);
    if (status.code != ErrorCode::OK) {
        std::cerr << "Get current state failed, code: " << status.code
            << ", message: " << status.message << std::endl;
        robot->Shutdown();
        return -1;
    }

    std::cout << "Health: " << robot_state.bms_data.battery_health
        << ", Percentage: " << robot_state.bms_data.battery_percentage

```

(下页继续)

(续上页)

```

        << ", State: " << static_cast<int>(robot_state.bms_data.battery_state)
        << ", Power supply status: " << static_cast<int>(robot_state.bms_data.power_
↪supply_status)
        << std::endl;

    auto& faults = robot_state.faults;
    for (const auto& fault : faults) {
        std::cout << "Code: " << fault.error_code
            << ", Message: " << fault.error_message << std::endl;
    }

    // Disconnect from robot
    status = robot->Disconnect();
    if (status.code != ErrorCode::OK) {
        std::cerr << "Disconnect robot failed, code: " << status.code
            << ", message: " << status.message << std::endl;
    } else {
        std::cout << "Robot disconnected" << std::endl;
    }

    robot->Shutdown();
    std::cout << "Robot shutdown" << std::endl;

    std::cout << "\nExample program execution completed!" << std::endl;

    return 0;
}

```

27.2 Python

示例文件: monitor_example.py

```

#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import sys
import time
import logging
from typing import Optional
import magicdog_w_python as magicdog_w

logging.basicConfig(
    level=logging.INFO, # Minimum log level

```

(下页继续)

(续上页)

```

        format="% (asctime)s [%(levelname)s] %(message)s",
        datefmt="%Y-%m-%d %H:%M:%S",
    )

def main():
    """Main function"""
    logging.info("MagicDog_W SDK Python Example Program")

    robot = magicdog_w.MagicRobot()
    if not robot.initialize("192.168.55.10"):
        logging.error("Initialization failed")
        return

    if not robot.connect():
        logging.error("Connection failed")
        robot.shutdown()
        return

    time.sleep(5)

    monitor = robot.get_state_monitor()

    status, state = monitor.get_current_state()
    if status.code != magicdog_w.ErrorCode.OK:
        logging.error(f"Get current state failed: {status.message}")
        robot.shutdown()
        return

    logging.info(
        f"health: {state.bms_data.battery_health}, percentage: {state.bms_data.battery_
↵percentage}, state: {state.bms_data.battery_state}, power_supply_status: {state.bms_data.
↵power_supply_status}"
    )

    for fault in state.faults:
        logging.info(f"code: {fault.error_code}, message: {fault.error_message}")

    robot.disconnect()
    robot.shutdown()

    logging.info("\nExample program execution completed!")

```

(下页继续)

(续上页)

```
if __name__ == "__main__":  
    main()
```

27.3 运行说明

27.3.1 环境准备

```
export PYTHONPATH=/opt/magic_robotics/magicdog_w_sdk/lib:$PYTHONPATH  
export LD_LIBRARY_PATH=/opt/magic_robotics/magicdog_w_sdk/lib:$LD_LIBRARY_PATH
```

27.3.2 运行示例

```
# C++  
./monitor_example  
# Python  
python3 monitor_example.py
```

27.3.3 停止程序

- 按 Ctrl+C 可以安全停止程序
- 程序会自动清理所有资源

该示例展示了如何使用 MagicDog_W SDK 进行 SLAM 建图、定位、导航等操作，包括地图管理、位姿获取等功能。

28.1 C++

示例文件：

- slam_example.cpp
- navigation_example.cpp

28.1.1 建图示例

```
#include "magic_robot.h"
#include "magic_sdk_version.h"

#include <fcntl.h>
#include <signal.h>
#include <termios.h>
#include <unistd.h>
#include <algorithm>
#include <atomic>
#include <chrono>
```

(下页继续)

(续上页)

```
#include <cmath>
#include <cstdlib>
#include <iostream>
#include <memory>
#include <sstream>
#include <string>
#include <thread>
#include <vector>

using namespace magic::dog_w;
using namespace std;

// Global variables
std::unique_ptr<MagicRobot> robot = nullptr;
std::atomic<bool> running(true);
std::string current_slam_mode = "IDLE";
NavMode current_nav_mode = NavMode::IDLE;

void signalHandler(int signum) {
    std::cout << "Interrupt signal ( " << signum << " ) received.\n";
    running = false;
    if (robot) {
        robot->Shutdown();
        std::cout << "Robot shutdown" << std::endl;
    }
    exit(-1);
}

void printHelp() {
    std::cout << "SLAM Function Demo Program" << std::endl;
    std::cout << " " << std::endl;
    std::cout << "preparation Functions:" << std::endl;
    std::cout << " 1      Function 1: Recovery stand" << std::endl;
    std::cout << " " << std::endl;
    std::cout << "SLAM Functions:" << std::endl;
    std::cout << " 2      Function 2: Start mapping" << std::endl;
    std::cout << " 3      Function 3: Cancel mapping" << std::endl;
    std::cout << " 4      Function 4: Save map" << std::endl;
    std::cout << " 5      Function 5: Load map" << std::endl;
    std::cout << " 6      Function 6: Delete map" << std::endl;
    std::cout << " 7      Function 7: Get all map information and save map image as PGM file" <
↵< std::endl;
    std::cout << " 8      Function 8: Get point cloud map" << std::endl;
```

(下页继续)

(续上页)

```
std::cout << "" << std::endl;
std::cout << "Joystick Functions:" << std::endl;
std::cout << " W      Function W: forward" << std::endl;
std::cout << " A      Function A: backward" << std::endl;
std::cout << " S      Function S: left" << std::endl;
std::cout << " D      Function D: right" << std::endl;
std::cout << " T      Function T: turn left" << std::endl;
std::cout << " G      Function G: turn right" << std::endl;
std::cout << " X      Function X: stop" << std::endl;
std::cout << "" << std::endl;
std::cout << "Close Functions:" << std::endl;
std::cout << " P      Function P: Close SLAM" << std::endl;
std::cout << "" << std::endl;
std::cout << " ?      Function H: Print help" << std::endl;
std::cout << " ESC     Exit program" << std::endl;
}

// ===== SLAM Functions =====

void loadMap(const std::string& mapToLoad) {
    try {
        if (mapToLoad.empty()) {
            std::cerr << "Map to load is not provided" << std::endl;
            return;
        }

        // Get SLAM navigation controller
        auto& controller = robot->GetSlamNavController();

        std::cout << "Loading map: " << mapToLoad << std::endl;
        controller.LoadMap(mapToLoad);
        std::cout << "Successfully loaded map: " << mapToLoad << std::endl;
    } catch (const std::exception& e) {
        std::cerr << "Exception occurred while loading map: " << e.what() << std::endl;
    }
}

void startMapping() {
    try {
        // Get SLAM navigation controller
        auto& controller = robot->GetSlamNavController();

        // Start mapping
    }
}
```

(下页继续)

(续上页)

```

    auto status = controller.StartMapping();
    if (status.code != ErrorCode::OK) {
        std::cerr << "Failed to start mapping, code: " << status.code
                    << ", message: " << status.message << std::endl;

        return;
    }

    current_slam_mode = "MAPPING";
    std::cout << "Successfully started mapping" << std::endl;
} catch (const std::exception& e) {
    std::cerr << "Exception occurred while starting mapping: " << e.what() << std::endl;
}
}

void cancelMapping() {
    try {
        // Get SLAM navigation controller
        auto& controller = robot->GetSlamNavController();

        // Cancel mapping
        auto status = controller.CancelMapping();
        if (status.code != ErrorCode::OK) {
            std::cerr << "Failed to cancel mapping, code: " << status.code
                        << ", message: " << status.message << std::endl;

            return;
        }

        std::cout << "Successfully cancelled mapping" << std::endl;
    } catch (const std::exception& e) {
        std::cerr << "Exception occurred while cancelling mapping: " << e.what() << std::endl;
    }
}

void saveMap() {
    try {
        // Get SLAM navigation controller
        auto& controller = robot->GetSlamNavController();

        // Check if in mapping mode
        if (current_slam_mode != "MAPPING") {
            std::cout << "Warning: Currently not in mapping mode, may not be able to save map" <<
↵std::endl;
        }
    }
}

```

(下页继续)

(续上页)

```

// Generate map name with timestamp
auto now = std::chrono::system_clock::now();
auto timestamp = std::chrono::duration_cast<std::chrono::seconds>(now.time_since_epoch()).
↵count();

std::string mapName = "map_" + std::to_string(timestamp);

std::cout << "Saving map: " << mapName << std::endl;

// Save map
auto status = controller.SaveMap(mapName);
if (status.code != ErrorCode::OK) {
    std::cerr << "Failed to save map, code: " << status.code
                << ", message: " << status.message << std::endl;
    return;
}

std::cout << "Successfully saved map: " << mapName << std::endl;
} catch (const std::exception& e) {
    std::cerr << "Exception occurred while saving map: " << e.what() << std::endl;
}
}

void deleteMap(const std::string& mapToDelete) {
    try {
        if (mapToDelete.empty()) {
            std::cerr << "Map to delete is not provided" << std::endl;
            return;
        }

        // Get SLAM navigation controller
        auto& controller = robot->GetSlamNavController();

        std::cout << "Deleting map: " << mapToDelete << std::endl;

        // Delete map
        auto status = controller.DeleteMap(mapToDelete);
        if (status.code != ErrorCode::OK) {
            std::cerr << "Failed to delete map, code: " << status.code
                        << ", message: " << status.message << std::endl;
            return;
        }
    }
}

```

(下页继续)

(续上页)

```

        std::cout << "Successfully deleted map: " << mapToDelete << std::endl;
    } catch (const std::exception& e) {
        std::cerr << "Exception occurred while deleting map: " << e.what() << std::endl;
    }
}

void getAllMapInfo() {
    try {
        // Get SLAM navigation controller
        auto& controller = robot->GetSlamNavController();

        // Get all map information
        AllMapInfo allMapInfo;
        auto status = controller.GetAllMapInfo(allMapInfo);
        if (status.code != ErrorCode::OK) {
            std::cerr << "Failed to get map information, code: " << status.code
                << ", message: " << status.message << std::endl;
            return;
        }

        std::cout << "Successfully retrieved map information" << std::endl;
        std::cout << "Current map: " << allMapInfo.current_map_name << std::endl;
        std::cout << "Total maps: " << allMapInfo.map_infos.size() << std::endl;

        if (!allMapInfo.map_infos.empty()) {
            std::cout << "Map details:" << std::endl;
            for (size_t i = 0; i < allMapInfo.map_infos.size(); ++i) {
                const auto& mapInfo = allMapInfo.map_infos[i];
                std::cout << "  Map " << (i + 1) << ": " << mapInfo.map_name << std::endl;
                std::cout << "    Origin: ["
                    << mapInfo.map_meta_data.origin.position[0] << ", "
                    << mapInfo.map_meta_data.origin.position[1] << ", "
                    << mapInfo.map_meta_data.origin.position[2] << "]" << std::endl;
                std::cout << "    Orientation: ["
                    << mapInfo.map_meta_data.origin.orientation[0] << ", "
                    << mapInfo.map_meta_data.origin.orientation[1] << ", "
                    << mapInfo.map_meta_data.origin.orientation[2] << "]" << std::endl;
                std::cout << "    Resolution: " << mapInfo.map_meta_data.resolution << " m/pixel" <<
↪std::endl;
                std::cout << "    Size: " << mapInfo.map_meta_data.map_image_data.width
                    << " x " << mapInfo.map_meta_data.map_image_data.height << std::endl;
                std::cout << "    Max gray value: " << mapInfo.map_meta_data.map_image_data.max_gray_
↪value << std::endl;
            }
        }
    }
}

```

(下页继续)

(续上页)

```

        std::cout << "    Image type: " << mapInfo.map_meta_data.map_image_data.type <<
↵std::endl;
    }
    } else {
        std::cout << "No available maps" << std::endl;
    }
} catch (const std::exception& e) {
    std::cerr << "Exception occurred while getting map information: " << e.what() << std::endl;
}
}

void getPointCloudMap() {
    try {
        // Get SLAM navigation controller
        auto& controller = robot->GetSlamNavController();

        // Get point cloud map
        PointCloud2 pointCloudMap;
        auto status = controller.GetPointCloudMap(pointCloudMap);
        if (status.code != ErrorCode::OK) {
            std::cerr << "Failed to get point cloud map, code: " << status.code
                << ", message: " << status.message << std::endl;
            return;
        }

        std::cout << "Successfully retrieved point cloud map" << std::endl;

        std::cout << "Point cloud map: " << pointCloudMap.data.size() << " points" << std::endl;
        std::cout << "Point cloud map: width: " << pointCloudMap.width << ", height: " <<
↵pointCloudMap.height << std::endl;
    } catch (const std::exception& e) {
        std::cerr << "Exception occurred while getting point cloud map: " << e.what() << std::endl;
    }
}

void closeSlam() {
    try {
        // Get SLAM navigation controller
        auto& controller = robot->GetSlamNavController();

        // Switch to idle mode to close SLAM
        auto status = controller.SwitchToIdle();
        if (status.code != ErrorCode::OK) {

```

(下页继续)

(续上页)

```

        std::cerr << "Failed to close SLAM, code: " << status.code
                    << ", message: " << status.message << std::endl;

        return;
    }

    current_slam_mode = "IDLE";
    std::cout << "Successfully closed SLAM system" << std::endl;
} catch (const std::exception& e) {
    std::cerr << "Exception occurred while closing SLAM: " << e.what() << std::endl;
}
}

// Switch gait to down climb stairs mode
bool changeGaitToDownClimbStairs() {
    try {
        auto& highController = robot->GetHighLevelMotionController();
        GaitMode currentGait;
        auto status = highController.GetGait(currentGait);
        if (status.code != ErrorCode::OK) {
            std::cerr << "Failed to get gait: " << status.message << std::endl;
            return false;
        }

        if (currentGait != GaitMode::GAIT_RL_TERRAIN) {
            auto status = highController.SetGait(GaitMode::GAIT_RL_TERRAIN);
            if (status.code != ErrorCode::OK) {
                std::cerr << "Failed to set down climb stairs gait: " << status.message << std::endl;
                return false;
            }
        }

        // Wait for gait switch to complete
        while (currentGait != GaitMode::GAIT_RL_TERRAIN) {
            std::this_thread::sleep_for(std::chrono::milliseconds(10));
            status = highController.GetGait(currentGait);
            if (status.code != ErrorCode::OK) {
                std::cerr << "Failed to get gait during transition: " << status.message << std::endl;
                return false;
            }
            std::this_thread::sleep_for(std::chrono::milliseconds(10));
        }
    }

    std::cout << "Gait changed to down climb stairs" << std::endl;
}

```

(下页继续)

(续上页)

```

        return true;
    } catch (const std::exception& e) {
        std::cerr << "Error changing gait: " << e.what() << std::endl;
        return false;
    }
}

void sendJoystickCommand(double leftX, double leftY, double rightX, double rightY) {
    if (!changeGaitToDownClimbStairs()) {
        return;
    }

    auto& highController = robot->GetHighLevelMotionController();
    JoystickCommand joyCommand;
    joyCommand.left_x_axis = leftX;
    joyCommand.left_y_axis = leftY;
    joyCommand.right_x_axis = rightX;
    joyCommand.right_y_axis = rightY;

    auto status = highController.SendJoyStickCommand(joyCommand);
    if (status.code != ErrorCode::OK) {
        std::cerr << "Failed to send joystick command: " << status.message << std::endl;
    }
}

// Position control standing
void recoveryStand() {
    try {
        auto& highController = robot->GetHighLevelMotionController();

        // Set gait to position control standing
        auto status = highController.SetGait(GaitMode::GAIT_STAND_R);
        if (status.code != ErrorCode::OK) {
            std::cerr << "Failed to set position control standing: " << status.message << std::endl;
        } else {
            std::cout << "Robot set to position control standing" << std::endl;
        }
    } catch (const std::exception& e) {
        std::cerr << "Error executing position control standing: " << e.what() << std::endl;
    }
}

// Get single character input (no echo)

```

(下页继续)

(续上页)

```
char getch() {
    char ch;
    struct termios oldt, newt;

    tcgetattr(STDIN_FILENO, &oldt);
    newt = oldt;
    newt.c_lflag &= ~(ICANON | ECHO);
    tcsetattr(STDIN_FILENO, TCSANOW, &newt);

    ch = getchar();

    tcsetattr(STDIN_FILENO, TCSANOW, &oldt);

    std::cout << "Received character: " << ch << std::endl;
    return ch;
}

int main(int argc, char* argv[]) {
    // Bind signal handler
    signal(SIGINT, signalHandler);

    // Create robot instance
    robot = std::make_unique<MagicRobot>();

    printHelp();
    std::cout << "Press any key to continue (ESC to exit)..." << std::endl;

    try {
        // Configure local IP address for direct network connection and initialize SDK
        std::string localIp = "192.168.55.10";
        if (!robot->Initialize(localIp)) {
            std::cerr << "Failed to initialize robot SDK" << std::endl;
            robot->Shutdown();
            return -1;
        }

        // Connect to robot
        auto status = robot->Connect();
        if (status.code != ErrorCode::OK) {
            std::cerr << "Failed to connect to robot, code: " << status.code
                << ", message: " << status.message << std::endl;
            robot->Shutdown();
            return -1;
        }
    }
}
```

(下页继续)

(续上页)

```

}

std::cout << "Successfully connected to robot" << std::endl;

// Set motion control level to high level
status = robot->SetMotionControlLevel(ControllerLevel::HighLevel);
if (status.code != ErrorCode::OK) {
    std::cerr << "Failed to set motion control level: " << status.message << std::endl;
    robot->Shutdown();
    return -1;
}

// Initialize SLAM navigation controller
auto& slamNavController = robot->GetSlamNavController();
if (!slamNavController.Initialize()) {
    std::cerr << "Failed to initialize SLAM navigation controller" << std::endl;
    robot->Disconnect();
    robot->Shutdown();
    return -1;
}

std::cout << "Successfully initialized SLAM navigation controller" << std::endl;

// Main loop
while (running.load()) {
    try {
        std::cout << "Enter command: ";
        char key = getch();
        if (key == 27) { // ESC key
            break;
        }

        // 1. Preparation Functions
        // 1.1 Recovery stand
        if (key == '1') {
            recoveryStand();
        }

        // 2. SLAM Functions
        // 2.1 Start mapping
        else if (key == '2') {
            startMapping();
        }

        // 2.2 Cancel mapping
        else if (key == '3') {

```

(下页继续)

(续上页)

```

        cancelMapping();
    }
    // 2.3 Save map
    else if (key == '4') {
        saveMap();
    }
    // 2.4 Load map
    else if (key == '5') {
        std::string strInput;
        std::cout << "Enter parameters: ";
        std::getline(std::cin, strInput);
        if (strInput.empty()) {
            continue;
        }
        // Split input parameters by space
        std::vector<std::string> parts;
        std::stringstream ss(strInput);
        std::string segment;
        while (std::getline(ss, segment, ' ')) {
            parts.push_back(segment);
        }
        // Parse parameters
        std::string mapToLoad = parts.empty() ? "" : parts[0];
        loadMap(mapToLoad);
    }
    // 2.5 Delete map
    else if (key == '6') {
        std::string strInput;
        std::cout << "Enter parameters: ";
        std::getline(std::cin, strInput);
        if (strInput.empty()) {
            continue;
        }
        // Split input parameters by space
        std::vector<std::string> parts;
        std::stringstream ss(strInput);
        std::string segment;
        while (std::getline(ss, segment, ' ')) {
            parts.push_back(segment);
        }
        // Parse parameters
        std::string mapToDelete = parts.empty() ? "" : parts[0];
        deleteMap(mapToDelete);
    }

```

(下页继续)

(续上页)

```

    }
    // 2.6 Get all map information
    else if (key == '7') {
        getAllMapInfo();
    }
    // 2.7 Get point cloud map
    else if (key == '8') {
        getPointCloudMap();
    }
    // 4. Joystick Functions
    // 4.1 Move forward
    else if (key == 'w' || key == 'W') {
        sendJoystickCommand(0.0, 1.0, 0.0, 0.0); // Move forward
    }
    // 4.2 Move backward
    else if (key == 's' || key == 'S') {
        sendJoystickCommand(0.0, -1.0, 0.0, 0.0); // Move backward
    }
    // 4.3 Move left
    else if (key == 'a' || key == 'A') {
        sendJoystickCommand(-1.0, 0.0, 0.0, 0.0); // Move left
    }
    // 4.4 Move right
    else if (key == 'd' || key == 'D') {
        sendJoystickCommand(1.0, 0.0, 0.0, 0.0); // Move right
    }
    // 4.5 Turn left
    else if (key == 't' || key == 'T') {
        sendJoystickCommand(0.0, 0.0, -1.0, 0.0); // Turn left
    }
    // 4.6 Turn right
    else if (key == 'g' || key == 'G') {
        sendJoystickCommand(0.0, 0.0, 1.0, 0.0); // Turn right
    }
    // 4.7 Stop movement
    else if (key == 'x' || key == 'X') {
        sendJoystickCommand(0.0, 0.0, 0.0, 0.0); // Stop movement
    }
    // 5. Close Functions
    // 5.1 Close SLAM
    else if (key == 'p' || key == 'P') {
        closeSlam();
    }

```

(下页继续)

(续上页)

```

        // Help
        else if (key == '?' || key == 'h' || key == 'H') {
            printHelp();
        } else {
            std::cout << "Unknown key: " << key << std::endl;
        }

        std::this_thread::sleep_for(std::chrono::milliseconds(10)); // Brief delay

    } catch (const std::exception& e) {
        std::cerr << "Exception occurred while processing user input: " << e.what() << std::endl;
        return -1;
    }

    // Clean up resources
    try {
        std::cout << "Clean up resources" << std::endl;

        // Close SLAM navigation controller
        auto& slamNavController = robot->GetSlamNavController();
        slamNavController.Shutdown();
        std::cout << "SLAM navigation controller closed" << std::endl;

        // Disconnect
        robot->Disconnect();
        std::cout << "Robot connection disconnected" << std::endl;

        // Shutdown robot
        robot->Shutdown();
        std::cout << "Robot shutdown" << std::endl;

    } catch (const std::exception& e) {
        std::cerr << "Exception occurred while cleaning up resources: " << e.what() << std::endl;
    }

    return 0;
}

```

28.1.2 导航示例

```
#include "magic_robot.h"
#include "magic_sdk_version.h"

#include <signal.h>
#include <algorithm>
#include <atomic>
#include <chrono>
#include <cmath>
#include <cstdlib>
#include <iostream>
#include <memory>
#include <sstream>
#include <string>
#include <thread>
#include <vector>

using namespace magic::dog_w;
using namespace std;

// Global variables
std::unique_ptr<MagicRobot> robot = nullptr;
std::atomic<bool> running(true);
std::string current_slam_mode = "IDLE";
NavMode current_nav_mode = NavMode::IDLE;
int odometry_counter = 0;

void signalHandler(int signum) {
    std::cout << "Interrupt signal (" << signum << ") received.\n";
    running = false;
    if (robot) {
        robot->Shutdown();
        std::cout << "Robot shutdown" << std::endl;
    }
    exit(-1);
}

void printHelp() {
    std::cout << "SLAM and Navigation Function Demo Program" << std::endl;
    std::cout << "" << std::endl;
    std::cout << "preparation Functions:" << std::endl;
    std::cout << "  1          Function 1: Recovery stand" << std::endl;
    std::cout << "" << std::endl;
    std::cout << "Localization Functions:" << std::endl;
```

(下页继续)

(续上页)

```
std::cout << " 2      Function 2: Switch to localization mode" << std::endl;
std::cout << " 4      Function 4: Initialize pose" << std::endl;
std::cout << " 5      Function 5: Get current pose information" << std::endl;
std::cout << "" << std::endl;
std::cout << "Navigation Functions:" << std::endl;
std::cout << " 3      Function 3: Switch to navigation mode" << std::endl;
std::cout << " 6      Function 6: Set navigation target goal" << std::endl;
std::cout << " 7      Function 7: Pause navigation" << std::endl;
std::cout << " 8      Function 8: Resume navigation" << std::endl;
std::cout << " 9      Function 9: Cancel navigation" << std::endl;
std::cout << " 0      Function 0: Get navigation status" << std::endl;
std::cout << "" << std::endl;
std::cout << "Odometry Functions:" << std::endl;
std::cout << " C      Function C: Subscribe odometry stream" << std::endl;
std::cout << " V      Function V: Unsubscribe odometry stream" << std::endl;
std::cout << "" << std::endl;
std::cout << "Close Functions:" << std::endl;
std::cout << " L      Function L: Close navigation" << std::endl;
std::cout << " P      Function P: Close SLAM" << std::endl;
std::cout << "" << std::endl;
std::cout << " ?      Function ?: Print help" << std::endl;
std::cout << " ESC    Exit program" << std::endl;
}

// ===== Navigation Functions =====

// Position control standing
void recoveryStand() {
    try {
        auto& highController = robot->GetHighLevelMotionController();

        // Set gait to position control standing
        auto status = highController.SetGait(GaitMode::GAIT_STAND_R);
        if (status.code != ErrorCode::OK) {
            std::cerr << "Failed to set position control standing: " << status.message << std::endl;
        } else {
            std::cout << "Robot set to position control standing" << std::endl;
        }
    } catch (const std::exception& e) {
        std::cerr << "Error executing position control standing: " << e.what() << std::endl;
    }
}
```

(下页继续)

(续上页)

```

void switchToLocalizationMode() {
    try {
        // Get SLAM navigation controller
        auto& controller = robot->GetSlamNavController();

        // Switch to localization mode
        auto status = controller.SwitchToLocation();
        if (status.code != ErrorCode::OK) {
            std::cerr << "Failed to switch to localization mode, code: " << status.code
                << ", message: " << status.message << std::endl;

            return;
        }

        current_slam_mode = "LOCALIZATION";
        std::cout << "Successfully switched to localization mode" << std::endl;
        std::cout << "Robot is now in localization mode, ready to localize on existing maps" <<
↪std::endl;
    } catch (const std::exception& e) {
        std::cerr << "Exception occurred while switching to localization mode: " << e.what() <<
↪std::endl;
    }
}

void initializePose(double x, double y, double yaw) {
    try {
        // Get SLAM navigation controller
        auto& controller = robot->GetSlamNavController();

        // Create initial pose (set to origin)
        Pose3DEuler initialPose;
        initialPose.position = {x, y, 0.0};           // x, y, z
        initialPose.orientation = {0.0, 0.0, yaw};    // roll, pitch, yaw

        std::cout << "Initializing robot pose to origin..." << std::endl;

        // Initialize pose
        auto status = controller.InitPose(initialPose);
        if (status.code != ErrorCode::OK) {
            std::cerr << "Failed to initialize pose, code: " << status.code
                << ", message: " << status.message << std::endl;

            return;
        }
    }
}

```

(下页继续)

(续上页)

```

        std::cout << "Successfully initialized pose" << std::endl;
        std::cout << "Robot pose has been set to origin (" << x << ", " << y << ", " << yaw << ")" <
↵< std::endl;
    } catch (const std::exception& e) {
        std::cerr << "Exception occurred while initializing pose: " << e.what() << std::endl;
    }
}

void getCurrentLocalizationInfo() {
    try {
        // Get SLAM navigation controller
        auto& controller = robot->GetSlamNavController();

        // Get current pose information
        LocalizationInfo poseInfo;
        auto status = controller.GetCurrentLocalizationInfo(poseInfo);
        if (status.code != ErrorCode::OK) {
            std::cerr << "Failed to get current pose information, code: " << status.code
                << ", message: " << status.message << std::endl;
            return;
        }

        std::cout << "Successfully retrieved current pose information" << std::endl;
        std::cout << "Localization status: " << (poseInfo.is_localization ? "Localized" : "Not_
↵localized") << std::endl;
        std::cout << "Position: [" << poseInfo.pose.position[0] << ", "
            << poseInfo.pose.position[1] << ", " << poseInfo.pose.position[2] << "]" <<
↵std::endl;
        std::cout << "Orientation: [" << poseInfo.pose.orientation[0] << ", "
            << poseInfo.pose.orientation[1] << ", " << poseInfo.pose.orientation[2] << "]" <<
↵std::endl;
    } catch (const std::exception& e) {
        std::cerr << "Exception occurred while getting current pose information: " << e.what() <<
↵std::endl;
    }
}

void switchToNavigationMode() {
    try {
        auto& highController = robot->GetHighLevelMotionController();
        highController.DisableJoyStick();
        usleep(100000);
    }
}

```

(下页继续)

(续上页)

```
// Get SLAM navigation controller
auto& controller = robot->GetSlamNavController();

// Switch to navigation mode
auto status = controller.ActivateNavMode(NavMode::GRID_MAP);
if (status.code != ErrorCode::OK) {
    std::cerr << "Failed to switch to navigation mode, code: " << status.code
                << ", message: " << status.message << std::endl;
    return;
}

current_nav_mode = NavMode::GRID_MAP;
std::cout << "Successfully switched to navigation mode" << std::endl;
} catch (const std::exception& e) {
    std::cerr << "Exception occurred while switching to navigation mode: " << e.what() <<
std::endl;
}
}

void setNavigationTarget(double x, double y, double yaw) {
    try {
        // Get SLAM navigation controller
        auto& controller = robot->GetSlamNavController();
        auto& highController = robot->GetHighLevelMotionController();

        // Disable joy stick
        highController.DisableJoyStick();
        std::cout << "Successfully disabled joy stick" << std::endl;

        // change gait to slow
        // To use Navigation, you must switch to the down stairs gait
        auto status = highController.SetGait(GaitMode::GAIT_RL_TERRAIN);
        if (status.code != ErrorCode::OK) {
            std::cerr << "Failed to set gait to slow: " << status.message << std::endl;
            return;
        }
        std::cout << "Successfully set gait to slow" << std::endl;

        // Create target goal
        NavTarget targetGoal;
        targetGoal.id = 1;
        targetGoal.frame_id = "map";
    }
}
```

(下页继续)

(续上页)

```

targetGoal.goal.position = {x, y, 0.0};
targetGoal.goal.orientation = {0.0, 0.0, yaw};

// Set target goal
status = controller.SetNavTarget(targetGoal);
if (status.code != ErrorCode::OK) {
    std::cerr << "Failed to set navigation target, code: " << status.code
                << ", message: " << status.message << std::endl;
    return;
}

std::cout << "Successfully set navigation target: position=("
            << targetGoal.goal.position[0] << ", " << targetGoal.goal.position[1] << ", "
            << targetGoal.goal.position[2] << "), orientation=("
            << targetGoal.goal.orientation[0] << ", " << targetGoal.goal.orientation[1] << ",
↵"
            << targetGoal.goal.orientation[2] << ")" << std::endl;
} catch (const std::exception& e) {
    std::cerr << "Exception occurred while setting navigation target: " << e.what() << ↵
↵std::endl;
}
}

void pauseNavigation() {
    try {
        // Get SLAM navigation controller
        auto& controller = robot->GetSlamNavController();

        // Pause navigation
        auto status = controller.PauseNavTask();
        if (status.code != ErrorCode::OK) {
            std::cerr << "Failed to pause navigation, code: " << status.code
                        << ", message: " << status.message << std::endl;
            return;
        }

        std::cout << "Successfully paused navigation" << std::endl;
    } catch (const std::exception& e) {
        std::cerr << "Exception occurred while pausing navigation: " << e.what() << std::endl;
    }
}

void resumeNavigation() {

```

(下页继续)

(续上页)

```

try {
    // Get SLAM navigation controller
    auto& controller = robot->GetSlamNavController();

    // Resume navigation
    auto status = controller.ResumeNavTask();
    if (status.code != ErrorCode::OK) {
        std::cerr << "Failed to resume navigation, code: " << status.code
                    << ", message: " << status.message << std::endl;
        return;
    }

    std::cout << "Successfully resumed navigation" << std::endl;
} catch (const std::exception& e) {
    std::cerr << "Exception occurred while resuming navigation: " << e.what() << std::endl;
}
}

void cancelNavigation() {
    try {
        // Get SLAM navigation controller
        auto& controller = robot->GetSlamNavController();

        // Cancel navigation
        auto status = controller.CancelNavTask();
        if (status.code != ErrorCode::OK) {
            std::cerr << "Failed to cancel navigation, code: " << status.code
                        << ", message: " << status.message << std::endl;
            return;
        }

        std::cout << "Successfully cancelled navigation" << std::endl;
    } catch (const std::exception& e) {
        std::cerr << "Exception occurred while cancelling navigation: " << e.what() << std::endl;
    }
}

void getNavigationStatus() {
    try {
        // Get SLAM navigation controller
        auto& controller = robot->GetSlamNavController();

        // Get navigation status
    }
}

```

(下页继续)

(续上页)

```

NavStatus navStatus;

auto status = controller.GetNavTaskStatus(navStatus);
if (status.code != ErrorCode::OK) {
    std::cerr << "Failed to get navigation status, code: " << status.code
                << ", message: " << status.message << std::endl;

    return;
}

// Display navigation status information
std::cout << "=== Navigation Status ===" << std::endl;
std::cout << "Target ID: " << navStatus.id << std::endl;
std::cout << "Status: " << static_cast<int>(navStatus.status) << std::endl;
std::cout << "Message: " << navStatus.message << std::endl;

// Provide status interpretation
std::string statusMeaning;
switch (navStatus.status) {
    case NavStatusType::NONE:
        statusMeaning = "No navigation target set";
        break;
    case NavStatusType::RUNNING:
        statusMeaning = "Navigation is running";
        break;
    case NavStatusType::END_SUCCESS:
        statusMeaning = "Navigation completed successfully";
        break;
    case NavStatusType::END_FAILED:
        statusMeaning = "Navigation failed";
        break;
    case NavStatusType::PAUSE:
        statusMeaning = "Navigation is paused";
        break;
    default:
        statusMeaning = "Unknown status value";
        break;
}

std::cout << "Status meaning: " << statusMeaning << std::endl;
std::cout << "===== " << std::endl;

} catch (const std::exception& e) {
    std::cerr << "Exception occurred while getting navigation status: " << e.what() <<
    ↪std::endl;

```

(下页继续)

(续上页)

```

    }
}

void closeNavigation() {
    try {
        // Get SLAM navigation controller
        auto& controller = robot->GetSlamNavController();

        // Switch to idle mode to close navigation
        auto status = controller.ActivateNavMode(NavMode::IDLE);
        if (status.code != ErrorCode::OK) {
            std::cerr << "Failed to close navigation, code: " << status.code
                << ", message: " << status.message << std::endl;

            return;
        }

        current_nav_mode = NavMode::IDLE;
        std::cout << "Successfully closed navigation system" << std::endl;
    } catch (const std::exception& e) {
        std::cerr << "Exception occurred while closing navigation: " << e.what() << std::endl;
    }
}

void openOdometryStream() {
    try {
        // Get SLAM navigation controller
        auto& controller = robot->GetSlamNavController();

        // Open odometry stream
        auto status = robot->OpenChannelSwitch();
        if (status.code != ErrorCode::OK) {
            std::cerr << "Failed to open odometry stream, code: " << status.code
                << ", message: " << status.message << std::endl;

            return;
        }

        std::cout << "Successfully opened odometry stream" << std::endl;
    } catch (const std::exception& e) {
        std::cerr << "Exception occurred while opening odometry stream: " << e.what() << std::endl;
    }
}

void closeOdometryStream() {
    try {

```

(下页继续)

(续上页)

```
// Get SLAM navigation controller
auto& controller = robot->GetSlamNavController();

// Close odometry stream
auto status = robot->CloseChannelSwitch();
if (status.code != ErrorCode::OK) {
    std::cerr << "Failed to close odometry stream, code: " << status.code
                << ", message: " << status.message << std::endl;
    return;
}
std::cout << "Successfully closed odometry stream" << std::endl;
} catch (const std::exception& e) {
    std::cerr << "Exception occurred while closing odometry stream: " << e.what() << std::endl;
}
}

void subscribeOdometryStream() {
    try {
        // Open channel switch
        robot->OpenChannelSwitch();

        // Get SLAM navigation controller
        auto& controller = robot->GetSlamNavController();

        // Define odometry callback
        auto odometryCallback = [](const std::shared_ptr<Odometry> data) {
            if (odometry_counter % 10 == 0) {
                std::cout << "Odometry position data: " << data->position[0] << ", "
                            << data->position[1] << ", " << data->position[2] << std::endl;
                std::cout << "Odometry orientation data: " << data->orientation[0] << ", "
                            << data->orientation[1] << ", " << data->orientation[2] << ", "
                            << data->orientation[3] << std::endl;
                std::cout << "Odometry linear velocity data: " << data->linear_velocity[0] << ", "
                            << data->linear_velocity[1] << ", " << data->linear_velocity[2] << std::endl;
                std::cout << "Odometry angular velocity data: " << data->angular_velocity[0] << ", "
                            << data->angular_velocity[1] << ", " << data->angular_velocity[2] <<
std::endl;
            }
            odometry_counter++;
        };

        // Subscribe odometry stream
        controller.SubscribeOdometry(odometryCallback);
    }
}
```

(下页继续)

(续上页)

```

        std::cout << "Successfully subscribed odometry stream" << std::endl;
    } catch (const std::exception& e) {
        std::cerr << "Exception occurred while subscribing odometry stream: " << e.what() << "\n";
    }
}

void unsubscribeOdometryStream() {
    try {
        // Get SLAM navigation controller
        auto& controller = robot->GetSlamNavController();
        robot->CloseChannelSwitch(); // Close channel switch
        std::cout << "Successfully unsubscribed odometry stream" << std::endl;
    } catch (const std::exception& e) {
        std::cerr << "Exception occurred while unsubscribing odometry stream: " << e.what() << "\n";
    }
}

void closeSlam() {
    try {
        // Get SLAM navigation controller
        auto& controller = robot->GetSlamNavController();

        // Switch to idle mode to close SLAM
        auto status = controller.SwitchToIdle();
        if (status.code != ErrorCode::OK) {
            std::cerr << "Failed to close SLAM, code: " << status.code
                << ", message: " << status.message << std::endl;
            return;
        }

        current_slam_mode = "IDLE";
        std::cout << "Successfully closed SLAM system" << std::endl;
    } catch (const std::exception& e) {
        std::cerr << "Exception occurred while closing SLAM: " << e.what() << std::endl;
    }
}

// ===== Utility Functions =====

std::string getUserInput() {
    std::string input;

```

(下页继续)

(续上页)

```

std::cout << "Enter command: ";
std::getline(std::cin, input);
return input;
}

int main(int argc, char* argv[]) {
    // Bind signal handler
    signal(SIGINT, signalHandler);

    // Create robot instance
    robot = std::make_unique<MagicRobot>();

    printHelp();
    std::cout << "Press any key to continue (ESC to exit)..." << std::endl;

    try {
        // Configure local IP address for direct network connection and initialize SDK
        std::string localIp = "192.168.55.10";
        if (!robot->Initialize(localIp)) {
            std::cerr << "Failed to initialize robot SDK" << std::endl;
            robot->Shutdown();
            return -1;
        }

        // Connect to robot
        auto status = robot->Connect();
        if (status.code != ErrorCode::OK) {
            std::cerr << "Failed to connect to robot, code: " << status.code
                << ", message: " << status.message << std::endl;
            robot->Shutdown();
            return -1;
        }

        std::cout << "Successfully connected to robot" << std::endl;

        // Set motion control level to high level
        status = robot->SetMotionControlLevel(ControllerLevel::HighLevel);
        if (status.code != ErrorCode::OK) {
            std::cerr << "Failed to set motion control level: " << status.message << std::endl;
            robot->Shutdown();
            return -1;
        }
    }
}

```

(下页继续)

(续上页)

```
// Initialize SLAM navigation controller
auto& slamNavController = robot->GetSlamNavController();
if (!slamNavController.Initialize()) {
    std::cerr << "Failed to initialize SLAM navigation controller" << std::endl;
    robot->Disconnect();
    robot->Shutdown();
    return -1;
}

std::cout << "Successfully initialized SLAM navigation controller" << std::endl;

// Main loop
while (running.load()) {
    try {
        std::string strInput = getUserInput();

        // Split input parameters by space
        std::vector<std::string> parts;
        std::stringstream ss(strInput);
        std::string segment;
        while (std::getline(ss, segment, ' ')) {
            parts.push_back(segment);
        }

        if (parts.empty()) {
            std::this_thread::sleep_for(std::chrono::milliseconds(10)); // Brief delay
            continue;
        }

        // Parse parameters
        std::string key = parts[0];
        std::vector<std::string> args(parts.begin() + 1, parts.end());

        if (key == "\x1b") { // ESC key
            break;
        }

        // 1. Preparation Functions
        if (key == "1") {
            // recovery stand
            recoveryStand();
        }

        // 2. Localization Functions
```

(下页继续)

(续上页)

```

// 2.1 Switch to localization mode
else if (key == "2") {
    switchToLocalizationMode();
}

// 2.2 Initialize pose
else if (key == "4") {
    double x = args.empty() ? 0.0 : std::stod(args[0]);
    double y = args.size() < 2 ? 0.0 : std::stod(args[1]);
    double yaw = args.size() < 3 ? 0.0 : std::stod(args[2]);
    std::cout << "input initial pose, x: " << x << ", y: " << y << ", yaw: " << yaw << "\n";
    initializePose(x, y, yaw);
}

// 2.3 Get current localization information
else if (key == "5") {
    getCurrentLocalizationInfo();
}

// 3. Navigation Functions
// 3.1 Switch to navigation mode
else if (key == "3") {
    switchToNavigationMode();
}

// 3.2 Set navigation target
else if (key == "6") {
    double x = args.empty() ? 0.0 : std::stod(args[0]);
    double y = args.size() < 2 ? 0.0 : std::stod(args[1]);
    double yaw = args.size() < 3 ? 0.0 : std::stod(args[2]);
    std::cout << "input navigation target, x: " << x << ", y: " << y << ", yaw: " << yaw << "\n";
    setNavigationTarget(x, y, yaw);
}

// 3.3 Pause navigation
else if (key == "7") {
    pauseNavigation();
}

// 3.4 Resume navigation
else if (key == "8") {
    resumeNavigation();
}

// 3.5 Cancel navigation
else if (key == "9") {
    cancelNavigation();
}
    
```

(下页继续)

(续上页)

```

// 3.6 Get navigation status
else if (key == "0") {
    getNavigationStatus();
}

// 4. Odometry Functions
// 4.1 Subscribe odometry stream
else if (key == "C" || key == "c") {
    subscribeOdometryStream();
}

// 4.2 Unsubscribe odometry stream
else if (key == "V" || key == "v") {
    unsubscribeOdometryStream();
}

// 5. Close Functions
// 5.1 Close navigation
else if (key == "L" || key == "l") {
    closeNavigation();
}

// 5.2 Close SLAM
else if (key == "P" || key == "p") {
    closeSlam();
}

// Help
else if (key == "?") {
    printHelp();
} else {
    std::cout << "Unknown key: " << key << std::endl;
}

std::this_thread::sleep_for(std::chrono::milliseconds(10)); // Brief delay

} catch (const std::exception& e) {
    std::cerr << "Exception occurred while processing user input: " << e.what() <<
↵std::endl;
}

}

} catch (const std::exception& e) {
    std::cerr << "Exception occurred during program execution: " << e.what() << std::endl;
    return -1;
}

// Clean up resources
try {

```

(下页继续)

(续上页)

```
std::cout << "Clean up resources" << std::endl;

// Close SLAM navigation controller
auto& slamNavController = robot->GetSlamNavController();
slamNavController.Shutdown();
std::cout << "SLAM navigation controller closed" << std::endl;

// Disconnect
robot->Disconnect();
std::cout << "Robot connection disconnected" << std::endl;

// Shutdown robot
robot->Shutdown();
std::cout << "Robot shutdown" << std::endl;

} catch (const std::exception& e) {
    std::cerr << "Exception occurred while cleaning up resources: " << e.what() << std::endl;
}

return 0;
}
```

28.2 Python

示例文件：

- slam_example.py
- navigation_example.py

28.2.1 建图示例

```
#!/usr/bin/env python3

import tty
import termios
import sys
import time
import signal
import threading
import logging
from typing import Optional
import numpy as np
```

(下页继续)

(续上页)

```
import cv2
import random
import magicdog_w_python as magicdog_w

# Configure logging format and level
logging.basicConfig(
    level=logging.INFO, # Minimum log level
    format="%(asctime)s [%(levelname)s] %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
)

# Global variables
robot: Optional[magicdog_w.MagicRobot] = None
running = True
current_slam_mode = None
current_nav_mode = magicdog_w.NavMode.IDLE
high_controller = None

def signal_handler(signum, frame):
    """Signal handler function for graceful exit"""
    global running, robot
    logging.info("Received interrupt signal (%s), exiting...", signum)
    running = False
    if robot:
        robot.shutdown()
        logging.info("Robot shutdown")
    exit(-1)

def print_help():
    """Print help information"""
    logging.info("SLAM and Navigation Function Demo Program")
    logging.info("")
    logging.info("preparation Functions:")
    logging.info("  1          Function 1: Recovery stand")
    logging.info("")
    logging.info("SLAM Functions:")
    logging.info("  2          Function 2: Start mapping")
    logging.info("  3          Function 3: Cancel mapping")
    logging.info("  4          Function 4: Save map")
    logging.info("  5          Function 5: Load map")
    logging.info("  6          Function 6: Delete map")
```

(下页继续)

(续上页)

```

logging.info(
    " 7          Function 7: Get all map information and save map image as PGM file"
)
logging.info("")
logging.info("Joystick Functions:")
logging.info(" W          Function W: forward")
logging.info(" A          Function A: backward")
logging.info(" S          Function S: left")
logging.info(" D          Function D: right")
logging.info(" T          Function T: turn left")
logging.info(" G          Function G: turn right")
logging.info(" X          Function X: stop")
logging.info("")
logging.info("Close Functions:")
logging.info(" P          Function P: Close SLAM")
logging.info("")
logging.info(" ?          Function ?: Print help")
logging.info(" ESC        Exit program")

# ===== SLAM Functions =====

def load_map(map_to_load):
    """Load map"""
    global robot
    try:
        if not map_to_load:
            logging.error("Map to load is not provided")
            return
        # Get SLAM navigation controller
        controller = robot.get_slam_nav_controller()

        logging.info("Loading map: %s", map_to_load)
        controller.load_map(map_to_load)
    except Exception as e:
        logging.error("Exception occurred while loading map: %s", e)
        return

    logging.info("Successfully loaded map: %s", map_to_load)

def start_mapping():

```

(下页继续)

(续上页)

```

"""Start mapping"""
global robot
try:
    # Get SLAM navigation controller
    controller = robot.get_slam_nav_controller()

    # Start mapping
    status = controller.start_mapping()
    if status.code != magicdog_w.ErrorCode.OK:
        logging.error(
            "Failed to start mapping, code: %s, message: %s",
            status.code,
            status.message,
        )
        return

    logging.info("Successfully started mapping")
except Exception as e:
    logging.error("Exception occurred while starting mapping: %s", e)

def cancel_mapping():
    """Cancel mapping"""
    global robot
    try:
        # Get SLAM navigation controller
        controller = robot.get_slam_nav_controller()

        # Cancel mapping
        status = controller.cancel_mapping()
        if status.code != magicdog_w.ErrorCode.OK:
            logging.error(
                "Failed to cancel mapping, code: %s, message: %s",
                status.code,
                status.message,
            )
            return

        logging.info("Successfully cancelled mapping")
    except Exception as e:
        logging.error("Exception occurred while cancelling mapping: %s", e)

```

(下页继续)

(续上页)

```
def save_map():
    """Save map"""
    global robot, current_slam_mode
    try:
        # Get SLAM navigation controller
        controller = robot.get_slam_nav_controller()

        # Check if in mapping mode
        if current_slam_mode != "MAPPING":
            logging.warning(
                "Warning: Currently not in mapping mode, may not be able to save map"
            )

        # Generate map name with timestamp
        map_name = f"map_{int(time.time())}"
        logging.info("Saving map: %s", map_name)

        # Save map
        status = controller.save_map(map_name)
        if status.code != magicdog_w.ErrorCode.OK:
            logging.error(
                "Failed to save map, code: %s, message: %s",
                status.code,
                status.message,
            )
            return

        logging.info("Successfully saved map: %s", map_name)
    except Exception as e:
        logging.error("Exception occurred while saving map: %s", e)

def delete_map(map_to_delete):
    """Delete map"""
    global robot
    try:
        if not map_to_delete:
            logging.error("Map to delete is not provided")
            return

        # Get SLAM navigation controller
        controller = robot.get_slam_nav_controller()

        # Delete the first map as an example
```

(下页继续)

(续上页)

```

logging.info("Deleting map: %s", map_to_delete)

# Delete map
status = controller.delete_map(map_to_delete)
if status.code != magicdog_w.ErrorCode.OK:
    logging.error(
        "Failed to delete map, code: %s, message: %s",
        status.code,
        status.message,
    )
    return

logging.info("Successfully deleted map: %s", map_to_delete)
except Exception as e:
    logging.error("Exception occurred while deleting map: %s", e)

def get_all_map_info():
    """Get all map information"""
    global robot
    try:
        # Get SLAM navigation controller
        controller = robot.get_slam_nav_controller()

        # Get all map information
        status, all_map_info = controller.get_all_map_info()
        if status.code != magicdog_w.ErrorCode.OK:
            logging.error(
                "Failed to get map information, code: %s, message: %s",
                status.code,
                status.message,
            )
            return

        logging.info("Successfully retrieved map information")
        logging.info("Current map: %s", all_map_info.current_map_name)
        logging.info("Total maps: %d", len(all_map_info.map_infos))

        if all_map_info.map_infos:
            logging.info("Map details:")
            for i, map_info in enumerate(all_map_info.map_infos):
                logging.info("  Map %d: %s", i + 1, map_info.map_name)
                logging.info(

```

(下页继续)

(续上页)

```

        "    Origin: [%f, %f, %f]",
        map_info.map_meta_data.origin.position[0],
        map_info.map_meta_data.origin.position[1],
        map_info.map_meta_data.origin.position[2],
    )
    logging.info(
        "    Orientation: [%f, %f, %f]",
        map_info.map_meta_data.origin.orientation[0],
        map_info.map_meta_data.origin.orientation[1],
        map_info.map_meta_data.origin.orientation[2],
    )
    logging.info(
        "    Resolution: %f m/pixel", map_info.map_meta_data.resolution
    )
    logging.info(
        "    Size: %d x %d",
        map_info.map_meta_data.map_image_data.width,
        map_info.map_meta_data.map_image_data.height,
    )
    logging.info(
        "    Max gray value: %d",
        map_info.map_meta_data.map_image_data.max_gray_value,
    )
    logging.info(
        "    Image type: %s", map_info.map_meta_data.map_image_data.type
    )

    save_map_image_to_file(map_info)

else:
    logging.info("No available maps")
except Exception as e:
    logging.error("Exception occurred while getting map information: %s", e)

def save_map_image_to_file(map_info):
    """Save map image to current directory"""
    try:
        # Extract image data
        map_data = map_info.map_meta_data.map_image_data
        width = map_data.width
        height = map_data.height
        max_gray_value = map_data.max_gray_value

```

(下页继续)

(续上页)

```

image_bytes = map_data.image

logging.info(
    "Saving map image: %dx%d, max_gray: %d", width, height, max_gray_value
)

# Convert bytes to numpy array
if len(image_bytes) != width * height:
    logging.error(
        "Image data size mismatch: expected %d, got %d",
        width * height,
        len(image_bytes),
    )
    return

# Convert Uint8Vector to bytes for numpy processing
try:
    # Try to convert Uint8Vector to bytes
    if hasattr(image_bytes, "__iter__") and not isinstance(
        image_bytes, (str, bytes)
    ):
        # Convert Uint8Vector or similar iterable to bytes
        image_bytes_data = bytes(image_bytes)
    else:
        image_bytes_data = image_bytes
except Exception as e:
    logging.error("Failed to convert image data to bytes: %s", e)
    return

# Create numpy array from image data
image_array = np.frombuffer(image_bytes_data, dtype=np.uint8).reshape(
    (height, width)
)

# Generate filename based on map name
safe_filename = "".join(
    c for c in map_info.map_name if c.isalnum() or c in (" ", "-", "_")
).rstrip()
safe_filename = safe_filename.replace(" ", "_")
if not safe_filename:
    safe_filename = f"map_{int(time.time())}"

# Save as PGM format using OpenCV

```

(下页继续)

(续上页)

```

pgm_filename = f"build/{safe_filename}.pgm"
success = cv2.imwrite(pgm_filename, image_array)

if success:
    logging.info("Map image saved successfully as PGM: %s", pgm_filename)
else:
    logging.error("Failed to save map image as PGM: %s", pgm_filename)

except ImportError:
    logging.error("OpenCV not available, cannot save image")
    logging.info(
        "Image data: %dx%d pixels, %d bytes", width, height, len(image_bytes)
    )
except Exception as e:
    logging.error("Exception occurred while saving map image: %s", e)

def close_slam():
    """Close SLAM system"""
    global robot, current_slam_mode
    try:
        # Get SLAM navigation controller
        controller = robot.get_slam_nav_controller()

        # Switch to idle mode to close SLAM
        status = controller.switch_to_idle()
        if status.code != magicdog_w.ErrorCode.OK:
            logging.error(
                "Failed to close SLAM, code: %s, message: %s",
                status.code,
                status.message,
            )
            return

        current_slam_mode = "IDLE"
        logging.info("Successfully closed SLAM system")
    except Exception as e:
        logging.error("Exception occurred while closing SLAM: %s", e)

# Switch gait to down climb stairs mode
def change_gait_to_down_climb_stairs():
    global high_controller

```

(下页继续)

(续上页)

```

try:
    current_gait = high_controller.get_gait()
    if current_gait != magicdog_w.GaitMode.GAIT_RL_TERRAIN:
        status = high_controller.set_gait(magicdog_w.GaitMode.GAIT_RL_TERRAIN)
        if status.code != magicdog_w.ErrorCode.OK:
            logging.error(f"Failed to set down climb stairs gait: {status.message}")
            return False

        # Wait for gait switch to complete
        while high_controller.get_gait() != magicdog_w.GaitMode.GAIT_RL_TERRAIN:
            time.sleep(0.01)
        logging.info("Gait changed to down climb stairs")
        return True
except Exception as e:
    logging.error(f"Error changing gait: {e}")
    return False

def send_joystick_command(high_controller, left_x, left_y, right_x, right_y):
    if not change_gait_to_down_climb_stairs():
        return

    joy_command = magicdog_w.JoystickCommand()
    joy_command.left_x_axis = left_x
    joy_command.left_y_axis = left_y
    joy_command.right_x_axis = right_x
    joy_command.right_y_axis = right_y

    status = high_controller.send_joystick_command(joy_command)
    if status.code != magicdog_w.ErrorCode.OK:
        logging.error(f"Failed to send joystick command: {status.message}")

# Position control standing
def recovery_stand(high_controller):
    try:
        # Set gait to position control standing
        status = high_controller.set_gait(magicdog_w.GaitMode.GAIT_STAND_R)
        if status.code != magicdog_w.ErrorCode.OK:
            logging.error(f"Failed to set position control standing: {status.message}")
        else:
            logging.info("Robot set to position control standing")
    except Exception as e:
        logging.error(f"Error executing position control standing: {e}")

```

(下页继续)

(续上页)

```
# Get single character input (no echo)
def getch():
    fd = sys.stdin.fileno()
    old_settings = termios.tcgetattr(fd)
    try:
        tty.setraw(sys.stdin.fileno())
        ch = sys.stdin.read(1)
        logging.info(f"Received character: {ch}")

        sys.stdout.write("\r")
        sys.stdout.flush()
    finally:
        termios.tcsetattr(fd, termios.TCSADRAIN, old_settings)
    return ch

# ===== Utility Functions =====

def main():
    """Main function"""
    global robot, running
    global high_controller

    # Bind signal handler
    signal.signal(signal.SIGINT, signal_handler)

    # Create robot instance
    robot = magicrobot_w.MagicRobot()

    print_help()
    logging.info("Press any key to continue (ESC to exit)...")

    try:
        # Configure local IP address for direct network connection and initialize SDK
        local_ip = "192.168.55.10"
        if not robot.initialize(local_ip):
            logging.error("Failed to initialize robot SDK")
            robot.shutdown()
            return -1
```

(下页继续)

(续上页)

```
# Connect to robot
status = robot.connect()
if status.code != magicdog_w.ErrorCode.OK:
    logging.error(
        "Failed to connect to robot, code: %s, message: %s",
        status.code,
        status.message,
    )
    robot.shutdown()
    return -1

logging.info("Successfully connected to robot")

# Set motion control level to high level
status = robot.set_motion_control_level(magicdog_w.ControllerLevel.HIGH_LEVEL)
if status.code != magicdog_w.ErrorCode.OK:
    logging.error(f"Failed to set motion control level: {status.message}")
    robot.shutdown()
    return

# Get high level motion controller
high_controller = robot.get_high_level_motion_controller()

# Initialize SLAM navigation controller
slam_nav_controller = robot.get_slam_nav_controller()
if not slam_nav_controller.initialize():
    logging.error("Failed to initialize SLAM navigation controller")
    robot.disconnect()
    robot.shutdown()
    return -1

logging.info("Successfully initialized SLAM navigation controller")

# Main loop
while running:
    try:
        logging.info("Enter command: ")
        key = getch()
        if key == "\x1b": # ESC key
            break

        # 1. Preparation Functions
        # 1.1 Recovery stand
```

(下页继续)

(续上页)

```

if key == "1":
    # recovery stand
    recovery_stand(high_controller)
# 2. SLAM Functions
# 2.1 Start mapping
elif key == "2":
    start_mapping()
# 2.2 Cancel mapping
elif key == "3":
    cancel_mapping()
# 2.3 Save map
elif key == "4":
    save_map()
# 2.4 Load map
elif key == "5":
    str_input = input("Enter parameters: ").strip()
    if not str_input:
        continue
    # Split input parameters by space
    parts = str_input.strip().split()
    # Parse parameters
    map_to_load = parts[0]
    load_map(map_to_load)
# 2.5 Delete map
elif key == "6":
    str_input = input("Enter parameters: ").strip()
    if not str_input:
        continue
    # Split input parameters by space
    parts = str_input.strip().split()
    # Parse parameters
    map_to_delete = parts[0]
    delete_map(map_to_delete)
# 2.6 Get all map information
elif key == "7":
    get_all_map_info()
# 3. Joystick Functions
# 3.1 Move forward
elif key.lower() == "w":
    left_y = 1.0
    left_x = 0.0
    right_x = 0.0
    right_y = 0.0

```

(下页继续)

(续上页)

```

        send_joystick_command(
            high_controller, left_x, left_y, right_x, right_y
        )
# 3.2 Move backward
elif key.lower() == "s":
    left_y = -1.0
    left_x = 0.0
    right_x = 0.0
    right_y = 0.0
    send_joystick_command(
        high_controller, left_x, left_y, right_x, right_y
    )
# 3.3 Move left
elif key.lower() == "a":
    left_x = -1.0
    left_y = 0.0
    right_x = 0.0
    right_y = 0.0
    send_joystick_command(
        high_controller, left_x, left_y, right_x, right_y
    )
# 3.4 Move right
elif key.lower() == "d":
    left_x = 1.0
    left_y = 0.0
    right_x = 0.0
    right_y = 0.0
    send_joystick_command(
        high_controller, left_x, left_y, right_x, right_y
    )
# 3.5 Turn left
elif key.lower() == "t":
    left_x = 0.0
    left_y = 0.0
    right_x = -1.0
    right_y = 0.0
    send_joystick_command(
        high_controller, left_x, left_y, right_x, right_y
    )
# 3.6 Turn right
elif key.lower() == "g":
    left_x = 0.0
    left_y = 0.0

```

(下页继续)

(续上页)

```

        right_x = 1.0
        right_y = 0.0
        send_joystick_command(
            high_controller, left_x, left_y, right_x, right_y
        )
        # 3.7 Stop movement
        elif key.lower() == "x":
            left_x = 0.0
            left_y = 0.0
            right_x = 0.0
            right_y = 0.0
            send_joystick_command(
                high_controller, left_x, left_y, right_x, right_y
            )
        # 4. Close Functions
        # 4.1 Close SLAM
        elif key.upper() == "P":
            close_slam()
        # Help
        elif key.upper() == "?":
            print_help()
        else:
            logging.warning("Unknown key: %s", key)

        time.sleep(0.01) # Brief delay

    except KeyboardInterrupt:
        break
    except Exception as e:
        logging.error("Exception occurred while processing user input: %s", e)
except Exception as e:
    logging.error("Exception occurred during program execution: %s", e)
return -1

finally:
    # Clean up resources
    try:
        logging.info("Clean up resources")
        # Close SLAM navigation controller
        slam_nav_controller = robot.get_slam_nav_controller()
        slam_nav_controller.shutdown()
        logging.info("SLAM navigation controller closed")
    
```

(下页继续)

(续上页)

```

        # Disconnect
        robot.disconnect()
        logging.info("Robot connection disconnected")

        # Shutdown robot
        robot.shutdown()
        logging.info("Robot shutdown")

    except Exception as e:
        logging.error("Exception occurred while cleaning up resources: %s", e)

if __name__ == "__main__":
    sys.exit(main())

```

28.2.2 导航示例

```

#!/usr/bin/env python3

import sys
import time
import signal
import threading
import logging
from typing import Optional
import numpy as np
import cv2
import random

import magicdog_w_python as magicdog_w

# Configure logging format and level
logging.basicConfig(
    level=logging.INFO, # Minimum log level
    format="%(asctime)s [%(levelname)s] %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
)

# Global variables
robot: Optional[magicdog_w.MagicRobot] = None
running = True
current_slam_mode = None

```

(下页继续)

(续上页)

```
current_nav_mode = magicdog_w.NavMode.IDLE
high_controller = None
odometry_counter = 0

def signal_handler(signum, frame):
    """Signal handler function for graceful exit"""
    global running, robot
    logging.info("Received interrupt signal (%s), exiting...", signum)
    running = False
    if robot:
        robot.shutdown()
        logging.info("Robot shutdown")
    exit(-1)

def print_help():
    """Print help information"""
    logging.info("SLAM and Navigation Function Demo Program")
    logging.info("")
    logging.info("preparation Functions:")
    logging.info(" 1      Function 1: Recovery stand")
    logging.info("")
    logging.info("Localization Functions:")
    logging.info(" 2      Function 2: Switch to localization mode")
    logging.info(" 4      Function 4: Initialize pose")
    logging.info(" 5      Function 5: Get current pose information")
    logging.info("")
    logging.info("Navigation Functions:")
    logging.info(" 3      Function 3: Switch to navigation mode")
    logging.info(" 6      Function 6: Set navigation target goal")
    logging.info(" 7      Function 7: Pause navigation")
    logging.info(" 8      Function 8: Resume navigation")
    logging.info(" 9      Function 9: Cancel navigation")
    logging.info(" 0      Function 0: Get navigation status")
    logging.info("")
    logging.info("Odometry Functions:")
    logging.info(" C      Function C: Subscribe odometry stream")
    logging.info(" V      Function V: Unsubscribe odometry stream")
    logging.info("")
    logging.info("Close Functions:")
    logging.info(" L      Function L: Close navigation")
    logging.info(" P      Function P: Close SLAM")
```

(下页继续)

(续上页)

```

logging.info("")
logging.info(" ?          Function?: Print help")
logging.info(" ESC       Exit program")

# Recovery stand
def recovery_stand(high_controller):
    try:
        # Set gait to position control standing
        status = high_controller.set_gait(magicdog_w.GaitMode.GAIT_STAND_R)
        if status.code != magicdog_w.ErrorCode.OK:
            logging.error(f"Failed to set position control standing: {status.message}")
        else:
            logging.info("Robot set to position control standing")
    except Exception as e:
        logging.error(f"Error executing position control standing: {e}")

# switch to down climb stairs gait
def switch_to_down_climb_stairs_gait(high_controller):
    try:
        status = high_controller.set_gait(magicdog_w.GaitMode.GAIT_RL_TERRAIN)
        if status.code != magicdog_w.ErrorCode.OK:
            logging.error(f"Failed to set down climb stairs gait: {status.message}")
        else:
            logging.info("Successfully set down climb stairs gait")
    except Exception as e:
        logging.error(f"Error setting down climb stairs gait: {e}")

def switch_to_localization_mode():
    """Switch to localization mode"""
    global robot, current_slam_mode
    try:
        # Get SLAM navigation controller
        controller = robot.get_slam_nav_controller()

        # Switch to localization mode
        status = controller.switch_to_location()
        if status.code != magicdog_w.ErrorCode.OK:
            logging.error(
                "Failed to switch to localization mode, code: %s, message: %s",
                status.code,

```

(下页继续)

(续上页)

```

        status.message,
    )
    return

    current_slam_mode = "LOCALIZATION"
    logging.info("Successfully switched to localization mode")
    logging.info(
        "Robot is now in localization mode, ready to localize on existing maps"
    )
except Exception as e:
    logging.error("Exception occurred while switching to localization mode: %s", e)

def initialize_pose(x, y, yaw):
    """Initialize pose"""
    global robot
    try:
        # Get SLAM navigation controller
        controller = robot.get_slam_nav_controller()

        # Create initial pose (set to origin)
        initial_pose = magicdog_w.Pose3DEuler()
        initial_pose.position = [x, y, 0.0] # x, y, z
        initial_pose.orientation = [0.0, 0.0, yaw] # roll, pitch, yaw

        logging.info("Initializing robot pose to origin...")

        # Initialize pose
        status = controller.init_pose(initial_pose)
        if status.code != magicdog_w.ErrorCode.OK:
            logging.error(
                "Failed to initialize pose, code: %s, message: %s",
                status.code,
                status.message,
            )
            return

        logging.info("Successfully initialized pose")
        logging.info("Robot pose has been set to origin (0, 0, 0)")
    except Exception as e:
        logging.error("Exception occurred while initializing pose: %s", e)

```

(下页继续)

(续上页)

```
def get_current_localization_info():
    """Get current pose information"""
    global robot
    try:
        # Get SLAM navigation controller
        controller = robot.get_slam_nav_controller()

        # Get current pose information
        status, localization_info = controller.get_current_localization_info()
        if status.code != magicdog_w.ErrorCode.OK:
            logging.error(
                "Failed to get current pose information, code: %s, message: %s",
                status.code,
                status.message,
            )
            return

        logging.info("Successfully retrieved current pose information")
        logging.info(
            "Localization status: %s",
            "Localized" if localization_info.is_localization else "Not localized",
        )
        logging.info(
            "Position: [%.3f, %.3f, %.3f]",
            localization_info.pose.position[0],
            localization_info.pose.position[1],
            localization_info.pose.position[2],
        )
        logging.info(
            "Orientation: [%.3f, %.3f, %.3f]",
            localization_info.pose.orientation[0],
            localization_info.pose.orientation[1],
            localization_info.pose.orientation[2],
        )
    except Exception as e:
        logging.error(
            "Exception occurred while getting current pose information: %s", e
        )

def switch_to_navigation_mode():
    """Switch to navigation mode"""
    global robot, current_nav_mode
```

(下页继续)

(续上页)

```

try:
    # Disable joy stick
    high_controller.disable_joy_stick()
    logging.info("Successfully disabled joy stick")
    time.sleep(1)

    # Get SLAM navigation controller
    controller = robot.get_slam_nav_controller()

    # Switch to navigation mode
    status = controller.activate_nav_mode(magicdog_w.NavMode.GRID_MAP)
    if status.code != magicdog_w.ErrorCode.OK:
        logging.error(
            "Failed to switch to navigation mode, code: %s, message: %s",
            status.code,
            status.message,
        )
        return

    current_nav_mode = magicdog_w.NavMode.GRID_MAP
    logging.info("Successfully switched to navigation mode")
except Exception as e:
    logging.error("Exception occurred while switching to navigation mode: %s", e)

pose_id = 0

def set_navigation_target(x, y, yaw):
    """Set navigation target goal"""
    global robot, high_controller
    global pose_id
    try:
        # Get SLAM navigation controller
        controller = robot.get_slam_nav_controller()

        # Disable joy stick
        high_controller.disable_joy_stick()
        logging.info("Successfully disabled joy stick")

        # change gait to down climb stairs
        # To use Navigation, you must switch to the down stairs gait
        switch_to_down_climb_stairs_gait(high_controller)
    
```

(下页继续)

(续上页)

```

    # Create target goal
    target_goal = magicdog_w.NavTarget()
    target_goal.id = pose_id
    pose_id = pose_id + 1
    target_goal.frame_id = "map"

    target_goal.goal.position = [x, y, 0.0]
    target_goal.goal.orientation = [0.0, 0.0, yaw]

    # Set target goal
    status = controller.set_nav_target(target_goal)
    if status.code != magicdog_w.ErrorCode.OK:
        logging.error(
            "Failed to set navigation target, code: %s, message: %s",
            status.code,
            status.message,
        )
        return

    logging.info(
        "Successfully set navigation target: position=(%.2f, %.2f, %.2f), orientation=(%.2f,
↪ %.2f, %.2f)",
        target_goal.goal.position[0],
        target_goal.goal.position[1],
        target_goal.goal.position[2],
        target_goal.goal.orientation[0],
        target_goal.goal.orientation[1],
        target_goal.goal.orientation[2],
    )
except Exception as e:
    logging.error("Exception occurred while setting navigation target: %s", e)

def pause_navigation():
    """Pause navigation"""
    global robot
    try:
        # Get SLAM navigation controller
        controller = robot.get_slam_nav_controller()

        # Pause navigation
        status = controller.pause_nav_task()

```

(下页继续)

(续上页)

```

    if status.code != magicdog_w.ErrorCode.OK:
        logging.error(
            "Failed to pause navigation, code: %s, message: %s",
            status.code,
            status.message,
        )
        return

    logging.info("Successfully paused navigation")
except Exception as e:
    logging.error("Exception occurred while pausing navigation: %s", e)

def resume_navigation():
    """Resume navigation"""
    global robot
    try:
        # Get SLAM navigation controller
        controller = robot.get_slam_nav_controller()

        # Resume navigation
        status = controller.resume_nav_task()
        if status.code != magicdog_w.ErrorCode.OK:
            logging.error(
                "Failed to resume navigation, code: %s, message: %s",
                status.code,
                status.message,
            )
            return

        logging.info("Successfully resumed navigation")
    except Exception as e:
        logging.error("Exception occurred while resuming navigation: %s", e)

def cancel_navigation():
    """Cancel navigation"""
    global robot
    try:
        # Get SLAM navigation controller
        controller = robot.get_slam_nav_controller()

        # Cancel navigation

```

(下页继续)

(续上页)

```

status = controller.cancel_nav_task()
if status.code != magicdog_w.ErrorCode.OK:
    logging.error(
        "Failed to cancel navigation, code: %s, message: %s",
        status.code,
        status.message,
    )
    return

logging.info("Successfully cancelled navigation")
except Exception as e:
    logging.error("Exception occurred while cancelling navigation: %s", e)

def get_navigation_status():
    """Get current navigation status"""
    global robot
    try:
        # Get SLAM navigation controller
        controller = robot.get_slam_nav_controller()

        # Get navigation status
        status, nav_status = controller.get_nav_task_status()
        if status.code != magicdog_w.ErrorCode.OK:
            logging.error(
                "Failed to get navigation status, code: %s, message: %s",
                status.code,
                status.message,
            )
            return

        # Display navigation status information
        logging.info("=== Navigation Status ===")
        logging.info("Target ID: %d", nav_status.id)
        logging.info("Status: %s", nav_status.status)
        logging.info("Message: %s", nav_status.message)

        # Provide status interpretation
        status_meaning = {
            magicdog_w.NavStatusType.NONE: "No navigation target set",
            magicdog_w.NavStatusType.RUNNING: "Navigation is running",
            magicdog_w.NavStatusType.END_SUCCESS: "Navigation completed successfully",
            magicdog_w.NavStatusType.END_FAILED: "Navigation failed",
        }

```

(下页继续)

(续上页)

```

        magicdog_w.NavStatusType.PAUSE: "Navigation is paused",
    }

    if nav_status.status in status_meaning:
        logging.info("Status meaning: %s", status_meaning[nav_status.status])
    else:
        logging.warning("Unknown status value: %s", nav_status.status)

    logging.info("=====")

    except Exception as e:
        logging.error("Exception occurred while getting navigation status: %s", e)

def close_navigation():
    """Close navigation system"""
    global robot, current_nav_mode
    try:
        # Get SLAM navigation controller
        controller = robot.get_slam_nav_controller()

        # Switch to idle mode to close navigation
        status = controller.activate_nav_mode(magicdog_w.NavMode.IDLE)
        if status.code != magicdog_w.ErrorCode.OK:
            logging.error(
                "Failed to close navigation, code: %s, message: %s",
                status.code,
                status.message,
            )
            return

        current_nav_mode = magicdog_w.NavMode.IDLE
        logging.info("Successfully closed navigation system")
    except Exception as e:
        logging.error("Exception occurred while closing navigation: %s", e)

def open_odometry_stream():
    """Open odometry stream"""
    global robot
    try:
        # Get SLAM navigation controller
        controller = robot.get_slam_nav_controller()

```

(下页继续)

(续上页)

```

        # Open odometry stream
        status = controller.open_odometry_stream()
        if status.code != magicdog_w.ErrorCode.OK:
            logging.error(
                "Failed to open odometry stream, code: %s, message: %s",
                status.code,
                status.message,
            )
            return
        logging.info("Successfully opened odometry stream")
    except Exception as e:
        logging.error("Exception occurred while opening odometry stream: %s", e)

def close_odometry_stream():
    """Close odometry stream"""
    global robot
    try:
        # Get SLAM navigation controller
        controller = robot.get_slam_nav_controller()

        # Close odometry stream
        status = controller.close_odometry_stream()
        if status.code != magicdog_w.ErrorCode.OK:
            logging.error(
                "Failed to close odometry stream, code: %s, message: %s",
                status.code,
                status.message,
            )
            return
        logging.info("Successfully closed odometry stream")
    except Exception as e:
        logging.error("Exception occurred while closing odometry stream: %s", e)

def subscribe_odometry_stream():
    """Subscribe odometry stream"""
    global robot
    try:
        # Open channel switch
        robot.open_channel_switch()
        # Get SLAM navigation controller

```

(下页继续)

(续上页)

```

controller = robot.get_slam_nav_controller()

def odometry_callback(data):
    global odometry_counter
    if odometry_counter % 10 == 0:
        logging.info(
            "Odometry position data: %f, %f, %f",
            data.position[0],
            data.position[1],
            data.position[2],
        )
        logging.info(
            "Odometry orientation data: %f, %f, %f, %f",
            data.orientation[0],
            data.orientation[1],
            data.orientation[2],
            data.orientation[3],
        )
        logging.info(
            "Odometry linear velocity data: %f, %f, %f",
            data.linear_velocity[0],
            data.linear_velocity[1],
            data.linear_velocity[2],
        )
        logging.info(
            "Odometry angular velocity data: %f, %f, %f",
            data.angular_velocity[0],
            data.angular_velocity[1],
            data.angular_velocity[2],
        )
        odometry_counter += 1

    # Subscribe odometry stream
    controller.subscribe_odometry(odometry_callback)
    logging.info("Successfully subscribed odometry stream")
except Exception as e:
    logging.error("Exception occurred while subscribing odometry stream: %s", e)

def unsubscribe_odometry_stream():
    """Unsubscribe odometry stream"""
    global robot
    try:

```

(下页继续)

(续上页)

```

    # Get SLAM navigation controller
    controller = robot.get_slam_nav_controller()
    robot.close_channel_switch() # Close channel switch

    # Unsubscribe odometry stream
    controller.unsubscribe_odometry()
    logging.info("Successfully unsubscribed odometry stream")
except Exception as e:
    logging.error("Exception occurred while unsubscribing odometry stream: %s", e)

def disable_joy_stick(high_controller):
    """Disable joy stick"""
    high_controller.disable_joy_stick()
    logging.info("Successfully disabled joy stick")

def close_slam():
    """Close SLAM system"""
    global robot, current_slam_mode
    try:
        # Get SLAM navigation controller
        controller = robot.get_slam_nav_controller()

        # Switch to idle mode to close SLAM
        status = controller.switch_to_idle()
        if status.code != magicdog_w.ErrorCode.OK:
            logging.error(
                "Failed to close SLAM, code: %s, message: %s",
                status.code,
                status.message,
            )
            return

        current_slam_mode = "IDLE"
        logging.info("Successfully closed SLAM system")
    except Exception as e:
        logging.error("Exception occurred while closing SLAM: %s", e)

# ===== Utility Functions =====

```

(下页继续)

(续上页)

```
def get_user_input():
    """Get user input - Read a line of data"""
    try:
        # Method 1: Use input() to read a line (recommended)
        return input("Enter command: ").strip()
    except (EOFError, KeyboardInterrupt):
        return ""

def main():
    """Main function"""
    global robot, running
    global high_controller

    # Bind signal handler
    signal.signal(signal.SIGINT, signal_handler)

    # Create robot instance
    robot = magicdog_w.MagicRobot()

    print_help()
    logging.info("Press any key to continue (ESC to exit)...")

    try:
        # Configure local IP address for direct network connection and initialize SDK
        local_ip = "192.168.55.10"
        if not robot.initialize(local_ip):
            logging.error("Failed to initialize robot SDK")
            robot.shutdown()
            return -1

        # Connect to robot
        status = robot.connect()
        if status.code != magicdog_w.ErrorCode.OK:
            logging.error(
                "Failed to connect to robot, code: %s, message: %s",
                status.code,
                status.message,
            )
            robot.shutdown()
            return -1

        logging.info("Successfully connected to robot")
```

(下页继续)

(续上页)

```
# Set motion control level to high level
status = robot.set_motion_control_level(magicdog_w.ControllerLevel.HIGH_LEVEL)
if status.code != magicdog_w.ErrorCode.OK:
    logging.error(f"Failed to set motion control level: {status.message}")
    robot.shutdown()
    return

# Get high level motion controller
high_controller = robot.get_high_level_motion_controller()

# Initialize SLAM navigation controller
slam_nav_controller = robot.get_slam_nav_controller()
if not slam_nav_controller.initialize():
    logging.error("Failed to initialize SLAM navigation controller")
    robot.disconnect()
    robot.shutdown()
    return -1

logging.info("Successfully initialized SLAM navigation controller")

# Main loop
while running:
    try:
        str_input = get_user_input()

        # Split input parameters by space
        parts = str_input.strip().split()

        if not parts:
            time.sleep(0.01) # Brief delay
            continue

        # Parse parameters
        key = parts[0]
        args = parts[1:] if len(parts) > 1 else []
        if key == "\x1b": # ESC key
            break

        # 1. Preparation Functions
        if key.upper() == "1":
            # recovery stand
            recovery_stand(high_controller)
```

(下页继续)

(续上页)

```
# 2. Localization Functions
# 2.1 Switch to localization mode
elif key == "2":
    switch_to_localization_mode()
# 2.2 Initialize pose
elif key == "4":
    x = float(args[0]) if args else 0.0
    y = float(args[1]) if args else 0.0
    yaw = float(args[2]) if args else 0.0
    logging.info("input initial pose, x: %f, y: %f, yaw: %f", x, y, yaw)
    initialize_pose(x, y, yaw)
# 2.3 Get current localization information
elif key == "5":
    get_current_localization_info()
# 3. Navigation Functions
# 3.1 Switch to navigation mode
elif key.upper() == "3":
    switch_to_navigation_mode()
# 3.2 Set navigation target
elif key.upper() == "6":
    x = float(args[0]) if args else 0.0
    y = float(args[1]) if args else 0.0
    yaw = float(args[2]) if args else 0.0
    logging.info(
        "input navigation target, x: %f, y: %f, yaw: %f", x, y, yaw
    )
    set_navigation_target(x, y, yaw)
# 3.3 Pause navigation
elif key.upper() == "7":
    pause_navigation()
# 3.4 Resume navigation
elif key.upper() == "8":
    resume_navigation()
# 3.5 Cancel navigation
elif key.upper() == "9":
    cancel_navigation()
# 3.6 Get navigation status
elif key.upper() == "0":
    get_navigation_status()
# 4. Odometry Functions
# 4.1 Subscribe odometry stream
elif key.upper() == "C":
    subscribe_odometry_stream()
```

(下页继续)

(续上页)

```

# 4.2 Unsubscribe odometry stream
elif key.upper() == "V":
    unsubscribe_odometry_stream()

# 5. Close Functions
# 5.1 Close navigation
elif key.upper() == "L":
    close_navigation()

# 5.2 Close SLAM
elif key.upper() == "P":
    close_slam()

# Help
elif key.upper() == "?":
    print_help()
else:
    logging.warning("Unknown key: %s", key)

time.sleep(0.01) # Brief delay

except KeyboardInterrupt:
    break
except Exception as e:
    logging.error("Exception occurred while processing user input: %s", e)
except Exception as e:
    logging.error("Exception occurred during program execution: %s", e)
return -1

finally:
    # Clean up resources
    try:
        logging.info("Clean up resources")
        # Close SLAM navigation controller
        slam_nav_controller = robot.get_slam_nav_controller()
        slam_nav_controller.shutdown()
        logging.info("SLAM navigation controller closed")

        # Disconnect
        robot.disconnect()
        logging.info("Robot connection disconnected")

        # Shutdown robot
        robot.shutdown()
        logging.info("Robot shutdown")

```

(下页继续)

(续上页)

```
except Exception as e:
    logging.error("Exception occurred while cleaning up resources: %s", e)

if __name__ == "__main__":
    sys.exit(main())
```

28.3 运行说明

28.3.1 环境准备

```
export PYTHONPATH=/opt/magic_robotics/magickdog_w_sdk/lib:$PYTHONPATH
export LD_LIBRARY_PATH=/opt/magic_robotics/magickdog_w_sdk/lib:$LD_LIBRARY_PATH
```

28.3.2 运行示例

```
# C++
./slam_example
./navigation_example
# Python
python3 slam_example.py
python3 navigation_example.py
```

28.3.3 控制说明

建图示例

- 准备工作：
 - 按键 1 - 恢复站立
- SLAM 功能：
 - 按键 2 - 开始建图
 - 按键 3 - 取消建图
 - 按键 4 - 保存地图
 - 按键 5 - 加载地图
 - 按键 6 - 删除地图
 - 按键 7 - 获取所有地图信息并保存为 PGM 文件
- 手柄控制功能：

- 按键 W - 前进
- 按键 A - 后退
- 按键 S - 左移
- 按键 D - 右移
- 按键 T - 左转
- 按键 G - 右转
- 按键 X - 停止
- 关闭功能：
 - 按键 P - 关闭 SLAM
- 系统命令：
 - 按键 ? - 显示帮助信息

导航

- 准备工作：
 - 按键 1 - 恢复站立
- 定位功能：
 - 按键 2 - 切换到定位模式
 - 按键 4 - 初始化位姿
 - 按键 5 - 获取当前位置信息
- 导航功能：
 - 按键 3 - 切换到导航模式
 - 按键 6 - 设置导航目标点
 - 按键 7 - 暂停导航
 - 按键 8 - 恢复导航
 - 按键 9 - 取消导航
 - 按键 0 - 获取导航状态
- 里程计功能：
 - 按键 C - 订阅里程计流
 - 按键 V - 取消订阅里程计流
- 关闭功能：
 - 按键 L - 关闭导航

- 按键 P - 关闭 SLAM
- 系统命令：
 - 按键 ? - 显示帮助信息

28.3.4 停止程序

- 按 ESC 可以安全停止程序
- 程序会自动清理所有资源并保存当前状态

28.3.5 注意事项

- 建图示例中，至少需要切换到 Recovery_stand 才可以进行建图，可以在 Recovery_stand 模式下推着机器人进行建图，也可以在 Balance_stand 模式下遥控控制机器人运动进行建图；
- 建图示例中，可以通过 GetAllMapInfo 获取当前机器人上存储地图的信息（包括二维地图 pgm, 地图大小，分辨率等信息）；
- 建图示例中，可以通过 SaveMap/DeleteMap 进行地图管理，机器人上存储地图尽量不要超过 3 张地图；
- 导航示例中，需要提前将机器人切换到 Balance_stand 模式；
- 导航示例中，导航目标点最好是基于 GetCurrentLocalizationInfo 获取的当前定位状态和当前定位位姿进行设置；
- 导航示例中，进行导航任务前需要正常切换到定位模式 (SwitchToLocation()) 和导航模式 (ActivateNavMode (NavMode::GRID_MAP))；
- 导航示例中，定位模式切换和导航模式切换都需要传入地图的绝对路径，可以通过 GetMapPath 获取指定地图的路径信息；
- 建图或者导航结束后，需要切换将 SLAM 切换到 IDLE 模式 (SwitchToIdle())，导航也切换到 IDLE 模式 (ActivateNavMode (NavMode::IDLE))

显示示例

该示例展示了如何使用 MagicDog SDK 进行初始化、连接机器人、显示控制等基本操作。

29.1 C++

示例文件: display_example.cpp

```
#include "magic_robot.h"
#include "magic_sdk_version.h"

#include <termios.h>
#include <unistd.h>
#include <chrono>
#include <csignal>
#include <cstdlib>
#include <iostream>
#include <memory>
#include <thread>

using namespace magic::dog_w;

// Global robot instance
std::unique_ptr<MagicRobot> robot = nullptr;
```

(下页继续)

(续上页)

```

void signalHandler(int signum) {
    std::cout << "\nInterrupt signal ( " << signum << " ) received." << std::endl;
    if (robot) {
        robot->Shutdown();
    }
    exit(signum);
}

void print_help() {
    std::cout << "Key Function Demo Program\n"
                << std::endl;
    std::cout << "Key Function Description:" << std::endl;
    std::cout << "Face expression Functions:" << std::endl;
    std::cout << " 1      Function 1: Get all face expressions" << std::endl;
    std::cout << " 2      Function 2: Set face expression" << std::endl;
    std::cout << " 3      Function 3: Get current face expression" << std::endl;
    std::cout << "" << std::endl;
    std::cout << " ?      Function ?: Print help" << std::endl;
    std::cout << " ESC    Exit program" << std::endl;
}

int getch() {
    struct termios oldt, newt;
    int ch;
    tcgetattr(STDIN_FILENO, &oldt); // Get current terminal settings
    newt = oldt;
    newt.c_lflag &= ~(ICANON | ECHO); // Disable buffering and echo
    tcsetattr(STDIN_FILENO, TCSANOW, &newt);
    ch = getchar(); // Read key press
    tcsetattr(STDIN_FILENO, TCSANOW, &oldt); // Restore settings
    return ch;
}

void get_all_face_expressions() {
    auto& display_controller = robot->GetDisplayController();

    std::vector<FaceExpression> face_expressions;
    auto status = display_controller.GetAllFaceExpressions(face_expressions);
    if (status.code != ErrorCode::OK) {
        std::cerr << "get all face expressions failed, code: " << status.code
                    << ", message: " << status.message << std::endl;
        return;
    }
}

```

(下页继续)

(续上页)

```

    for (auto& face_expression : face_expressions) {
        std::cout << "Face Expression ID: " << face_expression.id << std::endl;
        std::cout << "Face Expression Name: " << face_expression.name << std::endl;
        std::cout << "Face Expression Description: " << face_expression.description << std::endl;
        std::cout << std::endl;
    }
}

void get_current_face_expression() {
    auto& display_controller = robot->GetDisplayController();

    FaceExpression face_expression;
    auto status = display_controller.GetFaceExpression(face_expression);
    if (status.code != ErrorCode::OK) {
        std::cerr << "get current face expression failed, code: " << status.code
                    << ", message: " << status.message << std::endl;
        return;
    }

    std::cout << "Face Expression ID: " << face_expression.id << std::endl;
    std::cout << "Face Expression Name: " << face_expression.name << std::endl;
    std::cout << "Face Expression Description: " << face_expression.description << std::endl;
}

void set_face_expression(int face_expression_id) {
    auto& display_controller = robot->GetDisplayController();

    auto status = display_controller.SetFaceExpression(face_expression_id);
    if (status.code != ErrorCode::OK) {
        std::cerr << "set face expression failed, code: " << status.code
                    << ", message: " << status.message << std::endl;
        return;
    }

    std::cout << "set face expression success" << std::endl;
}

int main(int argc, char* argv[]) {
    // Bind SIGINT (Ctrl+C)
    signal(SIGINT, signalHandler);

    print_help();
}

```

(下页继续)

(续上页)

```
std::string local_ip = "192.168.55.10";
robot = std::make_unique<MagicRobot>();

// Configure local IP address for direct network connection and initialize SDK
if (!robot->Initialize(local_ip)) {
    std::cerr << "robot sdk initialize failed." << std::endl;
    robot->Shutdown();
    return -1;
}

// Connect to robot
auto status = robot->Connect();
if (status.code != ErrorCode::OK) {
    std::cerr << "connect robot failed, code: " << status.code
                << ", message: " << status.message << std::endl;
    robot->Shutdown();
    return -1;
}

std::cout << "Press any key to continue (ESC to exit)..." << std::endl;

// Wait for user input
while (true) {
    int key = getch();
    if (key == 27) { // ESC key
        break;
    }

    std::cout << "Key ASCII: " << key << ", Character: " << static_cast<char>(key) << std::endl;

    switch (key) {
        case '1':
            get_all_face_expressions();
            break;
        case '2':{
            int face_expression_id = 16;
            std::cout << "Please input face expression id: ";
            std::cin >> face_expression_id;
            set_face_expression(face_expression_id);
            std::cin.ignore();
        }
        break;
        case '3':
```

(下页继续)

(续上页)

```

        get_current_face_expression();
        break;
    case '?':
        print_help();
        break;
    default:
        std::cout << "Unknown key: " << key << std::endl;
        break;
    }

    // Sleep 10ms, equivalent to usleep(10000)
    std::this_thread::sleep_for(std::chrono::milliseconds(10));
}

// Disconnect from robot
status = robot->Disconnect();
if (status.code != ErrorCode::OK) {
    std::cerr << "disconnect robot failed, code: " << status.code
                << ", message: " << status.message << std::endl;
    robot->Shutdown();
    return -1;
}

std::cout << "disconnect robot success" << std::endl;

robot->Shutdown();
std::cout << "robot shutdown" << std::endl;

return 0;
}

```

29.2 Python

示例文件: display_example.py

```

#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import sys
import time
import signal
import termios
import tty

```

(下页继续)

(续上页)

```
import logging
from typing import Optional
import magicdog_w_python as magicdog_w

# Configure logging
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
)

# Global robot instance
robot = None

def signal_handler(signum, frame):
    """Handle Ctrl+C signal"""
    logging.info(f"\nInterrupt signal ({signum}) received.")
    if robot:
        robot.shutdown()
    sys.exit(signum)

def print_help():
    """Print help information"""
    logging.info("Key Function Description:")
    logging.info("")
    logging.info("Face expression Functions:")
    logging.info(" 1      Function 1: Get all face expressions")
    logging.info(" 2      Function 2: Set face expression")
    logging.info(" 3      Function 3: Get current face expression")
    logging.info("")
    logging.info(" ?      Function ?: Print help")
    logging.info(" ESC    Exit program")

def getch():
    """Get a single character from standard input"""
    fd = sys.stdin.fileno()
    old_settings = termios.tcgetattr(fd)
    try:
        tty.setraw(fd)
        ch = sys.stdin.read(1)
```

(下页继续)

(续上页)

```

finally:
    termios.tcsetattr(fd, termios.TCSADRAIN, old_settings)
return ch

def get_all_face_expressions(display_controller):
    """Get all face expressions"""
    status, all_face_expressions = display_controller.get_all_face_expressions()
    if status.code != magicdog_w.ErrorCode.OK:
        logging.error(
            "Failed to get all face expressions, code: %s, message: %s",
            status.code,
            status.message,
        )
        return
    logging.info("Total face expressions: %d", len(all_face_expressions))

    for i, face_expression in enumerate(all_face_expressions):
        logging.info("Face expression %d: %d", i + 1, face_expression.id)
        logging.info("  Name: %s", face_expression.name)
        logging.info("  Description: %s", face_expression.description)

def set_face_expression(display_controller, face_expression_id):
    """Set face expression"""
    status = display_controller.set_face_expression(face_expression_id)
    if status.code != magicdog_w.ErrorCode.OK:
        logging.error(
            "Failed to set face expression, code: %s, message: %s",
            status.code,
            status.message,
        )
        return

def get_current_face_expression(display_controller):
    """Get all face expressions"""
    status, face_expression = display_controller.get_current_face_expression()
    if status.code != magicdog_w.ErrorCode.OK:
        logging.error(
            "Failed to get current face expression, code: %s, message: %s",
            status.code,
            status.message,

```

(下页继续)

(续上页)

```

    )
    return

logging.info("Face expression: %d", face_expression.id)
logging.info("  Name: %s", face_expression.name)
logging.info("  Description: %s", face_expression.description)

def main():
    """Main function"""
    global robot

    # Bind SIGINT (Ctrl+C)
    signal.signal(signal.SIGINT, signal_handler)

    print_help()

    local_ip = "192.168.55.10"
    robot = magicdog_w.MagicRobot()

    # Configure local IP address for direct network connection and initialize SDK
    if not robot.initialize(local_ip):
        logging.error("robot sdk initialize failed.")
        robot.shutdown()
        return -1

    # Connect to robot
    status = robot.connect()
    if status.code != magicdog_w.ErrorCode.OK:
        logging.error(
            f"connect robot failed, code: {status.code}, message: {status.message}"
        )
        robot.shutdown()
        return -1

    logging.info("Press any key to continue (ESC to exit)...")

    display_controller = robot.get_display_controller()

    # Wait for user input
    while True:
        key = getch()
        if key == "\x1b": # ESC key

```

(下页继续)

(续上页)

```

        break

    key_ascii = ord(key)
    logging.info(f"Key ASCII: {key_ascii}, Character: {key}")

    # 1. Face expression Functions
    # 1.1 Get all face expressions
    if key == "1":
        get_all_face_expressions(display_controller)
    # 1.2 Set face expression
    elif key == "2":
        face_expression_id = 16
        try:
            user_input = input("Please input face expression id: ")
            face_expression_id = int(user_input)
        except ValueError:
            logging.info("invalid input, using default value 16.")
            set_face_expression(display_controller, face_expression_id)
    # 1.3 Get current face expression
    elif key == "3":
        get_current_face_expression(display_controller)
    # Help
    elif key == "?":
        print_help()
    else:
        logging.info(f"Unknown key: {key_ascii}")

    # Sleep 10ms, equivalent to usleep(10000)
    time.sleep(0.01)

display_controller.shutdown()
logging.info("display controller shutdown")

# Disconnect from robot
robot.disconnect()
logging.info("disconnect robot success")

robot.shutdown()
logging.info("robot shutdown")

return 0

```

(下页继续)

(续上页)

```
if __name__ == "__main__":
    sys.exit(main())
```

29.3 运行说明

29.3.1 环境准备

```
export PYTHONPATH=/opt/magic_robotics/magickdog_sdk/lib:$PYTHONPATH
export LD_LIBRARY_PATH=/opt/magic_robotics/magickdog_sdk/lib:$LD_LIBRARY_PATH
```

29.3.2 运行示例

```
# C++
./display_example
# Python
python3 display_example.py
```

29.3.3 控制说明:

- 面部表情功能:
 - 按键 1 - 获取所有面部表情
 - 按键 2 - 设置面部表情
 - 按键 3 - 获取当前面部表情
- 其他功能:
 - 按键 ? - 显示帮助菜单

29.3.4 停止程序

- 按 ESC 可以安全停止程序
- 程序会自动清理所有资源

30.1 SDK 介绍

MagicDog_W_sdk 是魔法原子为新一代机器人开发的 SDK。SDK 将底层电机控制、高层运动控制、雷达点云数据、音视频图传、SLAM、里程计等接口进行封装，并提供相关的函数接口。您可以按照我们提供的接口和例程，完成机器狗的二次开发。

30.1.1 SDK 支持情况

MagicDog_W_sdk 目前支持的机器人为 MagicDog_W 巡航版。其他型号或版本的机器人不支持使用该 SDK 进行二次开发。所有相关的 ros2 msg 均已安装到算力背包 nx 板子上。用户开发电脑连接机器人 wifi 后，执行

注：ROS2 SDK 后续不再更新，如果想要进行二开的话，建议使用 C++ API/Python API 开发

```
ssh eame@192.168.55.199
source /etc/eame/ros2.env

# 可以执行相关 ros2 命令

ros2 node list # 查看 ros2 node 节点列表

ros2 topic list # 查看 ros2 topic 节点列表

ros2 interface show # 查看 ros2 idl 详细的定位
```


机器狗 ROS2 SDK 文档

提供机器人系统 ROS2 接口服务，通过 ROS2 话题和服务方式实现对机器人的传感器数据获取、语音控制、相机管理、导航控制等功能。

本 SDK 仅支持消费级轮足巡航版（算力背包 + 3D Lidar）

注：ROS2 SDK 后续不再更新，如果想要进行二开的话，建议使用 C++ API/Python API 开发

31.1 文档概述

本手册面向机器人集成商和工程师，系统性地介绍机器狗 ROS2 SDK 的功能与接口，方便用户在算力背包上进行二次开发和集成。

适用对象：机器人集成商、行业开发者、科研人员

主要内容：系统结构、通讯协议、开发环境配置、接口说明、实用案例、常见问题

覆盖场景：工厂巡检、教育科研、定制应用等

31.2 系统架构

机器狗系统由底层硬件（主控、执行器、传感器）、嵌入式软件（固件）、中间件（ROS2 + DDS）、以及上层应用共同组成。

- **算力背包：**运行 SLAM 和 SDK 相关模块
- **主控板：**运算与调度核心，运行 ROS2
- **传感器：**包含 TOF、超声、IMU、鱼眼相机、激光雷达等

- **执行器**：用于运动与交互（如电机、舵机、显示等）
- **上位机/外部应用**：通过有线或 Wi-Fi 与主控进行交互

31.3 通信协议

SDK 所有通信基于 ROS2 的分布式架构，底层采用 DDS 中间件：

- **Topic**：适用于传感器数据、状态信息等”发布-订阅”模式的数据流转
- **Service**：用于参数配置、控制指令、触发操作等一对一请求-响应事务
- **Action**：适合需要过程反馈和结果通知的复杂任务（如导航、动作执行）
- **QoS**：可根据实际需求灵活设置，保障消息的实时性与可靠性

31.4 接口定义

31.4.1 传感器接口

超声波传感器输出

覆盖侧向与正前方低速避障场景。

话题	消息类型	index/顺序	位置说明
/ultra	std_msgs/Float32MultiArray	0	左侧
/ultra	std_msgs/Float32MultiArray	1	前方
/ultra	std_msgs/Float32MultiArray	2	右侧

头部触摸传感器

用于人机交互，可区分多种手势事件。

话题	消息类型	值	手势类型
/head_touch_state	std_msgs/Int8	0	未触发
/head_touch_state	std_msgs/Int8	1	单击
/head_touch_state	std_msgs/Int8	2	双击
/head_touch_state	std_msgs/Int8	3	上划
/head_touch_state	std_msgs/Int8	4	下划
/head_touch_state	std_msgs/Int8	5	长按
/head_touch_state	std_msgs/Int8	16	错误

电池信息

电池状态监控，包括电压、电流、温度、SOC 等。

话题	消息类型	主要字段
/	motion_msg	batt_volt (电压) batt_curr (电流) batt_temp (温度) batt_soc (剩余电量)
motion/	BMSResponse	status (电源状态) key (关机信号) batt_health (健康度) batt_loop_number
bms_dat		(循环次数) power_board_status (电源板故障位)

消息结构:

```
builtin_interfaces/Time timestamp
int16 batt_volt # 单位 mV
int16 batt_curr # 单位 mA
int16 batt_temp # 单位 C
int8 batt_soc # 剩余电量 %
int8 status # 电源状态 位字段
int8 key # 关机信号: 1 关机; 0 正常
int8 batt_health # 健康度 %
int16 batt_loop_number # 循环次数
int8 power_board_status # 电源板故障位
```

31.4.2 语音接口

获取音量

获取机器人当前的音量等级。

项目	内容
服务名	/voice/get_volume
服务类型	voice_msgs/srv/GetVolume
请求参数	{}
返回值	volume: int (取值范围 0~10)
备注	获取当前音量等级

服务调用:

```
ros2 service call /voice/get_volume voice_msgs/srv/GetVolume "{}"
```

设置音量

设置机器人音箱音量。

项目	内容
服务名	/voice/set_volume
服务类型	voice_msgs/srv/SetVolume
请求参数	{volume: int}
参数说明	volume: 音量值 (0~10)
备注	设置音箱音量

服务调用：

```
ros2 service call /voice/set_volume voice_msgs/srv/SetVolume "{volume: 7}"
```

播放 TTS

向机器人发送文本内容，并立即转为语音播报。

项目	内容
服务名	/voice/set_tts
服务类型	voice_msgs/srv/SetTTS
请求参数	{tts_id: string, content: string, priority: int, mode: int}
参数说明	tts_id: TTS 任务 IDcontent: 文本内容 priority: 优先级 mode: 播放模式
备注	文本转语音播报

服务调用：

```
ros2 service call /voice/set_tts voice_msgs/srv/SetTTS "{tts_id: 'id_1', content: '你好',
↵priority: 1, mode: 0}"
```

31.4.3 相机接口

相机开关接口

通过服务启动或关闭摄像头节点。

项目	内容
服务名	/berxel_camera_ros2/enable
服务类型	std_srvs/srv/SetBool
请求参数	{data: bool}
参数说明	data: true 开启, false 关闭
备注	控制相机节点启停

开启相机：

```
ros2 service call /berxel_camera_ros2/enable std_srvs/srv/SetBool "data: true"
```

关闭相机：

```
ros2 service call /berxel_camera_ros2/enable std_srvs/srv/SetBool "data: false"
```

数据流话题与格式

图像数据通常以 topic 形式实时输出。

话题名	消息类型	说明
/sensor_msgs	sensor_msgs::msg::Image	主摄原始图像
/depth	sensor_msgs::msg::Image + /depth/camera_info	深度图像 + 相机内参
/color	sensor_msgs::msg::Image + /color/camera_info	彩色图像 + 相机内参

31.4.4 导航接口

切换导航模式

可根据业务需求切换多种导航模式。

项目	内容
服务名	/switch_navigate_mode
服务类型	pp_msgs/srv/ChangeNavigateMode
请求参数	{set_mode: int}
参数说明	set_mode: 0 关, 1 无地图, 6 自主探索, 12 有地图
备注	切换导航工作模式

服务调用：

```
ros2 service call /switch_navigate_mode pp_msgs/srv/ChangeNavigateMode "{set_mode: 0}" # 关
ros2 service call /switch_navigate_mode pp_msgs/srv/ChangeNavigateMode "{set_mode: 1}" # 无地图
ros2 service call /switch_navigate_mode pp_msgs/srv/ChangeNavigateMode "{set_mode: 6}" # 自主探索
ros2 service call /switch_navigate_mode pp_msgs/srv/ChangeNavigateMode "{set_mode: 12}" # 有地图
```

速度输入

下发机器人期望速度。

项目	内容
话题名	/vel_app
消息类型	motion_msgs/msg/VelCommand
发布频率	建议 10Hz
参数说明	x_vel_des: X 方向速度 y_vel_des: Y 方向速度 yaw_vel_des: 旋转速度
备注	下发机器人期望速度

话题发布：

```
ros2 topic pub -r 10 /vel_app motion_msgs/msg/VelCommand "{mode_stamped:{timestamp:{sec:0,
nanosec:0},mode:2},x_vel_des:0.0,y_vel_des:0.0,yaw_vel_des:0.0}"
```

31.4.5 激光雷达接口

启动/关闭雷达节点

启动/关闭激光雷达话题。

项目	内容
服务名	/lds/enable
服务类型	std_srvs/srv/SetBool
请求参数	{data: bool}
参数说明	data: true 开启, false 关闭
备注	控制激光雷达节点启停

服务调用：

```
ros2 service call /lds/enable std_srvs/srv/SetBool "data: true" # 开启
ros2 service call /lds/enable std_srvs/srv/SetBool "data: false" # 关闭
```

激光建图/保存/定位

执行 SLAM 建图、保存当前地图、载入地图定位。

项目	内容
服务名	/set_slam_model
服务类型	cartographer_ros_msgs/srv/SetSlamModel
请求参数	{slam_model: int, path: string}
参数说明	slam_model: 0 停止, 1 建图, 3 定位 path: 地图路径
备注	SLAM 模式切换

服务调用：

注：调用建图命令，地图只会保存在算力背包的 /home/eame/cust_para/maps 目录下，无法与手机 app 联动

```
# 启动建图
ros2 service call /set_slam_model cartographer_ros_msgs/srv/SetSlamModel "{slam_model:1,path:'/
↪home/eame/cust_para/maps/123'}"

# 保存地图
ros2 service call /save_map cartographer_ros_msgs/srv/SaveMap "{path:'/home/eame/maps/test'}"

# 切换为定位模式
ros2 service call /set_slam_model cartographer_ros_msgs/srv/SetSlamModel "{slam_model:3,path:'/
↪home/eame/cust_para/maps/test'}"

# 停止 SLAM
ros2 service call /set_slam_model cartographer_ros_msgs/srv/SetSlamModel "{slam_model:0,path:'/
↪home/eame/cust_para/maps/123'}"
```

31.5 常见问题与故障排查

问题类型	解决方案
接口无响应	检查 topic/service 名称、参数格式，以及相关依赖节点是否已正常启动
数据异常	使用 ros2 topic echo 调试原始输出，关注是否有异常值（如 nan、inf）
网络丢包	优先采用有线连接，或调整 QoS 策略以提升可靠性
权限/认证失败	确认当前账户具备访问硬件和串口的足够权限

当出现 SDK 不可用时，请首先检查 SDK 版本是否符合要求，SDK 版本 tag 与机器人本体版本号对应记录，参考: [ChangeLog](#)

32.1 开发准备环境

操作系统：Ubuntu22.04，具体开发环境可以参考：《快速开始》

32.2 MagicDog_W 支持无线开发吗？

不支持，MagicDog_W 目前只支持有线连接进行开发

32.3 MagicDog_W 目前底层通讯频率是多少？

底层关节状态上报通信频率默认是 500Hz，底层关节指令下发 Publish 频率由用户控制，建议 500HZ

32.4 SDK 接口调用报错：Deadline Exceeded

SDK 接口内部 RPC 调用默认超时时间是 5s，部分步态或者特技的执行接口的执行时间可能不止 5s，如果出现该问题，可以通过设置对应接口的超时时间参数来规避该问题。

32.5 报错：Failed to disable SDK.

检查组播是否正常配置

```
ping 192.168.55.200 # 确认网卡配置是否正确，与机器狗连线是否正常
ifconfig # 确认与机器狗直连的 ethercat 网卡名称
sudo ifconfig <与机器狗直连的 ethercat 网卡名称> multicast
sudo route add -net 224.0.0.0 netmask 240.0.0.0 dev <与机器狗直连的 ethercat 网卡名称>
```

32.6 SDK 内部配置和日志如何查看？

SDK 程序执行时，默认会在/tmp 目录下生成/tmp/magicrobot_w_mjrrt.yaml 配置文件，在/tmp/logs 目录下生成 magicrobot_w_sdk.log 日志文件，SDK 内部的错误信息可以通过该日志文件查看；

32.7 调用 high_level_motion_example 时，打印以下错误 Fail initializing after {} seconds. \nPlease check the lcm connection between remote and hardware PC then try again.\n

检查组播是否正常配置

```
ping 192.168.55.200 # 确认网卡配置是否正确，与机器狗连线是否正常
ifconfig # 确认与机器狗直连的 ethercat 网卡名称
sudo ifconfig <与机器狗直连的 ethercat 网卡名称> multicast
sudo route add -net 224.0.0.0 netmask 240.0.0.0 dev <与机器狗直连的 ethercat 网卡名称>
```

32.8 调用 audio_example 或者 sensor_example 无法拿到对应的数据

检查组播是否正常配置

```
ping 192.168.55.200 # 确认网卡配置是否正确，与机器狗连线是否正常
ifconfig # 确认与机器狗直连的 ethercat 网卡名称
sudo ifconfig <与机器狗直连的 ethercat 网卡名称> multicast
sudo route add -net 224.0.0.0 netmask 240.0.0.0 dev <与机器狗直连的 ethercat 网卡名称>
```

32.9 如何检查双目相机图像延时

CompressedImage 或者 Image 数据结构的 header.stamp 字段记录图片发出时间，单位：纳秒

32.10 如何获取鱼眼相机或者 4K 相机的数据

```
# 鱼眼相机：
ffplay rtsp://192.168.12.1:8080/
# 中央的4k相机：
ffplay rtsp://192.168.12.1:8082/
```

演示上可以使用 ffplay 测试，如果想要获取每帧图像，需要自行编码

32.11 SLAM 导航不执行？

1. SLAM 导航执行前提条件一，需要进入 SLAM 定位模式和导航模式：

- SwitchToLocation()
- ActivateNavMode(NavMode::GRID_MAP)

2. SLAM 导航执行前提条件二，重定位成功：

需要通过 InitPose 进行重定位，并调取 GetCurrentLocalizationInfo 获取当前重定位状态，如果 is_localization=true，则代表重定位成功；否则，重定位失败，需要检查地图和重定位位姿，重新 InitPose 进行定位；

3. SLAM 导航目标点设置：

因为 InitPose 设置的重定位位姿是预估的，所以目标位置点建议基于 GetCurrentLocalizationInfo 获取的当前位姿进行设置目标点位姿；

32.12 运行报错：Invalid IP address

当前机器人不支持 APP 与 SDK 同时控制机器人，也不支持多个 SDK 客户端同时控制机器人呢，当出现 Invalid IP address 请求错误时，请检查是否有其他 APP 或 SDK 客户端连接；

32.13 SDK 无法订阅话题数据或者话题数据不生效？

- 检查相关话题数据流是否打开；

部分接口需要调用对应接口进行打开，比如 OpenLaserScan 打开雷达数据；

- 检查 udp 多播配置：

假设 SDK 二次开发 PC 与机器人链接的网口为 enp0s31f6，需要进行如下配置以便 SDK 与机器人的底层通信：

```
sudo ifconfig enp0s31f6 multicast
sudo route add -net 224.0.0.0 netmask 240.0.0.0 dev enp0s31f6
```

- 检查 SDK 版本与本体软件版本是否对齐：

SDK 版本 tag 与机器人本体版本号对应记录，参考: [ChangeLog](#)

- 清理残留 SDK 配置文件：

SDK 默认内置配置生成在/tmp/magicedog_w_mjrrt.yaml 文件，正常开机重启会自动清理掉；但是如果本次开机，先使用旧版本的 SDK，在/tmp 目录下生成旧版配置文件，后使用新版本 SDK 使用，默认会读取旧版本的配置（SDK 内部如果检查 magicedog_w_mjrrt.yaml 文件存在，则不重新创建，为的是方便调试）。从而导致话题订阅与发布存在问题。

最后，需要注意的是，在使用 Subscribe 等订阅类 SDK 接口时，请避免在回调函数中执行阻塞式的数据处理操作。否则，可能导致消息堆积、处理延迟，甚至引发不可预期的错误。

32.14 SDK 订阅话题数据频率不稳定？

检查 SDK 运行主机 socket 链接的接收缓存系统配置/etc/sysctl.conf，sysctl -p 或者重启主机立即生效：

```
net.core.rmem_max=20971520
net.core.rmem_default=20971520
net.core.wmem_max=20971520
net.core.wmem_default=20971520
```

32.15 日志报错打印: Call of pthread_setschedparam with insufficient privileges!

主要原因是 SDK 进行在普通用户执行，该报错根本上不影响程序运行，为解决该报错，需要在普通用户下配置系统文件/etc/security/limits.conf，确保进程拥有线程优先级设置权限：

```
*      -   rtprio    98
```

32.16 connect 返回报错: Failed to connect to robot, code: ErrorCode.SERVICE_ERROR, message: failed to connect to all addresses; last error: UNKNOWN: ipv4:192.168.55.200:50051: Failed to connect to remote host: Connection refused

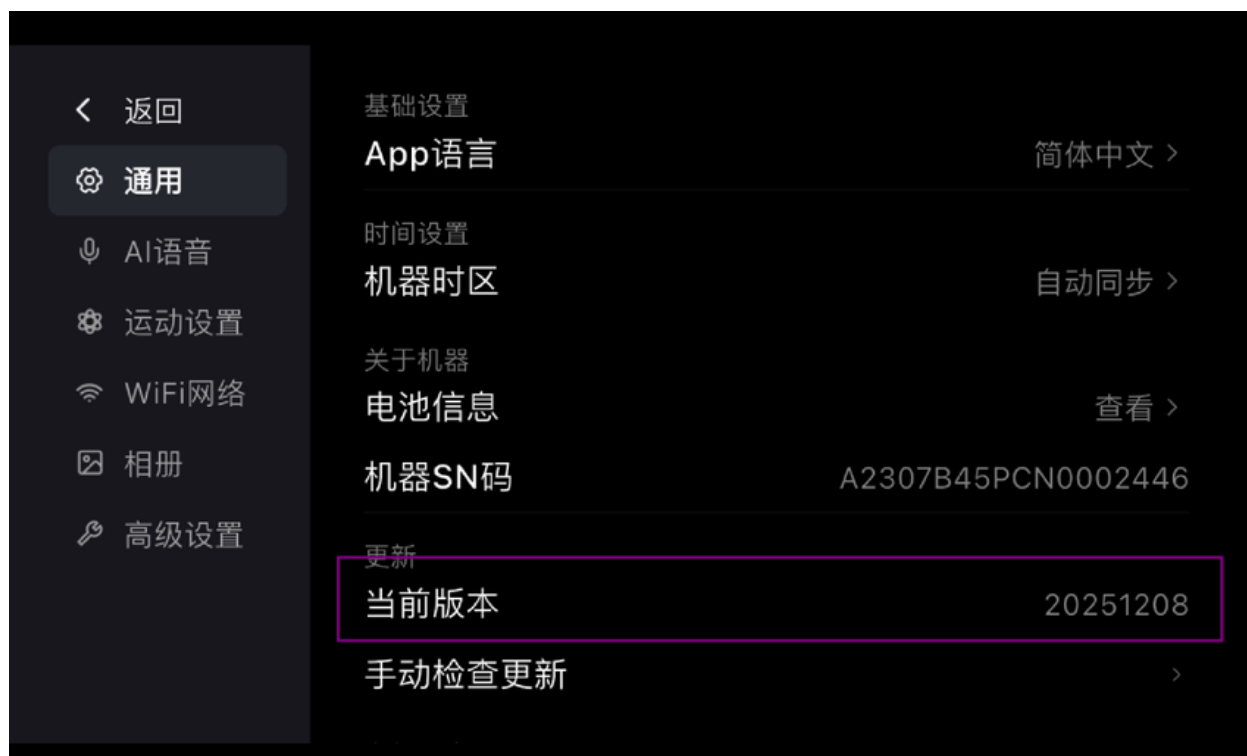
1. 无法 ping 通 192.168.55.200:

- 网线连接异常，检查硬件连接；

- 算力包 IP 配置不是 192.168.55.xx 网段;
2. 本体网关服务异常:
- 尝试重启, 恢复本体软件正常运行;

32.17 如何查看机器本体软件版本 ?

方法一: 通过 APP 查看机器人 OTA 软件版本:



方法二: 登录到机器本体内查看 OTA 软件版本:

```
# 登录到机器本体
ssh eame@192.168.55.200
# 查看机器上次OTA软件版本
cat /home/eame/devID/local_version.json
```